

Claude Multi-Account Fine Tuning

Running multiple Claude Code accounts on one host with shared history, memory, and settings

milos85vasic

2026-07-27

Claude Multi-Account Fine Tuning

1. Motivation
2. How Claude Code stores state
3. Architecture after fine-tuning
4. Reproducing this on any host
 - 4.0 Clean-slate host: `claude-bootstrap`
 - 4.1 Prerequisites
 - 4.2 Lay down the scripts directory
 - 4.3 Run the installer
 - 4.4 Add a third (or fourth, ...) account
 - 4.5 Verify the result
 - 4.6 Rollback if needed
5. Operational notes
 - 5.1 Concurrent use of two accounts
 - 5.2 Plugin path rewriting
 - 5.3 Adding a new account from scratch (no existing dir)
 - 5.4 Removing an account
 - 5.5 Backups created during install
6. Cross-platform reproduction
 - Linux
 - macOS
 - WSL / Windows Subsystem for Linux
7. Tests
 - 7.1 Running the suite
 - 7.2 What each test covers
 - 7.3 How the test harness works
 - 7.4 Adding a new test
8. The complete script source
 - 8.1 `lib.sh`
 - 8.2 `claude-unify.sh`
 - 8.3 `claude-add-account.sh`
 - 8.4 `claude-remove-account.sh`

- 8.5 `claude-list-accounts.sh`
- 8.6 `claude-rollback.sh`
- 8.7 `claude-export-docs.sh`
- 8.8 `install.sh`
- 8.9 `tests/lib/assert.sh`
- 8.10 `tests/lib/sandbox.sh`
- 8.11 `tests/run-all.sh`
- 8.12 `tests/test_lib.sh`
- 8.13 `tests/test_unify.sh`
- 8.14 `tests/test_add_remove.sh`
- 8.15 `tests/test_list.sh`
- 8.16 `tests/test_export.sh`
- 11. Provider aliases — running other LLMs through Claude Code
 - Data flow (nothing hardcoded)
 - Model tier — strongest paid if funded, strongest free if not
 - Transports
 - Router selector semantics — the / that killed 16 of 24 aliases
 - Token-limit guards — the arithmetic, and which direction is dangerous
 - HelixAgent: a local GPU backend, and why auto-start is opt-in
 - Safety
 - Commands
 - Session color — auto-applied (v1.10.0)
- 12. Per-project auto-session & per-alias color
- 13. TOON utility
- 14. Authoring rule: the generated alias file is self-contained
- 15. Bundled submodules and their pins
- 16. Kimi Code: accounts and the family model
 - 16.1 The namespace contract
 - 16.2 Kimi accounts
 - 16.3 `kimi-<id>` provider aliases
 - 16.4 Legacy rename (`kimi-*` → `kc-*`)

Claude Multi-Account Fine Tuning

Corresponds to toolkit release `claude_toolkit-1.26.7`.

This document captures, in reproducible detail, how a single host can run multiple Claude Code accounts (e.g. `claude1`, `claude2`, ..., `claudeN`) while sharing conversation history, project memory, todos, plans, plugins, and settings between them. Only account identity (`.credentials.json`, `.claude.json`, MCP auth cache) is kept private per account.

Companion artifacts:

- `scripts/` in this repository — the automation scripts, and the source of truth for the procedure described here. (Earlier revisions of this document pointed at `~/Documents/scripts/`; that was the author's pre-repo location and is no longer correct.)
- `Claude_Multi_Account_Fine_Tuning.html` — same content as this file, rendered as a standalone HTML page.
- `Claude_Multi_Account_Fine_Tuning.pdf` — same content, paginated PDF.
- `Claude_Multi_Account_Fine_Tuning.docx` — same content, Word format.

All three siblings are regenerated by `claude-export-docs.sh`, so editing the markdown is the only place changes need to be made:

```
MD_FILE="$PWD/Claude_Multi_Account_Fine_Tuning.md" bash scripts/claude-export-docs.sh
```

The `MD_FILE` override matters: the script's default is `$DOC_DIR/Claude_Multi_Account_Fine_Tuning.md` with `DOC_DIR` defaulting to `~/Documents`, so a bare invocation inside the repo renders the wrong copy.

1. Motivation

The Claude Code CLI is invoked as `claude`. By default it writes all of its state — credentials, conversation transcripts, project memory, todos, plans, plugin installs, settings — under a single config directory (`~/.claude` on Linux/macOS).

When you want to use **multiple Anthropic accounts** (for example, one personal Max subscription and one team/work account), you typically run two separate config dirs and switch between them with an environment variable:

```
# ~/.bashrc (before this setup)
export CLAUDE_BIN="$HOME/.local/bin/claude"
alias claude1="CLAUDE_CONFIG_DIR=$HOME/.claude-milos85vasic
               $CLAUDE_BIN"
alias claude2="CLAUDE_CONFIG_DIR=$HOME/.claude-milos85vasic2nd
               $CLAUDE_BIN"
```

That works for isolation, but the two accounts now have **completely separate** project memory, history, todos, and plugin sets. Switching accounts means losing the working context you built up under the other.

Goal: keep the two-alias UX (cheap account switch) while ensuring both accounts read and write the same project memory, conversation transcripts, todos, plans, plugins, and settings.

2. How Claude Code stores state

Every key piece of state lives under the directory pointed at by `CLAUDE_CONFIG_DIR` (with `~/.claude` as the default). Items fall into three categories:

Category	Path under <code>CLAUDE_CONFIG_DIR</code>	What it is
Identity (private)	<code>.credentials.json</code>	OAuth tokens for the Anthropic account.
	<code>.claude.json</code>	Account onboarding/migration markers, <code>numStartups</code> , etc.
	<code>mcp-needs-auth-cache.json</code>	Per-account MCP auth state.
Work product (share)	<code>projects/<slug>/<uuid>.jsonl</code>	Per-session conversation transcript.
	<code>projects/<slug>/memory/*.md</code>	Project memory entries (what <code>Skill</code> auto-memory writes).
	<code>history.jsonl</code>	Prompt-line history (Up/Down recall).
	<code>todos/, tasks/, plans/</code>	UUID-named files for TaskCreate/Plan tooling.
	<code>file-history/</code>	Snapshots of files edited via the Edit tool.
	<code>paste-cache/</code>	Long-paste buffer storage.
	<code>plugins/</code>	Installed plugins, marketplaces, manifest JSON.
	<code>daemon/, jobs/</code>	Background-agent registry (<code>roster.json</code> + <code>dispatch</code>) and its job store.
	<code>CLAUDE.md</code>	User-scope memory / instructions.
		Top-level config:
Per-dir own file (partially synced)	<code>settings.json</code>	<code>enabledPlugins</code> , <code>effortLevel</code> , <code>permissions</code> , <code>hooks</code> , <code>model</code> . Not a

Category	Path under CLAUDE_CONFIG_DIR	What it is
Runtime (share-able with care)		shared symlink — see below.
	sessions/	Active session lockfiles. Not safe to share between concurrent processes.
	telemetry/	Per-account telemetry batches.
	cache/, backups/, session-env/, shell- snapshots/	Ephemeral runtime caches.
	stats-cache.json	Per-account usage / quota stats.

The key observation: every **work-product** item is keyed either by UUID (session, todo, plan, file edit) or by project slug. That means combining two accounts' work-product directories is a **set-union operation**: copying both into the same destination never overwrites either's data, because each file's path is already unique.

The handful of files that genuinely need merging are:

- settings.json — **each config dir keeps its OWN real file; it is deliberately not in the shared set** (CMA_SHARED_ITEMS in lib.sh omits it, and claude-list-accounts excludes it from the link check for the same reason). Only the enabledPlugins map is unified: the unifier extracts that map from every account's own file, unions it into the shared *template* at \$SHARED_DIR/settings.json, and propagates the union back into each dir (seed_own_settings → cma_own_settings_seed). Every other key — effortLevel, permissions, hooks, model, skipDangerousModePermissionPrompt — stays local to the dir that set it, so per-alias configuration never leaks across aliases or providers. The test suite asserts both halves: the union arrives everywhere, and a non-plugin key set on acct1 is absent from acct2 *and* from the shared template. An earlier revision of this document described settings.json as a symlink into the shared store with "most permissive keys win"; that is no longer how it works.
- history.jsonl — append-only prompt log. We concatenate both files and de-duplicate by line, preserving order of first appearance.
- plugins/installed_plugins.json and plugins/known_marketplaces.json — these embed **absolute paths** to plugin code. We rewrite those paths to live under the shared store regardless of which account they came from.

3. Architecture after fine-tuning

```
~/ .claude-shared/                                <-- single source of truth
  projects/                                       (transcripts + memory; per-slug)
  todos/  tasks/  plans/
  file-history/  paste-cache/
  plugins/
    installed_plugins.json                       (paths rewritten -> ~/.claude-
shared/...)
    known_marketplaces.json                     (paths rewritten -> ~/.claude-
shared/...)
    cache/<marketplace>/<plugin>/<version>/
    marketplaces/<marketplace>/
  shell-snapshots/  session-env/
  telemetry/  sessions/  backups/  cache/
  daemon/  jobs/                                (background-agent registry + job
store)
  history.jsonl  stats-cache.json
  settings.json                                (TEMPLATE only – enabledPlugins
union)
  CLAUDE.md

~/ .claude-milos85vasic/                          <-- account 1 (claude1 alias)
  .credentials.json    (PRIVATE – claude1's OAuth tokens)
  .claude.json         (PRIVATE – claude1's onboarding state)
  mcp-needs-auth-cache.json (PRIVATE)
  settings.json        (OWN real file – enabledPlugins synced from
the
                                shared template; every other key stays
local)
  projects      -> ~/.claude-shared/projects
  todos         -> ~/.claude-shared/todos
  tasks         -> ~/.claude-shared/tasks
  plans         -> ~/.claude-shared/plans
  plugins       -> ~/.claude-shared/plugins
  daemon        -> ~/.claude-shared/daemon
  jobs          -> ~/.claude-shared/jobs
  history.jsonl -> ~/.claude-shared/history.jsonl
  CLAUDE.md     -> ~/.claude-shared/CLAUDE.md
  ... (every shared item is a symlink)

~/ .claude-milos85vasic2nd/                       <-- account 2 (claude2 alias)
  .credentials.json    (PRIVATE – claude2's OAuth tokens)
  .claude.json         (PRIVATE – claude2's onboarding state)
  mcp-needs-auth-cache.json (PRIVATE)
  ... (same symlink set as above)

~/ .claude/                                         <-- user-scope default root
  CLAUDE.md     -> ~/.claude-shared/CLAUDE.md
  plugins/
```

```
cache          -> ~/.claude-shared/plugins/cache
marketplaces  -> ~/.claude-shared/plugins/marketplaces
```

Properties of this layout:

1. **Account switching is purely a credentials swap.** Running `claude1` vs `claude2` changes `CLAUDE_CONFIG_DIR`, which changes which `.credentials.json` is loaded — everything else (history, memory, plugins) resolves through symlinks to the same shared store.
 2. **Adding a new account is the same operation, parameterised.** A new `~/.claude-<name>` dir with the same symlink set + its own `.credentials.json` slots in seamlessly. See `claude-add-account.sh`.
 3. **Account identity stays compartmentalised.** OAuth tokens are not in the shared store, so leaking the shared store does not leak account credentials.
 4. **Rollback is mechanical.** Every original directory was moved to `<path>.preunify.<timestamp>` before the symlink replaced it. A single script (`claude-rollback.sh`) reverses the migration.
-

4. Reproducing this on any host

This section is the actual step-by-step procedure. It assumes you have two (or more) Claude Code accounts already set up the conventional way — one config directory per account, each with its own `.credentials.json`. If you don't have a second account yet, run the first three steps and then use `claude-add-account.sh` to create one.

4.0 Clean-slate host: `claude-bootstrap`

If you are setting up a **brand-new machine** where Claude Code is installed but no accounts have been created or logged in yet, use `claude-bootstrap` instead of the §4.2–§4.3 flow. It provisions N fresh accounts in one shot:

```
git clone <this repo> claude-toolkit
cd claude-toolkit
bash scripts/claude-bootstrap.sh --count 2 --yes
exec $SHELL -l
claude1 /login
claude2 /login
```

Key flags:

Flag	Effect
<code>--count N</code>	Create N accounts named <code>claude1...</code> <code>claudeN</code> .

Flag	Effect
--yes	Non-interactive; skip confirmation prompts.
--aliases personal,work	Use custom alias names instead of --count.
--dir-of NAME=PATH	Override the config-dir path for a specific alias.

claude-bootstrap is **idempotent** — safe to re-run after pulling updates. It calls `install.sh` internally, so you do not need to run both.

4.1 Prerequisites

```
# Linux (Debian/Ubuntu/Fedora/Arch)
sudo apt-get install -y rsync jq gawk pandoc weasyprint #
# Debian/Ubuntu

# or
sudo dnf install -y rsync jq gawk pandoc weasyprint #
# Fedora

# or
sudo pacman -S rsync jq gawk pandoc weasyprint # Arch

# macOS (Homebrew)
brew install rsync jq gawk pandoc weasyprint
```

weasyprint is optional but produces the best-looking PDF; the export script will fall back to `wkhtmltopdf` or `chromium --headless` if weasyprint isn't available.

4.2 Lay down the scripts directory

Clone or copy the entire `scripts/` directory; all scripts listed in `scripts/README.md` are required:

```
git clone <this repo> claude-toolkit
cd claude-toolkit
```

All scripts are set executable in the repository; no `chmod` step is needed.

4.3 Run the installer

```
bash scripts/install.sh
```

What `install.sh` does, in order:

1. Verifies `rsync`, `jq`, `awk` are on `PATH`; warns if `pandoc` is missing.

2. Symlinks every `claude-*.sh` script into `~/.local/bin/` so they're invokable as plain commands (`claude-unify`, `claude-add-account`, ...).
3. Adds `export PATH="$HOME/.local/bin:$PATH"` to `~/.bashrc` and `~/.zshrc` if missing.
4. Creates the managed alias file at `~/.local/share/claude-multi-account/aliases.sh` and ensures it's sourced from `~/.bashrc` and `~/.zshrc`.
5. Migrates any pre-existing inline alias `claude<N>=...` lines from `~/.bashrc` / `~/.zshrc` into the managed alias file, commenting out the originals (and saving a `.preunify.<timestamp>` backup of the rc file first).
6. Builds the bundled Go router (`claude-ccr-build.sh`). A failure here is recorded as a hard install failure, because provider aliases on the **router** transport cannot launch without it.
7. Builds the compatibility proxy (`claude-proxy-build.sh`). A failure here is recorded as *degraded*, not fatal: the affected aliases (`helixagent`, `poe`, `kim*`, `sarvam`) still launch, just without their compatibility shims.
8. Installs the shell session hook and refreshes provider aliases.
9. Auto-detects every `~/.claude-*` directory and runs `claude-unify.sh` to merge them into `~/.claude-shared/`.
10. Re-renders the doc siblings via `claude-export-docs.sh` — **only if** `pandoc` is present *and* `~/Documents/Claude_Multi_Account_Fine_Tuning.md` exists. On a plain repository checkout that file is not there, so this step is skipped and the siblings must be regenerated explicitly with the `MD_FILE=` invocation shown at the top of this document.

It does **not** build the vendored Go toolchain (§15): `claude-go-build` is symlinked onto `PATH` but never invoked, so compiling Go from source stays an explicit operator decision.

After it finishes, reload your shell:

```
exec $SHELL -l
claude-list-accounts
```

You should see every account dir, with `CRED`S: yes and a `LINK`S: 5/5 indicator showing the shared symlinks are in place. The denominator is the size of `CHECK_LINK`S in `claude-list-accounts.sh` — currently `projects`, `history.jsonl`, `plugins`, `todos`, `CLAUDE.md`. `settings.json` is **not** in that list, because each dir owns its own copy (§2); a tool that reported it as "not linked" would be reporting the intended design as a fault.

4.4 Add a third (or fourth, ...) account

```
claude-add-account
# prompts:
```

```
# Alias name      [claude3]:  
# Config directory [/home/you/.claude-claude3]:
```

If you accept both defaults, you get:

- `~/.claude-claude3/` created with the full set of shared-state symlinks already in place.
- `alias claude3="CLAUDE_CONFIG_DIR=$HOME/.claude-claude3 \ $CLAUDE_BIN"` appended to the managed alias file.
- A reminder to authenticate: `claude3 /login`.

Custom alias names work just as well:

```
claude-add-account --alias work --dir ~/.claude-work --yes  
work /login
```

4.5 Verify the result

```
claude-list-accounts
```

```
# Expected output (your alias names may differ):  
# ALIAS      CONFIG DIR  
#          CRED$  LINK$  NOT$  
# claude1    /home/you/.claude-milos85vasic  
#          yes    5/5  
# claude2    /home/you/.claude-milos85vasic2nd  
#          yes    5/5  
# claude3    /home/you/.claude-claude3  
#          no     5/5  
#  
# Shared store: /home/you/.claude-shared  
# Alias file:   /home/you/.local/share/claude-multi-account/  
#               aliases.sh
```

You can confirm symlinks resolve correctly:

```
readlink -f ~/.claude-milos85vasic/projects  
# /home/you/.claude-shared/projects  
  
readlink -f ~/.claude-milos85vasic2nd/projects  
# /home/you/.claude-shared/projects  
  
# Both should print the same path. The same is true for todos,  
# plans,  
# history.jsonl, CLAUDE.md, daemon, jobs, etc. – but NOT  
# settings.json,  
# which is each dir's own real file by design.
```

4.6 Rollback if needed

Every directory or file the unifier touched was moved to `<path>.preunify.<timestamp>` before the symlink was created. To restore:

```
claude-rollback
# moves ~/.claude-shared aside and restores every .preunify.*
# backup
```

The shared store isn't deleted — it's renamed to `~/.claude-shared.removed.<timestamp>` so you can manually recover any data written through the symlinks before rolling back.

5. Operational notes

5.1 Concurrent use of two accounts

`sessions/` is shared in this configuration. A live Claude Code session holds its lockfile under `sessions/`, and two concurrent processes writing to the same lockfile path can confuse each other. **Avoid running `claude1` and `claude2` simultaneously in two terminals.** If you need true concurrent use, override the unifier to keep `sessions/` private:

```
# in claude-unify.sh, remove "sessions" from SHARED_ITEMS
# then run claude-unify again – it'll relink sessions/ as per-account
```

`history.jsonl` is append-only and is mostly safe under concurrent writes (worst case: a small number of duplicate lines, which the next unify run will de-dupe).

5.2 Plugin path rewriting

Claude Code records absolute paths in `plugins/installed_plugins.json` and `plugins/known_marketplaces.json`. These paths can point to one of:

- `~/.claude/plugins/cache/...` (user-scope installs from any account)
- `~/.claude-<name>/plugins/cache/...` (project-scope installs from that account)

The unifier rsyncs plugin code from all three roots (`~/.claude/plugins/`, plus every account's `plugins/`) into `~/.claude-shared/plugins/`, then rewrites every `installPath` / `installLocation` to point

under `~/.claude-shared/plugins/`. It also symlinks `~/.claude/plugins/cache` and `~/.claude/plugins/marketplaces` into the shared store, so any future user-scope install lands in the unified spot.

If you ever install a plugin and Claude reports "plugin code not found," re-run `claude-unify.sh` to repath the manifest.

5.3 Adding a new account from scratch (no existing dir)

The `claude-add-account.sh` script works whether or not the target dir exists yet. If it doesn't exist, the script creates an empty dir, links every shared item into it, and registers the alias. The new account has no credentials of its own — your next step is `claudeN / login`.

If you want to also pre-populate the new account with a `.claude.json` template (e.g., to skip the new-user warmup tips), copy one from an existing account *after* the add-account script finishes:

```
cp ~/.claude-milos85vasic2nd/.claude.json ~/.claude-
claude3/.claude.json
# but DO NOT copy .credentials.json — that would leak the source
# account's OAuth tokens to the new account
```

5.4 Removing an account

```
claude-remove-account --alias claude3
# default: archives ~/.claude-claude3 -> ~/.claude-
#             claude3.removed.<timestamp>
# add --delete to remove it instead
```

The shared store is left intact; other accounts continue to function.

5.5 Backups created during install

The installer never deletes existing data. Every directory or file it replaced lives at `<original-path>.preunify.<timestamp>` (or `<original-path>.removed.<timestamp>` for rollback). You can safely delete those backups once you've confirmed the new setup works:

```
find ~ -maxdepth 3 -name '*.preunify.*' -print      # see what's
there
find ~ -maxdepth 3 -name '*.preunify.*' -delete     # delete once
happy
```

A pre-migration tarball is also placed in `~/Documents/claude-config-backups/` by the original migration. Keep it until you're confident in the new layout.

6. Cross-platform reproduction

Linux

Everything in this document was developed and tested on Linux 6.12.41 with bash 5.2.37, jq 1.8.1, rsync 3.2.7, pandoc 3.10. No Linux-specific behaviours are relied on; the scripts use only POSIX-portable tools plus jq.

On lint, be precise about the severity — an earlier revision of this document claimed the scripts "pass shellcheck clean", which is true only at one severity level and ambiguous at every other. Measured with shellcheck 0.11.0 over scripts/: **0 findings at error severity**; warning and style severities do report findings (62 and 203 respectively at the v1.26.7 measurement). The suite gate is the error level.

macOS

macOS ships an older bash (3.2) by default. Install GNU bash via Homebrew (`brew install bash`) and update the shebangs only if you hit syntax issues — the scripts use bash 4+ features only where the fallback is explicit (mapfile, associative arrays). On a default macOS:

```
brew install bash jq rsync gawk pandoc weasyprint
echo "$(brew --prefix)/bin/bash" | sudo tee -a /etc/shells
chsh -s "$(brew --prefix)/bin/bash"
```

Everything else (paths, \$HOME, ~/.claude-* convention) is identical to Linux.

WSL / Windows Subsystem for Linux

WSL is just Linux for our purposes. Use the Linux instructions. If you also run Claude Code natively on Windows, the Windows version uses a different config root entirely (%APPDATA%\claude) and is not covered by this toolkit.

7. Tests

Everything in this toolkit is covered by a hermetic test suite under `scripts/tests/` — **58 `test_*.sh` files** at the time of writing (`ls scripts/tests/test_*.sh | wc -l` is the authoritative count; the orchestrator auto-discovers, so the number moves with every feature). Each test runs against a temporary `$HOME` created by `mktemp -d -t cma-test.XXXX`; the real `~/.claude-*` directories are never touched. After the test process exits, the sandbox is `rm -rf'd` (with a safety check that the path basename starts with `cma-test.`).

7.1 Running the suite

```
# All tests:
bash scripts/tests/run-all.sh

# A single file (no test_ prefix, no .sh suffix):
bash scripts/tests/run-all.sh unify
bash scripts/tests/run-all.sh add_remove
```

Output is line-based. Each file prints its own `[PASS]/[FAIL]` lines and ends with a per-file summary; the orchestrator then prints a tally across files:

```
==> test_unify.sh
```

```
• every shared item becomes a symlink into the shared store
  [PASS] .claude-acct1 projects linked: ...
  [PASS] .claude-acct2 projects linked: ...
• shared TEMPLATE settings.json enabledPlugins is the union
{a,b,c}
  [PASS] union (.enabledPlugins | keys | sort | join(",") =
a,b,c)
  ...
```

```
✓ 42 passed, 0 failed
```

```
=====
Test files: 58   passed: 58   failed: 0   (skipped-prereq: 0)
ALL GREEN
```

The orchestrator's exit code is 0 only if every `test_*.sh` file's own summary returned 0. Two guards sit on top of that, because an exit code alone is not trustworthy evidence:

- a file that exits 0 while having printed `[FAIL]` (a missing final summary, for instance) is forcibly re-scored as a failure;
- a file that exits 0 after printing `SKIP` on a missing prerequisite is counted in a separate `skipped-prereq` column, so an

environment-driven skip can never masquerade as a genuine pass inside "ALL GREEN".

Only one suite run at a time. `run-all.sh` and `run-proof.sh` both take a serialization lock (`tests/lib/suite-lock.sh`) before doing anything. A repo that mutates while its own tests execute cannot produce reproducible results — a second agent adding a test file mid-run is enough to make a set of deterministic tests look flaky — so overlapping runs are refused on purpose. The behaviour you will actually see:

- **Contention:** the run announces the holder's PID and waits, up to `CMA_SUITE_LOCK_WAIT` seconds (default 600). Usually the other run finishes and yours proceeds; that is the common case and needs no action.
- **Timeout:** it gives up with exit **75** (`EX_TEMPFAIL`) rather than hanging. Set `CMA_SUITE_LOCK_WAIT=0` if you would rather fail immediately.
- **Nesting:** `run-proof.sh` invokes `run-all.sh` as a child, and both are documented entry points, so both lock. The child *inherits* the parent's lock instead of deadlocking against it — and the inheritance is verified (live PID, matching lock path, PID actually recorded in the lock), so a stale or forged environment variable cannot quietly turn locking off.
- **A crashed run does not wedge the suite:** `flock(1)` releases automatically, and the portable `mkdir` fallback (macOS ships no `flock`) breaks a stale lock via `rename(2)`, which exactly one contender can win.

The lock file lives in the git dir (or `$TMPDIR`), never inside the tracked tree, so it can never show up in `git status` or be committed by accident.

7.2 What each test covers

Every `test_*.sh` file is auto-discovered by `run-all.sh`. The table below describes the core set — ten of the current 58 files. The suite has grown well beyond what any table can usefully mirror (it now also covers the router build and conformance, the `cma-proxy`, output-token arithmetic, route attribution, alias-file concurrency and self-containment, redaction, the release gate, suite locking, and more), so `ls scripts/tests/test_*.sh` is the authoritative list:

File	What it asserts
<code>test_lib.sh</code>	<code>cma_validate_alias</code> accepts/rejects correctly; <code>cma_suggest_alias</code> returns the next free <code>claudeN</code> (skipping past existing aliases, not just counting); <code>cma_write_alias</code> is idempotent; <code>cma_remove_alias</code> deletes; <code>cma_ensure_alias_file</code> patches <code>.bashrc</code> ;

File	What it asserts
	cma_detect_accounts excludes the shared store. Every shared item becomes a symlink to \$SHARED_DIR; settings.json is each dir's OWN real file and not a symlink; the shared <i>template</i>'s enabledPlugins is a strict union of every account's set and that union is propagated back into each dir; every non-enabledPlugins key stays local (an effortLevel set on acct1 is absent from acct2 and from the template, and vice versa); history.jsonl is concatenated and de-duplicated; on memory-file conflicts, the most recently touched account wins; plugins/installed_plugins.json and known_marketplaces.json have absolute paths rewritten to \$SHARED_DIR/plugins/; .credentials.json and .claude.json are NOT symlinked; re-running claude-unify.sh produces a byte-identical settings.json (idempotency); N>2 accounts (adding acct3 and re-unifying) works; --rollback restores every .preunify.* backup and moves the shared store aside.
test_unify.sh	
test_add_remove.sh	claude-add-account.sh --yes creates the dir with full symlink set and writes a shell-safe alias line; refuses to overwrite an existing dir; accepts a custom alias name + custom dir; validates alias names against the safe regex; claude-remove-account.sh --archive moves the dir aside and removes the alias; --delete actually deletes; unknown alias errors out.
test_list.sh	Empty host (no accounts) renders just the header; populated host renders one row per detected account dir; the shared store path is printed; accounts missing credentials show CREDs: no.
test_export.sh	claude-export-docs.sh (with MD_FILE=override) produces both a .html and a .pdf; <!-- INCLUDE: <relative path> --> markers are expanded in the output; output PDF starts with the %PDF- magic bytes; re-running is non-destructive.
test_opencode.sh	Builds a fake Claude plugin cache in a throwaway \$HOME and asserts the generated OpenCode config has the right shape: skills wired, MCP servers translated (wrapped + bare formats), dedup, collision-rename,

File	What it asserts
	secret/runtime enable gating, \$ {CLAUDE_PLUGIN_ROOT} expansion, instruction inclusion, pre-existing config preserved, idempotency, and --dry-run safety. No network, no real ~/.claude, no real opencode binary required.
test_sessions.sh	Verifies sessions, project memory, and todos created under one account become physically visible to every other account after unify. Covers: .claude.json projects subtree union; auth-private keys (userID, oauthAccount, etc.) preserved per-account; session JSONL byte-identical from every account's symlinked dir; project memory cross-account visibility; cma_merge_claude_json idempotency; claude-sync-state.sh push/pull round-trip.
test_coverage.sh	Coverage for lib.sh behaviors not exercised in test_lib.sh (new in v1.7.8): cma_ensure_alias_file fresh creation + idempotency + old-format migration; cma_can_prompt under CMA_NONINTERACTIVE=1 and no-tty; cma_enable_plugins JSON shape and additive behaviour; cma_link_shared_items turning all CMA_SHARED_ITEMS into symlinks; stats-cache.json newest-by-mtime selection via run_unify.
test_providers.sh	Tests for the provider-alias feature (claude-providers). Primary focus: account-detection regression — provider dirs (~/.claude-prov-*) must be invisible to cma_detect_accounts so they never merge into real-account auth or disturb claude-unify / claude-add-account.
test_claude.sh	Proves claudeN aliases are untouched by provider code: cma_run includes claude-sync-state pull/push hooks, contains no proxy or transformer code; cma_run_provider has proxy detection. Reads the live alias file; SKIPS if no alias file is present.

7.3 How the test harness works

Two small libraries sit under tests/lib/:

- **assert.sh** — it, assert_eq, assert_file, assert_dir, assert_symlink_to, assert_not_symlink, assert_file_contains, assert_file_not_contains, assert_jq, assert_lines, assert_exit.

Each helper prints a [PASS] or [FAIL] line and adjusts the counters used by summary at end-of-file.

- **sandbox.sh** — `make_sandbox` (creates a tempdir and overrides `HOME`, `SHARED_DIR`, `ALIAS_FILE`, `DEFAULT_DIR`, `CLAUDE_BIN`), `make_account` `NAME` [`--plugins`] [`--settings JSON`] [`--history a|b|c`] [`--memory KEY:VALUE`] [`--todo SUBJECT`] (builds a realistic per-account fixture dir), plus thin wrappers `run_unify`, `run_add_account`, etc. It also carries two hygiene helpers described below.

Sandbox hygiene: `assert_sandboxed` and `sandbox_stub`. These cover two *different* failure modes, and the distinction matters.

`assert_sandboxed` is called by `make_sandbox` itself and exits 99 unless `$HOME`'s basename matches `cma-test.*`. It proves the `HOME` switch actually took before any test writes a byte — it catches a test that never sandboxed at all.

`sandbox_stub` `PATH` (stub content on stdin) is the required way to install a fake executable under `$HOME/.local/bin`. It asserts the sandbox, refuses any target that escapes `$HOME`, and — the guarantee a bare `cat >` does not give — **deletes an existing symlink before writing, rather than writing through it.**

The reason this second helper exists is a real incident, not a hypothetical. `install.sh` symlinks every `claude-*.sh` into `~/.local/bin`, and those links point at the real repo. A shell redirect follows symlinks. A test that did `cat > "$HOME/.local/bin/claude-session"` therefore wrote *through* the link and truncated the production `scripts/claude-session.sh` from 201 lines to an 8-line stub — and `scripts/claude-sync-state.sh` from 103 lines to 2 — which broke every provider alias with `No conversation found with session ID: ...`

The part to internalise: **`$HOME` was a perfectly valid sandbox at the time.** `install.sh` had created those links *inside* the sandbox, so `assert_sandboxed` passed, exactly as designed, and could never have caught this. The sandbox assertion is not the protection here — breaking the symlink before writing is. `test_sandbox_hygiene.sh` lints the whole suite mechanically for this pattern (bare redirects into `.local/bin`) and for hardcoded `/tmp` write targets, reporting `file:line` so a violation is directly actionable.

One subtlety worth knowing: `lib.sh` (the production helper) has `set -euo pipefail`. When a test sources it for unit-testing the helper functions, the `set -e` flag stays active in the test shell. The unit test file (`test_lib.sh`) therefore runs `set +e` immediately after sourcing `lib.sh` so that intentional failures (e.g. `cma_validate_alias "bad name"`) don't abort the whole test process. End-to-end tests don't have this problem because they invoke the scripts as external processes (via `run_unify`, etc.), which run in their own shell.

7.4 Adding a new test

1. Create tests/test_<feature>.sh. Mark it executable.
2. Source the two libs:

```
TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"
```

3. Call make_sandbox once at the top.
 4. Use it "... " to label each scenario, then a sequence of assert_* calls.
 5. Never write to \$HOME/.local/bin with a bare redirect — use sandbox_stub "\$HOME/.local/bin/<name>" with the stub body on stdin, and keep every other temporary path under \$SANDBOX_HOME or mktemp, never a literal /tmp/.... test_sandbox_hygiene.sh enforces both mechanically.
 6. End the file with summary so the orchestrator picks up the result.
 7. Re-run bash tests/run-all.sh — auto-discovery will pick it up. If another suite run is already in progress it will wait for the lock (and exit 75 if that run never finishes); that is expected, not a failure of your test.
-

8. The complete script source

For convenience and audit, part of the toolkit source is reproduced below. The authoritative copies live in this repository's scripts/ directory.

How the reproduction works, and what it covers. Each fenced block below holds a single <!-- INCLUDE: <relative path> --> marker rather than a copy of the file. claude-export-docs.sh preprocesses the markdown with awk before handing it to pandoc, replacing each marker with the referenced file's contents (paths are resolved relative to the directory containing the .md). So the markdown stays small and reviewable while the generated HTML/PDF/DOCX are self-contained snapshots of the real code at render time — there is no copy to drift.

Two consequences worth knowing:

- **The included set is a subset, not the whole toolkit.** The sixteen markers below cover lib.sh, the six account-management commands, the installer, and the original test core. scripts/ currently holds 26 *.sh entrypoints; the provider stack (claude-providers.sh, providers-verify.sh, providers-semantic.sh, verify_superpowers_tui.sh), the build entrypoints (claude-ccr-build.sh, claude-proxy-build.sh, claude-go-build.sh),

claude-session.sh, claude-sync-state.sh, claude-release-gate.sh, claude-gc.sh and others are **not** inlined here. Read them in the repository.

- **A marker is consumed wherever it appears**, including inside prose or a code span. If the referenced path does not exist, awk prints nothing and skips the line — the whole line silently disappears from the rendered output. That is why the test_export.sh row in §7.2 writes the marker with a spaced placeholder (<relative path>): a literal INCLUDE: path --> there would have deleted that table row from the HTML and PDF. Keep any future prose mention of the marker un-matchable the same way.

One pre-existing wart, recorded rather than worked around: the export script hardcodes --metadata title="Claude Multi-Account Fine Tuning" for the HTML render, so the HTML <title> is fixed regardless of the YAML front matter of whatever MD_FILE is passed. It is correct for *this* document and wrong for any other one rendered through the same script.

8.1 lib.sh

```
#!/usr/bin/env bash
# lib.sh – shared functions for the Claude multi-account toolkit.
# Sourced by the other scripts; no side effects on its own.

set -euo pipefail

# Which rc files to source the alias file from. macOS interactive
# shell is
# zsh; touching .bashrc there just creates noise the user has to
# clean up.
# On Linux we keep both because either may be the login shell.
if [[ "$(uname -s)" == "Darwin" ]]; then
    CMA_RC_FILES=( "$HOME/.zshrc" )
else
    CMA_RC_FILES=( "$HOME/.bashrc" "$HOME/.zshrc" )
fi

# Resolve paths the user can override. SHARED_DIR is the single
# canonical
# location for cross-account state; ALIAS_FILE is the rc-sourced
# file we
# manage aliases through; ACCOUNT_PREFIX is the dir-name prefix
# for new
# per-account config directories.
: "${SHARED_DIR:= $HOME/.claude-shared}"
: "${ALIAS_FILE:= $HOME/.local/share/claude-multi-account/
aliases.sh}"
: "${ACCOUNT_PREFIX:= .claude-}"
```

```

# Resolve the Claude Code binary for the alias wrappers. Prefer an
  explicit
# CLAUDE_BIN, then $PATH, then the common install locations. npm's
  global prefix
# varies per host (e.g. ~/.npm-global vs ~/.local vs Homebrew), so
  a fixed
# ~/.local/bin default mis-points on hosts where `npm i -g
  @anthropic-ai/...`
# landed elsewhere – making EVERY alias launch fail "No such
  file". Checking
# the real locations keeps a fresh install working without a
  manual symlink.
cma_resolve_claude_bin() {
  if [ -n "${CLAUDE_BIN:-}" ]; then printf '%s\n' "$CLAUDE_BIN";
    return 0; fi
  local c; if c="$(command -v claude 2>/dev/null)"; then printf
    '%s\n' "$c"; return 0; fi
  local p
  for p in "$HOME/.local/bin/claude" "$HOME/.npm-global/bin/
    claude" \
    /opt/homebrew/bin/claude /usr/local/bin/claude; do
    [ -x "$p" ] && { printf '%s\n' "$p"; return 0; }
  done
  printf '%s\n' "$HOME/.local/bin/claude"
    # fallback (created by install/symlink)
}
# Resolved once at source time for callers that want the default
  without paying
# for a re-probe. The alias renderer does NOT use it – it prefers
  the value
# already recorded in the alias file when that still resolves (see
# _cma_alias_claude_bin), so a working per-host path is never
  clobbered.
# shellcheck disable=SC2034 # part of lib.sh's public surface for
  sourcing scripts
CLAUDE_BIN_DEFAULT="$(cma_resolve_claude_bin)"

cma_log() { printf '\033[36m[cma]\033[0m %s\n' "$*" >&2; }
cma_warn() { printf '\033[33m[cma warn]\033[0m %s\n' "$*" >&2; }
cma_err() { printf '\033[31m[cma err]\033[0m %s\n' "$*" >&2; }
cma_die() { cma_err "$*"; exit 1; }

cma_require() {
  command -v "$1" >/dev/null 2>&1 || cma_die "missing required
    tool: $1"
}

# Keys inside .claude.json that must NEVER leak across accounts
  (auth + identity).
# Everything else in .claude.json – projects map, UX state,
  caches, etc. – is
# safely shareable. New auth keys should be added here, not
  assumed shareable.
CMA_CLAUDE_JSON_PRIVATE_KEYS='["userID","oauthAccount","firstStartTime","claudeCoo

```

```

# Deep-merge every account's .claude.json so each account's file
# ends up with
# its OWN auth keys + the UNION of all other top-level keys
# (rightmost-wins
# for scalar conflicts, recursive object merge for `projects` and
# friends).
#
# Args: one or more account config dirs.
# Side effect: rewrites each $acct/.claude.json in place. Skips an
# account
# that doesn't yet have a .claude.json (e.g. brand-new dir before
# first login).
cma_merge_claude_json() {
    local accts=("$@")
    (( ${#accts[@]} >= 1 )) || return 0
    cma_require jq

    # Build the merged "shared portion" (everything except private
    # keys).
    local shared_tmp; shared_tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    printf '{}\n' > "$shared_tmp"
    local acct prev="$shared_tmp"
    for acct in "${accts[@]}; do
        local f="$acct/.claude.json"
        [[ -s "$f" ]] || continue
        local next; next="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
        # $a (accumulator) * ($b stripped of private keys). jq's `*`
        # is recursive
        # deep-merge for objects; rightmost wins on scalar conflicts
        # AND on array
        # values (arrays are replaced, not element-unioned – e.g. a
        # per-project

        # prompt-history array takes the last account's copy).
        # This is a deliberate

        # deep-merge trade-off: the projects subtree is unioned at
        # the object-key
        # level (which is what the cross-account session/MCP/memory
        # index needs);
        # blind element-level array union would be wrong for config-
        # style arrays.
        if ! jq -s --argjson priv "$CMA_CLAUDE_JSON_PRIVATE_KEYS" '
            . as [$a, $b]
            | ($b | with_entries(select(.key as $k | $priv | index($k) |
                not))) as $shared_b
            | ($a // {}) * ($shared_b // {})
        ' "$prev" "$f" > "$next"; then
            rm -f "$next"
            cma_warn ".claude.json in
                $acct is not valid JSON – skipping its contribution"
            continue
        fi
        rm -f "$prev"
    done
}

```

```

prev="$next"
done

# Sticky-true TRUST preservation (§11.4 anti-bluff config
# hygiene): the jq `*`
# merge above is last-writer-wins on scalars, so a per-project
# `projects[<path>].hasTrustDialogAccepted` bit trusted under
# one alias could be
# DEMOTED true->false by a later account that lacks it – which
# made Claude Code's
# per-workspace trust dialog ("read, edit, and execute files
# here") reappear on
# every provider alias. Fix: OR the trust bit across all
# accounts – once a
# project path is trusted anywhere, it stays trusted in the
# merged portion.
local _tr_files=() _tr_acct
for _tr_acct in "${accts[@]"; do [[ -s
"$tr_acct/.claude.json" ]] &&
_tr_files+=("$tr_acct/.claude.json"); done
if (( ${#_tr_files[@]} >= 1 )); then
local _trusted
_trusted="$(jq -s '[ .[] | (.projects // {}) | to_entries[]
| select(.value.hasTrustDialogAccepted ==
true) | .key ] | unique' \
"${_tr_files[@]}" 2>/dev/null || printf '[]')"
if [[ -n "$_trusted" && "$_trusted" != "[]" ]]; then
local _tt; _tt="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
if jq --argjson trusted "$_trusted" '
.projects // = {}
| reduce ($trusted[]) as $p
(.; .projects[$p].hasTrustDialogAccepted = true)
' "$prev" > "$_tt" 2>/dev/null && jq -e . "$_tt" >/dev/
null 2>&1; then
command mv -f "$_tt" "$prev"
else rm -f "$_tt"; fi
fi
fi

# Now $prev holds the merged shared portion. Write each
# account's file with
# its own private keys overlaid on top of the shared portion.
for acct in "${accts[@]"; do
local f="$acct/.claude.json" out
out="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
if [[ -s "$f" ]]; then
# Guard against set -e: a corrupt $f makes jq fail, which
# would abort
# the whole loop and leave the remaining accounts unsynced.
if ! jq -s --argjson priv "$CMA_CLAUDE_JSON_PRIVATE_KEYS" '
. as [$shared, $own]
| ($own | with_entries(select(.key as $k | $priv |
index($k)))) as $priv_only
| ($shared // {}) * $priv_only

```

```

    ' "$prev" "$f" > "$out" 2>/dev/null; then
        rm -f "$out"
        cma_warn ".claude.json for $acct could not be merged
        (invalid JSON?) – leaving original alone"
        continue
    fi
else
    # New account, no existing .claude.json. Seed with shared
    content only.
    cp "$prev" "$out"
fi
# Final sanity: only replace if the output is parseable JSON.
if jq -e . "$out" >/dev/null 2>&1; then
    command mv -f "$out" "$f"
else
    rm -f "$out"
    cma_warn "merged .claude.json for $acct was invalid –
    leaving original file untouched"
fi
done
rm -f "$prev"
}

# Detect Linux vs macOS for platform-specific commands.
cma_os() {
    case "$(uname -s)" in
        Linux*)    echo linux ;;
        Darwin*)  echo macos ;;
        *)         echo unknown ;;
    esac
}

# Portable realpath. BSD/macOS `readlink` has no `-f`, so
# `readlink -f` is a hard
# error there (prints "illegal option -- f", returns empty) even
# after a bash-4
# re-exec, because /usr/bin/readlink stays BSD. This resolves a
# path to its
# canonical absolute form by walking the symlink chain with
# single-arg
# `readlink` (supported everywhere) plus `pwd -P` – the same
# technique every
# script's LIB_DIR resolver uses. Safe under `set -e`.
cma_realpath() {
    local p="$1" t dir base
    while [ -L "$p" ]; do
        t="$(readlink "$p")"
        case "$t" in /*) p="$t" ;; *) p="$(dirname "$p")/$t" ;; esac
    done
    dir="$(cd "$(dirname "$p")" 2>/dev/null && pwd -P)" || dir="$(
    (dirname "$p")"
    base="$(basename "$p")"
    printf '%s/%s\n' "$dir" "$base"
}

```



```

}

# cma_probe_run SECONDS BIN [ARG...] – BOUNDED run of a binary
# with any probe
# grammar; prints the first 20 lines of its combined output, never
# hangs, and
# never stalls its caller.
#
# Why not `timeout 15 "$bin" ... | head -20` (the previous shape):
# 1. `timeout` is absent on macOS without coreutils, and the
#    fallback that
#    shipped there ran the binary UNBOUNDED – a wedged router
#    hung the
#    installer forever (review finding I1, 2026-07-22).
# 2. Even where `timeout` exists, a probed binary that spawns a
#    child which
#    inherits stdout keeps the PIPE open after the parent is
#    killed, so the
#    `head` reader can still wait forever on EOF. The PROBE's
#    output goes to
#    a temp FILE instead: a file read cannot block on a
#    straggler's open
#    descriptor. That protects against stragglers of the PROBE –
#    the
#    watchdog arm needs its own discipline, next point.
# 3. The WATCHDOG below is backgrounded, so it inherits the
#    CALLER's stdio –
#    and when the caller captures this function through a
#    command
#    substitution, that stdout IS the $( ) pipe. After a fast
#    probe the
#    disarm kill orphaned the watchdog's `sleep SECONDS` child,
#    which kept
#    the pipe write-end open, so the caller blocked for the FULL
#    budget
#    even though the verdict was already correct (+15s on every
#    healthy
#    install/verify – review finding I3, 2026-07-23, measured).
#    The
#    watchdog's stdio is therefore detached to /dev/null: a
#    background
#    helper must never hold a descriptor its caller waits on.
#    The guard for
#    this class is behavioural – test_install_clean_host_ccr.sh
#    case I
#    times a fast probe through a command substitution.
# One implementation for every host and every probe grammar – a
# single
# instrument to validate, instead of a proven branch plus an
# unproven fallback
# ($11.4.201: the path is part of the instrument).
#
# Returns the probe's own exit code, or 124 when the watchdog had
# to kill it
# (deliberately the same convention coreutils `timeout` uses, so
# callers can

```

```

# classify "wedged" with one check). Safe under `set -e` callers.
cma_probe_run() {
    local _budget="$1" _bin="$2"
    shift 2
    local _tmp _rc _pid _wpid
    _tmp="$(mktemp "${TMPDIR:-/tmp}/cma-probe.XXXXXX")" || return 1
    "$_bin" "$@" </dev/null >"$_tmp" 2>&1 &
    _pid=$!
    # Watchdog: TERM at the budget, KILL 2s later for a TERM-
    # ignoring probe. It
    # drops a marker file BEFORE killing so "watchdog fired" is a
    # recorded fact,
    # not inferred from an exit status the probe could also produce
    # on its own.
    # stdio detached – header point 3 (I3): its orphaned sleep must
    # not hold
    # the caller's command-substitution pipe.
    ( sleep "$_budget"; : >"${_tmp}.killed"; kill -TERM "$_pid" 2>/
    dev/null
        sleep 2; kill -KILL "$_pid" 2>/dev/null ) >/dev/null 2>&1 &
    _wpid=$!
    _rc=0; wait "$_pid" 2>/dev/null || _rc=$?
    kill "$_wpid" 2>/dev/null || true      # probe finished: disarm
    the watchdog
    wait "$_wpid" 2>/dev/null || true
    [ -e "${_tmp}.killed" ] && _rc=124
    head -20 "$_tmp"
    rm -f "$_tmp" "${_tmp}.killed"
    return "$_rc"
}

# cma_probe_help BIN [SECONDS] – BOUNDED `--help` probe:
#   cma_probe_run with
#   the one grammar every binary is expected to answer. Kept as the
#   named entry
#   point because "--help, default budget 15" is the contract the
#   install-seam
#   callers (claude-install-verify.sh, claude-ccr-build.sh) share.
cma_probe_help() {
    cma_probe_run "${2:-15}" "$1" --help
}

# cma_proxy_transform_family ID – does provider ID belong to a
#   family cma-proxy
#   transforms? Exit 0 when yes. Used by cma_run_provider to decide
#   whether a
#   MISSING cma-proxy is worth a warning: a provider with no shim
#   loses nothing,
#   so warning there would be a false positive (§11.4.201(1)).
#
#   Normally the proxy binary itself answers (`cma-proxy --has-
#   transform ID`),
#   but with no binary present we cannot ask it – this list mirrors
#   the

```

```

# registrations in scripts/proxy/*.go (registerRequest/
#   registerResponse).
# KEEP IN SYNC with that registry: test_cma_proxy.sh asserts, in
#   BOTH
# directions, that these families EQUAL the registered set, so a
#   new transform
# provider added to the proxy without a family here fails the
#   suite instead of
# silently missing its warning. Prefix-matched because ids carry
#   variant
# suffixes (poe2 -> poe, kimi-k2 -> kimi) – the same folds
#   providerKey() does.
cma_proxy_transform_family() {
    case "$1" in
        helixagent*|poe*|kimi*|kc*|sarvam*|nvidia*) return 0 ;;
        *) return 1 ;;
    esac
}

# Find all existing Claude account config directories under $HOME,
#   matching
# the convention `.claude-<name>`. Echoes absolute paths, one per
#   line,
# sorted. The default `~/.claude` is intentionally excluded
#   because we treat
# it as the shared user-scope spot, not an account dir.
cma_detect_accounts() {
    local d
    while IFS= read -r d; do
        [[ "$d" == *"-shared" ]] && continue
        # Provider-alias dirs (~/.claude-prov-<id>, created by claude-
        #   providers)
        # are account-like for SHARED state but must NEVER be merged
        #   into
        # real-account auth/identity or unify, so they're excluded
        #   from detection
        # exactly like *-shared. This is the linchpin that keeps the
        #   existing
        # claudeN accounts and add-account untouched by the provider
        #   feature.
        [[ "$(basename "$d")" == "${ACCOUNT_PREFIX}prov-*" ]] &&
            continue

        # The claude-code-router config dir and any *.lock dir are
        #   not accounts.
        [[ "$(basename "$d")" == "${ACCOUNT_PREFIX}code-router" ]] &&
            continue
        [[ "$(basename "$d")" == *.lock ]] && continue
        # Archived removals (claude-remove-account's DEFAULT mode
        #   renames the dir to
        # `<dir>.removed.<timestamp>`) are NOT accounts. They keep
        #   their `projects/`

        # marker, so without this they kept counting forever – and
        #   the alias-commit

```

```

# floor at _cma_alias_gate arms only while `src_acct >=
  n_acct`, so one
# ordinary removal (2 aliases vs 3 "detected") silently
  disarmed that floor
# PERMANENTLY on that host. It failed open rather than wrong,
  but a guard
# that quietly stops guarding is not a guard.
[[ "$(basename "$d")" == *.removed.* ]] && continue

    # Same reasoning for the pre-unify backups
    `backup_and_remove` leaves behind.
[[ "$(basename "$d")" == *.preunify.* ]] && continue
# Empty dirs always count (a brand-new account before any
  claude run).
if [[ -z "$(ls -A "$d" 2>/dev/null)" ]]; then
    echo "$d"
    continue
fi
# Non-empty: must look like a Claude account (at least one
  tell-tale
# file/dir). Filters out dirs that merely match the `.claude-
  *` prefix
# but belong to other tools (e.g. `.claude-server-commander`).
if [[ -d "$d/projects" || -d "$d/todos" || -d "$d/plugins" \
    || -f "$d/.claude.json" || -f "$d/.credentials.json" \
    || -f "$d/history.jsonl" ]]; then
    echo "$d"
fi
done < <(find "$HOME" -maxdepth 1 -type d -name "$
  {ACCOUNT_PREFIX}*" 2>/dev/null | sort)
}

# Read all aliases of the form `alias name=...` from $ALIAS_FILE
  and print
# their names (one per line). Returns nothing if the file is
  absent.
cma_existing_aliases() {
    [[ -f "$ALIAS_FILE" ]] || return 0
    awk -F'[ =]+' '/^[[:space:]]*alias[[:space:]]+/{print $2}'
      "$ALIAS_FILE"
}

# Suggest the next free `claude<N>` alias by scanning current
  aliases.
# E.g., if claude1 and claude2 exist, returns "claude3".
cma_suggest_alias() {
    local highest=0 n
    for a in $(cma_existing_aliases); do
        if [[ "$a" =~ ^claude([0-9]+)$ ]]; then
            n="${BASH_REMATCH[1]}"
            (( n > highest )) && highest="$n"
        fi
    done
    echo "claude$((highest + 1))"
}

```

```

}

# Validate that an alias name is safe to embed in a generated
#   `alias ...=`
# line – strictly alphanumeric + underscore/hyphen, starting with
# a letter.
cma_validate_alias() {
    [[ "$1" =~ ^[a-zA-Z][a-zA-Z0-9_-]*$ ]] || cma_die
    "invalid alias name: $1"
}

# Match an rc-file line that sources an aliases.sh, capturing the
# path:
#   BASH_REMATCH[2] = the (possibly $HOME/~ /quoted) target path.
# Anchored so leading-# comment lines (e.g. "# migrated to .../
#   aliases.sh: ...")
# never match. Used by the prune + dedup helpers below.
CMA_ALIAS_SRC_RE='^[[[:space:]]*(source|\.)[[[:space:]]+]"?
    ([^"[:space:]]*aliases\.sh)"?[[[:space:]]*$'

# The one managed header that heads the `source "<alias-file>"`
# block in every
# rc file. Spelled once here so the writer (append) and the reader
# (prune) agree
# on exactly what a "managed block" is – the bug that lost the
# operator's
# ~/.bashrc was the two disagreeing: the append wrote this header,
# the prune did
# not know to remove it, so every dropped source line left the
# header ORPHANED.
CMA_ALIAS_RC_HEADER='# Claude multi-account aliases'

# True if $1 is an `aliases.sh` source line (source|. form). Sets
# BASH_REMATCH.
_cma_is_alias_src_line() { [[ "$1" =~ $CMA_ALIAS_SRC_RE ]]; }
# Echo the resolved target path an alias-source line ($1) points
# at, or nothing.
_cma_alias_src_target() {
    [[ "$1" =~ $CMA_ALIAS_SRC_RE ]] || return 1
    local t="${BASH_REMATCH[2]}"; t="${t/#\$HOME/\$HOME}"; t="${t/
        #\~/\$HOME}"
    printf '%s' "$t"
}

# Portable mtime (epoch seconds). GNU `stat -c %Y`; BSD/macOS
#   `stat -f %m`
# (which has no `-c`). Deliberately NOT `date -r <path>`: on BSD
#   `date -r` reads
# its argument as epoch seconds rather than a file, silently
# returning 0 for
# every path – the macOS breakage that made this a $11.4.68 /
# $11.4.81 finding.
_cma_mtime() { stat -c %Y "$1" 2>/dev/null || stat -f %m "$1" 2>/
    dev/null || echo 0; }

```

```

# Newest existing rolling backup for $rc, by mtime (empty if
  none).
cma_newest_rc_backup() {
  local rc="$1" f newest="" ntime=-1 t
  for f in "$rc".cma-backup.*; do
    [[ -e "$f" ]] || continue
    t="$(_cma_mtime "$f")"
    if (( t >= ntime )); then ntime="$t"; newest="$f"; fi
  done
  printf '%s' "$newest"
}

# cma_backup_rc_file <rc>
# Protect a user rc file BEFORE the toolkit modifies it. Two
  backups:
#   * <rc>.cma-orig – the PRISTINE original, written exactly ONCE
    and NEVER
#   overwritten. This is the load-bearing recovery point: once a
    corrupted
#   state exists on disk it can never bury the true original.
#   * <rc>.cma-backup.<epoch> – a rolling snapshot of recent
    state, RATE-LIMITED:
#   skipped when byte-identical to the newest existing rolling
    backup. Without
#   this, BASH_ENV=~/.bashrc firing the prune->ensure cycle on
    every
#   non-interactive bash would spray hundreds of identical
    copies.
# Returns 0 when the rc is protected (or does not exist yet, so a
  fresh create
# loses nothing). Returns 1 when the PRISTINE backup could not be
  taken – the
# caller MUST then refuse to modify rather than proceed
  unprotected.
cma_backup_rc_file() {
  local rc="$1" orig="$1.cma-orig" newest ts cand i=0
  [[ -f "$rc" ]] || return 0 # nothing on disk =>
    nothing to lose

  # 1. Pristine original: create once, never overwrite. A failure
    here is fatal
  #   to the modification – an unwritable dir can still allow an
    append (which
  #   needs only file-write), so refusing here is what actually
    prevents an
  #   unprotected write.
  if [[ ! -e "$orig" ]]; then
    if ! cp -p -- "$rc" "$orig" 2>/dev/null; then
      cma_warn "cannot write pristine backup $orig – refusing to
        modify $rc"
      return 1
    fi
  fi
}

```

```

# 2. Rolling snapshot, rate-limited. Best-effort: the pristine
#    copy above is
#    the guarantee; a rolling-backup failure does not block the
#    modification
#    (the recovery point already exists).
newest="$(cma_newest_rc_backup "$rc")"
if [[ -n "$newest" ]] && cmp -s -- "$rc" "$newest"; then
    return 0
fi
ts="$(date +%s)"
cand="$rc.cma-backup.$ts"
while [[ -e "$cand" ]]; do i=$((i+1)); cand="$rc.cma-backup.$ts.$i"; done
cp -p -- "$rc" "$cand" 2>/dev/null || true
return 0
}

# _cma_rc_rewrite_ok <rc> <candidate> <intended_removed_lines>
# Sanity gate for any rc rewrite, mirroring cma_alias_commit's
# alias-file gate.
# Refuse to publish a candidate that: is empty while it was meant
# to keep lines;
# fails a shell syntax parse (bash -n for a .bashrc, zsh -n for
# a .zshrc when zsh
# is present); or lost more lines than the operation intended to
# remove.
_cma_rc_rewrite_ok() {
    local rc="$1" cand="$2" removed="$3" src_lines cand_lines
    expected
    src_lines="$(grep -c '' "$rc" 2>/dev/null || echo 0)"
    cand_lines="$(grep -c '' "$cand" 2>/dev/null || echo 0)"
    expected=$(( src_lines - removed ))
    (( expected < 0 )) && expected=0

    # 1. Empty candidate that was supposed to preserve content =
    # truncation.
    if [[ ! -s "$cand" ]] && (( expected > 0 )); then
        cma_warn "rc rewrite rejected: empty candidate for $rc would
        drop $expected intended line(s)"
        return 1
    fi

    # 2. Must still parse in the target shell.
    case "$rc" in
        *.zshrc)
            if command -v zsh >/dev/null 2>&1; then
                zsh -n "$cand" 2>/dev/null || { cma_warn "rc rewrite
                rejected: zsh -n failed on candidate for $rc"; return 1; }
            fi ;;
        *)
            bash -n "$cand" 2>/dev/null || { cma_warn "rc rewrite
            rejected: bash -n failed on candidate for $rc"; return
            1; } ;;
    esac
}

```



```

# 3. Must not have lost lines the operation did not intend to
    remove.
if (( cand_lines < expected )); then
    cma_warn "rc rewrite rejected: candidate for $src lost $((
        expected - cand_lines )) unintended line(s)"
    return 1
fi
return 0
}

# — Managed rc-block sentinels + the single rc-rewrite committer

# A managed rc block is delimited by a BEGIN/END sentinel PAIR
    (unlike the legacy
# single `CMA_ALIAS_RC_HEADER` comment, which had NO END marker –
    that is exactly
# what ORPHANED when its source line was pruned, piling up ~93
    headers). One
# stable marker id per block ("path", ...); begin/end are derived
    from the marker
# through one builder so the writer and the reader can never
    disagree.
_cma_rc_begin() { printf '# cma-rc:%s BEGIN – managed by claude-
    multi-account; do not edit inside' "$1"; }
_cma_rc_end()   { printf '# cma-rc:%s END' "$1"; }
# Named sentinels for the markers in use (tests + code share one
    source of truth).
# shellcheck disable=SC2034 # public surface: consumed by
    test_rc_safety.sh, which sources lib.sh
CMA_RC_PATH_BEGIN="$(_cma_rc_begin path)"
# shellcheck disable=SC2034 # public surface: consumed by
    test_rc_safety.sh, which sources lib.sh
CMA_RC_PATH_END="$(_cma_rc_end path)"

# cma_rc_safe_rewrite <rc> <candidate_tmp>
    <intended_removed_lines>
# THE single committer for any WHOLE-FILE rc rewrite – the rc
    analogue of
# cma_alias_commit. Every path that would `mv` a rebuilt file over
    a live rc
# funnels through here. Order is load-bearing and mirrors the
    proven prune path:
# 1. no-op guard          – a byte-identical candidate ⇒ discard,
    touch nothing;
# 2. sanity gate          – _cma_rc_rewrite_ok (empty/parse/
    content-loss). A reject
#                          is PARKED as <rc>.rejected.<ts>; the
    live rc is untouched;
# 3. backup-or-refuse     – cma_backup_rc_file; no pristine backup
    ⇒ DO NOT modify
#                          ($1.4.167(D));
# 4. publish              – one command mv, INT/TERM masked across
    the rename.
# Consumes <candidate_tmp> (moved or removed) on every return
    path.
# Returns 0 published (or no-op), 1 rejected/refused.

```



```

cma_rc_safe_rewrite() {
    local rc="$1" cand="$2" removed="${3:-0}" prev_int prev_term
    rc2=0
    [[ -f "$cand" ]] || return 1

    # 1. No-op guard (before any backup): a settled host writes
    #      nothing.
    if [[ -f "$rc" ]] && cmp -s -- "$cand" "$rc"; then rm -f
        "$cand"; return 0; fi

    # 2. Sanity gate – refuse a content-losing / unparseable
    #      candidate; park it.
    if ! _cma_rc_rewrite_ok "$rc" "$cand" "$removed"; then
        command mv -f "$cand" "$rc.rejected.$(date +%s)" 2>/dev/null
        || rm -f "$cand"
        cma_warn "$rc left untouched; rewrite candidate kept as
            $rc.rejected.*"
        return 1
    fi

    # 3. Backup-or-refuse – no pristine backup ⇒ no modify.
    if ! cma_backup_rc_file "$rc"; then rm -f "$cand"; return 1; fi

    # 4. Publish – one rename, INT/TERM masked (mirrors
    #      cma_alias_commit F4).
    prev_int="$(trap -p INT || true)"
    prev_term="$(trap -p TERM || true)"
    trap '' INT TERM
    if command mv -f "$cand" "$rc"; then rc2=0; else rm -f "$cand";
        rc2=1; fi
    trap - INT TERM
    if [[ -n "$prev_int" ]]; then eval "$prev_int"; fi
    if [[ -n "$prev_term" ]]; then eval "$prev_term"; fi
    return "$rc2"
}

# cma_rc_append_managed <rc> <marker> <payload-line...>
# Append a BEGIN/
#      END-delimited managed block to <rc>, backup-first and
#      idempotent:
# * no-op if a live block for <marker> is already present;
# * refuses to modify if the pristine backup cannot be taken
#   ($11.4.167(D));
# * an absent rc is first-touched (cma_backup_rc_file returns 0
#   for absent, and
#   there is nothing to lose).
cma_rc_append_managed() {
    local rc="$1" marker="$2"; shift 2
    local begin end
    begin="$(_cma_rc_begin "$marker")"
    end="$(_cma_rc_end "$marker")"
    if [[ -f "$rc" ]] && grep -qxF -- "$begin" "$rc"; then return
        0; fi # idempotent
    if ! cma_backup_rc_file "$rc"; then

```

```

    cma_warn "skipped managed '$marker' block in $rc (could not
        back it up)"
    return 1
fi
{ printf '\n%s\n' "$begin"; printf '%s\n' "$@"; printf '%s\n'
    "$end"; } >> "$rc"
cma_log "added managed '$marker' block to $rc"
}

# cma_rc_remove_block <rc> <marker>
# Remove the managed <marker> block (BEGIN..END) as a UNIT; no-op
# if absent. An
# unterminated BEGIN (file truncated inside the block by an
# outside tool) removes
# from BEGIN to EOF – never leaving a half block / orphan header.
# The rewrite is
# published through cma_rc_safe_rewrite (gate + backup + mv).
cma_rc_remove_block() {
    local rc="$1" marker="$2" begin end tmp removed
    [[ -f "$rc" ]] || return 0
    begin="$(_cma_rc_begin "$marker")"
    end="$(_cma_rc_end "$marker")"
    grep -qxF -- "$begin" "$rc" || return 0
    tmp="$({mktemp "${TMPDIR:-/tmp}/cma.XXXXXX}")" || return 1
    awk -v b="$begin" -v e="$end" '
        $0==b {skip=1; c++; next}
        skip && $0==e {skip=0; c++; next}
        skip {c++; next}
        {print}
        END {print c+0 > "/dev/stderr"}' "$rc" 2>"$tmp.n" >"$tmp" ||
        true
    removed="$({cat "$tmp.n" 2>/dev/null || echo 0}"); rm -f "$tmp.n"
    cma_rc_safe_rewrite "$rc" "$tmp" "${removed:-0}"
}

# Remove rc-file lines that source an aliases.sh whose target no
# longer exists
# (a stale path from a moved install, or a transient ALIAS_FILE
# used in testing).
# Without this, a deleted alias file leaves a dangling line that
# errors
# "-bash: ../aliases.sh: No such file or directory" on every new
# login shell.
#
# The managed `# Claude multi-account aliases` header and its
# source line are
# treated as ONE block: a dead block drops both (never orphaning
# the header), and
# a header that no longer heads a live source line is collapsed
# away (self-heal
# for the orphans a pre-fix toolkit already accumulated). A bare
# source line with
# no managed header is still pruned when its target is gone. Every
# rewrite is

```

```

# guarded by the sanity gate + a mandatory backup: a bad candidate
# is parked as
# <rc>.rejected.<ts> and the live rc is left untouched; if the
# backup cannot be
# taken the rc is not modified at all.
cma_prune_stale_alias_sources() {
    local rc="$1" tmp removed=0 n i line target
    [[ -f "$rc" ]] || return 0

    local lines=()
    while IFS= read -r line || [[ -n "$line" ]]; do
        lines+=("$line"); done < "$rc"
    n=${#lines[@]}

    tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")" || return 0
    i=0
    while (( i < n )); do
        line="${lines[$i]}"
        if [[ "$line" == "$CMA_ALIAS_RC_HEADER" ]]; then
            if (( i + 1 < n )) && _cma_is_alias_src_line "${lines[$
                ((i+1))]}"; then
                target="$(_cma_alias_src_target "${lines[$((i+1))]}")"
                if [[ -f "$target" ]]; then
                    printf '%s\n' "$line" >> "$tmp"                # live
                    block: keep header
                    printf '%s\n' "${lines[$((i+1))]}" >> "$tmp"
                    # ...and its source line
                else
                    removed=$((removed + 2))                        # dead
                    block: drop both
                fi
                i=$((i + 2)); continue
            fi
            removed=$((removed + 1)); i=$((i + 1)); continue      #
            orphan header: collapse
        fi
        if _cma_is_alias_src_line "$line"; then
            target="$(_cma_alias_src_target "$line")"
            if [[ ! -f "$target" ]]; then removed=$((removed + 1)); i=$
                ((i + 1)); continue; fi
        fi
        printf '%s\n' "$line" >> "$tmp"
        i=$((i + 1))
    done

    if (( removed == 0 )); then rm -f "$tmp"; return 0; fi

    # Publish through the single rc-rewrite committer (gate →
    # backup-or-refuse →
    # INT/TERM-masked mv, with a .rejected.<ts> park on a bad
    # candidate). The
    # candidate-building block/orphan logic above is unchanged.
    cma_rc_safe_rewrite "$rc" "$tmp" "$removed" \

```

```

    && cma_log "pruned stale/orphaned managed alias block(s) from
    $rc"
}

# True if $rc already sources a file resolving to $2 (across `./`
# `source` and
# $HOME/~ /absolute forms), so we never append a duplicate source
# line.
cma_rc_sources_alias_file() {
    local rc="$1" want="$2" line target want_real
    [[ -f "$rc" ]] || return 1
    want_real="$(cma_realpath "$want" 2>/dev/null || printf '%s'
    "$want")"
    while IFS= read -r line || [[ -n "$line" ]]; do
        if [[ "$line" =~ $CMA_ALIAS_SRC_RE ]]; then
            target="${BASH_REMATCH[2]}"; target="${target/#\$HOME/
            $HOME}"; target="${target/#\~/\$HOME}"
            [[ "$(cma_realpath "$target" 2>/dev/null || printf '%s'
            "$target")" == "$want_real" ]] && return 0
        fi
    done < "$rc"
    return 1
}

```

```

# =====
# Alias file: one renderer, one lock, one rename.
# =====
# WHY THIS SHAPE – forensics of the 2026-07-20 alias-file
# destruction. The live
# aliases.sh collapsed 37288 -> 282 -> 32112 -> 974 -> 909 ->
# 25682 -> 130 bytes
# inside four seconds, losing the header, BOTH wrapper functions
# and every
# `alias claudeN=` line. No single writer was buggy:
# `cma_ensure_alias_file`
# performed SIX sequential whole-file read-modify-write migrations
# plus two
# direct `cat >> "$ALIAS_FILE"` appends, while `claude-providers
# list
# --refresh-aliases` – fired by the session hook on EVERY shell
# start –
# performed 21 more whole-file rewrites. Each was atomic and
# content-preserving
# in isolation; together, with no lock anywhere on the file, one
# writer's stale
# snapshot silently overwrote another writer's committed result
# (lost update).
# The corruption timestamp preceded the incidental SIGTERM by ~14
# minutes, so
# this is clean-run reachable, not an interrupt-only hazard.
#
# The three properties that fix it, in order of leverage:
#
# 1. RENDER ONCE. The complete file (managed block + carried
# lines + the

```

```
# session-hook block) is built in ONE temp and committed with
# a SINGLE
# rename. There is no intermediate on-disk state for a
# concurrent reader
# or writer to snapshot, and the drop-then-re-append
# migrations disappear
# entirely – the wrappers are always emitted at their current
# version, so
# there is nothing to "migrate". That also kills the orphaned
# `# Wrapper:`
# comment leak: the old drop-awk started at the `cma_run() {`
# line and left
# the 4-line comment above it behind while the re-append put
# the function
# at the end, leaking 5 dead lines and reordering the file on
# every firing
# (~15 orphans had accumulated on the live host).
#
# 2. NO-OP GUARD. If the render is byte-identical to what is on
# disk, nothing
# is written and the lock is never even taken. On a steady-
# state host this
# turns 21 renames per shell start into zero writes, so the
# common path
# cannot enter the race at all. This is the highest-leverage
# part.
#
# 3. EXCLUSIVE LOCK. The remaining read-render-rename is still a
# read-modify-write, so it runs under an $ALIAS_FILE-scoped
# exclusive lock.
# Contention policy is caller-set via CMA_ALIAS_LOCK_WAIT:
# the session hook
# uses 0 (give up instantly – a shell start must NEVER block,
# and a skipped
# refresh is harmless because the next writer re-renders from
# the file).
#
# Plus a sanity gate (never commit a file missing the header,
# either wrapper, or
# any alias it did not explicitly drop).
#
# What protects WHAT, stated precisely, INCLUDING what the test
# suite does and
# does not cover. Two earlier versions of this comment overclaimed
# here; both
# corrections below were established by deleting the mechanism and
# re-running.
#
# * No half-file can be published because of RENDER-ONCE + a
# single mv(2).
# That property holds with the signal mask removed, and a 15-
# iteration
# SIGTERM test cannot tell the difference. The mask is not
# what earns it.
#
```

```
# * The INT/TERM mask is DEFENCE IN DEPTH WITH NO TEST-VISIBLE
#   EFFECT, and
#   saying otherwise was the second overclaim. The narrow thing
#   it was
#   credited with – stopping a signal between the rename and the
#   lock release
#   from leaking the lock – is not, in fact, a hazard either
#   backend suffers:
#   - flock: the lock is an fd. Process death releases it in
#     the kernel,
#     mask or no mask. There is nothing to leak.
#   - mkdir: a leaked lock dir names a dead pid, and the very
#     next acquire
#     reclaims it via _cma_alias_lock_break_stale – inside the
#     SAME call,
#     at any wait including 0. It does not make later writers
#     skip.
#   So deleting `trap '' INT TERM` leaves the suite green, and
#   no honest test
#   distinguishes it. What it genuinely buys is that the saved
#   INT/TERM traps
#   are restored rather than left disarmed for the caller. It is
#   kept for
#   that and for the narrow interrupt-safety it costs nothing to
#   have – NOT
#   because anything here proves it necessary. Do not cite it as
#   tested.
#
# * MUTUAL EXCLUSION is the lock's job and nothing else's. The
#   storm in
#   test_alias_file_concurrency.sh section 1 does NOT measure
#   it: its writers
#   all write the SAME content, so a lost update is re-derived
#   on the next
#   iteration, and deleting the lock acquire outright leaves
#   that section
#   passing 3/3. Section 1b is what measures it – N one-shot
#   DISTINCT writes,
#   nothing to re-derive a loss – and that is the case which
#   found the mkdir
#   backend was not excluding at all (see
#   _cma_alias_lock_break_stale).

# Sentinels bracketing the machine-owned region. Everything
#   between them is
#   regenerated verbatim on every write; everything outside is
#   carried over
#   untouched (user additions survive – see the "unrelated user line
#   preserved"
#   coverage test). Pre-sentinel files are recognised by content
#   instead, once,
#   by _cma_alias_carryover's legacy rules.
CMA_ALIAS_MANAGED_BEGIN='# cma-managed BEGIN – regenerated by
#   claude-multi-account; do not edit inside'
CMA_ALIAS_MANAGED_END='# cma-managed END'
CMA_ALIAS_HOOK_BEGIN='# cma-providers-session-refresh BEGIN'
```

```

CMA_ALIAS_HOOK_END='# cma-providers-session-refresh END'

# --- exclusive lock on the alias file
# -----
# This deliberately MIRRORS scripts/tests/lib/suite-lock.sh
# (flock(1) where
# present, otherwise an atomic mkdir(2) lock with rename-verified
# stale
# breaking – macOS ships no flock) rather than sourcing it. suite-
# lock.sh is
# test infrastructure: its acquire() `exit 75`'s the process on
# contention and
# installs its own EXIT/INT/TERM traps. Both are unacceptable in a
# library that
# runs inside install scripts and, transitively, shell startup –
# so the same
# primitive is re-implemented here with a returns-a-status
# contract, no process
# exit, and no trap hijacking.
#
# KNOBS
#   CMA_ALIAS_LOCK_WAIT          seconds to wait on contention
#                               (default 30);
#                               0 = fail fast, used by the session
#                               hook
#   CMA_ALIAS_LOCK_NO_FLOCK=1    force the portable fallback (tests
#                               exercise the
#                               macOS path on Linux)
#   CMA_ALIAS_LOCK_STALE_GRACE seconds a pid-less lock dir may
#                               exist before it
#                               is treated as stale (default 10)
CMA_ALIAS_LOCK_WAIT="${CMA_ALIAS_LOCK_WAIT:-30}"
CMA_ALIAS_LOCK_STALE_GRACE="${CMA_ALIAS_LOCK_STALE_GRACE:-10}"
_cma_alias_lock_depth=0
_cma_alias_lock_mode=""
_cma_alias_lock_file=""

# Sanitised wait, in seconds. A junk value must not abort the
# caller under
# `set -e` when it reaches an arithmetic test.
_cma_alias_lock_wait() {
  case "${CMA_ALIAS_LOCK_WAIT:-}" in
    ''|*[*!0-9]*) printf '30\n' ;;
    *)             printf '%s\n' "$CMA_ALIAS_LOCK_WAIT" ;;
  esac
}

_cma_alias_lock_pid() {
  local p=""
  if [[ "$_cma_alias_lock_mode" == "mkdir" ]]; then
    p="$(head -1 "$_cma_alias_lock_file/pid" 2>/dev/null || true)"
  else
    p="$(head -1 "$_cma_alias_lock_file" 2>/dev/null || true)"
  fi
}

```

```

printf '%s' "$p" | tr -d '[:space:]'
}

# Discard a lock whose owner is gone.
#
# WHY THIS IS NOT THE OBVIOUS "rename it aside and look at it".
# It used to be. That version renamed the lock directory to a
#     private path,
# read the pid inside, and renamed it BACK when the pid was not
#     the one it had
# judged dead. Two flaws compounded, and together they cost the
#     mkdir backend
# mutual exclusion outright – measured, not theorised: 12 one-shot
#     concurrent
# writers each adding a DIFFERENT alias lost at least one
#     COMMITTED alias in
# roughly 40% of runs, with the caller told rc 0. That is a lost
#     update, the
# very failure that destroyed the live alias file.
#
# 1. TOCTOU on the holder's identity. The caller samples the
#     pid, then tests
#     liveness. A holder that RELEASED NORMALLY and then exited
#     is
#     indistinguishable from one that died holding the lock: by
#     the time
#     `kill -0` runs the pid is gone, while the directory has
#     already been
#     re-created by a different, LIVE writer. The audit trace for
#     this bug
#     shows the judged holder exiting 1ms before the test that
#     condemned it.
# 2. The restore was not atomic. Between `mv lock aside` and `mv
#     aside lock`
#     the lock DOES NOT EXIST, so any contender's `mkdir`
#     succeeds there. The
#     displaced holder never learns it lost the lock, and two
#     processes enter
#     the critical section together. Every double-hold in the
#     audit is
#     preceded, within ~15ms, by exactly this restore.
#
# So the directory is never moved. It is inspected IN PLACE and
#     removed only
# once proven stale, under a short-lived breaker lock that makes
# inspect-then-remove exclusive among breakers. That closes the
#     window: a
# directory whose recorded pid is dead cannot be released by its
#     owner (it is
# dead) and cannot be removed by another breaker (we hold the
#     breaker lock), so
# nothing can re-create it under a LIVE owner between the check
#     and the removal.
#
# NOTE: tests/lib/suite-lock.sh carries the older shape. It is
#     deliberately NOT

```



```

# converged with this one – it is global-by-design test
# infrastructure with a
# different contract (it exits the process on contention) – but it
# has the same
# hazard, at far lower stakes (one suite run per checkout).
_cma_alias_lock_break_stale() {
    local observed="${1:-}" brk="{_cma_alias_lock_file}.breaker"
    bpid got
    # Breaker exclusivity: mkdir(2) is the atomic claim. Losing it
    # is not an
    # error – someone else is already breaking, so we simply retry
    # the acquire.
    if ! mkdir "$brk" 2>/dev/null; then
        # A breaker that died mid-break would wedge every later one,
        # so a breaker
        # lock whose owner is gone is reaped. This is the ONLY thing
        # this branch
        # may remove; it never touches the alias lock itself.
        bpid="$(head -1 "$brk/pid" 2>/dev/null || true)"
        bpid="$(printf '%s' "$bpid" | tr -d '[:space:]')"
        if [[ -n "$bpid" ]] && ! kill -0 "$bpid" 2>/dev/null; then
            rm -rf "$brk" 2>/dev/null || true
        fi
        return 0
    fi
    printf '%s\n' "$$" > "$brk/pid" 2>/dev/null || true

    # Re-read the evidence as late as possible and grade the
    # directory we are
    # about to delete, never a pid the caller sampled an iteration
    # ago.
    got="$(_cma_alias_lock_pid)"
    if [[ -n "$observed" ]]; then
        # Still the same holder we judged, and still gone.
        if [[ "$got" == "$observed" ]] && ! kill -0 "$got" 2>/dev/
            null; then
            rm -rf "$_cma_alias_lock_file" 2>/dev/null || true
        fi
    elif [[ -z "$got" ]]; then
        # Pid-less. The caller only asks for this after
        # CMA_ALIAS_LOCK_STALE_GRACE
        # seconds of CONTINUOUS emptiness, which is what separates
        # "the winner has
        # not written its pid yet" (microseconds) from "the winner
        # died in between".
        # That grace, not this re-read, is the guard for the empty
        # case.
        rm -rf "$_cma_alias_lock_file" 2>/dev/null || true
    fi

    rm -rf "$brk" 2>/dev/null || true
    return 0
}

```

```

# Acquire (or re-enter) the alias-file lock. Returns 1 on
# contention timeout -
# never exits, never blocks longer than CMA_ALIAS_LOCK_WAIT.
_cma_alias_lock_acquire() {
    _cma_alias_lock_depth=$(( _cma_alias_lock_depth + 1 ))
    if (( _cma_alias_lock_depth > 1 )); then return 0; fi # re-entrant

    local dir wait_s deadline holder empty_since=""
    dir="$(dirname "$ALIAS_FILE")"
    mkdir -p "$dir" 2>/dev/null || true
    wait_s="$(_cma_alias_lock_wait)"

    if command -v flock >/dev/null 2>&1 && [[ -z "$
        {CMA_ALIAS_LOCK_NO_FLOCK:-}" ]] ; then
        _cma_alias_lock_mode="flock"
        _cma_alias_lock_file="$dir/.aliases.lock"
        # APPEND mode on purpose: `exec 8>file` truncates at open,
        # erasing a live
        # holder's PID record before we even contend.
        if ! exec 8>>"$_cma_alias_lock_file"; then
            _cma_alias_lock_depth=0; return 1
        fi
        if ! flock -n 8; then
            if (( wait_s <= 0 )) || ! flock -w "$wait_s" 8; then
                # BRACES ARE LOAD-BEARING. `exec 8>&- 2>/dev/null` is NOT
                # "close fd 8,"
                # quietly": a command-less `exec` applies EVERY
                # redirection on the line
                # to the shell PERMANENTLY, so the `2>/dev/null` silences
                # this
                # process's stderr for the rest of its life. That fired
                # here on every
                # contended acquire - precisely when the caller has
                # something to say -

                # and it swallowed claude-add-account's whole "here is how
                # to finish
                # the job" message, leaving a user with a config dir, no
                # alias, and no
                # explanation. Scoping the redirection to a group closes
                # the fd and
                # leaves stderr alone.
                { exec 8>&-; } 2>/dev/null || true
                _cma_alias_lock_depth=0
                return 1
            fi
        fi
        printf '%s\n' "$$" > "$_cma_alias_lock_file" 2>/dev/null ||
            true
        return 0
    fi

    _cma_alias_lock_mode="mkdir"

```

```

_cma_alias_lock_file="$dir/.aliases.lockdir"
deadline=$(( $(date +%s) + wait_s ))
while ;; do
    if mkdir "$_cma_alias_lock_file" 2>/dev/null; then
        printf '%s\n' "$$" > "$_cma_alias_lock_file/pid" 2>/dev/
        null || true

        # A contender mid stale-break could have swapped the dir
        out from under
        # us; the recorded PID is the tiebreaker.
        if [[ "$(_cma_alias_lock_pid)" != "$$" ]]; then continue; fi
        return 0
    fi
    holder="$(_cma_alias_lock_pid)"
    if [[ -z "$holder" ]]; then
        # mkdir won but the pid write has not landed yet – or the
        winner died in
        # between. Give it a grace window before declaring the lock
        stale.
        [[ -n "$empty_since" ]] || empty_since="$(date +%s)"
        if (( $(date +%s) - empty_since >=
            CMA_ALIAS_LOCK_STALE_GRACE )); then
            _cma_alias_lock_break_stale ""
            empty_since=""
            continue
        fi
    else
        empty_since=""
        if ! kill -0 "$holder" 2>/dev/null; then
            _cma_alias_lock_break_stale "$holder"
            continue
        fi
    fi
    if (( $(date +%s) >= deadline )); then
        _cma_alias_lock_depth=0
        return 1
    fi
    sleep 0.1 2>/dev/null || sleep 1
done
}

_cma_alias_lock_release() {
    (( _cma_alias_lock_depth > 0 )) || return 0
    _cma_alias_lock_depth=$(( _cma_alias_lock_depth - 1 ))
    if (( _cma_alias_lock_depth > 0 )); then return 0; fi
    if [[ "$_cma_alias_lock_mode" == "mkdir" ]]; then
        # Only remove a lock we still own – after a stale break the
        directory may
        # belong to someone else.
        if [[ "$(_cma_alias_lock_pid)" == "$$" ]]; then
            rm -rf "$_cma_alias_lock_file" 2>/dev/null || true
        fi
    else

```

```

# The file is deliberately NOT unlinked: unlinking a flock
# target races a
# contender who already opened the old inode.
flock -u 8 2>/dev/null || true
# Braces, for the reason spelled out in
# _cma_alias_lock_acquire. This site
# was the worse of the two: it is on the SUCCESS path, so
# every process
# that committed an alias – install.sh, claude-add-account,

# claude-providers sync – ran the rest of its life with
# stderr pointed at

# /dev/null and lost every warning and error it later
# tried to report.
{ exec 8>&-; } 2>/dev/null || true
fi
return 0
}

# --- the managed block
# -----
# One emitter per piece. These are the ONLY definitions of the
# wrappers; the
# renderer always emits the current text, which is why no drop/re-
# append
# migration (and no orphan-comment leak) exists any more.

# The ccr-self-reference test, as ONE definition serving THREE
# call sites:
# cma_run_provider (which lives inside the emitted alias file and
# therefore has
# no access to lib.sh's functions), providers-verify.sh, and
# lib.sh itself. The
# emitter below is the single source of the text; the alias file
# gets it via
# _cma_emit_managed, and this shell gets it via the `eval`
# immediately after, so
# the two copies cannot drift. The previous arrangement – a `case`
# duplicated at
# lib.sh's launch gate and providers-verify.sh's Gate 0 – under-
# matched every
# spelling except the four it literally listed, so
# `127.0.1.1:3456` (Debian's
# DEFAULT loopback for the local hostname), any other 127/8
# address, the
# v4-mapped and fully-expanded IPv6 loopbacks, `LOCALHOST`, a
# `user@` prefix and
# a `?query`/`#fragment` suffix all sailed through the guard and
# re-armed the
# exact helixagent hazard the guard exists to stop.
_cma_emit_ccr_gateway_guard() {
    cat <<'CMA_CCRGW_EOF'
# True when $1 is a URL pointing at the LOCAL ccr gateway (port
# $2, default

```

```

# $CMA_CCR_PORT or 3456). Such a base_url has no upstream: routing
    through it
# means serving whatever provider ccr last routed to, with no
    error anywhere.
# Matches the whole 127/8 range, 0.0.0.0, `localhost` in any case,
    and IPv6
# loopback in every spelling (::1,
    0:0:0:0:0:0:0:1, ::ffff:127.0.0.1, ...),
# tolerating userinfo, path, query and fragment. A different port,
    a
# non-loopback host, or a remote https base is NOT a match.
_cma_is_ccr_gateway() {
    local url="${1:-}" want="${2:-${CMA_CCR_PORT:-3456}}" hp host
    port v4 squash
    [[ -n "$url" ]] || return 1
    hp="${url#*://}"          # drop the scheme
    hp="${hp%/*}"            # drop the path
    hp="${hp%\?*}"           # drop a query that followed no path
    hp="${hp%#*}"            # drop a fragment that followed no
        path
    hp="${hp##*@}"           # drop userinfo (greedy: the LAST '@'
        wins)
    case "$hp" in
        \[*\]:*) host="${hp%:*}"; host="${host#[]}"; port="$
            {hp##*}:}" ;;
        *:*)      host="${hp%:*}"; port="${hp##*}:}" ;;
        *)        return 1 ;;    # no explicit port -> not the gateway
    esac
    [[ "$port" == "$want" ]] || return 1
    host="$(printf '%s' "$host" | tr '[:upper:]' '[:lower:]')"
    case "$host" in
        localhost|localhost.localdomain) return 0 ;;
        0.0.0.0) return 0 ;;
        # 127/8, but only as a real dotted quad: `127.example.com` is
            a hostname.
        127.*) case "$host" in *(!0-9.)* ) return 1 ;; *) return 0 ;;
            esac ;;
        # IPv4-mapped/compatible IPv6 (::ffff:127.0.0.1): grade the
            embedded v4.
        *:*.*.*.*)
            v4="${host##*:}"
            case "$v4" in
                *(!0-9.)* ) return 1 ;;
                127.*|0.0.0.0) return 0 ;;
                *) return 1 ;;
            esac ;;
        # Pure IPv6: loopback (::1) and unspecified (::) in any zero-
            compression.
        *:*)
            squash="${host//:/}"; squash="${squash//0/}"
            [[ -z "$squash" || "$squash" == "1" ]] && return 0
            return 1 ;;
    esac
    return 1

```

```

}
CMA_CCRGW_EOF
}
# Define it in THIS shell from the exact same bytes the alias file
  receives.
eval "$(_cma_emit_ccr_gateway_guard)"

# Dynamic account dispatcher (live defect 2026-07-23, `claude5`):
  a newly
# added account's alias is INVISIBLE to every shell that sourced
  the alias
# file BEFORE the add – and long-lived shells are the norm (tmux
  panes
# survive SSH re-logins; re-attaching does NOT restart the pane's
  shell, so
# "log out and back in" does not pick the alias up). The static
  per-account
# `alias claudeN=...` lines can never fix that class: an alias must
  exist at
# shell start.
#
# The dispatcher closes the class STRUCTURALLY: a command-not-
  found handler
# (bash: command_not_found_handle; zsh: command_not_found_handler)
  resolves
# `` -> $HOME/<prefix><name> at INVOCATION time from the
  config dirs,
# so ANY account added AFTER a shell sourced a dispatcher-bearing
  alias file
# works in that shell immediately, with no re-source. It is a pure
  FALLBACK:
# defined aliases/functions/binaries always win (the handler only
  fires when
# nothing else matched), a pre-existing handler (e.g. Ubuntu's
  command-not-found) is preserved and chained for non-account
  names, and a
# non-account miss still reports "command not found" with rc 127.
#
# Honest boundary: shells that sourced a PRE-dispatcher alias file
  remain
# stale until they re-source once (claude-add-account warns about
  exactly
# this) – the dispatcher eliminates the staleness class from then
  on, it
# cannot retroactively appear in old shells. Accounts created with
  a custom
# --dir that does not follow the $HOME/<prefix><alias> pattern are
  resolved
# by their (re-sourced) alias line only; the dispatcher cannot
  infer such a
# mapping and does not guess (§11.4.6).
_cma_emit_account_dispatch() {
  # The account-dir prefix is embedded at render time (like
    CLAUDE_BIN above)
  # so the emitted file is self-contained; the runtime fallback
    keeps a

```

```

# hand-truncated header from breaking dispatch entirely.
printf '_CMA_ACCOUNT_PREFIX=%q\n' "${ACCOUNT_PREFIX:-.claude-}"
cat <<'CMA_DISPATCH_EOF'
# True when $1 names a launchable ACCOUNT: a plain command word
# (no path
# separator, not an option, not dot-prefixed) whose $HOME/
# <prefix>$1 config
# dir exists. Provider config dirs (<prefix>prov-*) and the
# router's own dir
# (<prefix>code-router) are NOT accounts – providers launch via
# cma_run_provider and must never be hijacked by the account
# fallback.
_cma_is_account_cmd() {
  case "${1:-}" in ''|*/*|-*|.*) return 1 ;; esac
  case "$1" in prov-*|code-router) return 1 ;; esac
  [ -d "$HOME/${_CMA_ACCOUNT_PREFIX:-.claude-}$1" ]
}
_cma_cnf_impl() {
  if _cma_is_account_cmd "${1:-}"; then
    local _cma_d="$HOME/${_CMA_ACCOUNT_PREFIX:-.claude-}$1"
    shift
    CLAUDE_CONFIG_DIR=$_cma_d cma_run "$@"
    return $?
  fi
  # Not an account: hand off to whatever handler existed before
  # this file was
  # sourced (captured below), else reproduce the shell's stock
  # behaviour.
  if declare -f _cma_prev_cnf_handle >/dev/null 2>&1; then
    _cma_prev_cnf_handle "$@"
    return $?
  fi
  printf '%s: command not found\n' "${1:-}" >&2
  return 127
}
# Preserve a PRE-EXISTING handler exactly once. Idempotent across
# re-sources:
# a definition that already routes through _cma_cnf_impl is OURS
# and must not
# be captured as "previous" (that would chain the handler to
# itself).
if [ -n "${BASH_VERSION:-}" ]; then
  _cma_cnf_name=command_not_found_handle
else
  _cma_cnf_name=command_not_found_handler
fi
_cma_cnf_def="$(declare -f "$_cma_cnf_name" 2>/dev/null || true)"
case "$_cma_cnf_def" in
  *_cma_cnf_impl*) : ;;
  '') : ;;
  *) eval "_cma_prev_cnf_handle() ${_cma_cnf_def#*'}'" ;;
esac
unset _cma_cnf_def _cma_cnf_name

```

```

# Define BOTH spellings; each shell consults only its own, the
    other is inert.
command_not_found_handle() { _cma_cnf_impl "$@"; }
command_not_found_handler() { _cma_cnf_impl "$@"; }
CMA_DISPATCH_EOF
}

# ATM-850 remediation half 2 (2026-07-23): operator-facing tmux
    staleness
# notice. The dispatcher above closes the new-account staleness
    class for
# every shell that sourced a dispatcher-bearing alias file; shells
    older than
# that render still hold pre-dispatcher definitions until they re-
    source ONCE.
# tmux pane shells are the canonical long-lived case (five servers
    from 00:00
# survived the 15:14 add in the live incident; SSH re-login re-
    attaches the
# SAME server). This function only DETECTS and PRINTS – the exact
    re-source
# command plus a per-server broadcast one-liner scoped to idle
    SHELL panes –
# and NEVER sends keys itself: an autonomous send-keys could type
    into a pane
# the operator is mid-edit in. Callers may offer an interactive,
# default-No broadcast on top (claude-add-account does).
cma_tmux_stale_shell_notice() {
    local alias_file="${1:-$ALIAS_FILE}"
    command -v tmux >/dev/null 2>&1 || return 0
    local tdir="${TMUX_TMPDIR:-/tmp}/tmux-$(id -u 2>/dev/null ||
        echo 0)"
    [[ -d "$tdir" ]] || return 0
    local sock cmds n total=0
    local -a servers=()
    for sock in "$tdir"/*; do
        [[ -S "$sock" ]] || continue
        cmds="$(tmux -S "$sock" list-panes -a -F
            '#{pane_current_command}' 2>/dev/null)" || continue
        [[ -n "$cmds" ]] || continue
        n="$(printf '%s\n' "$cmds" | grep -cE '^(?(bash|zsh|sh|dash|
            ksh|fish)$)'" || n=0
        servers+=("$n:$sock"); total=$(( total + n ))
    done
    (( ${#servers[@]} )) || return 0
    printf
        '\n[tmux] %d running tmux server(s) detected – their pane
        shells read the alias file when THEY started, not now.\n'
        "${#servers[@]}"
    if grep -q '_cma_cnf_impl' "$alias_file" 2>/dev/null; then
        printf '[tmux] This alias file carries the dynamic account
            dispatcher: shells that sourced a\n'
        printf '[tmux] dispatcher-bearing render resolve NEW accounts
            at invocation time – no re-source needed there.\n'
    fi
}

```



```

    printf '[tmux] Only shells that sourced an OLDER (pre-
        dispatcher) render still need ONE re-source:\n'
else
    printf '[tmux] Every pre-existing shell needs a re-source to
        see this account:\n'
fi
printf '
        source %s\n' "$alias_file"
printf '[tmux] %d shell pane(s) detected across those servers.
        To push the re-source into the idle\n' "$total"
printf '[tmux] SHELL panes of a server (editors/agents/panes
        running programs are skipped):\n'
local ent
for ent in "${servers[@]}; do
    sock="${ent#*:}"
    printf "
        tmux -S %s list-panes -a -F '#{pane_id}
        #{pane_current_command}' | awk '\$2 ~ /^-?(bash|zsh|sh|
        dash|ksh|fish)\$/ {print \$1}' | xargs -r -I{} tmux -S %s
        send-keys -t {} C-u ' source %s' Enter\n" "$sock" "$sock"
        "$alias_file"
done
return 0
}

_cma_emit_cma_run() {
    cat <<'CMA_RUN_BODY_EOF'
# Wrapper: keeps .claude.json projects/session index synced across
    every
# logged-in account. Pulls merged state from every account into
    the launching
# one before claude runs; pushes the post-session state back out
    after exit.
# Cheap (jq deep-merge of one ~50KB file per account), runs
    unconditionally.
cma_run() {
    # Self-heal CLAUDE_BIN: the alias file normally exports it at
        the top, but if
    # that header line is missing (corrupted or hand-edited alias
        file) every
    # invocation would silently expand to an empty command ("-
        bash: : command
    # not found"). Mirrors cma_resolve_claude_bin inline so the
        function body is
    # self-contained regardless of the header state. §11.4.185.
    if ! command -v "${CLAUDE_BIN:-}" >/dev/null 2>&1; then
        if command -v claude >/dev/null 2>&1; then
            CLAUDE_BIN="$(command -v claude)"
        else
            for _cma_cb in "$HOME/.local/bin/claude" "$HOME/.npm-global/
                bin/claude" \
                    /opt/homebrew/bin/claude /usr/local/bin/
                claude; do
                [ -x "$_cma_cb" ] && { CLAUDE_BIN="$_cma_cb"; break; }
            done
            if ! command -v "${CLAUDE_BIN:-}" >/dev/null 2>&1; then

```

```

        printf 'cma_run: claude binary not found – check PATH or
        re-run install.sh\n' >&2
        return 127
    fi
fi
fi
# Provider-env isolation: native claudeN must talk to the real
  Anthropic API.
# A provider alias run earlier in THIS shell exports
  ANTHROPIC_BASE_URL etc.;
# those persist and would otherwise leak into this native launch
  (claude1
# silently using a provider's endpoint). Clear them so native is
  always clean.
# The 4 ANTHROPIC_DEFAULT_*_MODEL tier-map vars are exported by
# cma_run_provider (native transport) and PERSIST after that
  alias returns; a
# subsequent native claudeN launch MUST clear them too, else the
  opus/sonnet/
# haiku/fable tier resolution silently points at the previous
  provider's
# serving model instead of the real Anthropic tier.
unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL
  ANTHROPIC_SMALL_FAST_MODEL
unset ANTHROPIC_DEFAULT_OPUS_MODEL
  ANTHROPIC_DEFAULT_SONNET_MODEL
  ANTHROPIC_DEFAULT_HAIKU_MODEL
  ANTHROPIC_DEFAULT_FABLE_MODEL
# Token-guard isolation: cma_run_provider exports
  CLAUDE_CODE_MAX_OUTPUT_TOKENS
# (output cap, clamped <=128000),
  CLAUDE_CODE_AUTO_COMPACT_WINDOW (input
# compact trigger) and CLAUDE_CODE_MAX_CONTEXT_TOKENS (the
  provider's real
# context size for client-side accounting) for every provider
  alias; ALL
# persist in this shell after that alias returns. A subsequent
  native claudeN
# launch MUST clear them, else native inherits a provider's
  (possibly small)
# output cap, compact window, or context size instead of the
  real Anthropic
# per-model defaults – silently capping native's output or
  early-compacting
# its context. Parallels the ANTHROPIC_DEFAULT_* tier-map
  isolation above.
unset CLAUDE_CODE_MAX_OUTPUT_TOKENS
  CLAUDE_CODE_AUTO_COMPACT_WINDOW
  CLAUDE_CODE_MAX_CONTEXT_TOKENS
# Working-dir hook (opt-in; no-op when absent). Resolution
  order:
#   1. CMA_CWD_HOOK env var – explicit user
  override
#   2. <git-toplevel>/.claude-cwd-hook – per-project hook
  (each repo

```

```

#     gets its own multitrack resolver, preventing a single
#     global hook
#     from hijacking every project's sessions)
# 3. ~/.local/bin/claude-cwd-hook           - global fallback
# The hook runs before claude-session (below) so auto-session
#     keys to the
# resolved workspace root. Escape hatch: MULTITRACK_DISABLE=1
#     (honored
# inside the hook itself; the toolkit does not check it).
local _cma_cwd_hook _cma_cwd_label _cma_cwd_target
if [[ -n "${CMA_CWD_HOOK:-}" ]]; then
    _cma_cwd_hook="$CMA_CWD_HOOK"
else
    local _cma_hook_root
    _cma_hook_root="$(git rev-parse --show-toplevel 2>/dev/null ||
        pwd)"
    if [[ -x "$_cma_hook_root/.claude-cwd-hook" ]]; then
        _cma_cwd_hook="$_cma_hook_root/.claude-cwd-hook"
    else
        _cma_cwd_hook="$HOME/.local/bin/claude-cwd-hook"
    fi
fi
if [[ -x "$_cma_cwd_hook" ]] && ! git rev-parse --show-toplevel
    >/dev/null 2>&1; then
    _cma_cwd_label="$(basename "${CLAUDE_CONFIG_DIR:-claude}")";
    _cma_cwd_label="${_cma_cwd_label#.claude-}"
    _cma_cwd_target="$("$_cma_cwd_hook" "$_cma_cwd_label" 2>/dev/
        null || true)"
    if [[ -n "$_cma_cwd_target" && -d "$_cma_cwd_target" ]]; then
        cd "$_cma_cwd_target" 2>/dev/null || true; fi
fi
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" pull "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
# Auto session-per-project: when launched with NO args, resume
#     (or create) the
# one long-lived session for this project root, name it after
# the root dir,
# trust the workspace, and hint the alias color. Only when bare
# so explicit
# user flags (-p, --resume, a prompt, ...) are always respected
# verbatim.
# claude-session emits only "--resume <uuid>" or "--session-id
# <uuid> --name
# <kebab>" (no shell metacharacters), so eval-splitting is safe
# and works in
# both bash and zsh (zsh does not word-split unquoted
# expansions).
local _cma_label=""
if [[ $# -eq 0 && -x "$HOME/.local/bin/claude-session" ]]; then
    local _cma_sf
    _cma_sf="$("$HOME/.local/bin/claude-session" flags
        "$CLAUDE_CONFIG_DIR" 2>/dev/null || true)"

```

```

    _cma_label="$(basename "${CLAUDE_CONFIG_DIR:-claude}")";
    _cma_label="${_cma_label#.claude-}"
    "$HOME/.local/bin/claude-session" hint "$_cma_label" 2>/dev/
    null || true
    eval "set -- $_cma_sf"
    # Auto-apply the per-alias color: a resumable session's jsonl
    exists now, so
    # colour it before launch; a brand-new session's file appears
    during launch,
    # so we colour it again after exit (see post-launch call
    below).
    "$HOME/.local/bin/claude-session" apply-color
    "$CLAUDE_CONFIG_DIR" "$_cma_label" 2>/dev/null || true
fi
"$CLAUDE_BIN" "$@"
local rc=$?
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" push "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
[[ -n "$_cma_label" && -x "$HOME/.local/bin/claude-session" ]]
&& \
    "$HOME/.local/bin/claude-session" apply-color
    "$CLAUDE_CONFIG_DIR" "$_cma_label" 2>/dev/null || true
return $rc
}
CMA_RUN_BODY_EOF
}

_cma_emit_cma_run_provider() {
    cat <<'CMA_PROV_BODY_EOF'
cma_run_provider() {
    # Self-heal CLAUDE_BIN (same as cma_run – §11.4.185). Prevents
    "-bash: : command
    # not found" when the alias-file header export line is missing.
    Also resolves
    # the binary for the native-transport path ("$CLAUDE_BIN" "$@" )
    below.
    if ! command -v "${CLAUDE_BIN:-}" >/dev/null 2>&1; then
        if command -v claude >/dev/null 2>&1; then
            CLAUDE_BIN="$(command -v claude)"
        else
            for _cma_cb in "$HOME/.local/bin/claude" "$HOME/.npm-global/
            bin/claude" \
                /opt/homebrew/bin/claude /usr/local/bin/
            claude; do
                [ -x "$_cma_cb" ] && { CLAUDE_BIN="$_cma_cb"; break; }
            done
        fi
    fi
    if ! command -v "${CLAUDE_BIN:-}" >/dev/null 2>&1; then
        printf 'claude-providers: claude binary not found – check PATH
        or re-run install.sh\n' >&2
        return 127
    fi
}

```

```

fi
# --force bypasses the activation gate (operator override).
#   Accepted BOTH as
# the very first arg (direct call: cma_run_provider --force
#   <id>) and as the
# first arg after the id (alias path: `<alias> --force` expands
#   to
# cma_run_provider <id> --force). Either way it is consumed, not
#   forwarded.
local _cma_force=0
if [[ "${1:-}" == "--force" ]]; then _cma_force=1; shift; fi
local id="$1"; shift 2>/dev/null || true
if [[ "${1:-}" == "--force" ]]; then _cma_force=1; shift; fi
local pdir="$HOME/.local/share/claude-multi-account/providers"
local envf="$pdir/$id.env"
if [[ ! -f "$envf" ]]; then
    printf 'claude-providers: unknown provider %s (missing %s)\n'
        "$id" "$envf" >&2
    return 1
fi
# shellcheck disable=SC1090
source "$envf"
# Cross-alias env isolation: unset any ANTHROPIC_*/CLAUDE_CODE_*
#   vars that
# leaked from a PREVIOUS cma_run_provider invocation in this
#   shell. The
# transport-specific branches below re-export them from this
#   provider's
# CMA_PROVIDER_* vars (fresh from source "$envf"), so the unset
#   here is
# only clearing the leftover from the previous alias – identical
#   to how
# cma_run (the native claudeN wrapper) isolates its ANTHROPIC_*
#   vars.
unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL
    ANTHROPIC_SMALL_FAST_MODEL
# The 4 tier-default-model vars this same wrapper exports
#   (native branch below)
# also persist into a following alias invocation; clear the
#   previous run's
# values so this provider re-exports its own from
#   CMA_PROVIDER_MODEL below.
unset ANTHROPIC_DEFAULT_OPUS_MODEL
    ANTHROPIC_DEFAULT_SONNET_MODEL
    ANTHROPIC_DEFAULT_HAIKU_MODEL
    ANTHROPIC_DEFAULT_FABLE_MODEL
unset CLAUDE_CODE_AUTO_COMPACT_WINDOW
    CLAUDE_CODE_MAX_OUTPUT_TOKENS
# Working-dir hook (same 3-tier resolution as cma_run). This
#   must run
# BEFORE sync-state pull + session flags so the resolved
#   directory is the
# session's canonical cwd. Without this, provider aliases ignore
#   the
# multitrack resolver and launch in whatever $PWD the user
#   happened to be in.

```

```

local _cma_cwd_hook _cma_cwd_label _cma_cwd_target
if [[ -n "${CMA_CWD_HOOK:-}" ]]; then
    _cma_cwd_hook="${CMA_CWD_HOOK}"
else
    local _cma_hook_root
    _cma_hook_root="$(git rev-parse --show-toplevel 2>/dev/null ||
        pwd)"
    if [[ -x "$_cma_hook_root/.claude-cwd-hook" ]]; then
        _cma_cwd_hook="$_cma_hook_root/.claude-cwd-hook"
    else
        _cma_cwd_hook="$HOME/.local/bin/claude-cwd-hook"
    fi
fi
if [[ -x "$_cma_cwd_hook" ]] && ! git rev-parse --show-toplevel
    >/dev/null 2>&1; then
    _cma_cwd_label="$(basename "${CLAUDE_CONFIG_DIR:-.claude}")";
    _cma_cwd_label="${_cma_cwd_label#}.claude-}"
    _cma_cwd_target="$("$_cma_cwd_hook" "$_cma_cwd_label" 2>/dev/
        null || true)"
    if [[ -n "$_cma_cwd_target" && -d "$_cma_cwd_target" ]]; then
        cd "$_cma_cwd_target" 2>/dev/null || true; fi
fi
# Activation gate: only a 'verified' alias launches Claude Code.
# A non-verified
# alias (unverified / failed / pending) prints a clear,
# actionable message and
# refuses to launch, so a broken provider never surfaces as a
# confusing
# in-session error. Status comes from the non-secret status
# cache; this body is
# self-contained (no cma_* helpers) so it reads status.json with
# jq inline.
if (( ! _cma_force )); then
    local _cma_sf="$pdir/status.json" _cma_st="pending"
    if command -v jq >/dev/null 2>&1 && [[ -s "$_cma_sf" ]]; then
        _cma_st="$(jq -r --arg i "$CMA_PROVIDER_ID" '.[$i].status //
            "pending"' "$_cma_sf" 2>/dev/null)"
        [[ -n "$_cma_st" && "$_cma_st" != "null" ]] ||
            _cma_st="pending"
    fi
    if [[ "$_cma_st" != "verified" ]]; then
        printf 'claude-providers: alias %s is %s – not launching.\n'
            "$CMA_PROVIDER_ID" "$_cma_st" >&2
        printf '  Re-verify: claude-providers sync    (re-runs
            verification for %s)\n' "$CMA_PROVIDER_ID" >&2
        printf '  Override (operator): run the alias with --force\n'
            >&2
        return 3
    fi
fi
local keysf="${CMA_KEYS_FILE:-$HOME/api_keys.sh}"
# Disable nounset while sourcing the user-controlled keys file:
# it may have
# dangling refs (e.g. `export X=$UNSET`) that would abort the
# source under a

```

```

# caller's `set -u`, leaving the token empty. Save/restore so we
# never change
# the user's interactive shell options.
if [[ -f "$keysf" ]]; then
    case $- in *) local _cma_had_u=1 ;; *) local _cma_had_u=0 ;;
    esac
    set -a +u; source "$keysf"; set +a
    (( _cma_had_u )) && set -u
fi
# Indirect-expand the key var name. ${!var} is bash-only and a
# fatal error in
# zsh (the default macOS interactive shell this alias file is
# sourced into),
# so use eval, which works in both. CMA_PROVIDER_KEYVAR is a
# validated env
# var name ([A-Za-z_][A-Za-z0-9_]*), so this eval is safe.
local token="" _cma_xt=""
# Suppress xtrace around the indirect key read so an active `set
# -x` in the
# user's shell can't echo the secret to the terminal or a
# redirected log.
case $- in *) _cma_xt=1; set +x ;; esac
# Kimi Code OAuth sentinel: the OAuth token is SHORT-LIVED (~15
# min), so a
# sync-time snapshot is stale by the next launch. Freshness
# order:
# 1. the LIVE kimi-code credentials file, when unexpired (60s
# skew);
# 2. a CLI-triggered refresh (kimi -p hi) followed by a re-read
# of 1;
# 3. the token-file snapshot written at sync (last resort
# only).
if [[ "$CMA_PROVIDER_KEYVAR" == "_CMA_KIMICODE_OAUTH_" ]]; then
    local _cma_kcred="$HOME/.kimi-code/credentials/kimi-code.json"
    if [[ -f "$_cma_kcred" ]] && command -v jq >/dev/null 2>&1;
    then
        local _cma_kexp; _cma_kexp="$(jq -r '.expires_at // 0'
        "$_cma_kcred" 2>/dev/null || echo 0)"
        if (( _cma_kexp > $(date +%s) + 60 )); then
            token="$(jq -r '.access_token // ""' "$_cma_kcred" 2>/dev/
            null)"
        fi
    fi
fi
if [[ -z "$token" && -f "$_cma_kcred" ]] && command -v kimi >/
dev/null 2>&1; then
    timeout 20 kimi -p "hi" --output-format text >/dev/null 2>&1
    || true
    token="$(jq -r '.access_token // ""' "$_cma_kcred" 2>/dev/
    null)"
fi
if [[ -z "$token" ]]; then
    local _cma_ktok="$pdir/${CMA_PROVIDER_ID}.token"
    [[ -f "$_cma_ktok" ]] && token="$(cat "$_cma_ktok" 2>/dev/
    null)" || token=""
fi

```

```

else
    eval "token="\${$CMA_PROVIDER_KEYVAR:-}"
fi
[[ -n "$_cma_xt" ]] && set -x
if [[ -z "$token" ]]; then
    printf 'claude-providers: $%s is empty (set it in %s)\n'
        "$CMA_PROVIDER_KEYVAR" "$keysf" >&2
    return 1
fi
export CLAUDE_CONFIG_DIR="$CMA_PROVIDER_CONFIG_DIR"
# Input-context guard (fixes "400 exceeded model token limit:
# 262144
# (requested: 311786)": tell Claude Code this provider's REAL
# context window
# so it auto-compacts (at window-13000) before a request
# overshoots the
# provider's hard input limit. Without this, Claude Code assumes
# Anthropic's
# own large (~1M) window and lets the prompt grow past a smaller
# provider's
# cap. Fully dynamic – the value is CMA_PROVIDER_CONTEXT_LIMIT,
# resolved from
# the models.dev catalog (limit.context) per selected model.
# Applies to BOTH
# transports (native + router), so every provider alias is
# protected.
# NOTE: this caps INPUT context; CLAUDE_CODE_MAX_OUTPUT_TOKENS
# (set just
# below, before the transport branch, BOTH transports, clamped
# <=128000)
# caps OUTPUT – the two are independent halves of the guard.
# Auto-compact cap: only lower the window; never raise it above
# the cap.
# The DEFAULT cap is context-proportional – max(200000, context/
# 2) – because
# a fixed 200000 was correct for 200K-class providers but
# produced the
# 2026-09-03 endless-compaction loop on 1M-class ones: a native-
# alias session
# grows to ~977K-1M tokens (native unsets every guard, see
# cma_run), and
# resumed under a provider alias whose window the 200000 cap
# forced to
# 176000, ONE substantive turn of real work (~150K tokens of
# tool results,
# measured live on the helix_code session) refilled past the
# trigger and
# compacted AGAIN – a compact on every turn, indistinguishable
# from an
# infinite loop. context/2 keeps the cap's original intent
# (compact around
# ~50% of the model's window, never near the hard limit) while
# giving
# large-context providers enough post-compact headroom for a
# resumed native

```



```

# session to actually continue. CMA_AUTO_COMPACT_CAP overrides
  outright.
#
# _cma_in_guard (v1.24.0): the input guard is exported for EVERY
  provider
# with a known context, CLAMPED – never skipped. The previous
  gate
# ("export only when CONTEXT_LIMIT <= cap") was FAIL-OPEN and
  backwards: a
# provider whose context was LARGER than the cap got no input
  guard at all,
# which is precisely the case that needs one. openrouter
  (catalog context
# 1000000) therefore ran unguarded on the input side while the
  output side
# reserved 128000, and the request overshoot the endpoint's real
  262144
# window: 33796 + 103687 + 128000 = 265483. See _cma_out_guard
  below for the
# matching output half; the window is computed AFTER it, from
  ctx - out, so
# the two halves cannot sum past the context.
# --- cma-token-guards:begin --- (extracted verbatim by
  test_providers.sh;
# the guards are pure arithmetic over
  CMA_PROVIDER_{CONTEXT_LIMIT,MAX_OUTPUT},
# so the suite can exercise the SHIPPED source without launching
  anything.)
local _cma_compact_cap="${CMA_AUTO_COMPACT_CAP:-}"
# Sanitize the override: a non-numeric CMA_AUTO_COMPACT_CAP must
  fall back to
# the context-proportional default, never reach an integer
  comparison under
# set -e (which would abort every provider alias launch).
case "$_cma_compact_cap" in
  ''|*[^0-9]*) _cma_compact_cap="" ;;
esac
# _cma_out_guard (v1.16.0) + <=128000 clamp ($11.4.108/
  $11.4.111): output-
# token cap for BOTH transports, not just native. Without it,
  router
# providers run with Claude Code's generic default output cap
  (128000 for
# models it does not know) and long reasoning responses die with
  "Claude's
# response exceeded the 128000 output token maximum". The value
  starts from
# the provider model's REAL output limit (models.dev
  limit.output via
# CMA_PROVIDER_MAX_OUTPUT); proxies may clamp further API-side
# (sarvam_proxy's tier clamp). Exported ONCE here, before the
  transport
# branch, so router AND native behave identically (previously
  only the
# native branch re-exported it – an unclamped, transport-
  asymmetric raw

```

```

# value).
# Catalog caveat (live-proven on nvidia5): when limit.output >=
# limit.context the "output" number is really the context size –
# exporting
# it makes Claude Code request that many completion tokens, and
# input+request overshoots the shared window (400 "maximum
# context length
# is N ... you requested M"). Only a genuinely separate output
# budget
# (output < context) is exported.
# Clamp caveat (live-proven, 128k Tier-1): exporting the model's
# THEORETICAL
# limit.output when it exceeds the CLI's own custom-model
# ceiling (deepseek
# 384000, xiaomi 131072) makes Claude Code request its OWN
# unknown-model
# ceiling (128000) and then FATALLY abort any length-truncated
# response:
# "...exceeded the 128000 output token maximum... set
# CLAUDE_CODE_MAX_OUTPUT_TOKENS". The CLI hard-caps custom
# models to 128000
# regardless, so any value >128000 is pointless – clamp to
# min(CMA_PROVIDER_MAX_OUTPUT, 128000).
# Sanitize-then-decide order is load-bearing (POSIX-shape so it
# behaves
# identically whether this body is sourced by bash or zsh), and
# NO
# arithmetic ever runs on an unsanitized value ([ N -gt .. ]
# errors past
# 2^63-1, and (( )) errors on non-integers –
# CMA_PROVIDER_MAX_OUTPUT traces
# to the user-settable CMA_HELIXAGENT_MAX_OUTPUT, fed via 'jq --
# argjson'
# which preserves huge-int digits verbatim, so both shapes are
# reachable):
# 1. empty / non-plain-integer (negatives, "1e6", "12.5") /
# zero -> NO
# export: no real output budget is known, and the CLI's own
# unknown-model default (128000) applies exactly as if the
# catalog had
# no entry. (The pre-merge always-export-128000 default was
# effect-equivalent for known models but could resurrect
# the nvidia5
# overshoot on small-context catalog-gap models – the
# conditional
# no-export subsumes it safely.)
# 2. >18 digits: past intmax – no test/(( )) arithmetic is
# safe. Any real
# context (<=18 digits) is smaller, so with a usable
# context this is
# the mislabel shape (-> NO export); with no usable context
# it
# collapses to the 128000 cap WITHOUT arithmetic on the raw
# value.
# 3. <=18 digits (test-safe): floor 0/00/000 to no-export,
# apply the

```

```

#     nvidia5 mislabel skip (output >= context -> NO export),
#     then the
#     128000 clamp. A leading-zero form like 007 tests as 7
#     here, stays
#     <=128000, and exports as "007" (Claude Code parses it as
#     decimal 7 -
#     min-semantics); a leading-zero 19+ digit form was already
#     collapsed
#     by rule 2, so it is NEVER re-read as octal.
local _cma_out="${CMA_PROVIDER_MAX_OUTPUT:-}" _cma_octx="$
{CMA_PROVIDER_CONTEXT_LIMIT:-}"
case "$_cma_octx" in
    ''|*(!0-9)*) _cma_octx="" ;;
    *) [ "${#_cma_octx}" -le 18 ] || _cma_octx="" ;;
esac
case "$_cma_out" in
    ''|*(!0-9)*) _cma_out="" ;;
    *) if [ "${#_cma_out}" -gt 18 ]; then
        # Past intmax no test/(( )) arithmetic is safe. With a
        # known context
        # the carve-out below supplies the cap; with none, fall
        # back to the
        # CLI ceiling WITHOUT arithmetic on the raw value (F2:
        # never leak it
        # unclamped).
        if [ -n "$_cma_octx" ]; then _cma_out=""; else
            _cma_out=128000; fi
        elif [ "$_cma_out" -lt 1 ]; then _cma_out=""
        elif [ -n "$_cma_octx" ] && [ "$_cma_out" -ge
            "$_cma_octx" ]; then _cma_out=""
        fi ;;
esac
# Carve the output cap OUT of the context instead of trusting it
# alongside
# the context (v1.24.0). Leaving it unset is NOT the safe
# fallback it looks
# like: Claude Code's own default for an unknown model is
# 128000, so "no
# export" IS a request for 128000 output tokens. kilo proves the
# point - its
# catalog row is the impossible ctx==out==262144, the mislabel
# branch above
# blanks the value, and the CLI default then reserves 128000
# against a 262144
# window that must also carry Claude Code's ~137K system-prompt
# + tool-schema
# floor. Whenever the context is known we therefore always emit
# a cap that
# provably fits: min(catalog output, context - input floor),
# bounded by the
# CLI's own 128000 ceiling and floored at 8192.
# Keep this in lockstep with CLAUDE_CODE_INPUT_FLOOR /
# MIN_SAFE_OUTPUT /
# CLI_MAX_OUTPUT_TOKENS in providers_resolve.py: the resolver
# bakes these

```

```

# numbers into the .env, and this block is the last line of
# defence for a
# stale or hand-edited one.
local _cma_in_floor="${CMA_INPUT_FLOOR:-160000}" _cma_cap=""
if [ -n "$_cma_octx" ]; then
    _cma_cap=$(( _cma_octx - _cma_in_floor ))
    if [ "$_cma_cap" -gt 128000 ]; then _cma_cap=128000; fi
    if [ "$_cma_cap" -lt 8192 ]; then _cma_cap=8192; fi
    if [ -z "$_cma_out" ] || [ "$_cma_out" -gt "$_cma_cap" ]; then
        _cma_out="$_cma_cap"; fi
elif [ -n "$_cma_out" ] && [ "$_cma_out" -gt 128000 ]; then
    _cma_out=128000
fi
if [ -n "$_cma_out" ]; then
    export CLAUDE_CODE_MAX_OUTPUT_TOKENS="$_cma_out"
fi
# Input half, computed from what the output half actually
# reserved so the two
# can never sum past the context. Clamped to the auto-compact
# cap, and
# exported for every known context – including the >cap ones the
# old gate
# silently skipped. This also closes the 200K-270K "dead zone",
# where a real
# window sat above the cap yet below cap+output and so got no
# guard at all.
#
# TOOL-SCHEMA BUDGET (live defect 2026-07-26, openrouter3): the
# old
# co-derivation window = context - output pretended the request
# is only
# text + output. The endpoint counts tools too – the provider's
# own 400
# named it: "maximum context length is 262144 ... you requested
# about 371727
# tokens (199347 of text input, 70236 of tool input, 102144 in
# the output)".
# With window = 262144 - 102144 = 160000 the compact trigger sat
# at ~147000,
# and 147000 + 70236 tools + 102144 output = 319380 > 262144 –
# the endpoint
# rejected BEFORE compaction could ever help, on the FIRST
# request. The
# trigger must reserve for the tool payload as well:
# window = context - output - CMA_TOOL_TOKEN_BUDGET (default
# 80000).
# BE EXACT ABOUT WHICH DIRECTION IS DANGEROUS – the earlier note
# here had it
# backwards. An UNDERestimate does NOT merely "compact earlier":
# because the
# output cap is derived as ctx - min_win - budget, too small a
# budget INFLATES
# the output cap, and the request then overflows and the launch
# dies with a

```

```

# 400. Measured live 2026-07-27 on a host with 227 enabled
# plugins: real tool
# input 158112 against this 80000 default gave
# 46536 text + 158112 tools + 62144 output = 266792 on a
# 262144 window.
# Note further that 158112 + 120000 (min window) already exceeds
# 262144, so on
# such a host NO output cap makes a 262144-context provider fit:
# the enabled
# plugin surface has to shrink, or a wider-context provider must
# be used
# (nvidia 1000000 and xiaomi 1048576 fit it comfortably).
# Override per host
# via CMA_TOOL_TOKEN_BUDGET in the provider env or the shell. A
# context too
# small to reserve the budget yields no window at all: that
# provider cannot
# serve Claude Code's tool prefix regardless, and the launch
# must fail
# loudly upstream (the proof's context-inadequate class) rather
# than be
# lied to by a guard that cannot hold.
if [ -n "$_cma_octx" ]; then
    local _cma_win="$_cma_octx" _cma_tool_budget="$
        {CMA_TOOL_TOKEN_BUDGET:-80000}"
    local _cma_min_win="{CMA_MIN_COMPACT_WINDOW:-120000}"
    if [ -n "$_cma_out" ]; then _cma_win=$(( _cma_octx -
        _cma_out )); fi
    _cma_win=$(( _cma_win - _cma_tool_budget ))
    # Compression-loop guard: if the window is below the minimum
    # viable size,
    # Claude Code triggers compaction on every request (system
    # prompt + tool
    # schemas alone exceed the window), enters an infinite
    # compression loop,
    # and never gets any work done. Raise the window by reducing
    # the output
    # cap when possible, rather than exporting a window that
    # cannot hold the
    # static overhead.
    if [ "$_cma_win" -lt "$_cma_min_win" ] && [ -n "$_cma_out" ]
        && [ "$_cma_out" -gt 8192 ]; then
        local _cma_new_out=$(( _cma_octx - _cma_min_win -
            _cma_tool_budget ))
        if [ "$_cma_new_out" -ge 8192 ]; then
            _cma_out="$_cma_new_out"
            _cma_win="$_cma_min_win"
            export CLAUDE_CODE_MAX_OUTPUT_TOKENS="$_cma_out"
        fi
    fi
    # Context-proportional compact cap (the 2026-09-03 loop fix):
    # resolve the
    # default HERE, where the sanitized integer context is in
    # scope. The floor
    # of 200000 keeps every <=400K-context provider byte-identical
    # to the

```

```

# pre-fix behaviour; only a provider whose context/2 exceeds
    200000 gets a
# larger cap. An explicit CMA_AUTO_COMPACT_CAP always wins.
if [ -z "$_cma_compact_cap" ]; then
    _cma_compact_cap=200000
    local _cma_half_ctx=$(( _cma_ctx / 2 ))
    if [ "$_cma_half_ctx" -gt 200000 ]; then
        _cma_compact_cap="$_cma_half_ctx"; fi
fi
if [ "$_cma_win" -gt "$_cma_compact_cap" ]; then
    _cma_win="$_cma_compact_cap"; fi
# Autocompact thrashing guard (v1.26.8): the raw window above
    consumes every
# token the model can hold (text + tools + output = context).
    That leaves
# zero headroom for the compacted summary itself, so after
    Claude Code
# compacts the conversation the next large file read or tool
    output can
# refill the context to the limit within a few turns – the
    exact "Autocompact
# is thrashing" failure. Reserve a safety margin from the
    window so compaction
# fires earlier and the post-compact context has meaningful
    breathing room.
# The margin is applied ONLY when the provider can afford it
    without dropping
# below the minimum viable window; tight providers keep their
    existing window
# rather than trade one failure mode for another.
local _cma_safety_margin="$
    {CMA_AUTO_COMPACT_SAFETY_MARGIN:-24000}"
case "$_cma_safety_margin" in
    ''|0|*(!0-9)*) _cma_safety_margin=0 ;;
esac
if [ "$_cma_safety_margin" -gt 0 ] && [ "$_cma_win" -gt
    "$_cma_min_win" ]; then
    local _cma_max_margin=$(( _cma_win - _cma_min_win ))
    if [ "$_cma_safety_margin" -gt "$_cma_max_margin" ]; then
        _cma_safety_margin="$_cma_max_margin"
    fi
    _cma_win=$(( _cma_win - _cma_safety_margin ))
fi
if [ "$_cma_win" -gt 0 ]; then
    export CLAUDE_CODE_AUTO_COMPACT_WINDOW="$_cma_win"
fi
fi
# --- cma-token-guards:end ---
# Sync .claude.json projects/session index across ALL accounts
    and providers
# so sessions created under any alias are visible from every
    other alias.
# Pull merged state before launch; push post-session state after
    exit.
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then

```

```

"$HOME/.local/bin/claude-sync-state" pull "$CLAUDE_CONFIG_DIR"
2>/dev/null || true
fi
# _cma_session_flags (v1.17.0): per-project session resolution
# applies to
# BOTH transports – previously the flags block lived only in the
# native
# branch, so every router alias (kimi-*, poe, openrouter, ...)
# always opened
# a FRESH session and could never see the project session
# another alias
# left behind. It also now covers conversation args: `alias -p
# "...` used to
# skip resolution entirely (verbatim-args rule) and start a new
# session
# every time. Explicit session selectors and non-conversation
# subcommands
# are always left verbatim.
local _cma_psf=""
if [[ -x "$HOME/.local/bin/claude-session" ]]; then
  if [[ $# -eq 0 ]]; then
    # TRIM providers skip the stored session flags here too:
    # `flags` emits
    # `--resume <id> ...` for an existing project session, which
    # drags the
    # synced history into the INTERACTIVE path just like the
    # auto-resume
    # below does for conversation args (live issue 2026-07-22 –
    # both seams
    # must stay history-free or a local model's window
    # overflows).
    if [[ "${CMA_PROVIDER_TRIM:-}" != "bare" ]]; then
      _cma_psf="$("$HOME/.local/bin/claude-session" flags
        "$CLAUDE_CONFIG_DIR" 2>/dev/null || true)"
      # Resume-fit warning (same rationale as the conversation-
      # args path
      # below): the flags form is `--resume <id> --name
      # <kebab>`, so the
      # session id is word 2. Warn-only, never block.
      if [[ "$_cma_psf" == --resume\ * ]]; then
        local _cma_rsid _cma_est=""
        _cma_rsid="$(printf '%s\n' "$_cma_psf" | awk '{print
          $2}')"
        _cma_est="$("$HOME/.local/bin/claude-session" context-
          size "$CLAUDE_CONFIG_DIR" "$_cma_rsid" 2>/dev/null ||
          true)"
        case "$_cma_est" in ''|*[*!0-9]*) _cma_est="" ;; esac
        if [[ -n "$_cma_est" && -n "$
          {CLAUDE_CODE_AUTO_COMPACT_WINDOW:-}" ]] \
          && [ "$_cma_est" -gt
            "$CLAUDE_CODE_AUTO_COMPACT_WINDOW" ]; then
          printf 'claude-providers: resume target %s holds ~%s
            tokens; provider %s compact window is %s – it will be
            compacted immediately on resume (then continue normally).
            \n' \

```

```

        "$_cma_rsid" "$_cma_est" "$CMA_PROVIDER_ID"
"$CLAUDE_CODE_AUTO_COMPACT_WINDOW" >&2
    fi
fi
fi
"$HOME/.local/bin/claude-session" hint "$CMA_PROVIDER_ID"
2>/dev/null || true
eval "set -- $_cma_psf"
# Auto-apply this provider alias's color to the session
(idempotent).
"$HOME/.local/bin/claude-session" apply-color
"$CLAUDE_CONFIG_DIR" "$CMA_PROVIDER_ID" 2>/dev/null ||
true
_cma_pcolor=1
else
case "$1" in
--resume|--session-id|--continue|--fork-session|-c) ;;
agents|mcp|export|doctor|install|update|config|plugin|
setup|acp|server|web|provider) ;;
*)
    # existing-id (NOT latest-id): latest-id falls back to
the
    # deterministic UUID for never-used projects, and
injecting --resume
    # with a session that was never created fails hard ("No
conversation
    # found with session ID"). Inject only for a session
that EXISTS.
    # TRIM providers (CMA_PROVIDER_TRIM=bare) SKIP the auto-
resume: a
    # fresh session per launch keeps the synced session
history out of
    # the request (live issue 2026-07-22: a 229,376-token
local-model
    # window refused every helixagent launch under ~330k
tokens of
    # resumed history – history must not ride along by
default).
    if [[ "${CMA_PROVIDER_TRIM:-}" != "bare" ]]; then
        _cma_psf="$("$HOME/.local/bin/claude-session"
existing-id "$CLAUDE_CONFIG_DIR" 2>/dev/null || true)"
        if [[ -n "$_cma_psf" ]]; then
            # Resume-fit warning (2026-09-03 compaction-loop
fix): when the
            # session about to be resumed already holds MORE
tokens than
            # this provider's computed auto-compact window,
Claude Code
            # compacts it immediately on resume. That one-off
compact is
            # intended – but it must never be SILENT: the
operator otherwise
            # sees an unexplained "Compacting" stall on every
alias switch
            # (the endless-compaction report). Warn, never block
– the

```



```

        # resume still happens, the window-scaling guard
        above gives the
        # post-compact session real headroom.
        local _cma_est=""
        _cma_est="$("$HOME/.local/bin/claude-session"
context-size "$CLAUDE_CONFIG_DIR" "$_cma_psf" 2>/dev/null
|| true)"
        case "$_cma_est" in ''|*[!0-9]*) _cma_est="" ;; esac
        if [[ -n "$_cma_est" && -n "$
{CLAUDE_CODE_AUTO_COMPACT_WINDOW:-}" ]] \
        && [ "$_cma_est" -gt
"$CLAUDE_CODE_AUTO_COMPACT_WINDOW" ]; then
            printf 'claude-providers: resume target %s holds
~%s tokens; provider %s compact window is %s – it will be
compacted immediately on resume (then continue normally).
\n' \
                "$_cma_psf" "$_cma_est" "$CMA_PROVIDER_ID"
"$CLAUDE_CODE_AUTO_COMPACT_WINDOW" >&2
            fi
            set -- --resume "$_cma_psf" "$@"
        fi
    fi
    ;;
esac
fi
fi
# TRIM providers launch MINIMAL: --bare skips the hook/plugin/
MCP/CLAUDE.md
# surface (~110k tokens of fixed tool schemas on a plugin-heavy
host) so a
# session actually fits a local model's context window.
Conversation
# launches only – non-conversation subcommands are left
untouched.
if [[ "${CMA_PROVIDER_TRIM:-}" == "bare" ]]; then
    case "${1:-}" in
        agents|mcp|export|doctor|install|update|config|plugin|setup|
        acp|server|web|provider) ;;
        *) set -- --bare "$@" ;;
    esac
fi
local rc
local _proxy_pid=""
# Model + tier maps for BOTH transports (live defect 2026-07-26,
Claude Code
# 2.1.220): these used to be exported only in the native branch
below, so a
# ROUTER-transport alias launched with no ANTHROPIC_MODEL and no
tier maps.
# 2.1.220 then resolves the tiers client-side – the fast/compact
tier to
# "Fable 5", which the binary reports as CURRENTLY UNAVAILABLE
("Claude
# Fable 5 is currently unavailable. Please use Opus 4.8 or
another available

```

```

# model.")). Auto-compaction runs on that fast tier, so on the
# first session
# whose context crosses the compact trigger the compact call
# itself fails
# client-side: the context is never freed, compaction retries
# forever, and
# the alias is unusable – exactly the operator-reported compact
# loop, with
# ZERO requests reaching the gateway (verified: no /v1/messages
# in the
# per-alias service.log during the loop). Mapping every tier to
# the
# provider's real serving model makes compact/fast/background
# calls
# addressable; the gateway substitutes the route model
# regardless of the
# requested id (proven: bogus ids and literal claude-* ids both
# get routed
# and answered), so these exports change only the CLIENT-side
# resolution,
# never the upstream model.
export ANTHROPIC_MODEL="$CMA_PROVIDER_MODEL"
[[ -n "${CMA_PROVIDER_FAST_MODEL:-}" ]] && export
    ANTHROPIC_SMALL_FAST_MODEL="$CMA_PROVIDER_FAST_MODEL"
# ...and the REAL context size with it: an id Claude's table
# does not know
# makes its context accounting fall back to a tiny default
# window, against
# which the system+tool prefix alone overflows – "Prompt is too
# long" on a
# 20-character prompt, or the same compact-retry loop on
# anything larger
# (both reproduced live 2026-07-26 against 2.1.220 with
# ANTHROPIC_MODEL set
# and this unset).
[[ -n "${CMA_PROVIDER_CONTEXT_LIMIT:-}" ]] && export
    CLAUDE_CODE_MAX_CONTEXT_TOKENS="$CMA_PROVIDER_CONTEXT_LIMIT"
export ANTHROPIC_DEFAULT_OPUS_MODEL="$CMA_PROVIDER_MODEL"
export ANTHROPIC_DEFAULT_SONNET_MODEL="$CMA_PROVIDER_MODEL"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="$
    {CMA_PROVIDER_FAST_MODEL:-$CMA_PROVIDER_MODEL}"
export ANTHROPIC_DEFAULT_FABLE_MODEL="$CMA_PROVIDER_MODEL"
if [[ "${CMA_PROVIDER_TRANSPORT:-native}" == "router" ]]; then
    # Resolve OUR router by its stable install identity, NOT by
    # PATH order
    # (live issue 2026-07-22, §11.4.111 resolve-by-stable-name):
    # the npm
    # @musistudio/claude-code-router also installs a `ccr` (npm's
    # bin dir
    # precedes ~/.local/bin on PATH) whose --help carries the same
    # "ccr start" / "ccr serve" fingerprint – it passes the
    # identity gate
    # below – but has NO `restart` subcommand. Bare PATH
    # resolution picked
    # it, every route-apply failed exactly like a stale bundled
    # build, and

```

```

# the self-heal rebuild (which repairs the BUNDLED binary)
# could never
# fix it: a rebuild cannot fix PATH shadowing. claude-ccr-
# build installs
# the bundled router at $HOME/.local/bin/ccr; that symlink IS
# the stable
# identity (same idiom as claude-session above). CMA_CCR_BIN
# overrides
# for tests/power users; PATH is the LAST resort, only when
# the bundled
# install is absent.
local _ccr="${CMA_CCR_BIN:-$HOME/.local/bin/ccr}"
[[ -x "$_ccr" ]] || _ccr="$(command -v ccr 2>/dev/null ||
true)"
if [[ -z "$_ccr" ]]; then
    printf 'claude-providers: provider %s needs claude-code-
router (the `ccr` gateway).\n Build the bundled Go
router: claude-ccr-build\n' "$id" >&2
    return 127
fi
# Identity check: the bundled Go router MUST advertise `ccr
restart` in its
# --help. `ccr start`/`ccr serve` are CARRIER matches – the
# npm
# @musistudio/claude-code-router also prints them – and the
# `restart`
# subcommand is exactly what the npm build lacks and what
# makes route-apply
# work (§11.4.201(7)(a) match STRUCTURE, not a substring a
# carrier also
# carries). A binary at the stable install path that lacks it
# is STALE (built
# before `restart` existed); trigger a rebuild. A binary found
# via PATH
# fallback that lacks it is NOT ours – refuse cleanly (audit
# I3, 2026-07-23).
local _ccr_help; _ccr_help="$($_ccr --help 2>&1 | head -20)"
case "$_ccr_help" in
    *"ccr restart"*) ;;
    *)
        if [[ "$_ccr" == "$HOME/.local/bin/ccr" ]] && command -v
        claude-ccr-build >/dev/null 2>&1; then
            command -v cma_log >/dev/null 2>&1 \
            && cma_log "bundled ccr lacks 'ccr restart' in --help
– rebuilding via claude-ccr-build ..." \
            || printf 'claude-providers: bundled ccr lacks the
required "restart" subcommand – rebuilding ...\\n' >&2
            claude-ccr-build >/dev/null 2>&1 || true
            _ccr_help="$($_ccr --help 2>&1 | head -20)"
            case "$_ccr_help" in
                *"ccr restart"*) ;; # OK after rebuild
                *)

```

```

        printf 'claude-providers: the bundled ccr at %s
still lacks the "restart" subcommand after a rebuild.\n
Rebuild manually and retry: claude-ccr-build\n' "$_ccr"
>&2

        return 127 ;;

    esac
else
    printf 'claude-providers: resolved ccr (%s) is not the
bundled claude-code-router.\n Its --help does not show
the "ccr restart" subcommand that the bundled Go router
has.\n Fix: claude-ccr-build (builds and installs the
bundled router at %s)\n If an EARLIER PATH ccr shadows
the bundled one, remove it:\n    npm rm -g @musistudio/
claude-code-router\n' "$_ccr" "$HOME/.local/bin/ccr" >&2
    return 127
fi
;;

esac
# Upsert THIS provider into ccr config with the live key
# (regenerated each
# launch, chmod 600 – never stored by the toolkit), set it as
# the active
# route, then launch through ccr.
#
# Per-alias port isolation (Defect 3 architecture fix): every
# router-type
# provider alias gets its OWN ccr gateway instance on a unique
# port, so
# concurrent aliases never compete for one config.json and no
# alias
# silently inherits another's route.
# - CMA_CCR_PORT in the provider's .env assigns a pinned
# port.
# - When unset, we allocate a unique port deterministically
# from the
# provider id (base_port + hash), so the same alias always
# gets the
# same port and its own ccr instance survives across
# launches.
# - Per-alias config lives under ~/.claude-code-router/
# <provider-id>/
# so each alias has its own CCR_HOME with its own pidfile,
# config.json,
# and service.log – fully isolated from every other alias.
local _ccr_port="${CMA_CCR_PORT:-}"
# The hash is needed for BOTH ports, so compute it
# unconditionally: two
# disjoint 500-port spaces are derived from it – gateway
# [3460,3959] and
# management [3960,4459]. (Was 100-port spaces [3460,3559]/
# [3560,3659]:
# with ~50 router aliases a birthday collision was not
# theoretical but
# LIVE – xiaomi and opencode2 both hashed to 3519, the second
# child died

```

```

# on bind, a foreign /health answer falsely proved it ready
# (fixed in the
# submodule's waitForReady the same day), and the launch fell
# back to the
# shared default ports. 500 slots per space make a collision
# ~25x rarer;
# a surviving collision now fails LOUDLY (rc=78) instead of
# silently
# degrading.)
local _ccr_hash=0 _ccr_c _ccr_i=0
while (( _ccr_i < ${#CMA_PROVIDER_ID} )); do
    _ccr_c="$(printf '%d' "${CMA_PROVIDER_ID:_ccr_i:1}")";
    _ccr_hash=$(( ( _ccr_hash * 31 + _ccr_c ) & 0x7FFFFFFF ));
    _ccr_i=$(( _ccr_i + 1 ))
done
if [[ -z "$_ccr_port" ]]; then
    # Deterministic port from the provider name: hash the id
    # into a port
    # offset from base 3460, bounded to [3460, 3959] (500
    # ports).
    _ccr_port=$(( 3460 + ( _ccr_hash % 500 ) ))
fi
# Per-alias MANAGEMENT port (live defect 2026-07-25): the
# gateway port was
# isolated per alias but the management port was left at ccr's
# default
# 3458 for EVERY alias, so exactly ONE per-alias gateway could
# be up at a
# time – every second alias's `ccr restart` child died with
# "start management interface: listen on 127.0.0.1:3458: bind:
# address
# already in use" and the launch refused with rc=78 (live-
# captured on
# sarvam: 4 child starts, 4 bind failures, "did not become
# ready within
# 15s"). Worse, the child's gateway socket binds BEFORE the
# management
# bind fails, so waitForReady could WIN the race against the
# dying child –
# a launch that "succeeded" against a gateway that was already
# exiting.
# CMA_CCR_MGMT_PORT in the provider's .env pins it; otherwise
# derive from
# the same hash in the disjoint [3960,4459] space.
local _ccr_mgmt_port="${CMA_CCR_MGMT_PORT:-$(( 3960 +
    ( _ccr_hash % 500 ) )})"
local _ccr_dir="$HOME/.claude-code-router"
# Per-alias config isolation: each router-type alias gets its
# OWN
# CCR_HOME directory so its pidfile (service.json),
# config.json, and
# service.log live under ~/.claude-code-router/<provider-id>/
# instead
# of sharing the global ~/.claude-code-router/. This is load-
# bearing

```

```
# for concurrent aliases: a `ccr restart` for alias A must
# never see
# alias B's pidfile and kill its gateway. CCR_HOME is the Go
# router's
# override for config.Dir() (added for this fix).
local _ccr_home="$_ccr_dir/${CMA_PROVIDER_ID}"
local cfg="$_ccr_home/config.json"
base="$_CMA_PROVIDER_BASE_URL"
# Self-reference guard: when THIS provider's base_url IS the
# ccr gateway
# itself, there is no upstream to route to – upserting a
# provider whose
# api_base_url is ccr registers a ccr->ccr self-loop.
#
# This used to SKIP the upsert+restart and launch anyway,
# justified by
# "under ccr v3.0.6 the live route is app_config (config.json
# is not
# re-imported on restart), so the write is inert-for-routing".
# That claim
# described the RETIRED JS router and is INVERTED for the Go
# router that now
# serves every alias. The Go source documents the opposite –
# submodules/claude-code-router/cmd/ccr/service.go,
# cmdRestart: the toolkit
# "rewrites ~/.claude-code-router/config.json before every
# provider-alias
# launch and then runs `ccr restart` to make that rewrite take
# effect."
# Post-ATM-870 (`a6a24e0`) the mechanism is richer than that
# quote: the
# running gateway applies a validated hot-reload IN PLACE via
# `gw.SwapConfig` (atomic pointer swap; /health, /ready and
# routing follow
# it per-request – only the startup-built outbound proxy
# client and
# in-flight request snapshots do not follow). `ccr restart`
# nonetheless
# stays on this path: it is the DETERMINISTIC apply-point –
# the swap's
# timing is not observable from outside the process, so only
# the bounce
# gives the launch a provable receipt that the route it wrote
# is the
# route being served.
#
# So config.json IS the live route (once restarted), and
# skipping the
# rewrite does not make the launch inert – it makes it INHERIT
# whichever
# provider the gateway last served. That is exactly how
# `helixagent` earned
# a `verified` badge on a turn served by `deepseek`: 157,419
# tokens through
# a nominally 24,576-token alias, with no `helixagent`
# provider present in
```

```

# ccr's config at all. The verdict measured whoever ran last.
#
# There is no honest third state, so REFUSE. A ccr-self base
# cannot be
# routed, and inheriting a foreign route is a silent wrong-
# model launch.
# The provider must be pointed at its REAL backing endpoint
# (for the
# HelixAgent facade that is the HelixLLM server itself, not
# the ccr gateway
# standing in front of it).
# _cma_is_ccr_gateway is emitted into this same alias file
# (see
# _cma_emit_ccr_gateway_guard) – ONE definition shared with
# lib.sh and
# providers-verify.sh, so the launch gate and the verify gate
# cannot drift.
if _cma_is_ccr_gateway "$base"; then
    printf 'claude-providers: refusing to launch %s – its
    base_url (%s) IS the ccr gateway itself.\n' "$id" "$base"
    >&2
    printf ' A provider cannot route through the router it
    *is*. Skipping the route write (the old behaviour)\n
    silently inherited whatever provider the gateway last
    served: the wrong model, with no error.\n' >&2
    printf ' Reprint %s at its real backing endpoint (its pins
    file / CMA_PROVIDER_BASE_URL) and re-run:\n      claude-
    providers sync\n' "$id" >&2
    return 78
fi
# Create the per-alias config dir + config with restrictive
# perms from the
# start: this file will hold the live API key, so it must
# never be
# group/world readable, even transiently or if a later jq
# rewrite fails.
( umask 077; mkdir -p "$_ccr_home"
  [[ -f "$cfg" ]] || echo '{"Providers":[],"Router":{}}' >
  "$cfg" )
chmod 600 "$cfg" 2>/dev/null || true
# Upstream TLS trust anchor (2026-09-04, helixllm-anton live
# launch): the
# verify probe honors CMA_PROVIDER_CA_CERT, but the LAUNCH
# path wired no
# CA at all, so a `verified` alias behind a self-signed https
# gateway died
# on the first real request – "502 upstream request failed:
# tls: failed to
# verify certificate: x509: certificate signed by unknown
# authority". The
# router's outbound client is a Go &http.Transport{} on the
# SYSTEM pool;
# Go honors SSL_CERT_FILE, which REPLACES that pool, so the
# anchor is a
# COMBINED bundle (system roots + the extra CA) – a CA-only
# bundle would

```

```

# narrow trust and break every other TLS dial this gateway
  might make.
# _ccr_ca_env is consumed via env(1) with the ${arr[@]+"$
  {arr[@]}" guard
# so an empty array expands to NOTHING in both bash and zsh.
local _ccr_ca="" _ccr_ca_env=()
if [[ -n "${CMA_PROVIDER_CA_CERT:-}" && -r "$
  {CMA_PROVIDER_CA_CERT:-}" ]] \
  && [[ "$base" == https:///* ]]; then
  local _ccr_sys_ca=""
  for _c in /etc/ssl/certs/ca-certificates.crt \
    /etc/pki/tls/certs/ca-bundle.crt \
    /etc/ssl/cert.pem; do
    [[ -r "$_c" ]] && { _ccr_sys_ca="$_c"; break; }
  done
  if [[ -n "$_ccr_sys_ca" ]]; then
    cat "$_ccr_sys_ca" "$CMA_PROVIDER_CA_CERT" > "$_ccr_home/
    ca-bundle.pem" 2>/dev/null || true
  else
    cat "$CMA_PROVIDER_CA_CERT" > "$_ccr_home/ca-bundle.pem"
    2>/dev/null || true
  fi
# The bundle carries a private upstream CA: same 600
  discipline as the
# config dir it lives in (the shell's umask is not
  guaranteed here).
# The chmod stays BEST-EFFORT by deliberate threat model
  (code-review
# 2026-09-04 finding 8): (a) $_ccr_home itself was created
  under
# umask 077, so a 644 bundle inside a 700 directory is
  already
# group/world-inaccessible; (b) a CA certificate is PUBLIC
  material by
# design – it is distributed to every TLS client, unlike the
  API key in
# config.json; and (c) making chmod a hard precondition
  would convert a
# benign permission hiccup into a dead alias (no
  SSL_CERT_FILE -> the
# x509 failure this whole block exists to fix), a strictly
  worse failure
# mode. Belt-and-braces, never a launch gate.
[[ -s "$_ccr_home/ca-bundle.pem" ]] && _ccr_ca="$_ccr_home/
  ca-bundle.pem"
[[ -n "$_ccr_ca" ]] && { chmod 600 "$_ccr_ca" 2>/dev/null ||
  true; _ccr_ca_env=("SSL_CERT_FILE=$_ccr_ca"); }
fi
case "$base" in
  */chat/completions|*/v1beta/models|*/v1beta/models) ;;
  *) base="${base%}/chat/completions" ;;
esac
# Preserve the original CMA_PROVIDER_BASE_URL for the fossil
  marker.

```



```

# `base` is mutated above (appends /chat/completions) but the
# fossil
# marker must record the REAL backing endpoint, not the
# ephemeral proxy
# address. RC4 (2026-07-27): the fix was to capture
# $CMA_PROVIDER_BASE_URL
# BEFORE the case statement mutates it.
local _real_base="$CMA_PROVIDER_BASE_URL" _eph_base=""
local _eph_marker="$HOME/.claude-code-router/.cma-
ephemeral.json"
local _eph_lock="${_eph_marker}.lock"
local _prior_default="" _prior_background=""
local _wrote_default="" _wrote_background=""
# fd slots for the two launch locks below, filled by bash's
# dynamic fd
# allocation (`exec {var}>>FILE`). Declared here so the nested
# lock/unlock
# helpers assign into THIS scope rather than leaking a global
# into the
# operator's interactive shell.
local _cma_eph_fd="" _cma_cfg_fd=""
# Mutual exclusion for every read-modify-write touching
# $_eph_marker below
# (reap-on-start here, add-on-launch further down, delete-on-
# exit inside
# _cma_ccr_unfossilise). Two CONCURRENT cma_run_provider
# launches both
# touch this one file; without a lock the classic lost update
# is live:
# launch A reads the marker, adds its own key, and mv's the
# result in;
# launch B – already holding a snapshot read from BEFORE A's
# write –
# computes ITS OWN add against that stale snapshot and mv's it
# in too,
# silently overwriting the whole file and dropping A's entry
# (the entry
# A's own exit repair needs to prove, via CAS, that it still
# owns the
# route it wrote). flock is per-open-file-description and
# released by the
# KERNEL the instant the holding process dies (no stale-lock
# recovery is
# needed here the way the mkdir-based _cma_alias_lock_*
# fallback
# elsewhere in this file requires for its non-flock backend);
# an absent
# `flock` binary (no macOS) degrades to the pre-existing
# UNLOCKED
# behaviour rather than blocking a launch – consistent with
# this whole
# section's best-effort discipline (a missing `jq` is handled
# the same
# way, one line below).
_cma_eph_lock() {
    command -v flock >/dev/null 2>&1 || return 0

```

```

# The fd is DYNAMICALLY allocated (`exec {var}>>FILE`, bash
  >= 4.1 /
# zsh): the shell picks a guaranteed-free fd and stores the
  number in
# _cma_eph_fd. A HARDCODED fd number here was the defect,
  twice over:
#   (a) `exec 9>>FILE 2>/dev/null` – `exec` without a
  command applies
#   EVERY redirection on the line to the CURRENT SHELL
  permanently,
#   so one provider launch silenced the interactive
  shell's stderr
#   for the rest of its life (the original field
  failure).
#   (b) `{ exec 9>>FILE; } 2>/dev/null` – the "fix" for (a),
  and itself
#   a silent break: hardcoded LOW fds (9/10) collide
  with the
#   shell's OWN redirection bookkeeping (bash saves/
  restores fds
#   around compound-command redirections and command
  substitutions
#   onto low free fds), so the fd the inner exec opened
  did not
#   survive the group – proven live on bash 5.2.37: the
  very next
#   `flock -w 5 9` failed "Bad file descriptor" (hidden
  by the
#   group's own 2>/dev/null), and EVERY critical section
  below ran
#   UNLOCKED while claiming mutual exclusion.
# Dynamic allocation avoids both: no hardcoded number to
  collide, no
# stderr redirection to leak. It only ever runs where
  `flock` exists
# (the guard above), which is exactly the set of platforms
  where the
# lock can exist at all – macOS degrades to the documented
  unlocked
# best-effort path before reaching this line.
exec {_cma_eph_fd}>>"$_eph_lock" || return 0
# BOUNDED wait. Defensive hardening carried over from an
  earlier
# independent review, NOT a fix for the reported exit-hang
  (that was
# traced to the launch itself, and this lock/unlock pair
  provably
# completes). An unbounded flock can never time out while
  any process
# holds the lock, which on an interactive path is a latent
  stall by
# construction; the guarded critical section is a tiny
  marker
# read-modify-write, so a short bound is ample and a timeout
  degrades

```

```

# to the same unlocked best-effort path this section already
# takes
# when `flock` is absent entirely (macOS).
# On timeout, SAY SO (§11.4.201(5): a guard reports its
# resolved
# evidence). Swallowing it discards the acquired-vs-timed-
# out
# distinction, leaving a real contention event
# undiagnosable.
flock -w 5 "$_cma_eph_fd" 2>/dev/null || {
    command -v cma_log >/dev/null 2>&1 \
        && cma_log "eph-lock: 5s timeout – proceeding unlocked
        (best-effort)"
    true
}
return 0
}

_cma_eph_unlock() {
    [[ -n "$_cma_eph_fd" ]] || return 0
    flock -u "$_cma_eph_fd" 2>/dev/null || true
    exec {_cma_eph_fd}>&- || true
    _cma_eph_fd=""
    return 0
}

# Mutual exclusion for every read-modify-write touching THIS
# alias's
# config.json below – the stale-fossil reap, the main route
# upsert, and
# _cma_ccr_unfossilise. Two CONCURRENT cma_run_provider
# launches of the
# SAME provider id both touch the same per-alias $cfg; without
# a lock the
# classic lost update is live: launch A rewrites config.json
# with provider
# route A, launch B – already holding a snapshot read BEFORE
# A's write –
# rewrites it with provider route B, silently dropping A's
# entire config.
# This is the race fixed by this lock.
_cma_cfg_lock() {
    local _lock="$1.lock"
    command -v flock >/dev/null 2>&1 || return 0
    # Dynamic fd allocation for the same reason as _cma_eph_lock
    # above:
    # a hardcoded `exec 10>>FILE` collides with the shell's
    # redirection
    # save slots (live-proven: "flock: 10: Bad file descriptor",
    # lock
    # silently never held), and any 2>/dev/null ON the exec line
    # itself
    # would silence this interactive shell's stderr permanently.
    exec {_cma_cfg_fd}>>"$_lock" || return 0
    flock -w 5 "$_cma_cfg_fd" 2>/dev/null || {
        command -v cma_log >/dev/null 2>&1 \

```

```

        && cma_log "cfg-lock: 5s timeout on ${_lock} -
        proceeding unlocked (best-effort)"
        true
    }
    return 0
}
_cma_cfg_unlock() {
    [[ -n "$_cma_cfg_fd" ]] || return 0
    flock -u "$_cma_cfg_fd" 2>/dev/null || true
    exec {_cma_cfg_fd}>&- || true
    _cma_cfg_fd=""
    return 0
}
if command -v jq >/dev/null 2>&1; then
    _prior_default="$(jq -r '.Router.default // ""' "$cfg" 2>/
    dev/null || printf '')"
    _prior_background="$(jq -r '.Router.background // ""' "$cfg"
    2>/dev/null || printf '')"
    # (2) stale-fossil reap. Entirely best-effort: any failure
    here must never
    # block a launch, so every step is guarded and the loop
    simply moves on.
    if [[ -s "$eph_marker" ]]; then
        _cma_eph_lock
        local _rv_dead="" _rv_n _rv_pid _rv_tmp
        # `< <(jq ...)` (process substitution) feeding a `while IFS=
        read -r`,
        # NOT `for x in $(jq ...)` – an unquoted command
        substitution word-
        # splits on IFS, so a hypothetical whitespace-bearing
        provider id
        # would silently fan out into multiple bogus keys. NOT a
        trailing
        # `jq ... | while ...` pipe either: bash runs the LAST stage
        of a pipeline
        # in a subshell, so `_rv_dead` accumulated inside such a
        loop would
        # vanish the moment the loop exits and this reap would
        never see any
        # dead entry. Process substitution keeps the loop in THIS
        shell while
        # still reading one whole line at a time.
        while IFS= read -r _rv_n; do
            [[ -n "$_rv_n" ]] || continue
            _rv_pid="$(jq -r --arg k "$_rv_n" '[$k].pid // 0'
            "$eph_marker" 2>/dev/null || printf '0')"
            # Liveness PROVEN, never assumed – a live owner keeps
            its address.
            # Conservative-by-construction on PID reuse: if the
            ORIGINAL owner
            # died and the kernel later hands its pid to some
            UNRELATED later
            # process, `kill -0` reads that pid as alive, so this
            entry is kept

```

```

        # and NOT reaped this cycle. That is the SAFE direction
        - the worst
        # outcome is one stale fossil surviving an extra reap
        cycle, never a
        # live owner's address being ripped out from under it -
        and it
        # self-heals the next time the SAME provider launches:
        CAS-1 in
        # _cma_ccr_unfossilise (`_u_addr_now == _u_eph`) still
        fires on the
        # marker's own record, and a later reap catches it once
        the reused
        # pid itself is gone too.
        if [[ "$_rv_pid" != "0" ]] && kill -0 "$_rv_pid" 2>/dev/
        null; then continue; fi
        _rv_dead="$(jq -c --arg k "$_rv_n" '. + [$k]'
        <<<"$_rv_dead" 2>/dev/null || printf '%s' "$_rv_dead")"
        done < <(jq -r 'keys[]?' "$_eph_marker" 2>/dev/null)
        if [[ "$_rv_dead" != "[]" ]]; then
            _rv_tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX" 2>/dev/
            null)" && chmod 600 "$_rv_tmp" 2>/dev/null
            _cma_cfg_lock "$cfg"
            if [[ -n "$_rv_tmp" ]] && jq --argjson dead "$_rv_dead"
            --slurpfile mk "$_eph_marker" '
                .Providers = [ .Providers[]? as $p
                    | ($mk[0][$p.name] // null) as $m
                    | if ($m != null) and ([ $p.name ] |
inside($dead)) and ($p.api_base_url == ($m.eph // ""))
                        and (($m.real // "") != "")
                        then $p + {api_base_url: $m.real} else $p
                    end ]
                ' "$cfg" >| "$_rv_tmp" 2>/dev/null && [[ -s
            "$_rv_tmp" ]]; then
                command mv -f "$_rv_tmp" "$cfg" 2>/dev/null && chmod
            600 "$cfg" 2>/dev/null
                command -v cma_log >/dev/null 2>&1 && cma_log "reaped
            stale cma-proxy fossil(s) from ccr config: $_rv_dead" ||
            true
            else
                [[ -n "$_rv_tmp" ]] && rm -f "$_rv_tmp"
            fi
            _cma_cfg_unlock
            _rv_tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX" 2>/dev/
            null)" && chmod 600 "$_rv_tmp" 2>/dev/null
            if [[ -n "$_rv_tmp" ]] && jq --argjson dead "$_rv_dead"
            'delpaths([$dead[] | [.]])' \
                "$_eph_marker" >| "$_rv_tmp" 2>/dev/null && [[ -s
            "$_rv_tmp" ]]; then
                command mv -f "$_rv_tmp" "$_eph_marker" 2>/dev/null
            else
                [[ -n "$_rv_tmp" ]] && rm -f "$_rv_tmp"
            fi
        fi
        _cma_eph_unlock
    fi
fi

```

```

fi
# (1) Exit repair. Defined here (indented – a `}` at column 0
# would end the
# function body for _cma_alias_carryover's legacy scanner) so
# it travels with
# this body into the emitted, self-contained alias file. Its
# inputs arrive as
# positional args, with ONE deliberate exception: the resolved
# router binary
# is read from the enclosing `$_ccr` via bash dynamic scoping
# rather than
# being passed in. That is not laziness –
# test_ccr_conformance.sh derives the
# REQUIRED router-subcommand set by scanning this file for
# `$_ccr` <word>` at
# command position, so passing the binary as an argument would
# (a) make the
# call site read as a bogus invocation `ccr "$_eph_marker"`
# and (b) hide this
# function's REAL `$_ccr` restart` from the very gate that
# exists to catch an
# unsupported subcommand. Both call sites are inside
# cma_run_provider, so
# `$_ccr` is always in scope.
_cma_ccr_unfossilise() {
    local _u_cfg="$1" _u_n="$2" _u_eph="$3" _u_real="$4"
    _u_wd="$5" _u_wb="$6"
    local _u_pd="$7" _u_pb="$8" _u_marker="$9"
    [[ -n "$_u_eph" ]] || return 0 # no proxy ran =>
    nothing fossilised
    [[ -f "$_u_cfg" ]] || return 0
    command -v jq >/dev/null 2>&1 || return 0
    _cma_cfg_lock "$_u_cfg"
    local _u_addr_now _u_def_now _u_bg_now _u_mine=0 _u_tmp
    _u_addr_now="$(jq -r --arg n "$_u_n" \
        '[.Providers[]?|select(.name==$n)|.api_base_url][0] // ""'
        "$_u_cfg" 2>/dev/null || printf '')"
    # CAS-1 – repair ONLY while the stored address is still the
    # one WE wrote.
    # If a newer launch re-stamped it, that launch owns the
    # entry and will run
    # its own repair; touching it here would corrupt a live
    # route.
    # Every early return between _cma_cfg_lock and
    # _cma_cfg_unlock MUST pass
    # through the unlock: this runs in the operator's
    # INTERACTIVE shell, which
    # never dies, so the kernel never releases a leaked flock –
    # each leaked
    # acquisition stalls every later cfg-lock on this config for
    # the full 5s
    # budget and silently degrades it to unlocked (audit-proven
    # 2026-07-25:
    # a CAS-1 early return held the lock for the shell's life
    # and stacked
    # +1 leaked fd per occurrence).

```

```

if [[ "$_u_addr_now" != "$_u_eph" ]]; then _cma_cfg_unlock;
return 0; fi
_u_def_now="$(jq -r '.Router.default // ""' "$_u_cfg" 2>/
dev/null || printf '')"
_u_bg_now="$(jq -r '.Router.background // ""' "$_u_cfg" 2>/
dev/null || printf '')"
# CAS-2 – the route is still ours only if BOTH halves are
unchanged.
[[ "$_u_def_now" == "$_u_wd" && "$_u_bg_now" == "$_u_wb" ]]
&& _u_mine=1
_u_tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX" 2>/dev/null)"
|| { _cma_cfg_unlock; return 0; }
chmod 600 "$_u_tmp" 2>/dev/null || true
if jq --arg n "$_u_n" --arg eph "$_u_eph" --arg real
"$_u_real" \
--arg pd "$_u_pd" --arg pb "$_u_pb" --argjson mine
"$_u_mine" '
    .Providers = [ .Providers[]?
    | if .name == $n and .api_base_url == $eph
then .api_base_url = $real else . end ]
    | if ($mine == 1) and ($pd != "")
then .Router.default = $pd else . end
    | if ($mine == 1) and ($pb != "")
then .Router.background = $pb else . end
    ' "$_u_cfg" >| "$_u_tmp" 2>/dev/null && [[ -s
"$_u_tmp" ]]; then
command mv -f "$_u_tmp" "$_u_cfg" 2>/dev/null && chmod 600
"$_u_cfg" 2>/dev/null
command -v cma_log >/dev/null 2>&1 && \
    cma_log "un-fossilised ccr config for $_u_n:
api_base_url ephemeral -> real (route_restored=$_u_mine)"
|| true
# Bounce ONLY when the route was still ours: restarting
while ANOTHER
# launch owns the route would drop that launch's in-flight
requests.
# (Post-ATM-870 the running gateway also picks the edit up
live via
# SwapConfig; the bounce remains the deterministic,
receipt-provable
# apply-point – and here, the repair to an ephemeral
address that the
# dead proxy no longer serves, it is what makes the
restore effective
# for in-flight consumers regardless of swap timing.)
if [[ "$_u_mine" == "1" && -n "$_ccr" && -x "$_ccr" ]];
then
    CCR_HOME="$_ccr_home" "$_ccr" restart --gateway-port
"$_ccr_port" --port "$_ccr_mgmt_port" >/dev/null 2>&1 ||
true
fi
else
    rm -f "$_u_tmp"
fi
_cma_cfg_unlock

```

```

# Drop our marker entry last: until here it is the crash-
# recovery record.
# Locked ($11.4.180/marker-lost-update hardening): _u_marker
# is derived
# from $_eph_marker by the caller and $_eph_lock is visible
# here via the
# SAME bash dynamic-scoping this function already relies on
# for $_ccr
# (see the comment on this function's own definition) -
# _cma_eph_lock /
# _cma_eph_unlock are defined once, before this function's
# first call
# site, alongside $_eph_lock's declaration.
if [[ -n "$_u_marker" && -s "$_u_marker" ]]; then
    _cma_eph_lock
    local _u_mt; _u_mt="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX"
    2>/dev/null)"
    if [[ -n "$_u_mt" ]]; then
        chmod 600 "$_u_mt" 2>/dev/null || true
        if jq --arg k "$_u_n" 'del(.[ $k])' "$_u_marker" >|
        "$_u_mt" 2>/dev/null && [[ -s "$_u_mt" ]]; then
            command mv -f "$_u_mt" "$_u_marker" 2>/dev/null
        else
            rm -f "$_u_mt"
        fi
    fi
    _cma_eph_unlock
fi
return 0
}
# Start the Go compatibility proxy (cma-proxy) when it
# transforms this
# provider: helixagent Hermes tool-call recovery, or poe/kimi/
# sarvam request-
# schema fixes (e.g. Poe requires `parameters` in tool
# definitions; Claude
# Code sometimes omits it). cma-proxy resolves the family key
# itself
# (poe2->poe, kimi-*->kimi), so the wrapper just asks `--has-
# transform`. It
# lives in the INSTALLED share dir; this wrapper is self-
# contained in the
# alias file and has NO $LIB_DIR (repo-only), so resolve
# against SHARED_DIR
# with lib.sh's default. (Replaces the former per-provider
# python proxies.)
local _cma_proxy_dir="${SHARED_DIR:-$HOME/.claude-shared}/
proxy"
local _proxy_bin="$_cma_proxy_dir/cma-proxy"
local _proxy_script=""
if [[ -x "$_proxy_bin" ]] && "$_proxy_bin" --has-transform
"$CMA_PROVIDER_ID" >/dev/null 2>&1; then
    _proxy_script=$_proxy_bin
elif [[ ! -x "$_proxy_bin" ]]; then

```



```

# HONEST SKIP-with-reason, never a silent degrade
# ($11.4.69).
# This branch did not exist: with cma-proxy absent,
# _proxy_script stayed
# empty, the whole shim block was skipped WITHOUT A WORD,
# and the alias
# launched against the raw endpoint. The provider then
# failed in exactly
# the ways the shims exist to prevent (helixagent Hermes
# tool-call
# recovery; poe/kimi/sarvam request-schema fixes) with
# nothing anywhere
# saying the shims were off – from the outside,
# indistinguishable from a
# shim that ran and did not help.
#
# stderr only (never stdout, which is the agent's channel),
# and only for
# providers that actually HAVE a transform: a provider with
# no shim loses
# nothing, so warning there would be a false positive
# ($11.4.201(1)).
# We normally ask the binary via --has-transform, but with
# no binary we
# cannot – cma_proxy_transform_family mirrors the proxy's
# registry
# (prefix-matched: poe2 -> poe, kimi-k2 -> kimi) and
# test_cma_proxy.sh
# asserts the two stay EQUAL in both directions.
if cma_proxy_transform_family "$CMA_PROVIDER_ID"; then
    printf 'claude-providers: cma-proxy is NOT installed (%s)
    – launching %s WITHOUT its compatibility shims.\n Tool-
    call/request-schema fixes for this provider are inactive;
    failures here are expected until it is built.\n Fix:
    claude-proxy-build (then: claude-install-verify)\n' \
        "$_proxy_bin" "$CMA_PROVIDER_ID" >&2
fi
fi
if [[ -n "$_proxy_script" ]]; then
    # HelixAgent pre-flight: check HelixLLM availability and
    # mode.
    # If HelixLLM is not running and Helix Code is installed,
    # auto-start it
    # in claude mode (229376-token context). HelixLLM in coder
    # mode has only
    # 24576-token context; helixagent requires claude mode's
    # 229376 tokens.
    if [[ "$CMA_PROVIDER_ID" == helixagent* ]]; then
        # cma_log/cma_warn are lib.sh helpers. THIS BODY IS
        # EMITTED INTO THE
        # ALIAS FILE by the heredoc below, and that file never
        # sources lib.sh –
        # so calling them bare prints `cma_warn: command not
        # found` and DESTROYS
        # the message. Live-proven: providers-helixagent-
        # superpowers.txt carries

```

```

# exactly that line where the operator should have seen
the coder-mode
# remedy. Every older call site guards with `command -v
cma_log && ...`,
# but that pattern SILENTLY DROPS the text when the helper
is absent,
# which is wrong here: for this pre-flight the message IS
the feature –
# it is the only thing that tells an operator to flip
HelixLLM out of
# coder mode. So emit either way, through the helper when
it exists and
# through a plain printf when it does not.
_hl_log() { if command -v cma_log >/dev/null 2>&1; then
cma_log "$1"; else printf '[cma] %s\n' "$1" >&2; fi; }
_hl_warn() { if command -v cma_warn >/dev/null 2>&1; then
cma_warn "$1"; else printf '[cma warn] %s\n' "$1" >&2;
fi; }
# WHICH SERVICE THIS PRE-FLIGHT IS ABOUT – and why it is
no longer
# derived from the alias's own base_url.
#
# Everything below is a property of the llama.cpp HelixLLM
server: the
# /health liveness poll, the coder-vs-claude decision read
out of
# context_limit / n_ctx, and the auto-start, which runs
# helixllm-mode.sh – a script that binds :18434. While the
helixagent
# alias happened to point AT that server, `
{CMA_PROVIDER_BASE_URL}`
# was a fine way to find it. It no longer does: the alias
now points at
# HelixAgent's own gateway (:7061), which is a DIFFERENT
service with no
# n_ctx and no coder/claude mode. Deriving the URL from
the base_url
# would therefore poll the wrong service for a mode it
cannot report,
# and – worse – auto-start would boot a llama-server on
:18434 and then
# wait 120s for :7061 to come up, a service it never
started.
#
# So resolve it independently, defaulting to the port
helixllm-mode.sh
# actually binds (the same default that script's own
# HELIXLLM_HEALTH_URL uses), and keep it overridable
(CONST-045).
local _hl_url="${CMA_HELIXLLM_HEALTH_URL:-http://
127.0.0.1:18434}" _hl_mode=""
local _hl_healthy=0
if command -v curl >/dev/null 2>&1; then
    # Some HelixLLM builds expose context_limit in /health;
newer ones only

```

```

    # return {"status":"ok"}. Use /health for liveness and
    # fall back to
    # /v1/models for the serving context size (n_ctx) when
    # needed.
    if curl -sf --max-time 3 "${_hl_url}/health" >/dev/null
    2>&1; then
        _hl_healthy=1
        _hl_mode="$(curl -sf --max-time 3 "${_hl_url}/health"
    2>/dev/null | grep -o '"context_limit" *: *[0-9]*' | head
    -1 | sed 's/.*: */' || true)"
    fi
    if [[ -z "$_hl_mode" ]] && curl -sf --max-time 3 "$
    {_hl_url}/v1/models" >/dev/null 2>&1; then
        _hl_healthy=1
        _hl_mode="$(curl -sf --max-time 3 "${_hl_url}/v1/
    models" 2>/dev/null | grep -o '"n_ctx" *: *[0-9]*' | head
    -1 | sed 's/.*: */' || true)"
    fi
fi
# Auto-start HelixLLM if not running and Helix Code is
# installed.
#
# OPT-IN ONLY (operator mandate 2026-07-27). Booting a
# Helix service is
# a heavyweight, machine-wide side effect: HelixLLM claims
# the single
# GPU, and starting it in claude mode EVICTS whatever
# HelixCode had
# running in coder mode. A provider alias must not do that
# to a host
# merely because someone launched it. Default is therefore
# FALSE; set
# CMA_HELIX_AUTOSTART=1 (or true/yes/on) to allow it. When
# it is not
# enabled we still say so, and still print the exact
# command to run –
# the operator keeps the information and the choice.
local _hl_autostart="${CMA_HELIX_AUTOSTART:-0}"
case "${_hl_autostart,,}" in
    1|true|yes|on) _hl_autostart=1 ;;
    *)             _hl_autostart=0 ;;
esac
if [[ "$_hl_healthy" -eq 0 && "$_hl_autostart" -ne 1 ]];
then
    _hl_warn "helixagent: HelixLLM is not answering on $
    {_hl_url} and auto-start is OFF by default.
Start it yourself (it claims the GPU and evicts HelixCode's
coder mode):
    helix_code/scripts/helixllm-mode.sh claude
Or opt in for this launch:
    CMA_HELIX_AUTOSTART=1 <alias>"
fi
if [[ "$_hl_healthy" -eq 0 && "$_hl_autostart" -eq 1 ]];
then
    local _hl_mode_script=""

```

```

# Find helixllm-mode.sh in common locations
for _hl_candidate in \
"$HOME/projects/helix_code/scripts/helixllm-mode.sh" \
"$HOME/helix_code/scripts/helixllm-mode.sh" \
"/run/media/milosvasic/DATA4TB/Projects/helix_code/
scripts/helixllm-mode.sh" \
"${HELIX_CODE_DIR:-}/scripts/helixllm-mode.sh"; do
    if [[ -x "$_hl_candidate" ]]; then
        _hl_mode_script=$_hl_candidate
        break
    fi
done
if [[ -n "$_hl_mode_script" ]]; then
    _hl_log "helixagent: HelixLLM not running – auto-
starting in claude mode via $_hl_mode_script ..."
    if "$_hl_mode_script" claude 2>&1 | tail -5 >&2; then
        # Wait for HelixLLM to become healthy (up to 120s)
        local _hl_wait=0
        while [[ "$_hl_wait" -lt 24 ]]; do
            if curl -sf --max-time 5 "${_hl_url}/health" >/
dev/null 2>&1; then
                _hl_healthy=1
                _hl_log "helixagent: HelixLLM started
successfully"
                break
            fi
            sleep 5
            _hl_wait=$(( _hl_wait + 1 ))
        done
        if [[ "$_hl_healthy" -eq 0 ]]; then
            _hl_warn "helixagent: HelixLLM auto-start timed
out after 120s.

```

The container may still be loading the model. Check:

```

podman logs helixllm-coder
$_hl_mode_script status"

```

```

fi

```

```

else

```

```

    _hl_warn "helixagent: HelixLLM auto-start failed.

```

Fix: manually start HelixLLM:

```

$_hl_mode_script claude"

```

```

fi

```

```

else

```

```

    _hl_warn "helixagent: HelixLLM not running and
helixllm-mode.sh not found.

```

Install Helix Code, or start HelixLLM manually so it answers at
\${_hl_url}."

```

fi

```

```

fi

```

```

# Check for coder mode (even if running)

```

```

if [[ "$_hl_healthy" -eq 1 && "$_hl_mode" -eq 24576 ]] 2>/
dev/null; then

```

```

    _hl_warn "helixagent: HelixLLM detected in CODER MODE
(context_limit=24576).

```

```

helixagent REQUIRES CLAUDE MODE (context_limit=229376).
The first request (~67K system+tools) will immediately exceed
the limit.
Fix on the HelixLLM host:
    helix_code/scripts/helixllm-mode.sh claude
Then re-run the alias."
    fi
fi
# Port-squatter guard (live-proven 2026-07-19: `poe: FAIL
tools-params`).
# Find a genuinely free port, then confirm OUR pid owns it –
never point
# `base` at a squatter (ccr once held 3457 for 21h, silently
disabling the
# proxy so every request bypassed its shims).
local _proxy_port=3457 _pp_try=0
while lsof -i "::$_proxy_port" >/dev/null 2>&1 && (( _pp_try
< 20 )); do
    _proxy_port=$(( _proxy_port + 1 )); _pp_try=$(( _pp_try + 1 ))
done
# Export the upstream to the proxy child: the env file is
sourced WITHOUT
# `set -a`, so CMA_PROVIDER_BASE_URL is not otherwise
inherited, and
# cma-proxy needs it to reach the real backend (a bare --
port launch would
# fall back to its built-in default).
# STDIO REDIRECTION IS LOAD-BEARING, not tidiness. A
backgrounded daemon
# that inherits the caller's stdout/stderr holds the WRITE
END of whatever
# pipe the caller installed. Every caller that captures a
launch with
# command substitution – verify_superpowers_tui.sh, claude-
release-gate.sh
# – reads until EOF, and EOF never arrives while the proxy
is alive: the
# capture hangs for the proxy's whole lifetime (measured:
30005ms vs 5ms,
# both rc=0). It bites hardest on the SUCCESS path, where
`timeout` has
# already reaped its own child and can bound nothing, and it
is reached
# deterministically whenever the proxy is orphaned by the
`_proxy_pid=""`
# branch below (lsof is an undeclared dependency; absent,
that branch is
# always taken). Independently and unconditionally, cma-
proxy prints a
# startup banner to stderr, and callers apply `2>&1` to the
whole
# cma_run_provider call – so the banner lands INSIDE the
captured payload
# and breaks `jq` on captured JSON. Both go away by giving
the daemon its

```

```

# own log file and closing its stdin (mirrors the router's
  own precedent
# in cmd/ccr/spawn_unix.go). $_ccr_home is in scope here and
  already
# mkdir -p'd above.
CMA_PROVIDER_BASE_URL="$CMA_PROVIDER_BASE_URL" \
  "$_proxy_bin" --provider "$CMA_PROVIDER_ID" --port
  "$_proxy_port" \
  >>"$_ccr_home/cma-proxy.log" 2>&1 </dev/null &
_proxy_pid=$!
local _waited=0
# Wait for OUR process to be listening – not merely for the
  port to be busy.
while ! lsof -a -p "$_proxy_pid" -i "::$_proxy_port" >/dev/
  null 2>&1 && (( _waited < 25 )); do
  kill -0 "$_proxy_pid" 2>/dev/null || break    # proxy died:
  stop waiting
  sleep 0.2
  _waited=$(( _waited + 1 ))
done
if ! lsof -a -p "$_proxy_pid" -i "::$_proxy_port" >/dev/null
  2>&1; then
  # Never point at a foreign listener. Fall back to the
  provider's direct
  # endpoint and say so loudly, rather than silently losing
  the shims.
  command -v cma_log >/dev/null 2>&1 && \
    cma_log "WARNING: cma-proxy for $CMA_PROVIDER_ID did not
  start on port $_proxy_port – using the direct endpoint
  (compat shims INACTIVE)" || true
  # REAP BEFORE CLEARING. `$_proxy_pid` is the ONLY handle
  the exit path
  # (the `kill "$_proxy_pid"` reap further down) is guarded
  on, so
  # blanking it here without killing first leaks the child:
  it survives
  # the launch, keeps its port, and – pre-redirection – kept
  the caller's
  # pipe open forever. This branch is not exotic: `lsof` is
  an undeclared
  # dependency of this file, so on any host without it the
  probe ALWAYS
  # fails and this orphan is produced on EVERY proxied
  launch.
  if [[ -n "$_proxy_pid" ]] && kill -0 "$_proxy_pid" 2>/dev/
    null; then
    kill "$_proxy_pid" 2>/dev/null || true
  fi
  _proxy_pid=""
  _proxy_script=""
fi
fi
if [[ -n "$_proxy_script" ]]; then
  base="http://127.0.0.1:${_proxy_port}/v1/chat/completions"

```

```

# THE fossil-producing assignment: `base` is now an
# EPHEMERAL address whose
# lifetime is this launch. Record it (and the real endpoint
# it replaced) so
# both the exit repair and the next launch's stale-reap can
# undo it. The
# marker is a sidecar – never a field inside config.json,
# which the router
# parses.
_eph_base="$base"
# cma_log is a lib.sh helper; the self-contained alias file
# has no such
# function, so guard the call to avoid a 'cma_log: command
# not found' on
# every proxied launch.
command -v cma_log >/dev/null 2>&1 && cma_log "started cma-
proxy for $CMA_PROVIDER_ID on port $_proxy_port
(pid=$_proxy_pid)" || true
fi
# Route write + apply. EVERY failure here is FATAL, because
# the failure mode
# is silent-and-wrong: the launch below serves whatever route
# the gateway
# currently holds. The old code hid both halves – the jq/mv
# write ran under
# `2>/dev/null` with a bare `else rm -f` (a failed write was
# indistinguishable from a successful one) and `ccr restart`
# ran under
# `|| true` (a config written but never applied still
# launched, serving the
# PREVIOUS provider's model). That is what made the helixagent
# facade
# invisible for a whole release. A route we cannot prove we
# set is a route
# we must not launch against.
local _route_err=0 _route_msg=""
if ! command -v jq >/dev/null 2>&1; then
    _route_err=1
    _route_msg="jq is not on PATH, so the ccr route cannot be
    written"
else
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")";
    chmod 600 "$tmp" 2>/dev/null || true
    # Lock the config.json read-modify-write against concurrent
    # launches of
    # the SAME provider id. Two launches that both read the
    # config before
    # either writes it will each produce a rewrite from the same
    # stale
    # snapshot – the second mv silently overwrites the first,
    # and the first
    # launch's provider route is lost. The per-alias config is
    # isolated
    # ($cfg = $_ccr_home/config.json, one per provider-id), so
    # the lock
    # targets only this config's own file.

```

```

_cma_cfg_lock "$cfg"
# Pass the secret through the environment ($ENV.tok), never
  as a jq argv
# argument – argv is visible in ps/proc to other local
  users.
# `>|` (force-clobber), NOT `>`: cma_run_provider runs in
  the user's
# interactive shell, which may have `set -o noclobber`.
  Plain `>` onto the
# just-created mktemp file fails there ("cannot overwrite
  existing file"),
# silently dropping the router-config update so EVERY router
  provider breaks.
local _jq_err _jq_rc _mv_err _mv_rc _rst_out _rst_rc
_jq_err="$(CMA_TOK="$token" jq --arg n "$CMA_PROVIDER_ID" --
  arg u "$base" \
    --arg s "$CMA_PROVIDER_MODEL" --arg f "$
  {CMA_PROVIDER_FAST_MODEL:-$CMA_PROVIDER_MODEL}" '
    .Providers = ([{name:$n, api_base_url:$u, api_key:
  $ENV.CMA_TOK, models:[$s,$f],
    transformer:{use:
  ["cleancache","streamoptions"]}]))
    | .Router.default = ($n + "," + $s)
    | .Router.background = ($n + "," + $f)
  ' "$cfg" 2>&1 >| "$tmp")"
_jq_rc=$?
if (( _jq_rc != 0 )) || [[ ! -s "$tmp" ]]; then
  # An empty output file is a failed rewrite too: mv-ing it
  would leave an
  # unparseable config behind and the gateway would keep the
  old route.
  rm -f "$tmp"
  _route_err=1
  _route_msg="the jq rewrite of $cfg failed (rc=$_jq_rc)$
  {_jq_err:+: $_jq_err}"
  _cma_cfg_unlock
else
  _mv_err="$(command mv -f "$tmp" "$cfg" 2>&1)"; _mv_rc=$?
  if (( _mv_rc != 0 )); then
    rm -f "$tmp"
    _route_err=1
    _route_msg="installing the rewritten $cfg failed (mv
    rc=$_mv_rc){$_mv_err:+: $_mv_err}"
    _cma_cfg_unlock
  else
    chmod 600 "$cfg" 2>/dev/null || true
    _cma_cfg_unlock
    # Remember exactly what WE wrote, so the exit repair can
    compare-and-swap
    # instead of clobbering a newer launch's route.
    _wrote_default="$CMA_PROVIDER_ID,$CMA_PROVIDER_MODEL"
    _wrote_background="$CMA_PROVIDER_ID,$
    {CMA_PROVIDER_FAST_MODEL:-$CMA_PROVIDER_MODEL}"
    # Crash-recovery marker: written the moment an ephemeral
    address becomes

```



```

    # durable, so a launch killed before its exit repair
    (SIGKILL, reboot)
    # still leaves the next launch enough to reap the
    fossil. $$ is the
    # owning shell; the next launch reaps only when `kill
    -0` proves it dead.
    # Locked (marker-lost-update hardening – see _eph_lock's
    declaration
    # above): read-modify-write against $_eph_marker under
    the same
    # cross-launch mutual exclusion as the reap loop and the
    exit repair.
    if [[ -n "$_eph_base" ]] && command -v jq >/dev/null
    2>&1; then
        _cma_eph_lock
        local _mk_tmp
        [[ -f "$_eph_marker" ]] || ( umask 077; printf '{}\n'
    > "$_eph_marker" )
        _mk_tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX" 2>/dev/
    null)" && chmod 600 "$_mk_tmp" 2>/dev/null
        if [[ -n "$_mk_tmp" ]] && jq --arg k
    "$CMA_PROVIDER_ID" --arg e "$_eph_base" \
            --arg r "$_real_base" --argjson p "$$" \
            '.[$k] = {eph:$e, real:$r, pid:$p}'
    "$_eph_marker" >| "$_mk_tmp" 2>/dev/null \
            && [[ -s "$_mk_tmp" ]]; then
            command mv -f "$_mk_tmp" "$_eph_marker" 2>/dev/null
    && chmod 600 "$_eph_marker" 2>/dev/null
        else
            [[ -n "$_mk_tmp" ]] && rm -f "$_mk_tmp"
        fi
        _cma_eph_unlock
    fi
    # `ccr restart` is what makes the write LIVE
    (service.go:cmdRestart).
    # Its failure means the file is right and the gateway
    is still wrong –
    # the most dangerous state of all, and the one `||
    true` used to hide.
    _rst_out="$(env ${_ccr_ca_env[@]}+"${_ccr_ca_env[@]}")
    CCR_HOME="$_ccr_home" "$_ccr" restart --gateway-port
    "$_ccr_port" --port "$_ccr_mgmt_port" 2>&1); _rst_rc=$?
    if (( _rst_rc != 0 )); then
        # SELF-HEAL the commonest cause. A "Profile ... not
        found or is
        # disabled" reply means ccr parsed 'restart' as a
        profile NAME – the
        # installed ccr binary is STALE: it predates the
        'restart' subcommand
        # this launch depends on (a current build lists it in
        `ccr --help`).
        # The binary is a gitignored build artifact, so a
        submodule bump does
        # NOT rebuild it, and because EVERY router-transport
        alias hits this

```

```

        # same step, one stale ccr bricks all of them at once.
Rebuild once +
        # retry (bounded: one rebuild, one retry) so a launch
cannot be
        # silently refused for a condition the toolkit can fix
itself; if the
        # rebuild is unavailable or does not help, fall
through to the
        # self-diagnosing error below.
        case "$_rst_out" in
            *'not found or is disabled'*)
                if command -v claude-ccr-build >/dev/null 2>&1;
then
                    command -v cma_log >/dev/null 2>&1 \
                        && cma_log "bundled ccr is stale (no 'restart'
subcommand) – rebuilding once via claude-ccr-build ..." \
                        || printf 'claude-providers: bundled ccr is
stale – rebuilding it once (this may take ~30s) ...\\n' >&2
                        claude-ccr-build >/dev/null 2>&1 || true
                        _rst_out="$(env ${_ccr_ca_env[@]}+"$
${_ccr_ca_env[@]})" CCR_HOME="$_ccr_home" "$_ccr" restart
--gateway-port "$_ccr_port" --port "$_ccr_mgmt_port"
2>&1)"; _rst_rc=$?
                        fi ;;
                    esac
                fi
                if (( _rst_rc != 0 )); then
                    _route_err=1
                    # Single quotes, not backticks:
test_ccr_conformance.sh scans lib.sh
                    # for `ccr <subcommand>` invocations by splitting on
shell command
                    # separators – a backtick here reads as a command
substitution and
                    # the scanner extracts the bogus subcommand
'restart\''.
                    _route_msg="'ccr restart' failed (rc=$_rst_rc), so the
new route was written but never applied${_rst_out:+:
$_rst_out}"
                    case "$_rst_out" in
                        *'not found or is disabled'*)
                            _route_msg=$_route_msg
ROOT CAUSE: the bundled 'ccr' binary is STALE – it does not
recognize the 'restart' subcommand (it read 'restart' as a
profile), and an automatic rebuild was unavailable or did
not resolve it. Rebuild manually, then retry: claude-ccr-
build (needs the Go toolchain)." ;;
                            esac
                        fi
                    fi
                fi
            fi
        if (( _route_err )); then
            # Defensive scrub: diagnostics are echoed from tool stderr,
which must

```

```

# never be allowed to carry the live key back to the
terminal.
[[ -n "$token" ]] && _route_msg="${_route_msg//"$token"/
<redacted>}"
printf 'claude-providers: refusing to launch %s – its ccr
route was NOT applied.\n %s\n' "$id" "$_route_msg" >&2
printf ' Launching anyway would serve whichever provider
the gateway last routed to (wrong model, no error).\n' >&2
if [[ -n "$_proxy_pid" ]]; then
    kill "$_proxy_pid" 2>/dev/null || true
    command -v cma_log >/dev/null 2>&1 && cma_log "stopped
proxy for $CMA_PROVIDER_ID (pid=$_proxy_pid) after route
failure" || true
fi
# This path can ALSO leave a fossil: the jq+mv may have
succeeded and only
# `ccr restart` failed, in which case the ephemeral address
is already
# durable. Repair here too – the proxy we just killed is now
dead.
_cma_ccr_unfossilise "$cfg" "$CMA_PROVIDER_ID" "$_eph_base"
"$_real_base" \
"$_wrote_default" "$_wrote_background" "$_prior_default"
"$_prior_background" \
"$_eph_marker"
return 78
fi
env ${_ccr_ca_env[@]+"${_ccr_ca_env[@]}"}
CCR_HOME="$_ccr_home" "$_ccr" default-claude-code -- "$@";
rc=$?
# Stop proxy if we started one
if [[ -n "$_proxy_pid" ]]; then
    kill "$_proxy_pid" 2>/dev/null || true
    command -v cma_log >/dev/null 2>&1 && cma_log "stopped proxy
for $CMA_PROVIDER_ID (pid=$_proxy_pid)" || true
fi
# The proxy is dead as of the line above, so the address
written into the
# DURABLE config now names nothing. Undo it before returning –
compare-and-
# swap, so a launch that started after ours keeps its own
route untouched.
_cma_ccr_unfossilise "$cfg" "$CMA_PROVIDER_ID" "$_eph_base"
"$_real_base" \
"$_wrote_default" "$_wrote_background" "$_prior_default"
"$_prior_background" \
"$_eph_marker"
else
    export ANTHROPIC_BASE_URL="$CMA_PROVIDER_BASE_URL"
    export ANTHROPIC_AUTH_TOKEN="$token"
    # Native transport against a self-signed https upstream: the
verify probe
    # trusts CMA_PROVIDER_CA_CERT, and the launch must too. claude
is a Node

```

```

# process, and Node APPENDS NODE_EXTRA_CA_CERTS to its system
# roots – so
# the raw CA path is sufficient (no combined bundle needed,
# unlike the
# router transport's Go client where SSL_CERT_FILE REPLACES
# the pool).
if [[ -n "${CMA_PROVIDER_CA_CERT:-}" && -r "$
    {CMA_PROVIDER_CA_CERT:-}" ]] \
    && [[ "$CMA_PROVIDER_BASE_URL" == https://* ]]; then
    export NODE_EXTRA_CA_CERTS="$CMA_PROVIDER_CA_CERT"
fi
# ANTHROPIC_MODEL + the four ANTHROPIC_DEFAULT*_MODEL tier
# maps are
# exported ABOVE for BOTH transports (v1.26.1 – the 2.1.220
# compact-loop
# fix): a tier-pinned subagent dispatch or fast-tier call
# never leaks a
# literal claude-* id to a native provider endpoint (which
# rejects it –
# xiaomi HTTP 400 "Unsupported model" – or silently
# substitutes – deepseek
# 200), and the router transport's client-side tier resolution
# (fast/
# compact = Fable 5, unavailable in 2.1.220) maps to the
# serving model.
# Session flags are applied ABOVE for both transports
# (v1.17.0) – see
# _cma_session_flags. CLAUDE_CODE_MAX_OUTPUT_TOKENS is
# exported ABOVE too,
# before the transport branch, CLAMPED to <=128000 for BOTH
# transports –
# see the _cma_out_guard/output-token-clamp block. (Was
# formerly
# re-exported here as the RAW, unclamped
# CMA_PROVIDER_MAX_OUTPUT – the
# origin of the "128000" fatal.)
# CLAUDE_CODE_AUTO_COMPACT_WINDOW caps
# INPUT – the two are independent halves of the guard.
"$CLAUDE_BIN" "$@"; rc=$?
fi
# Push post-session state back to all accounts/providers for
# cross-alias visibility.
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" push "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
# Colour a freshly-created session too (its jsonl exists now),
# so the colour is
# in place on the next resume even on the very first launch.
[[ "${_cma_pcolor:-}" == 1 && -x "$HOME/.local/bin/claude-
# session" ]] && \
    "$HOME/.local/bin/claude-session" apply-color
    "$CLAUDE_CONFIG_DIR" "$CMA_PROVIDER_ID" 2>/dev/null ||
    true
return $rc
}

```

CMA_PROV_BODY_EOF

}

The provider session-refresh hook: on every interactive shell start it
re-writes provider aliases from cache (NO network) and, when the status cache
is older than CMA_PROVIDERS_SYNC_TTL (default 24h), kicks a detached full sync.
#
CMA_ALIAS_LOCK_WAIT=0 is LOAD-BEARING: this runs on every shell start, so it
must never wait on another writer. On contention each alias write is skipped
silently and the file is left exactly as the other writer leaves it.

_cma_emit_session_hook() {

printf '%s\n' "\$CMA_ALIAS_HOOK_BEGIN"

cat <<'HOOK'

cma_providers_session_refresh() {

command -v claude-providers >/dev/null 2>&1 || return 0

No-network: re-write alias functions from the cached env files. The

alias-file lock is taken with a ZERO wait so a shell start can never block

behind a concurrent sync; a skipped refresh is harmless.

CMA_ALIAS_SKIP_ON_CONTENTION=1 is the ONLY opt-in to "a contended write

reports success". It belongs here and nowhere else: a shell start must not

block, and a skipped refresh is genuinely harmless because the next writer

re-renders the same delta from the file. Every other caller gets rc 75.

CMA_ALIAS_LOCK_WAIT=0 CMA_ALIAS_SKIP_ON_CONTENTION=1 claude-providers list --quiet --refresh-aliases >/dev/null 2>&1 || true

TTL-triggered background full sync (detached; never blocks the shell).

local ttl="\${CMA_PROVIDERS_SYNC_TTL:-86400}"

local sf="\$HOME/.local/share/claude-multi-account/providers/status.json"

if [-f "\$sf"]; then

local now mtime age

now="\$(date +%s)"

mtime="\$(stat -c %Y "\$sf" 2>/dev/null || stat -f %m "\$sf" 2>/dev/null || echo "\$now")"

age=\$((now - mtime))

if ["\$age" -gt "\$ttl"]; then

(nohup claude-providers sync >/dev/null 2>&1 & disown) 2>/dev/null || true

fi

fi

}

cma_providers_session_refresh

HOOK

```
printf '%s\n' "$CMA_ALIAS_HOOK_END"
}

# Which CLAUDE_BIN the render should record. Keeps the value
# already on disk
# when it still resolves to an executable; otherwise re-resolves
# (an alias file
# carrying a stale path from another host makes EVERY alias launch
# an empty
# command). Only replaces when the replacement is actually
# executable.
_cma_alias_claude_bin() {
    local src="${1:-}" cur="" exp new
    if [[ -f "$src" ]]; then
        cur="$(grep -m1 '^export CLAUDE_BIN=' "$src" 2>/dev/null ||
            true)"
        cur="${cur#export CLAUDE_BIN=}"; cur="${cur#\}"; cur="${cur%
            \}"
        fi
        exp="${cur/#\${HOME}/${HOME}}"; exp="${exp/#\~/\$HOME}"
        if [[ -n "$cur" && -x "$exp" ]]; then printf '%s\n' "$cur";
            return 0; fi
        new="$(cma_resolve_claude_bin)"
        if [[ -n "$cur" && ! -x "${new/#\${HOME}/${HOME}}" ]]; then printf
            '%s\n' "$cur"; return 0; fi
        printf '%s\n' "$new"
    }

_cma_emit_managed() {
    local cb="$1"
    printf '%s\n' "$CMA_ALIAS_MANAGED_BEGIN"
    printf '# Managed by claude-multi-account. Do not edit by hand;
        use\n'
    printf '# ~/.local/bin/claude-add-account to add accounts.\n'
    printf 'export CLAUDE_BIN="%s"\n' "$cb"
    printf '\n'
    _cma_emit_ccr_gateway_guard
    printf '\n'
    _cma_emit_cma_run
    printf '\n'
    _cma_emit_cma_run_provider
    printf '\n'
    _cma_emit_cma_run_kimi
    printf '\n'
    _cma_emit_cma_run_kimi_provider
    printf '\n'
    _cma_emit_account_dispatch
    printf '%s\n' "$CMA_ALIAS_MANAGED_END"
}

# Everything the toolkit does NOT own, in original order: alias
# lines plus any
```

```

# user-added content. Strips the managed block, the hook block,
# and – for files
# written before the sentinels existed – the legacy header/wrapper
# text by
# content, including the orphaned `# Wrapper:` comment copies the
# old
# drop/re-append migration left behind.
_cma_alias_carryover() {
    local src="$1" region=0 hookr=0 mtrunc=0 htrunc=0
    [[ -f "$src" ]] || return 0
    # A block is only treated as a block when BOTH of its sentinels
    # are present.
    # An unterminated BEGIN (a file truncated by something outside
    # this toolkit)
    # would otherwise swallow every alias below it.
    if grep -qxF "$CMA_ALIAS_MANAGED_BEGIN" "$src"; then
        if grep -qxF "$CMA_ALIAS_MANAGED_END" "$src"; then region=1;
        else mtrunc=1; fi
    fi
    if grep -qxF "$CMA_ALIAS_HOOK_BEGIN" "$src"; then
        if grep -qxF "$CMA_ALIAS_HOOK_END" "$src"; then hookr=1; else
        htrunc=1; fi
    fi
    # TRUNCATION RECOVERY, stated as a property of the block rather
    # than of its
    # current contents. A BEGIN with no END means the file was cut
    # INSIDE the
    #
    # block, so what survives is a PREFIX of machine-owned
    # text – an unterminated
    # function, a half-written `case`, whatever the emitters
    # happened to open.
    # Carrying that prefix over as "user content" re-emits it above
    # the alias
    # lines and makes the whole file unparseable: `bash -n` fails,
    # so the rc file
    # sourcing it aborts and EVERY alias in it is dead. That is
    # strictly worse
    # than the truncation itself.
    #
    # The end of the salvage window is derived, not enumerated: no
    # emitted
    # managed/hook line is an `alias` at column 0, and no emitted
    # managed line is
    # the hook's BEGIN sentinel, so the first of those is provably
    # outside the
    #
    # truncated block. Everything before it goes; everything
    # from it on is kept.
    #
    # Deriving it this way is the point. The by-content legacy
    # rules below list
    # the wrapper functions by name, and that list SILENTLY went
    # stale when

```

```

# _cma_emit_ccr_gateway_guard was added to the managed block: a
# file cut
# inside the new guard (the block's first function, so the
# likeliest cut of
# all) carried an unterminated `case` straight into the rebuilt
# file. A rule
# phrased over the block's boundary cannot go stale that way.
awk -v mb="$CMA_ALIAS_MANAGED_BEGIN" -v
    me="$CMA_ALIAS_MANAGED_END" \
    -v hb="$CMA_ALIAS_HOOK_BEGIN" -v he="$CMA_ALIAS_HOOK_END" \
    -v region="$region" -v hookr="$hookr" \
    -v mtrunc="$mtrunc" -v htrunc="$htrunc" '
$0 == mb { if (region) { m = 1 } else if (mtrunc) { t = 1 };
    next }
$0 == me { m = 0; next }
m      { next }
# Salvage window ends at the first line that cannot be block
# content.
t && /^alias[[:space:]]/ { t = 0 }
t && $0 == hb           { t = 0 }
t      { next }
$0 == hb { if (hookr) { h = 1 } else if (htrunc) { t = 1 };
    next }
$0 == he { h = 0; next }
h      { next }
# --- legacy (pre-sentinel) managed content
# -----
# NOTE: the parens are escaped for awk ERE, where \( is a
# LITERAL paren, so
# /^cma_run\( \) ?\{/ cannot match "cma_run_provider() {}". (The
# same spelling
# in grep BRE would be an empty capture group and WOULD match
# it – that bug
# once dropped the function entirely.)
/^cma_run\( \) ?\{/           { f = 1 }
/^cma_run_provider\( \) ?\{/ { f = 1 }
# An `alias` at column 0 always ends the skip: no emitted
# wrapper body
# contains such a line, so this cannot swallow one, and it
# means a wrapper
# left UNTERMINATED by an external truncation can never eat
# the alias lines
# below it. Without this the recovery path drops exactly the
# data the
# recovery exists to save.
f && /^alias[[:space:]]/ { f = 0 }
f { if ($0 ~ /^\\}/) f = 0; next }
/^# Managed by claude-multi-account/ { next }
/^# ~\\/.local\\bin\\/claude-add-account to add/ { next }
/^export CLAUDE_BIN=/ { next }
/^# Wrapper: keeps \.claude\\.json/ { next }
/^# logged-in account\\. Pulls merged state/ { next }
/^# one before claude runs; pushes the post/ { next }
/^# Cheap \\(jq deep-merge of one/ { next }

```



```

    { print }
    ' "$src"
}

# The existing hook block, verbatim (first one only). Requires
# BOTH sentinels:
# an unterminated BEGIN would otherwise copy the rest of the file
# into the hook
# position, duplicating every alias below it.
_cma_alias_extract_hook() {
    local src="$1"
    [[ -f "$src" ]] || return 0
    grep -qxF "$CMA_ALIAS_HOOK_BEGIN" "$src" || return 0
    grep -qxF "$CMA_ALIAS_HOOK_END" "$src" || return 0
    awk -v b="$CMA_ALIAS_HOOK_BEGIN" -v e="$CMA_ALIAS_HOOK_END" '
        $0 == b && !seen { h = 1; seen = 1 }
        h { print }
        $0 == e { h = 0 }
    ' "$src"
}

# Alias names defined in a file, sorted and unique.
_cma_alias_names() {
    [[ -f "$1" ]] || return 0
    awk -F'[ =]+' '/^[[[:space:]]*alias[[[:space:]]+/{print $2}' "$1"
        | LC_ALL=C sort -u
}

# Names of the account aliases (the `CLAUDE_CONFIG_DIR=` form) in
# a file,
# sorted and unique – the input to the gate's account-alias floor.
_cma_alias_account_names() {
    [[ -f "$1" ]] || return 0
    grep '^alias[[[:space:]]][^=]*="CLAUDE_CONFIG_DIR=' "$1" 2>/dev/
        null \
        | awk -F'[ =]+' '{print $2}' | LC_ALL=C sort -u || true
}

# Count of account aliases (the `CLAUDE_CONFIG_DIR=` form) in a
# file.
_cma_alias_account_count() {
    local n=0
    if [[ -f "$1" ]]; then
        # `grep -c` prints 0 AND exits 1 on no-match, so the || must
        # not print too.
        n="$(grep -c '^alias[[[:space:]]][^=]*="CLAUDE_CONFIG_DIR='
            "$1" 2>/dev/null || true)"
        [[ -n "$n" ]] || n=0
    fi
    printf '%s\n' "$n"
}

_cma_alias_mktemp() {

```

```

local dir; dir="$(dirname "$ALIAS_FILE")"
# Same directory as the target on purpose: the commit is a
#   rename, which is
#   only atomic within one filesystem, and a leaked temp is then
#   visible where
# it matters instead of accumulating invisibly in $TMPDIR (7743
#   leaked
# $TMPDIR/cma.* snapshots were found during the incident).
mktemp "$dir/.aliases.render.XXXXXX" 2>/dev/null
}

# _cma_alias_render <src> <drop_name> <add_line> <hook_mode> <out>
#   drop_name  alias to remove (" " = none)
#   add_line   full alias line to add (" " = none); replaces an
#               existing
#               definition of the same name IN PLACE, else is
#               appended
#   hook_mode  keep | install
# Emits the COMPLETE file. This is the only place the file's shape
# is decided.
_cma_alias_render() {
    local src="$1" drop="$2" add="$3" hook="$4" out="$5"
    local carry cb
    carry="$(_cma_alias_mktemp)" || return 1

    # 1. carry over everything we do not own
    _cma_alias_carryover "$src" > "$carry"

    # 2. legacy alias migration: pre-wrapper lines invoking
    #   $CLAUDE_BIN directly
    #   are rewritten to go through cma_run (idempotent – a line
    #   already using
    #   cma_run is left alone).

    # shellcheck disable=SC2016 # $CLAUDE_BIN is literal
    # alias-file text here
    if grep -qE '^alias[[:space:]]+[^\s]=.*CLAUDE_CONFIG_DIR=[^\s]+
    [[:space:]]+\?\$CLAUDE_BIN"$' "$carry" 2>/dev/null; then
        # shellcheck disable=SC2016
        sed -E 's|(^alias[[:space:]]+[^\s]=)"(CLAUDE_CONFIG_DIR=[^\s]+)
        [[:space:]]+\?\$CLAUDE_BIN"$|\1"2 cma_run"|' \
        "$carry" > "$carry.mig" && command mv -f "$carry.mig"
        "$carry"
    fi

    # 3. removal delta. Skipped when the SAME name is being
    #   rewritten: that case

    #   is a replace-in-place below, which is what keeps a
    #   repeated write
    #   byte-identical instead of permuting the file's alias order
    #   (drop+append

    #   would move the rewritten alias to the end on every
    #   call, so a repeat of

```

```

# the same two writes never converged and the no-op guard
  could never fire).
local addname=""
if [[ -n "$add" ]]; then
    addname="$(printf '%s\n' "$add" | awk -F'[ =]+' '{print $2}')"
fi
if [[ -n "$drop" && "$drop" != "$addname" ]]; then
    grep -v -E "^alias[[:space:]]+${drop}=" "$carry" > "$carry.d"
    || true
    command mv -f "$carry.d" "$carry"
fi

# 4. de-duplicate alias definitions (last one wins) and apply
  the add delta:
# an existing definition of the same name is replaced at its
  original
# position; a genuinely new alias is appended. Two passes
  over one file.
# The add text goes through ENVIRON, not -v, because -v
  applies backslash
# escape processing to the value.
CMA_ADD_NAME="$addname" CMA_ADD_LINE="$add" awk -F'[ =]+' '
NR == FNR { if ($0 ~ /^alias[[:space:]]+/) last[$2] = FNR;
next }
{
  if ($0 ~ /^alias[[:space:]]+/) {
    if (FNR != last[$2]) next
    if (ENVIRON["CMA_ADD_NAME"] != "" && $2 ==
ENVIRON["CMA_ADD_NAME"]) {
      print ENVIRON["CMA_ADD_LINE"]; added = 1; next
    }
  }
  print
}
END { if (ENVIRON["CMA_ADD_NAME"] != "" && !added) print
ENVIRON["CMA_ADD_LINE"] }
' "$carry" "$carry" > "$carry.u" && command mv -f "$carry.u"
"$carry"

cb="$(_cma_alias_claude_bin "$src")"

# The write status is PROPAGATED, and the stages are &&-chained
  so a failure
# in any of them (not merely the last) is seen. Discarding it
  meant a

# candidate truncated by ENOSPC – or one whose output file
  could not be opened
# at all – was returned as a successful render. Against a zero-
  alias source
# such a stump also clears the sanity gate, which only requires
  the header and

# the two wrapper opening lines and has no aliases left to
  miss. A render we

```

```

# cannot prove we wrote is a render we must not offer for
  commit.
#
# The open is probed SEPARATELY, with a simple command, because
  the two
# failure modes reach us through different channels and the
  group form only
# reports one of them:
#   * ENOSPC – the open SUCCEEDS, the writes fail; the group's
    own exit
#           status carries it, and the &&-chain below sees
    it.
#   * ENOENT/EACCES on the output path – the redirection fails
    BEFORE the
#           group runs, and bash leaves the compound
    command's status at

#           0 (verified on bash 5.2: `if ! { ;; } > /
nope/x` takes the
#           else branch). A simple command with the same
    failed
#           redirection does return 1, so that is what
    grades the open.

# Without this probe an unwritable output path was
  reported as a successful
# render of a file that does not exist.
if ! : > "$out" 2>/dev/null; then
  rm -f "$carry" "$carry.mig" "$carry.d" "$carry.u" 2>/dev/null
  || true
  cma_warn "alias render: cannot open $out for writing (missing
    directory? permissions?)"
  return 1
fi
if ! { _cma_emit_managed "$cb" \
  && cat "$carry" \
  && if [[ "$hook" == "install" ]]; then
    _cma_emit_session_hook
  else
    _cma_alias_extract_hook "$src"
  fi
} > "$out"; then
  rm -f "$carry" "$carry.mig" "$carry.d" "$carry.u" 2>/dev/null
  || true
  cma_warn "alias render: writing the candidate to $out failed
    (out of space? unwritable path?)"
  return 1
fi

rm -f "$carry" "$carry.mig" "$carry.d" "$carry.u" 2>/dev/null
|| true
return 0
}

```

```

# Refuse to publish a candidate that lost something. Structural
# floor: the
# header and BOTH wrappers must be present. Content floor: the
# candidate's set
# of alias names must be exactly the source's set, minus an
# explicit drop, plus
# an explicit add – the incident's signature was account aliases
# silently
# vanishing from a file nobody asked to change. Account aliases
# are additionally
# floored against cma_detect_accounts, as far as the source file
# already
# satisfied it (a fresh install legitimately has fewer).
_cma_alias_gate() {
    local cand="$1" src="$2" drop="$3" add="$4"
    local want got n_acct src_acct cand_acct rc=0

    grep -q '^export CLAUDE_BIN=' "$cand" || { cma_warn "alias
        render rejected: no CLAUDE_BIN header"; return 1; }
    grep -q '^cma_run() {' "$cand" || { cma_warn "alias
        render rejected: no cma_run()"; return 1; }
    grep -q '^cma_run_provider() {' "$cand" || { cma_warn "alias
        render rejected: no cma_run_provider()"; return 1; }
    # The kimi wrappers are part of the emitted managed block
    # (family layer);
    # a render that lost either is as structurally broken as one
    # missing cma_run,

    # since the kimiN / kimi-<id> aliases would then exec a
    # function that does
    # not exist on shell start.
    grep -q '^cma_run_kimi() {' "$cand" || { cma_warn
        "alias render rejected: no cma_run_kimi()"; return 1; }
    grep -q '^cma_run_kimi_provider() {' "$cand" || { cma_warn
        "alias render rejected: no cma_run_kimi_provider()";
        return 1; }

    want="$(_cma_alias_mktemp)" || return 1
    got="$(_cma_alias_mktemp)" || { rm -f "$want"; return 1; }
    {
        if [[ -n "$drop" ]]; then
            _cma_alias_names "$src" | grep -v -x -F -- "$drop" || true
        else
            _cma_alias_names "$src"
        fi
        if [[ -n "$add" ]]; then printf '%s\n' "$add" | awk -F'[ =]+'
            '{print $2}'; fi
    } | LC_ALL=C sort -u > "$want"
    _cma_alias_names "$cand" > "$got"
    if ! cmp -s "$want" "$got"; then
        cma_warn "alias render rejected: alias set changed
            unexpectedly ($(wc -l < "$want" | tr -d ' ') wanted, $(wc
            -l < "$got" | tr -d ' ') rendered)"
        rc=1
    fi
}

```

```

rm -f "$want" "$got"
(( rc == 0 )) || return 1

    # Account aliases specifically: every `CLAUDE_CONFIG_DIR`
    # alias the source
    # had must still be there, minus one the caller explicitly
    # dropped. The
    # incident's signature was exactly these disappearing from a
    # file nobody
    # asked to change.
    want="$(_cma_alias_mktop)" || return 1
    got="$(_cma_alias_mktop)" || { rm -f "$want"; return 1; }
    if [[ -n "$drop" ]]; then
        _cma_alias_account_names "$src" | grep -v -x -F -- "$drop" >
            "$want" || true
    else
        _cma_alias_account_names "$src" > "$want"
    fi
    _cma_alias_account_names "$cand" > "$got"
    if [[ -n "$(LC_ALL=C comm -23 "$want" "$got")" ]]; then
        cma_warn "alias render rejected: account alias(es) lost: $
            (LC_ALL=C comm -23 "$want" "$got" | tr '\n' ' ')"
        rc=1
    fi
    rm -f "$want" "$got"
    (( rc == 0 )) || return 1

    # Secondary floor: cma_detect_accounts. Only meaningful
    # when the source
    # already satisfied it AND no account is being removed – claude-
    # remove-account
    # drops the alias BEFORE the directory, so the detected count
    # legitimately
    # leads the alias count by one for the duration of that call.
    src_acct="$(_cma_alias_account_count "$src")"
    cand_acct="$(_cma_alias_account_count "$cand")"
    n_acct="$(cma_detect_accounts | wc -l | tr -d ' ')"
    if [[ -z "$drop" ]] && (( src_acct >= n_acct && cand_acct <
        n_acct )); then
        cma_warn "alias render rejected: $cand_acct account aliases <
            $n_acct detected accounts"
        return 1
    fi
    return 0
}

# cma_alias_commit [drop_name] [add_line] [hook_mode]
# THE single committer. Every mutation of $ALIAS_FILE in this
# toolkit goes
# through here; nothing else may write, append to, or rename onto
# the file.
cma_alias_commit() {

```

```

local drop="${1:-}" add="${2:-}" hook="${3:-keep}"
local dir cand rc=0 prev_int prev_term
dir="$(dirname "$ALIAS_FILE")"
mkdir -p "$dir" 2>/dev/null || true

# (F3) No-op guard, taken WITHOUT the lock: render against the
#       current file
# and, if nothing would change, do nothing at all. Steady-state
#       shell starts
# perform zero writes and never contend.
cand="$(_cma_alias_mktemp)" || { cma_warn "cannot create temp
next to $ALIAS_FILE"; return 1; }
if _cma_alias_render "$ALIAS_FILE" "$drop" "$add" "$hook"
"$cand" \
    && [[ -f "$ALIAS_FILE" ]] && cmp -s "$cand" "$ALIAS_FILE";
then
    rm -f "$cand"
    return 0
fi
rm -f "$cand"

if ! _cma_alias_lock_acquire; then
    # CONTENTION. The write did NOT happen: the file is byte-
    #       unchanged and the
    # caller's delta is not in it.
    #
    # Skipping is the right policy for exactly one caller – the
    #       session-refresh
    # hook, which runs on every shell start with
    #       CMA_ALIAS_LOCK_WAIT=0 and whose

    # delta the next writer re-derives from the file anyway.
    # It is the WRONG

    # policy for everybody else, and reporting 0 to everybody
    # else is how it
    # became a data-integrity bug: claude-add-account created the
    #       config dir,
    # linked the shared items, got 0 from cma_write_alias and
    #       announced success
    # for an account with NO alias (whose retry then dies on
    #       "config dir already
    # exists"), and claude-remove-account got 0 from
    #       cma_remove_alias and went on
    # to ARCHIVE the directory out from under a still-live alias.
    #
    # So the skip is now an explicit opt-in
    #       (CMA_ALIAS_SKIP_ON_CONTENTION=1, set
    # by _cma_emit_session_hook and nothing else). Every other
    # caller gets 75
    # (EX_TEMPFAIL – "try again", as distinct from rc 1 "this
    #       render is bad") so
    # it can react instead of being told a lie.
    cma_warn "alias file busy – skipped update of $(basename
"$ALIAS_FILE")"

```

```

[[ "${CMA_ALIAS_SKIP_ON_CONTENTION:-}" == 1 ]] && return 0
return 75
fi

    # (F4) Mask INT/TERM across the critical section. NOT what
    # makes the commit
    # atomic – render-once + a single mv(2) does that, with or
    # without this mask
    # – and NOT what keeps the lock from leaking either: flock is
    # released by the
    # kernel on death, and a leaked mkdir lock is reclaimed by the
    # next acquire's
    # stale-breaker. Deleting these three lines leaves the suite
    # green. They are
    # kept so the caller's own INT/TERM traps are restored rather
    # than left

    # disarmed, and as cheap interrupt hygiene – not on the
    # strength of any test.

    # See the "What protects WHAT" note above. Saved/restored,
    # never clobbered.
prev_int="$(trap -p INT || true)"
prev_term="$(trap -p TERM || true)"
trap '' INT TERM

cand="$(_cma_alias_mktemp)" || cand=""
if [[ -z "$cand" ]]; then
    rc=1
elif ! _cma_alias_render "$ALIAS_FILE" "$drop" "$add" "$hook"
"$cand"; then
    rm -f "$cand"; cma_warn "alias render failed – $ALIAS_FILE
left untouched"; rc=1
elif [[ -f "$ALIAS_FILE" ]] && cmp -s "$cand" "$ALIAS_FILE";
then
    rm -f "$cand" # raced to
identical; nothing to do
elif ! _cma_alias_gate "$cand" "$ALIAS_FILE" "$drop" "$add";
then
    command mv -f "$cand" "$ALIAS_FILE.rejected.$(date +%s)" 2>/
dev/null || rm -f "$cand"
    cma_warn "$ALIAS_FILE left untouched; candidate kept as
$ALIAS_FILE.rejected.*"
    rc=1
else
    command mv -f "$cand" "$ALIAS_FILE" || { rm -f "$cand";
rc=1; }
fi

trap - INT TERM
if [[ -n "$prev_int" ]]; then eval "$prev_int"; fi
if [[ -n "$prev_term" ]]; then eval "$prev_term"; fi
_cma_alias_lock_release
return $rc

```



```

}

# Ensure $ALIAS_FILE exists, is current, and is sourced from the
# user's rc
# files. Idempotent – safe to call repeatedly, and a no-op on disk
# when the
# rendered content already matches.
cma_ensure_alias_file() {
    mkdir -p "$(dirname "$ALIAS_FILE")"
    cma_alias_commit "" "" keep || return 1
    local rc src_line="source \"$ALIAS_FILE\""
    # ${arr[@]+"${arr[@]}" } (not bare "${arr[@]}") so an EMPTY
    # CMA_RC_FILES does not
    # trip "unbound variable" under `set -u` on bash 3.2 (macOS
    # ships 3.2; it errors
    # on empty-array expansion where bash 4.4+ treats it as empty).
    # LOAD-BEARING.
    for rc in ${CMA_RC_FILES[@]+"${CMA_RC_FILES[@]"}"; do
        [[ -f "$rc" ]] || continue
        cma_prune_stale_alias_sources "$rc" # self-heal: drop
        # dangling aliases.sh source lines
        # Add the canonical source line only if no existing line
        # already sources THIS
        # alias file (matched across .|source and $HOME/~/.absolute
        # forms) – prevents
        # duplicate source lines accumulating across re-installs with
        # differing forms.

        # Back up BEFORE the append; refuse the write if the rc
        # cannot be protected.
        if ! cma_rc_sources_alias_file "$rc" "$ALIAS_FILE"; then
            if cma_backup_rc_file "$rc"; then
                printf '\n%s\n%s\n' "$CMA_ALIAS_RC_HEADER" "$src_line" >>
                "$rc"
                cma_log "added source line to $rc"
            else
                cma_warn "skipped adding source line to $rc (could not
                back it up)"
            fi
        fi
    done
}

# Add (or refresh) a single alias entry in $ALIAS_FILE.
# Idempotent.
# The generated alias wraps ` $CLAUDE_BIN ` with sync-pull (before
# launch) and
# sync-push (after exit) so the project/session index
# inside .claude.json
# stays merged across every logged-in account without manual unify
# runs.
cma_write_alias() {
    local alias_name="$1" config_dir="$2"
    cma_validate_alias "$alias_name"

```

```

# config_dir is interpolated into the alias body and re-parsed
# by the shell
# when the alias is invoked. Reject shell metacharacters
# (injection) and
# whitespace (an unquoted space would word-split the alias into
# a bogus
# command – fail loud rather than write a silently-broken
# alias). Matched
# literally per-char to avoid glob-bracket pitfalls.
local _cma_c
for _cma_c in "' '$ ' ` ' \ ' ; ' & ' | ' < ' > ' ( ' ) ' ' ; do
    case "$config_dir" in *"$_cma_c"*)
        cma_warn "refusing to write alias '$alias_name': unsafe
            config dir"
        return 1 ;;
    esac
done
case "$config_dir" in *[:space:]*)
    cma_warn "refusing to write alias '$alias_name': config dir
        must not contain whitespace"
    return 1 ;;
esac
cma_ensure_alias_file
# Note: bash aliases can't take args, so we use a quoted
# CLAUDE_CONFIG_DIR=
# prefix plus a wrapped invocation. The wrapper is a shell
# function reference
# (cma_run) emitted in the managed block by the renderer.
# One delta, one render, one rename – see cma_alias_commit.
cma_alias_commit "$alias_name" \
    "$((printf 'alias %s="CLAUDE_CONFIG_DIR=%s cma_run"'
        "$alias_name" "$config_dir"))" keep
}

# Remove an alias line. Idempotent.
cma_remove_alias() {
    local alias_name="$1"
    [[ -f "$ALIAS_FILE" ]] || return 0
    cma_alias_commit "$alias_name" "" keep
}

# =====
# Provider-alias helpers (used by claude-providers.sh)
# =====

# The shared items every account/provider dir symlinks into
# $SHARED_DIR.
# Kept here (single source) so claude-add-account and claude-
# providers agree.
CMA_SHARED_ITEMS=(
    projects todos tasks plans file-history paste-cache shell-
    snapshots
    session-env telemetry sessions backups cache plugins
    stats-cache.json history.jsonl CLAUDE.md

```

```

daemon jobs
)
# NOTE ($11.4 own-settings): settings.json is DELIBERATELY NOT in
    the shared set.
# Each config dir gets its OWN settings.json so per-alias
    permissions/model/hooks
# never leak across aliases/providers, while the plugin CACHE
    (`plugins`),
# history (`history.jsonl`), memory (`CLAUDE.md`) and sessions
    stay shared. Each
# dir's own settings.json is seeded from + kept enabledPlugins-
    synced with the
# shared template $SHARED_DIR/settings.json by
    cma_own_settings_seed (so
# superpowers et al. stay enabled everywhere). See
    cma_link_shared_items +
# cma_enable_plugins below.
# NOTE (daemon/jobs): `daemon` is Claude Code's background-agent
    registry
# (roster.json + dispatch). It MUST be shared or a background
    agent started
# under one alias is invisible to every other alias – the registry
    is
# config-dir-scoped, not session-scoped. `jobs` is its sibling job
    store.
# daemon/roster.json is union-merged (not last-wins) – see
# merge_daemon_roster in claude-unify.sh.

cma_providers_dir() { echo "$HOME/.local/share/claude-multi-
    account/providers"; }

# --- verification status cache
    -----
# Single source of truth for "is this provider alias usable".
    Holds ONLY
# non-secret metadata: provider id -> {status, model, checked_at,
    failing_layer}.
# status is one of: verified | unverified | failed | pending.
    Consumed by the
# list family (claude-providers list/list-all/list-faulty) and the
    launch-time
# activation gate in cma_run_provider. NO key material ever lands
    here.
cma_status_cache() { echo "$(cma_providers_dir)/status.json"; }

# cma_status_write <id> <status> [<model> [<failing_layer>]]
# Upserts one record. failing_layer is "" for verified/pending.
    Atomic write.
cma_status_write() {
    local id="$1" status="$2" model="${3:-}" layer="${4:-}"
    cma_require jq
    local f; f="$(cma_status_cache)"; mkdir -p "$(dirname "$f")"
    [[ -s "$f" ]] || printf '{}\n' > "$f"
    # checked_at: portable UTC ISO-8601 (GNU + BSD date both accept
        -u +fmt).

```

```

local now; now="$(date -u +%Y-%m-%dT%H:%M:%SZ)"
local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
if jq --arg id "$id" --arg s "$status" --arg m "$model" \
    --arg l "$layer" --arg t "$now" \
    '.[$id] = {status:$s, model:$m, checked_at:$t,
    failing_layer:$l}' \
    "$f" > "$tmp" 2>/dev/null; then
    command mv -f "$tmp" "$f"
else
    rm -f "$tmp"; cma_warn "could not update status cache $f"
fi
}

# cma_status_read <id> -> status word (pending if absent/
# unreadable).
cma_status_read() {
    local id="$1" f; f="$(cma_status_cache)"
    [[ -s "$f" ]] || { echo pending; return 0; }
    local s; s="$(jq -r --arg id "$id" '.[$id].status // "pending"
    "$f" 2>/dev/null)"
    [[ -n "$s" && "$s" != "null" ]] && echo "$s" || echo pending
}

# cma_status_all ->
# id<TAB>status<TAB>model<TAB>checked_at<TAB>failing_layer
# per record.
cma_status_all() {
    local f; f="$(cma_status_cache)"
    [[ -s "$f" ]] || return 0
    jq -r 'to_entries[] | [.key, .value.status, (.value.model //
    ""),
    (.value.checked_at // ""), (.value.failing_layer // "")]
    | @tsv' \
    "$f" 2>/dev/null || true
}

# Union daemon/roster.json files into one registry. workers are
# merged by id
# with the newer updatedAt winning per worker; proto and
# supervisorPid come
# from the newest roster; top-level updatedAt is the max. Used by
# claude-unify.sh (merge_daemon_roster) and
# cma_migrate_daemon_dirs_once.
# Usage: cma_union_rosters OUTFILE roster1.json [roster2.json ...]
cma_union_rosters() {
    local out="$1"; shift
    command -v jq >/dev/null 2>&1 || return 1
    (( $# >= 1 )) || return 1
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    if jq -s '
        def newer($a; $b): if (($a.updatedAt // 0) >= ($b.updatedAt //
        0)) then $a else $b end;
        ([.] | {u: (.updatedAt // 0), p: (.proto // 1), s:
        (.supervisorPid // null), w: (.workers // {})}])
    ' "$@" > "$tmp" 2>/dev/null; then
        command mv -f "$tmp" "$out"
    else
        rm -f "$tmp"; cma_warn "could not union rosters $@"
    fi
}

```

```

| (map(.u) | max // 0) as $maxu
| (sort_by(.u) | last) as $newest
| (reduce .[] as $r ({}; . as $acc
  | reduce ($r.w | to_entries[]) as $e ($acc;
    if ($acc[$e.key] == null) then . + {($e.key):
      $e.value}
    else . + {($e.key): newer($acc[$e.key]; $e.value)}
  end))) as $workers
| {proto: $newest.p, supervisorPid: $newest.s, updatedAt:
  $maxu, workers: $workers}
' "$@" >| "$tmp" 2>/dev/null && [[ -s "$tmp" ]]; then
  command mv -f "$tmp" "$out"
  return 0
fi
rm -f "$tmp"
return 1
}

# One-time migration for pre-v1.17.0 LOCAL daemon/jobs dirs under
  provider
# dirs: their contents (including background-agent rosters) must
  not be
# stranded when daemon/jobs become shared items. Merges every real
  provider
# daemon/jobs dir into $SHARED_DIR (roster.json excluded), backs
  it up,
# replaces it with the shared symlink, then union-merges every
  collected
# roster.json (incl. the shared one) with cma_union_rosters.
Idempotent via a
# marker file; cma_link_shared_items handles all NEW provider
  dirs.
cma_migrate_daemon_dirs_once() {
  command -v rsync >/dev/null 2>&1 || return 0
  local marker="$SHARED_DIR/.daemon-migration-done"
  [[ -e "$marker" ]] && return 0
  local d item tgt roster_tmp=""
  for d in "$HOME/${CMA_PROVIDER_DIR_PREFIX:-.claude-prov-}"/; do
    [[ -d "$d" ]] || continue
    for item in daemon jobs; do
      tgt="$d$item"
      [[ -d "$tgt" && ! -L "$tgt" ]] || continue
      mkdir -p "$SHARED_DIR/$item"
      rsync -a --exclude 'roster.json' "$tgt/" "$SHARED_DIR/
        $item/" 2>/dev/null || true
      if [[ -f "$tgt/roster.json" ]]; then
        # Stash roster CONTENT before the dir moves – collecting
        paths and
        # unioning afterwards would read the just-moved (missing)
        files.
        [[ -z "$roster_tmp" ]] && roster_tmp="$(mktemp -d "$
          {TMPDIR:-/tmp}/cma.XXXXXX)"
        cp "$tgt/roster.json" "$roster_tmp/$(printf '%s' "$d" |
          md5sum | cut -c1-12).json"

```

```

fi
# Same backup convention as unify's backup_and_remove
# (defined there,
# not in lib.sh): rename to <path>.preunify.<timestamp> -
# recoverable.
command mv -f "$tgt" "${tgt}.preunify.$(date +%Y%m%d%H%M%S)"
ln -s "$SHARED_DIR/$item" "$tgt"
done
done
local srcs=()
[[ -f "$SHARED_DIR/daemon/roster.json" && ! -L "$SHARED_DIR/
daemon/roster.json" ]] && \
srcs+=("$SHARED_DIR/daemon/roster.json")
[[ -n "$roster_tmp" ]] && srcs+=("$roster_tmp"/*.json)
if (( ${#srcs[@]} )); then
cma_union_rosters "$SHARED_DIR/daemon/roster.json" "$
{srcs[@]}" || \
cma_warn "daemon roster union failed during migration -
last-wins file kept"
fi
[[ -n "$roster_tmp" ]] && rm -rf "$roster_tmp"
# $SHARED_DIR may not exist yet (fresh host / first sync before
any shared
# item was linked). claude-providers.sh runs under `set -e`, so
an unguarded
# `: > $marker` onto a missing directory ABORTS the whole
cmd_sync (captured
# live: "line 1166: ../.claude-shared/.daemon-migration-done: No
such file or
# directory" -> sync exit 1, zero providers registered). The
marker is only
# a skip-optimization - the migration loop itself is idempotent
(symlinked
# dirs are skipped) - so a failed marker write must never kill
the sync.
mkdir -p "$SHARED_DIR" 2>/dev/null || true
: > "$marker" 2>/dev/null || true
}

# Symlink every shared item into a config dir (account or
provider), creating
# empty placeholders in $SHARED_DIR for any item that doesn't
exist yet.
# Idempotent: skips items already present in the target.
cma_link_shared_items() {
local cdir="$1" item src tgt
mkdir -p "$SHARED_DIR" "$cdir"
for item in "${CMA_SHARED_ITEMS[@]}; do
src="$SHARED_DIR/$item"; tgt="$cdir/$item"
if [[ ! -e "$src" ]]; then
case "$item" in
*.json|*.jsonl|*.md) : > "$src" ;;
*) mkdir -p "$src" ;;
esac

```

```

    fi
    [[ -e "$tgt" || -L "$tgt" ]] || ln -s "$src" "$tgt"
done

    # $11.4 own-settings: settings.json is NOT symlinked –
    # give this dir its OWN copy.
cma_own_settings_seed "$cdire"
}

# List all toolkit-managed config dirs (native accounts +
# providers). Used to
# fan out per-dir OWN settings.json enabledPlugins-sync.
cma_all_config_dirs() {
    local d
    for d in "$HOME"/.claude-claude* "$HOME"/.claude-prov-*; do
        [[ -d "$d" ]] && printf '%s\n' "$d"
    done
}

# Give a config dir its OWN settings.json ($11.4 own-settings). If
# it is a symlink
# (legacy shared layout) or absent, seed a REAL copy from the
# shared template so
# it inherits enabledPlugins + theme. If already a real file,
# additively merge the
# template's enabledPlugins in (own entries win – never clobber
# per-alias
# overrides). The shared template $SHARED_DIR/settings.json
# remains the single
# source of the always-on plugin set.
cma_own_settings_seed() {
    # NOTE: separate `local`s – a single `local a="$1" b="$a/x"`
    # expands $a before
    # it is assigned, which aborts under `set -u` ("a: unbound
    # variable").
    local cdire="${1:-}"
    [ -n "$cdire" ] || return 0
    local own="$cdire/settings.json"
    local tmp="$SHARED_DIR:-$HOME/.claude-shared/settings.json"
    local tmp=""
    command -v jq >/dev/null 2>&1 || return 0
    [[ -s "$tmp" ]] || printf '{}\n' > "$tmp"
    if [[ -L "$own" || ! -e "$own" ]]; then
        rm -f "$own" 2>/dev/null
        cp "$tmp" "$own" 2>/dev/null || printf '{}\n' > "$own"
        return 0
    fi
    tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    if jq -s '.[0] as $own | .[1] as $t
        | $own | .enabledPlugins = (($t.enabledPlugins // {})
        + ($own.enabledPlugins // {}))' \
        "$own" "$tmp" > "$tmp" 2>/dev/null && jq -e . "$tmp" >/
        dev/null 2>&1; then
        command mv -f "$tmp" "$own"
    fi
}

```

```

else rm -f "$tmp"; fi
}

# Force-enable the always-on plugins in the shared settings.json
#   enabledPlugins
# map (additive union – never removes a user's existing entries).
#   Each arg is a
#   plugin key as it appears in enabledPlugins (e.g.
#   "superpowers@anthropics").
cma_enable_plugins() {
    cma_require jq
    local settings="$SHARED_DIR/settings.json" tmp
    mkdir -p "$SHARED_DIR"
    [[ -s "$settings" ]] || printf '{}\n' > "$settings"
    tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    local args=() p i=0

    # Use a dedicated counter for the jq --arg names: each
    # iteration appends THREE
    # elements (--arg, "pN", value), so deriving the index from $
    # {#args[@]} drifts
    # (it produced p0,p1,p3,p4 for 4 plugins → $p2 undefined → jq
    # failed silently

    # → no plugins enabled). The counter matches the $pN refs
    # in the prog below.
    for p in "$@"; do args+=(--arg "p$i" "$p"); i=$((i+1)); done
    # Build a jq program that sets each provided key true if absent.
    local prog='.enabledPlugins //= {}'
    local i=0
    for p in "$@"; do
        prog+=" | .enabledPlugins[\$p$i] //= true"; i=$((i+1))
    done
    # ${args[@]+...} guards an empty array under set -u on bash 3.2
    # (reachable
    # via CMA_ALWAYS_ON_PLUGINS="" from non-re-exec'd claude-
    # providers.sh).
    if jq "${args[@]}+${args[@]}" "$prog" "$settings" > "$tmp" 2>/
    dev/null; then
        command mv -f "$tmp" "$settings"
    else
        rm -f "$tmp"; cma_warn "could not update enabledPlugins in
        $settings"
    fi
    # Fan the always-on plugin set out into every managed config
    # dir's OWN
    # settings.json (settings.json is no longer a shared symlink –
    # $11.4
    # own-settings) so enabling a plugin still reaches every alias/
    # provider.
    local _cd
    while IFS= read -r _cd; do [[ -n "$_cd" ]] &&
        cma_own_settings_seed "$_cd"; done <
        <(cma_all_config_dirs)
}

```



```

# Write the non-secret per-provider env file consumed by
  cma_run_provider.
# Args: id keyvar transport base_url model fast_model config_dir
  [context_limit [max_output]]
# context_limit and max_output are optional trailing args (default
  empty).
# cma_run_provider consumes both catalog limits to avoid the 400
  "exceeded"
# model token limit" error: CMA_PROVIDER_CONTEXT_LIMIT ->
  CLAUDE_CODE_AUTO_COMPACT_WINDOW
# (INPUT: compact before the prompt overshoots the provider's
  window) and
# CMA_PROVIDER_MAX_OUTPUT -> CLAUDE_CODE_MAX_OUTPUT_TOKENS (OUTPUT
  cap).
cma_provider_write_env() {
  local id="$1" keyvar="$2" transport="$3" base="$4" model="$5"
    fast="$6" cdir="$7"
  local context_limit="${8:-}" max_output="${9:-}"
    alias_name="${10:-}"
  # Normalize the literal "null" (from a missing JSON field) to
    empty so it

    # never leaks into the wrapper as a bogus value.
    transport+model were missed
  # originally – a null strong_model/transport wrote
    CMA_PROVIDER_MODEL='null'
  # (provider launches with a bogus model). Normalize every field
    for symmetry.
  [[ "$transport" == "null" ]] && transport=""
  [[ "$base" == "null" ]] && base=""
  [[ "$model" == "null" ]] && model=""
  [[ "$fast" == "null" ]] && fast=""
  [[ "$context_limit" == "null" ]] && context_limit=""
  [[ "$max_output" == "null" ]] && max_output=""
  [[ "$alias_name" == "null" ]] && alias_name=""
  local pdir; pdir="$(cma_providers_dir)"; mkdir -p "$pdir"

    # Preserve the opt-in per-provider TRIM knob
    (CMA_PROVIDER_TRIM) across
  # regeneration. It is NOT in the models.dev catalog nor the
    alias manifest, so
  # neither caller passes it – it is set on the env file itself.
    Because this
  # generator OVERWRITES the whole file (cat > below), an `add`/
    re-add would
  # otherwise DROP it. Read the current on-disk value BEFORE the
    cat truncates
  # the file and re-emit it below (same existing-value-from-env
    read as cmd_sync).
  # `unset CMA_PROVIDER_TRIM` first so only the file's value is
    preserved – never
  # a value that happens to be exported in the caller's shell.
  local trim=""
  if [[ -f "$pdir/$id.env" ]]; then

```

```

# shellcheck disable=SC1090,SC1091
trim="$( ( unset CMA_PROVIDER_TRIM; set -a; . "$pdir/$id.env"
        >/dev/null 2>&1; set +a; printf '%s' "$
        {CMA_PROVIDER_TRIM:-}" ) )"
fi
[[ "$trim" == "null" ]] && trim=""
# Values are single-quoted (with embedded-quote escaping) so
# sourcing the file
# in the user's shell is safe regardless of characters in URLs/
# model ids.
# POSIX single-quote escaping: replace each ' with '\'' via a
# loop.
# Portable across all bash versions – avoids ${var/pattern/
# replacement} with
# complex escape sequences that break on bash 3.2 and 5.3.
_cma_q() {
    local _r="" _rem="$1"
    while true; do
        case "$_rem" in *\'*)
            _r="${_r}${_rem%*\'}'\''"; _rem="${_rem#*\'}" ;;
        *) _r="${_r}${_rem}"; break ;;
    esac
done
printf "'%s'" "$_r"
}
cat > "$pdir/$id.env" <<EOF
# generated by claude-providers – non-secret. Do not edit by hand.
# Secrets are NEVER stored here; the key is read from the keys
# file at launch.
CMA_PROVIDER_ID=$( _cma_q "$id" )
CMA_PROVIDER_KEYVAR=$( _cma_q "$keyvar" )
CMA_PROVIDER_TRANSPORT=$( _cma_q "$transport" )
CMA_PROVIDER_BASE_URL=$( _cma_q "$base" )
CMA_PROVIDER_MODEL=$( _cma_q "$model" )
CMA_PROVIDER_FAST_MODEL=$( _cma_q "$fast" )
CMA_PROVIDER_CONFIG_DIR=$( _cma_q "$cdir" )
# Context-window limits from the models.dev catalog for the
# selected strong model.
# CMA_PROVIDER_CONTEXT_LIMIT: input context window (tokens); empty
# = unknown.
# -> exported as CLAUDE_CODE_AUTO_COMPACT_WINDOW (input-side
# guard).
# CMA_PROVIDER_MAX_OUTPUT: maximum output tokens; empty =
# unknown.
# -> exported as CLAUDE_CODE_MAX_OUTPUT_TOKENS (output-side
# guard).
CMA_PROVIDER_CONTEXT_LIMIT=$( _cma_q "$context_limit" )
CMA_PROVIDER_MAX_OUTPUT=$( _cma_q "$max_output" )
# Alias name for this provider (used by 'list --refresh-aliases'
# to rebuild the
# alias shell line with NO network – the session hook's fast
# path). Empty is OK;
# refresh falls back to the provider id as the alias name.
CMA_PROVIDER_ALIAS=$( _cma_q "$alias_name" )

```

EOF

```
# Re-emit the preserved TRIM knob, ONLY when set – a provider
  without it must
# stay without it (no spurious empty line).
  CMA_PROVIDER_TRIM=bare makes
# cma_run_provider launch minimal (--bare, fresh session, no
  auto-resume).
if [[ -n "$trim" ]]; then
{
  printf '# Opt-in per-provider TRIM knob (preserved across
    regeneration).\n'
  printf '# CMA_PROVIDER_TRIM=bare launches minimal: --bare +
    fresh session (no auto-resume).\n'
  printf 'CMA_PROVIDER_TRIM=%s\n' "$(_cma_q "$trim)"
} >> "$pdir/$id.env"
fi
unset -f _cma_q
}

# Write (or refresh) a provider alias: alias
  <name>="cma_run_provider <id>".
cma_provider_write_alias() {
  local alias_name="$1" id="$2"
  cma_validate_alias "$alias_name"
  # The provider id is interpolated into the alias body and re-
    parsed when the
  # alias is invoked. Provider ids are always [A-Za-z0-9._-];
    reject anything
  # else so a hostile catalog/--id value can't inject shell
    commands.
  case "$id" in
    ''|*[^A-Za-z0-9._-]*)
      cma_warn "refusing to write alias '$alias_name': unsafe
        provider id"
      return 1 ;;
  esac

  # One delta, one render, one rename. This used to
    bootstrap-only (never

  # calling cma_ensure_alias_file on an existing file)
    because the old
  # drop/re-append migrations could reposition cma_run_provider
    relative to the
  # alias lines, making '--refresh-aliases' non-idempotent (see
    test_providers.sh

  # "--refresh-aliases is idempotent"). The renderer emits a
    fixed canonical
  # order, so position is now invariant and that hazard is gone –
    which is why

  # this path can safely render the managed block like every
    other writer.
  cma_alias_commit "$alias_name" \
```

```

    "$(printf 'alias %s="cma_run_provider %s"' "$alias_name"
        "$id")" keep
}

# Install (idempotently) the provider session-refresh hook into
# $ALIAS_FILE. On
# every interactive shell start the hook re-writes provider
# aliases from cache
# (NO network) and, when the status cache is older than
# CMA_PROVIDERS_SYNC_TTL
# (default 24h), kicks a detached full sync ($11.4.89 background –
# never blocks
# the shell). Bracketed by markers so a re-install replaces the
# block atomically
# (no duplication across re-installs).
cma_install_session_hook() {
    cma_ensure_alias_file
    # The hook text itself lives in _cma_emit_session_hook (next to
    # the other
    # emitters); "install" tells the renderer to emit the current
    # version instead
    # of carrying the existing block over.
    cma_alias_commit "" "" install
}

# =====
# Kimi family layer (v1.27.0). The Kimi Code CLI is a first-class
# sibling of
# Claude Code. The family model is an orthogonal axis to the
# provider backend:
# the provider engine stays shared, and each family supplies its
# agent binary,
# home env var, account prefix, user-scope root, and launcher
# functions.
#
# |               | Claude family      | Kimi family
# | agent binary  | claude             | kimi
# | home env var  | CLAUDE_CONFIG_DIR  | KIMI_CODE_HOME
# | account prefix | .claude-           | .kimi-code-
# | user-scope root | ~/.claude (excluded) | ~/.kimi-code
# |               | (excluded)
# | account aliases | claude1...N        | kimil...N
# | account launcher | cma_run             | cma_run_kimi
# | provider launcher | cma_run_provider    | cma_run_kimi_provider
# | shared store   | $SHARED_DIR/**      | $SHARED_DIR/kimi/**
#
# Namespace contract (the invariant): claudeN/kimiN are native
# accounts; <id>
# (no prefix) is ALWAYS Claude Code over that backend; kimi-<id>
# is ALWAYS Kimi
# Code over the SAME backend; kc-<id> is Claude Code over a Kimi-
# native backend
# (legacy rename). kimiN account aliases must never be named kimi-
# */kc-* (those

```

```

# are reserved provider namespaces), and kc-* ids never get a
# kimi-kc-* twin.
# =====
KIMI_ACCOUNT_PREFIX=".kimi-code-"
KIMI_PROVIDER_PREFIX=".kimi-prov-"
KIMI_SHARED_SUBDIR="kimi"

cma_kimi_home() { printf '%s\n' "$HOME/.kimi-code"; }
cma_kimi_account_home() { printf '%s\n' "$HOME/
    $KIMI_ACCOUNT_PREFIX$1"; }
cma_kimi_provider_home() { printf '%s\n' "$HOME/
    $KIMI_PROVIDER_PREFIX$1"; }

CMA_KIMI_SHARED_ITEMS=(AGENTS.md plugins skills sessions
    session_index.jsonl)

# Resolve the Kimi Code binary for the kimi wrappers. Prefer an
# explicit
# KIMI_BIN, then the per-home bundled bin (installed by
# install.sh / logic), then
# ~/.local/bin, then PATH. Mirror of cma_resolve_claude_bin.
cma_resolve_kimi_bin() {
    if [ -n "${KIMI_BIN:-}" ]; then printf '%s\n' "$KIMI_BIN";
        return 0; fi
    if [ -x "$HOME/.kimi-code/bin/kimi" ]; then printf '%s\n'
        "$HOME/.kimi-code/bin/kimi"; return 0; fi
    if [ -x "$HOME/.local/bin/kimi" ]; then printf '%s\n'
        "$HOME/.local/bin/kimi"; return 0; fi
    local c; if c="$(command -v kimi 2>/dev/null)"; then printf
        '%s\n' "$c"; return 0; fi
    printf '%s\n' "$HOME/.local/bin/kimi" # fallback (created by
        install/symlink)
}

# Marker-based detection mirror of cma_detect_accounts for the
# Kimi family.
# Matches ${KIMI_ACCOUNT_PREFIX}* (~/.kimi-code-*). The user-scope
# root
# ~/.kimi-code (no trailing hyphen) and the provider homes
# ~/.kimi-prov-*
# never match the glob structurally. Skips *-shared, *.lock,
# *.removed.*,
# *.preunify.*, and non-empty dirs lacking any kimi marker.
cma_detect_kimi_accounts() {
    local d prefix="${KIMI_ACCOUNT_PREFIX:-.kimi-code-}"
    while IFS= read -r d; do
        [[ "$d" == *"-shared" ]] && continue
        [[ "$(basename "$d")" == *.lock ]] && continue
        [[ "$(basename "$d")" == *.removed.* ]] && continue
        [[ "$(basename "$d")" == *.preunify.* ]] && continue

        # Empty dirs always count (a brand-new account before any
        # kimi run).
        if [[ -z "$(ls -A "$d" 2>/dev/null)" ]]; then echo "$d";
            continue; fi
    done
}

```

```

# Non-empty: must look like a Kimi account (at least one tell-
tale
# file/dir). Filters out dirs that merely match the `.kimi-
code-*` prefix
# but belong to other tools.
if [[ -f "$d/config.toml" || -d "$d/credentials" || -d "$d/
sessions" \
|| -f "$d/session_index.jsonl" ]]; then
    echo "$d"
fi
done < <(find "$HOME" -maxdepth 1 -type d -name "${prefix}*" 2>/
dev/null | sort)
}

# Read all aliases of the form `alias name="KIMI_CODE_HOME=...
cma_run_kimi"`
# from $ALIAS_FILE and print their names (one per line).
cma_existing_kimi_aliases() {
    [[ -f "$ALIAS_FILE" ]] || return 0
    grep '^alias[[:space:]][^=]*="KIMI_CODE_HOME=' "$ALIAS_FILE" 2>/
dev/null \
    | awk -F'[ =]+' '{print $2}' 2>/dev/null || true
}

# Suggest the next free `kimi<N>` alias by scanning current kimi
aliases.
cma_suggest_kimi_alias() {
    local highest=0 n a
    for a in $(cma_existing_kimi_aliases); do
        if [[ "$a" =~ ^kimi([0-9]+)$ ]]; then
            n="${BASH_REMATCH[1]}"
            (( n > highest )) && highest="$n"
        fi
    done
    printf 'kimi%s\n' "$((highest + 1))"
}

# Validate a kimi ACCOUNT alias name: same charset as
cma_validate_alias, and
# additionally the two provider namespaces `kimi-*` and `kc-*` are
reserved for
# provider aliases – a kimiN account must never be named into
either.
cma_validate_kimi_alias() {
    cma_validate_alias "$1"
    case "$1" in
        kimi-*|kc-*)
            cma_die "invalid alias name:
            $1 (reserved provider namespace kimi-*/kc-*)" ;;
    esac
}

# Symlink every kimi shared item into a kimi account dir, creating
empty

```

```

# placeholders under $SHARED_DIR/kimi for items that don't exist
yet.
# Idempotent: skips items already present in the target. Mirror of
# cma_link_shared_items but into the family-specific $SHARED_DIR/
kimi area.
cma_link_kimi_shared_items() {
    local kdir="$1" item src tgt
    mkdir -p "$SHARED_DIR/$KIMI_SHARED_SUBDIR" "$kdir"
    for item in "${CMA_KIMI_SHARED_ITEMS[@]}; do
        src="$SHARED_DIR/$KIMI_SHARED_SUBDIR/$item"; tgt="$kdir/$item"
        if [[ ! -e "$src" ]]; then
            case "$item" in
                *.json|*.jsonl|*.md) : > "$src" ;;
                *) mkdir -p "$src" ;;
            esac
        fi
        [[ -e "$tgt" || -L "$tgt" ]] || ln -s "$src" "$tgt"
    done
}

# Add (or refresh) a single kimi ACCOUNT alias in $ALIAS_FILE.
# Idempotent.
# The alias wraps `cma_run_kimi` with a KIMI_CODE_HOME= prefix so
the account
# resolves its own home, exactly as claudeN aliases carry
CLAUDE_CONFIG_DIR=.
cma_write_kimi_alias() {
    local alias_name="$1" kdir="$2" _cma_c
    cma_validate_kimi_alias "$alias_name"
    # kdir is interpolated into the alias body and re-parsed by the
    shell when the

    # alias is invoked. Reject shell metacharacters and
    whitespace – same rules as
    # cma_write_alias.
    for _cma_c in "' '$' '`' '\\' ';' '&' '|' '<' '>' '(' ')'; do
        case "$kdir" in *"$_cma_c"*)
            cma_warn "refusing to write alias '$alias_name': unsafe
            config dir"
            return 1 ;;
        esac
    done
    case "$kdir" in *[:space:]*)
        cma_warn "refusing to write alias '$alias_name': config dir
        must not contain whitespace"
        return 1 ;;
    esac
    cma_ensure_alias_file
    cma_alias_commit "$alias_name" \
        "$(printf 'alias %s="KIMI_CODE_HOME=%s cma_run_kimi"'
        "$alias_name" "$kdir")" keep
}

# Remove a kimi account alias line. Idempotent.

```

```

cma_remove_kimi_alias() {
    local alias_name="$1"
    [[ -f "$ALIAS_FILE" ]] || return 0
    cma_alias_commit "$alias_name" "" keep
}

# ---- Emitted kimi wrappers
# -----
# These are emitted into the managed block and eval'd into THIS
# shell from the
# exact bytes the alias file receives (same pattern as the ccr
# guard), so the
# wrapper under test is always the same function a user's shell
# gets.
_cma_emit_cma_run_kimi() {
    cat <<'CMA_KIMI_RUN_EOF'
# Kimi Code account wrapper (kimiN). Resolves the kimi binary,
# scrubs the
# Claude/Anthropic env the Kimi CLI must never inherit, then runs
# kimi.
cma_run_kimi() {
    local _ck_bin _ck_khome
    _ck_bin="${KIMI_BIN:-}"
    if ! command -v "$_ck_bin" >/dev/null 2>&1; then
        if [ -x "$HOME/.kimi-code/bin/kimi" ]; then
            _ck_bin="$HOME/.kimi-code/bin/kimi"
        elif [ -x "$HOME/.local/bin/kimi" ]; then
            _ck_bin="$HOME/.local/bin/kimi"
        elif command -v kimi >/dev/null 2>&1; then _ck_bin="$(command
            -v kimi)"
        fi
    fi
    if ! command -v "$_ck_bin" >/dev/null 2>&1; then
        printf '%s\n' "cma_run_kimi: kimi binary not found (checked \
            $KIMI_BIN, ~/.kimi-code/bin, ~/.local/bin, PATH)" >&2
        return 127
    fi
# Family isolation: never let a Claude/Anthropic env left over
# from a previous
# alias leak into the Kimi CLI – base URL, auth token, model
# pins, and the
# Claude Code token-limit guards are all Claude-family only.
unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL
    ANTHROPIC_SMALL_FAST_MODEL
unset ANTHROPIC_DEFAULT_OPUS_MODEL
    ANTHROPIC_DEFAULT_SONNET_MODEL
    ANTHROPIC_DEFAULT_HAIKU_MODEL
    ANTHROPIC_DEFAULT_FABLE_MODEL
unset CLAUDE_CODE_MAX_OUTPUT_TOKENS
    CLAUDE_CODE_AUTO_COMPACT_WINDOW
    CLAUDE_CODE_MAX_CONTEXT_TOKENS
    "$_ck_bin" "$@"
}
CMA_KIMI_RUN_EOF
}

```



```

eval "$(_cma_emit_cma_run_kimi)"

_cma_emit_cma_run_kimi_provider() {
    cat <<'CMA_KIMI_PROV_EOF'
# Kimi Code over a verified provider backend (kimi-<id>). Shares
  the SAME
# status.json activation gate as the Claude twin (single shared
  record per id),
# reads default_model/base_url from the rendered config.toml, and
  refuses to
# launch on a missing or stale config. Self-contained: no lib.sh
  helpers.
cma_run_kimi_provider() {
    local _ckf=0 _ckpid _ckhome _ckconf _ckbin _ckdm _ckbase
    if [[ "${1:-}" == "--force" ]]; then _ckf=1; shift; fi
    _ckpid="${1:-}"
    shift 2>/dev/null || true
    if [[ "${1:-}" == "--force" ]]; then _ckf=1; shift; fi
    if [[ -z "$_ckpid" ]]; then
        printf '%s\n' "usage: cma_run_kimi_provider <id> [--force]
          [kimi args...]" >&2
        return 2
    fi
    _ckhome="${KIMI_CODE_HOME:-$HOME/.kimi-prov-$_ckpid}"
    _ckconf="$_ckhome/config.toml"
    _ckbin="${KIMI_BIN:-}"
    if ! command -v "$_ckbin" >/dev/null 2>&1; then
        if [ -x "$HOME/.kimi-code/bin/kimi" ]; then
            _ckbin="$HOME/.kimi-code/bin/kimi"
        elif [ -x "$HOME/.local/bin/kimi" ]; then
            _ckbin="$HOME/.local/bin/kimi"
        elif command -v kimi >/dev/null 2>&1; then _ckbin="$(command
          -v kimi)"
        fi
    fi
    if ! command -v "$_ckbin" >/dev/null 2>&1; then
        printf '%s\n' "cma_run_kimi_provider: kimi binary not found"
        >&2
        return 127
    fi
    # Shared activation gate (same record as the claude twin) – only
      a 'verified'
    # id launches, unless the operator passes --force.
    if (( ! _ckf )); then
        local _ck_sf="$HOME/.local/share/claude-multi-account/
          providers/status.json" _ck_st="pending"
        if command -v jq >/dev/null 2>&1 && [[ -s "$_ck_sf" ]]; then
            _ck_st="$(jq -r --arg i "$_ckpid" '.[${i}].status //
              "pending"' "$_ck_sf" 2>/dev/null)"
            [[ -n "$_ck_st" && "$_ck_st" != "null" ]] ||
              _ck_st="pending"
        fi
        if [[ "$_ck_st" != "verified" ]]; then

```

```

    printf 'claude-providers: alias kimi-%s is %s – not
    launching.\n' "$_ckpid" "$_ck_st" >&2
    printf '  Re-verify: claude-providers verify %s    (and
    claude-providers sync)\n' "$_ckpid" >&2
    printf '  Override (operator): run the alias with --force\n'
    >&2
    return 3
fi
fi
if [[ ! -f "$_ckconf" ]]; then
    printf 'claude-providers: kimi-%s has no config.toml at %s\n'
    "$_ckpid" "$_ckconf" >&2
    printf '  Run: claude-providers sync    (renders the kimi
    config for verified providers)\n' >&2
    return 1
fi
# Pick up per-id launch metadata from the provider env record
# (tls CA cert, ...
# ). config.toml carries the wire/API material; the env record
# carries host
# config. Optional – a record is not required for kimi launch.
local _ck_envf="$HOME/.local/share/claude-multi-account/
providers/$_ckpid.env"
if [[ -f "$_ck_envf" ]]; then source "$_ck_envf"; fi
# default_model / base_url are read at launch from what sync
# rendered – never
# hardcoded here. Portable strip: take the value after the first
# `=`, drop
# surrounding quotes/whitespace with tr (works for the double-
# quoted TOML the
# kimi renderer emits).
_ckdm="$("
grep -E '^[[:space:]]*default_model[[:space:]]*=' "$_ckconf"
2>/dev/null \
| head -n1 | cut -d= -f2- | tr -d ' "' \
)"
if [[ -z "$_ckdm" ]]; then
    printf 'claude-providers: kimi-%s config.toml has no
    default_model – stale config\n' "$_ckpid" >&2
    printf '  Run: claude-providers sync    to re-render\n' >&2
    return 1
fi
_ckbase="$("
grep -E '^[[:space:]]*base_url[[:space:]]*=' "$_ckconf" 2>/
dev/null \
| head -n1 | cut -d= -f2- | tr -d ' "' \
)"
# TLS trust for self-signed backends, mirrored from the claude-
# side wiring:
# Node APPENDS NODE_EXTRA_CA_CERTS and Go replaces the pool (a
# CA-only
# SSL_CERT_FILE would narrow trust for every other dial). Gated
# on the cert
# being set+readable and the base URL being https://.

```

```

if [[ -n "${CMA_PROVIDER_CA_CERT:-}" ]] && -r "${CMA_PROVIDER_CA_CERT:-}" && "$_ckbase" == https://* ]];
then
    export NODE_EXTRA_CA_CERTS="${CMA_PROVIDER_CA_CERT}"
    export SSL_CERT_FILE="${CMA_PROVIDER_CA_CERT}"
fi
# Same family isolation as cma_run_kimi: scrub a leftover
  Claude/Anthropic env.
unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL
  ANTHROPIC_SMALL_FAST_MODEL
unset ANTHROPIC_DEFAULT_OPUS_MODEL
  ANTHROPIC_DEFAULT_SONNET_MODEL
  ANTHROPIC_DEFAULT_HAIKU_MODEL
  ANTHROPIC_DEFAULT_FABLE_MODEL
unset CLAUDE_CODE_MAX_OUTPUT_TOKENS
  CLAUDE_CODE_AUTO_COMPACT_WINDOW
  CLAUDE_CODE_MAX_CONTEXT_TOKENS
KIMI_CODE_HOME=$_ckhome "$_ckbin" -m "$_ckdm" "$@"
}
CMA_KIMI_PROV_EOF
}
eval "$(_cma_emit_cma_run_kimi_provider)"

# True only when the toolkit may prompt the user interactively.
Scripts read
# confirmations from /dev/tty (so prompts survive `curl | bash`),
so this
# probes /dev/tty rather than stdin. It returns false when:
#   * CMA_NONINTERACTIVE=1 is exported – a global "never prompt"
switch for
#   automation, CI, and the test suite (deterministic regardless
of TTY); or
#   * no terminal is available – CI, a test sandbox, an SSH
command with no PTY.
# When it returns false, callers MUST fall back to their non-
interactive
# default instead of blocking or erroring on a failed read. This
is what makes
# toolkit execution always non-interactive off a terminal.
cma_can_prompt() {
  [[ "${CMA_NONINTERACTIVE:-}" == 1 ]] && return 1
  ( exec </dev/tty ) 2>/dev/null
}

```

8.2 claude-unify.sh

```

#!/usr/bin/env bash
# claude-unify.sh – Unify N Claude Code account config dirs into a
  single
# shared store so every account sees the same projects/
  conversations,
# memory, history, todos, plans, plugins, and settings.
#
# Inputs:

```

```

# * Positional args: one or more account config directories. If
#   none
#   are given, all `~/.claude-*` directories are auto-detected
#   (excluding
#   the shared store itself).
# * Env vars:
#   SHARED_DIR    target shared store (default: ~/.claude-
#                 shared)
#   DEFAULT_DIR   user-scope plugin root (default: ~/.claude)
#
# Use --rollback to restore the .preunify.* backups and remove the
# shared
# store. Safe to re-run any time.

set -euo pipefail

# macOS ships bash 3.2 which lacks `mapfile`. Re-exec under
# Homebrew bash
# if available; otherwise tell the user how to install it.
if (( BASH_VERSINFO[0] < 4 )); then
  for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
    [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
  done
  echo "claude-unify requires bash 4+. Install via: brew install
    bash" >&2
  exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
    (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

DEFAULT_DIR="${DEFAULT_DIR:-$HOME/.claude}"

cma_require rsync
cma_require jq
cma_require awk

# Items each account dir contributes to the shared store. Order
# matters
# only insofar as we want plugins last so the path-rewrite step
# has all
# manifest entries to rewrite at once.
SHARED_ITEMS=(

```

```

projects
todos
tasks
plans
file-history
paste-cache
shell-snapshots
session-env
telemetry
sessions
backups
cache
stats-cache.json
history.jsonl
plugins
daemon
jobs
)
# NOTE ($11.4 own-settings): settings.json is DELIBERATELY NOT in
    SHARED_ITEMS,
# so it is never symlinked into an account dir. Unify does NOT
    blanket-merge full
# account settings into the shared store (that re-leaks per-alias
    permissions/
# model/hooks to shared and thence to every dir). Instead it ONLY
    unions the
# enabledPlugins map into the shared TEMPLATE
    (union_enabled_plugins_into_template)
# and then re-seeds each dir's OWN settings.json from it
    (seed_own_settings) so
# newly-enabled plugins propagate while every other per-alias key
    stays local.
# This list MUST stay == CMA_SHARED_ITEMS (lib.sh) minus the
    CLAUDE.md special-
# case (enforced by the drift-guard test in test_unify.sh).

PRIVATE_ITEMS=(
    .credentials.json
    .claude.json
    mcp-needs-auth-cache.json
)

ts() { date +%Y%m%d%H%M%S; }

already_linked_to_shared() {
    [[ -L "$1" ]] || return 1
    # cma_realpath (not `readlink -f`, which is absent on BSD/macOS
    #   – there the
    # old check always failed, so unify re-linked every item on
    #   every re-run).
    [[ "$(cma_realpath "$1")" == "$(cma_realpath "$2")" ]]
}

```

```

backup_and_remove() {
    local p="$1"
    [[ -e "$p" || -L "$p" ]] || return 0
    mv "$p" "${p}.preunify.${ts}"
}

link_to_shared() {
    local item="$1" acct
    # Never create a dangling symlink: if the merge step did not
    # produce the
    # shared item (e.g. settings.json skipped because every account
    # file was
    # invalid JSON, or a stats-cache.json that no account had),
    # leave each
    # account's real file untouched rather than backing it up and
    # pointing it at a
    # missing target – that would silently replace a valid live
    # config with a
    # broken link.
    [[ -e "$SHARED_DIR/$item" ]] || return 0
    for acct in "${ACCOUNTS[@]}"; do
        [[ -d "$acct" ]] || continue
        local target="$acct/$item"
        already_linked_to_shared "$target" "$SHARED_DIR/$item" &&
            continue
        backup_and_remove "$target"
        mkdir -p "$(dirname "$target")"
        ln -s "$SHARED_DIR/$item" "$target"
    done
}

merge_dir_into_shared() {
    local item="$1" exclude="${2:-}" acct rc
    local excl=()
    [[ -n "$exclude" ]] && excl=(--exclude "$exclude")
    mkdir -p "$SHARED_DIR/$item"
    # First pass: each account fills only gaps (--ignore-existing)
    # so the
    # union is preserved across N accounts.
    for acct in "${ACCOUNTS[@]}"; do
        if [[ -d "$acct/$item" && ! -L "$acct/$item" ]]; then
            rc=0
            rsync -a --ignore-existing "${excl[@]}" "$acct/$item/"
                "$SHARED_DIR/$item/" || rc=$?
            # rsync exit 23/24 are partial transfers ("some files
            # vanished" / "some
            # files couldn't be transferred") that we treat as warnings
            # – common on
            # macOS for `unlinkat: Directory not empty` when symlinks
            # straddle the
            # tree. Anything else is fatal.
            (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
        fi
    done
}

```

```

        # Second pass: overlay every account with rsync -u
        (update-only) so the file
        # carrying the NEWEST mtime wins each conflict, regardless of
        how many accounts
        # there are or what they're named. The previous code overlaid
        only
        # ACCOUNTS[-1] – the alphabetically-last dir, not necessarily
        the most recently
        # active one – which let a stale lexically-last account clobber
        fresher content
        # (e.g. memory/*.md) from an earlier account.
        for acct in "${ACCOUNTS[@]"; do
            if [[ -d "$acct/$item" && ! -L "$acct/$item" ]]; then
                rc=0
                rsync -au "${excl[@]}" "$acct/$item/" "$SHARED_DIR/$item/"
                || rc=$?
                (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
            fi
        done
    }

    # daemon/roster.json is the background-agent WORKER REGISTRY. The
    generic
    # dir merge's per-file last-wins would keep only ONE alias's
    registry and
    # silently drop every other alias's workers. Union them instead
    via
    # cma_union_rosters (lib.sh): newer updatedAt wins per worker,
    proto and
    # supervisorPid from the newest roster, top-level updatedAt is the
    max.
    merge_daemon_roster() {
        local acct f srcs=()
        for acct in "${ACCOUNTS[@]"; do
            f="$acct/daemon/roster.json"
            [[ -f "$f" && ! -L "$f" ]] && srcs+=("$f")
        done
        f="$SHARED_DIR/daemon/roster.json"
        [[ -f "$f" && ! -L "$f" ]] && srcs+=("$f")
        (( ${#srcs[@]} )) || return 0
        cma_union_rosters "$SHARED_DIR/daemon/roster.json" "${srcs[@]}"
        || \
            cma_warn "daemon roster union failed (invalid roster.json?) –
            last-wins file kept"
        return 0
    }

    merge_history_jsonl() {
        local acct srcs=() tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/
        cma.XXXXXX")"
        for acct in "${ACCOUNTS[@]"; do
            local f="$acct/history.jsonl"
            if [[ -f "$f" && ! -L "$f" ]]; then srcs+=("$f"); fi

```

```

done
if [[ -f "$SHARED_DIR/history.jsonl" && ! -L "$SHARED_DIR/
    history.jsonl" ]]; then
    srcs+=("$SHARED_DIR/history.jsonl")
fi
# Feed the files straight to awk rather than cat'ing them
    together first: awk
# starts a fresh record at every file boundary, so a source
    missing its
# trailing newline cannot fuse its last line onto the next
    file's first line
# (plain `cat` would). Write via a temp file so we never
    truncate shared while
# it is also one of the inputs.
if (( ${#srcs[@]} )); then
    awk 'NF && !seen[$0]++' "${srcs[@]}" > "$tmp"
    mv "$tmp" "$SHARED_DIR/history.jsonl"
else
    : > "$SHARED_DIR/history.jsonl"
    rm -f "$tmp"
fi
return 0
}

# §11.4 own-settings: unify NEVER blanket-merges full account
    settings into the
# shared store – that would re-leak each alias's permissions/
    model/hooks to
# shared and, through the shared symlink, to every other alias.
    Instead it ONLY
# extracts the enabledPlugins map from every account's OWN
    settings.json and
# UNIONS it (any-true) into the shared TEMPLATE's enabledPlugins,
    leaving every
# other key of the template – and every key of each dir's OWN
    settings.json –
# untouched. seed_own_settings then propagates that union back
    into each dir.
union_enabled_plugins_into_template() {
    local acct files=() resolved f tpl="$SHARED_DIR/settings.json"
    for acct in "${ACCOUNTS[@]}; do
        f="$acct/settings.json"
        if [[ -L "$f" ]]; then
            resolved="$(cma_realpath "$f")" # readlink -f is
                unavailable on BSD/macOS
            if [[ -f "$resolved" ]]; then files+=("$resolved"); fi
        elif [[ -f "$f" ]]; then
            files+=("$f")
        fi
    done
    # Include the existing template so a plugin enabled on a prior
        run (any-true)
    # persists even if no current account carries it.
    [[ -f "$tpl" ]] && files+=("$tpl")
}

```



```

# De-duplicate. On re-runs two account paths can resolve to the
# same file, and

# passing the same path twice would double-count (harmless
# for a union, but we
# keep the set clean).
if (( ${#files[@]} > 1 )); then
    mapfile -t files < <(printf '%s\n' "${files[@]}" | awk '!
        seen[$0]++')
fi
# Drop any file that is not valid JSON so one hand-edited/
# corrupt account
# settings.json can't sink the whole union – jq -s slurps every
# file and fails
# wholesale on a single parse error. See test R3.
local validf=()
for f in "${files[@]}+${files[@]}"; do
    if jq empty "$f" 2>/dev/null; then
        validf+=("$f")
    else
        cma_warn "settings.json in $(dirname
            "$f") is not valid JSON – excluded from enabledPlugins
            union"
    fi
done
files=("${validf[@]}+${validf[@]}")
(( ${#files[@]} )) || return 0
mkdir -p "$SHARED_DIR"
[[ -f "$tmpl" ]] || printf '{}\n' > "$tmpl"
local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"

# Two-step so the shared template's OWN non-plugin keys
# (if any) survive and no
# per-account non-plugin key ever enters shared:
# 1. build the any-true union of enabledPlugins across every
# valid file,
# with keys SORTED so the template is byte-stable across
# re-runs
# regardless of per-account key order (idempotency, test
# line ~93);

# 2. set ONLY .enabledPlugins on the existing template
# to that union.
local union
if union="$(jq -s '
    reduce .[] as $x ({};
        reduce (($x.enabledPlugins // {}) | to_entries[]) as $e
        (.;
            .[$e.key] = (([$e.key] // false) or $e.value))
        | to_entries | sort_by(.key) | from_entries
    ' "${files[@]}" 2>/dev/null)" \
&& printf '%s' "$union" | jq empty 2>/dev/null \
&& jq --argjson u "$union" '.enabledPlugins = $u' "$tmpl" >
    "$tmp" 2>/dev/null \

```

```

    && jq -e . "$tmp" >/dev/null 2>&1; then
    mv "$tmp" "$tmpl"
else
    rm -f "$tmp"
    cma_warn "settings.json: enabledPlugins union skipped (jq
        error)"
fi
return 0
}

# Give every account dir its OWN real settings.json ($11.4 own-
# settings): seed
# from the shared TEMPLATE when absent/legacy-symlink, else
# additively merge the
# template's enabledPlugins in (own keys always win). Per-alias
# non-plugin keys
# stay local – they never leak to shared or to a sibling dir.
# Delegates to the
# single canonical seeder in lib.sh so unify/add-account/
# providers/bootstrap all
# agree.
seed_own_settings() {
    local acct
    for acct in "${ACCOUNTS[@]"; do
        [[ -d "$acct" ]] || continue
        cma_own_settings_seed "$acct"
    done
}

merge_file_into_shared() {
    local item="$1" acct newest=""
    for acct in "${ACCOUNTS[@]"; do
        local f="$acct/$item"
        if [[ -L "$f" ]]; then continue; fi
        if [[ ! -f "$f" ]]; then continue; fi
        if [[ -z "$newest" || "$f" -nt "$newest" ]]; then
            newest="$f"; fi
    done
    if [[ -n "$newest" ]]; then cp -p "$newest" "$SHARED_DIR/
        $item"; fi
    return 0
}

absorb_default_plugins() {
    [[ -d "$DEFAULT_DIR/plugins" && ! -L "$DEFAULT_DIR/plugins" ]]
        || return 0
    mkdir -p "$SHARED_DIR/plugins"
    local rc=0
    rsync -a --ignore-existing "$DEFAULT_DIR/plugins/" "$SHARED_DIR/
        plugins/" || rc=$?
    (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
    return 0
}

```

```

rewrite_plugin_paths() {
  local f="$SHARED_DIR/plugins/installed_plugins.json"
  if [[ -f "$f" ]]; then
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    jq --arg shared "$SHARED_DIR" '
      .plugins |= with_entries(
        .value |= map(
          if ((.installPath // "") | test(".*plugins/cache/"))
          then .installPath = ($shared + "/plugins/cache/" +
            (.installPath | sub(".*plugins/cache/"; "")))
          else . end
        )
      )' "$f" > "$tmp" && mv "$tmp" "$f"
  fi
  local km="$SHARED_DIR/plugins/known_marketplaces.json"
  if [[ -f "$km" ]]; then
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    jq --arg shared "$SHARED_DIR" '
      with_entries(
        if ((.value.installLocation // "") | test(".*plugins/
          marketplaces/"))
        then .value.installLocation = ($shared + "/plugins/
          marketplaces/" + (.value.installLocation | sub(".*
          plugins/marketplaces/"; "")))
        else . end
      )' "$km" > "$tmp" && mv "$tmp" "$km"
  fi
}

# Repoint ~/.claude/plugins/cache and ~/.claude/plugins/
# marketplaces at
# the shared store so future user-scope installs land in one
# place.
link_default_plugin_subdirs() {
  for sub in cache marketplaces; do
    local src="$SHARED_DIR/plugins/$sub" tgt="$DEFAULT_DIR/
      plugins/$sub"
    [[ -d "$src" ]] || continue
    already_linked_to_shared "$tgt" "$src" && continue
    mkdir -p "$DEFAULT_DIR/plugins"
    backup_and_remove "$tgt"
    ln -s "$src" "$tgt"
  done
}

# CLAUDE.md (user-scope memory) lives at ~/.claude/CLAUDE.md by
# default.
# We promote it to the shared store and symlink all three
# locations (each
# account dir + ~/.claude) so any account writing user memory
# updates the
# same file.

```

```

sync_claude_md() {
    local shared_md="$SHARED_DIR/CLAUDE.md" src
    if [[ ! -f "$shared_md" ]]; then
        if [[ -f "$DEFAULT_DIR/CLAUDE.md" && ! -L "$DEFAULT_DIR/CLAUDE.md" ]]; then
            cp -p "$DEFAULT_DIR/CLAUDE.md" "$shared_md"
        else
            for acct in "${ACCOUNTS[@]}; do
                if [[ -f "$acct/CLAUDE.md" && ! -L "$acct/CLAUDE.md" ]]; then
                    cp -p "$acct/CLAUDE.md" "$shared_md"; break
                fi
            done
        fi
        [[ -f "$shared_md" ]] || return 0
    fi
    local targets=("$DEFAULT_DIR/CLAUDE.md")
    for acct in "${ACCOUNTS[@]}; do targets+=("$acct/CLAUDE.md"); done
    for tgt in "${targets[@]}; do
        [[ -d "$(dirname "$tgt")" ]] || continue
        already_linked_to_shared "$tgt" "$shared_md" && continue
        backup_and_remove "$tgt"
        ln -s "$shared_md" "$tgt"
    done
}

rollback() {
    cma_log "rollback: restoring .preunify.* backups"
    local root
    local roots=("$DEFAULT_DIR" "$DEFAULT_DIR/plugins" "$SHARED_DIR")
    for root in "${ACCOUNTS[@]}; do roots+=("$root"); done
    for root in "${roots[@]}; do
        [[ -d "$root" ]] || continue

        # Restore backups oldest-first. The suffix is a
        YYYYMMDDHHMMSS timestamp, so
        # `sort -z` orders lexically = chronologically; the earliest
        backup is the
        # true pre-unify original and wins when a path has more than
        one .preunify.*.
        while IFS= read -r -d '' bk; do
            local orig="${bk%.preunify.*}"
            [[ -L "$orig" ]] && rm -f "$orig"
            [[ -e "$orig" ]] || { mv "$bk" "$orig"; cma_log "restored $orig"; }
        done <<(find "$root" -maxdepth 1 -name '*.preunify.*' -print0 2>/dev/null | sort -z)
        # Don't scan the shared store itself (its contents move aside intact below);
        # only external roots can hold links that would dangle.
        [[ "$root" == "$SHARED_DIR" ]] && continue
    done
}

```

```

# Remove leftover symlinks pointing into the shared store.
# unify created
# these for items an account never had (so there's no backup
# to restore);
# without this they dangle once SHARED_DIR is moved aside.
while IFS= read -r -d '' lnk; do
    local tgt; tgt="$(readlink "$lnk")"
    case "$tgt" in
        "$SHARED_DIR"|"${SHARED_DIR}/") rm -f "$lnk"; cma_log
            "removed shared-store symlink $lnk" ;;
    esac
done < <(find "$root" -maxdepth 1 -type l -print0 2>/dev/null)
done
if [[ -d "$SHARED_DIR" ]]; then
    mv "$SHARED_DIR" "${SHARED_DIR}.removed.${ts}"
    cma_log "moved $SHARED_DIR aside"
fi
}

usage() {
    cat <<EOF
Usage: $(basename "$0") [--rollback] [account-dir ...]

Without args, auto-detects all ~/${ACCOUNT_PREFIX}* directories.
With --rollback, restores .preunify.* backups and removes the
    shared store.

Env: SHARED_DIR=$SHARED_DIR DEFAULT_DIR=$DEFAULT_DIR
EOF
}

# === main ===

ACCOUNTS=()
DO_ROLLBACK=0
while (( $# )); do
    case "$1" in
        -h|--help) usage; exit 0 ;;
        --rollback) DO_ROLLBACK=1; shift ;;
        --) shift; while (( $# )); do ACCOUNTS+=("$1"); shift; done ;;
        -*) cma_die "unknown flag: $1" ;;
        *) ACCOUNTS+=("$1"); shift ;;
    esac
done

if (( ${#ACCOUNTS[@]} == 0 )); then
    mapfile -t ACCOUNTS < <(cma_detect_accounts)
fi

(( ${#ACCOUNTS[@]} >= 1 )) || cma_die "no account dirs given and
    none auto-detected at ~/${ACCOUNT_PREFIX}*"

cma_log "accounts: ${ACCOUNTS[*]}"

```

```

cma_log "shared:  $SHARED_DIR"

if (( DO_ROLLBACK )); then rollback; exit 0; fi

mkdir -p "$SHARED_DIR"
absorb_default_plugins

for item in "${SHARED_ITEMS[@]}; do
    case "$item" in
        history.jsonl)    merge_history_jsonl ;;
        stats-cache.json) merge_file_into_shared "$item" ;;
        daemon)           merge_dir_into_shared "$item" "roster.json"
                        && merge_daemon_roster ;;
        *)                merge_dir_into_shared "$item" ;;
    esac
    [[ "$item" == "plugins" ]] && rewrite_plugin_paths
    link_to_shared "$item"
    cma_log "ok: $item"
done

# $11.4 own-settings: settings.json is NOT a shared symlink. Union
# each account's
# enabledPlugins into the shared template, then re-seed every
# dir's OWN copy so
# the union propagates while per-alias non-plugin keys stay local.
union_enabled_plugins_into_template
seed_own_settings
cma_log
    "ok: settings.json (per-dir OWN copy; enabledPlugins union
    propagated; per-alias keys kept local)"

link_default_plugin_subdirs
sync_claude_md

# Merge the projects/session index inside .claude.json across
# every account.
# This is the single most important step for cross-account session
# resume:
# without it, account A's session UUIDs are invisible to account B
# even though
# the JSONL transcripts are already shared on disk via projects/.
cma_merge_claude_json "${ACCOUNTS[@]}"
cma_log "ok: .claude.json (projects/session index merged; auth
    keys preserved per-account)"

# Ensure every detected account has an invocable alias. install.sh
# and direct
# unify runs may discover pre-existing ~/.claude-* dirs that were
# never
# registered (e.g. the user ran install.sh after creating the
# dirs). Without
# this step `claude1` etc. are undefined even though the dirs
# exist.
cma_ensure_alias_file

```

```

declare -A ALIAS_OF_DIR=()
if [[ -f "$ALIAS_FILE" ]]; then
    while IFS= read -r line; do
        [[ "$line" =~ ^alias[[:space:]]+([a-zA-Z0-9_-]+)
            =.*CLAUDE_CONFIG_DIR=([[:space:]]+)$ ]] || continue
        ALIAS_OF_DIR["${BASH_REMATCH[2]}"]="${BASH_REMATCH[1]}"
    done < "$ALIAS_FILE"
fi
# First pass: dirs whose suffix (after ~/.claude-) is already
# "claude<N>"
# (e.g. ~/.claude-claude4) keep that number, so user expectations
# about named
# dirs are preserved.
for acct in "${ACCOUNTS[@]"; do
    if [[ -z "${ALIAS_OF_DIR[$acct]:-}" ]]; then
        base="$(basename "$acct")"
        suffix="${base}${ACCOUNT_PREFIX}"
        if [[ "$suffix" =~ ^claude([0-9]+)$ ]]; then
            wanted="claude${BASH_REMATCH[1]}"
            if ! cma_existing_aliases | grep -qx "$wanted"; then
                cma_write_alias "$wanted" "$acct"
                cma_log "registered alias: $wanted -> $acct"
                continue
            fi
        fi
    fi
done
# Lowest free claude<N> (fills gaps left by explicit claudeM
# basenames).
_cma_lowest_free_clauden() {
    local n=1
    while cma_existing_aliases | grep -qx "claude$n"; do
        n=$((n + 1))
        (( n < 1000 )) || { cma_warn "could not find a free claude<N>
            alias"; return 1; }
    done
    printf 'claude%s\n' "$n"
}

# Second pass: anything still unaliased gets the lowest free
# claude<N>.
for acct in "${ACCOUNTS[@]"; do
    if [[ -z "${ALIAS_OF_DIR[$acct]:-}" ]]; then
        if grep -qE "alias[[:space:]]+([^\s=]
            +=.*CLAUDE_CONFIG_DIR=$acct([[:space:]]|$)"
            "$ALIAS_FILE"; then
            # A newly-written pass-1 alias now points at this dir;
            # update our map.
            ALIAS_OF_DIR[$acct]="$(grep -E "alias[[:space:]]+([^\s=]
                +=.*CLAUDE_CONFIG_DIR=$acct([[:space:]]|$)" "$ALIAS_FILE"
                | head -1 | sed -E 's/^\s+alias[[:space:]]+([^\s=]+)=.*\/\1\/')
            continue
        fi
    fi
done

```

```

    next_alias="$(_cma_lowest_free_clauden)"
    cma_write_alias "$next_alias" "$acct"
    cma_log "registered alias: $next_alias -> $acct"
fi
done

cma_log "done. shared: $SHARED_DIR"
cma_log "private per-account: ${PRIVATE_ITEMS[*]}"

```

8.3 claude-add-account.sh

```

#!/usr/bin/env bash
# claude-add-account.sh – Add a new Claude Code account to the
# multi-account
# setup. Creates the per-account config directory, links every
# shared item
# to the shared store, and registers a shell alias so the new
# account is
# invocable as e.g. `claude3`.
#
# Modes:
# * Interactive (default): prompts for alias name and config dir
#   name,
#   showing defaults that follow the existing pattern.
# * Non-interactive: pass --alias NAME and optionally --dir
#   PATH, e.g.
#   `claude-add-account.sh --alias work --dir ~/.claude-work`.
#
# After this script runs you still need to authenticate the new
# account
# with Anthropic – the script prints the exact command for that.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

ALIAS_NAME=""
CONFIG_DIR=""
NONINTERACTIVE=0

usage() {

```



```

cat <<EOF
Usage: $(basename "$0") [--alias NAME] [--dir PATH] [--yes]

    --alias NAME    Shell alias to create (e.g. claude3 or work).
                    Default:
                        next free claudeN.
    --dir    PATH    Config directory for the new account. Default:
                    ~/${ACCOUNT_PREFIX}<alias>.
    --yes           Skip prompts and use defaults / passed values.
EOF
}

while (( $# )); do
    case "$1" in
        -h|--help) usage; exit 0 ;;
        --alias)    ALIAS_NAME="$2"; shift 2 ;;
        --dir)      CONFIG_DIR="$2"; shift 2 ;;
        --yes|-y)   NONINTERACTIVE=1; shift ;;
        *)          cma_die "unknown arg: $1" ;;
    esac
done

# Prompt with a default value, return whatever the user types (or
# default).
prompt() {
    local label="$1" default="$2" answer
    # Use the default when --yes was passed OR no terminal is
    # available to
    # prompt from (CI, test sandbox, SSH without a PTY) – never
    # block.
    if (( NONINTERACTIVE )) || ! cma_can_prompt; then echo
        "$default"; return; fi
    read -r -p "$label [$default]: " answer < /dev/tty
    echo "${answer:-$default}"
}

[[ -n "$ALIAS_NAME" ]] || ALIAS_NAME="$(cma_suggest_alias)"
ALIAS_NAME="$(prompt "Alias name" "$ALIAS_NAME")"
cma_validate_alias "$ALIAS_NAME"

# Reject if alias already exists in the alias file.
if cma_existing_aliases | grep -qx "$ALIAS_NAME"; then
    cma_die "alias '$ALIAS_NAME' already exists in $ALIAS_FILE"
fi

[[ -n "$CONFIG_DIR" ]] || CONFIG_DIR="$HOME/${ACCOUNT_PREFIX}$
    {ALIAS_NAME}"
CONFIG_DIR="$(prompt "Config directory" "$CONFIG_DIR")"
case "$CONFIG_DIR" in /*) ;; *) CONFIG_DIR="$HOME/$CONFIG_DIR" ;;
esac

[[ -d "$CONFIG_DIR" ]] && cma_die "config dir already exists:
    $CONFIG_DIR (refusing to overwrite)"

```

```

mkdir -p "$CONFIG_DIR"
cma_log "created $CONFIG_DIR"

# Symlink every shared item into the new account dir, using the
# single
# canonical list (CMA_SHARED_ITEMS in lib.sh) so accounts and
# provider
# aliases stay in lockstep without two copies of the list drifting
# apart.
cma_link_shared_items "$CONFIG_DIR"
cma_log "linked ${#CMA_SHARED_ITEMS[@]} shared items into
$CONFIG_DIR"

# Add the alias. lib.sh handles dedupe, rc-file sourcing, etc.
#
# The status is CHECKED. cma_write_alias used to report success
# even when the
# write was skipped on lock contention, which left the account on
# disk with no
# alias while this script printed "[done]" – and the obvious retry
# then died at
# the "config dir already exists" guard above, with no supported
# way forward.
# Say what actually happened and name the one command that
# finishes the job.
if ! cma_write_alias "$ALIAS_NAME" "$CONFIG_DIR"; then
    cma_warn "the alias for '$ALIAS_NAME' was NOT written to
$ALIAS_FILE"
    cma_warn "$CONFIG_DIR exists and its shared items are linked –
only the alias is missing."
    cma_die "Finish with: claude-unify (re-registers an alias
for every detected account)"
fi
cma_log "registered alias: $ALIAS_NAME -> $CONFIG_DIR"

cat <<EOF

[done] new account: $ALIAS_NAME

Next steps:
1. Reload your shell (or run: source $ALIAS_FILE).
2. Authenticate the new account:
$ALIAS_NAME /login
3. After login, run any project – history, memory, plugins, and
settings will already match your other accounts via
$SHARED_DIR.

To remove this account later:
$LIB_DIR/claude-remove-account.sh --alias $ALIAS_NAME
EOF

# ATM-850 (2026-07-23): long-lived shells (tmux panes above all)
# sourced the

```

```

# alias file BEFORE this add. Detect them, say exactly who is
# covered by the
# dynamic dispatcher vs who needs ONE re-source, and print the
# exact commands.
# The notice never sends keys; the broadcast below is interactive,
# default No,
# and scoped to idle SHELL panes only.
cma_tmux_stale_shell_notice "$ALIAS_FILE" || true
if ! (( NONINTERACTIVE )) && cma_can_prompt; then
    _bc="$(prompt "Broadcast the re-source into idle tmux SHELL
    panes now? (y/N)" "N")"
    case "$_bc" in
        [Yy]*)
            _tdir="${TMUX_TMPDIR:-/tmp}/tmux-${id -u}"
            for _sock in "$_tdir"/*; do
                [[ -S "$_sock" ]] || continue
                tmux -S "$_sock" list-panes -a -F '#{pane_id}
                #{pane_current_command}' 2>/dev/null \
                | awk '$2 ~ /^-?(bash|zsh|sh|dash|ksh|fish)$/{print
                $1}' \
                | while read -r _pane; do
                    tmux -S "$_sock" send-keys -t "$_pane" C-u " source
                    $ALIAS_FILE" Enter 2>/dev/null || true
                done
            done
            cma_log "re-source broadcast sent to idle shell panes" ;;
        *) : ;;
    esac
fi

```

8.4 claude-remove-account.sh

```

#!/usr/bin/env bash
# claude-remove-account.sh – Remove a Claude Code account from the
# multi-account setup. Drops the alias from the managed alias file
# and
# (optionally) deletes or archives the per-account config
# directory.
#
# The shared store is left untouched – only this one account's
# symlinks
# and credentials are removed. Other accounts continue using
# shared.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in
        /*) _cma_src=$_cma_tgt ;;
        *) _cma_src="$(dirname "$_cma_src")/$_cma_tgt" ;;
    esac
done

```

```

done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

ALIAS_NAME=""
DELETE_DIR=0
NONINTERACTIVE=0

usage() {
    cat <<EOF
Usage: $(basename "$0") --alias NAME [--delete | --archive] [--yes]

    --alias NAME    Required. Alias to remove (e.g. claude3).
    --delete        Permanently delete the per-account config
                    directory.
    --archive        Move the per-account dir to
                    <dir>.removed.<timestamp>
                    instead of deleting (default).
    --yes            Skip confirmation prompts.
EOF
}

while (( $# )); do
    case "$1" in
        -h|--help) usage; exit 0 ;;
        --alias) ALIAS_NAME="$2"; shift 2 ;;
        --delete) DELETE_DIR=1; shift ;;
        --archive) DELETE_DIR=0; shift ;;
        --yes|-y) NONINTERACTIVE=1; shift ;;
        *)        cma_die "unknown arg: $1" ;;
    esac
done

[[ -n "$ALIAS_NAME" ]] || { usage; exit 2; }
cma_validate_alias "$ALIAS_NAME"

# Resolve the config dir by inspecting the existing alias line.
CONFIG_DIR=""
if [[ -f "$ALIAS_FILE" ]]; then
    # POSIX/2-arg awk match() only: the 3-arg capture form is GNU-
awk-specific

    # and silently yields an empty CONFIG_DIR on the BSD awk
shipped with macOS.
    CONFIG_DIR="$(awk -v a="$ALIAS_NAME" '
        $0 ~ "^alias[[:space:]]+a=\"" {
            if (match($0, /CLAUDE_CONFIG_DIR=[^ ]+/)) {
                s = substr($0, RSTART, RLENGTH)
                sub(/^CLAUDE_CONFIG_DIR=/, "", s)
                print s
            }
        }'"

```

```

    }
    exit
}' "$ALIAS_FILE")"
fi
[[ -n "$CONFIG_DIR" ]] || cma_die "alias '$ALIAS_NAME' not found
    in $ALIAS_FILE"

cma_log "removing alias '$ALIAS_NAME' (config dir: $CONFIG_DIR)"

if (( ! NONINTERACTIVE )); then
    # Destructive op: when no terminal is available to confirm from
    # (CI, SSH

        # without a PTY), refuse rather than block or guess – pass
        --yes to proceed
    # non-interactively.
    cma_can_prompt || cma_die "no terminal to confirm removal; pass
        --yes to proceed non-interactively"
    read -r -p "Proceed? [y/N] " ans < /dev/tty
    [[ "$ans" =~ ^[Yy]$ ]] || cma_die "aborted"
fi

# The status is CHECKED, and the directory move below is gated on
# it. This used
# to report success even when the write was skipped on lock
# contention, so the
# script logged "alias removed", then archived/deleted the config
# dir anyway –
# leaving a LIVE alias pointing at a directory that no longer
# exists. Removing
# nothing is recoverable; a dangling alias over a moved dir is
# not.
if ! cma_remove_alias "$ALIAS_NAME"; then
    cma_warn "the alias '$ALIAS_NAME' was NOT removed from
        $ALIAS_FILE"
    cma_die "$CONFIG_DIR left in place on purpose – a live alias
        must never point at a moved dir. Re-run to retry."
fi
cma_log "alias removed from $ALIAS_FILE"

if [[ -d "$CONFIG_DIR" ]]; then
    if (( DELETE_DIR )); then
        rm -rf -- "$CONFIG_DIR"
        cma_log "deleted $CONFIG_DIR"
    else
        mv -- "$CONFIG_DIR" "${CONFIG_DIR}.removed.${date +
            %Y%m%d%H%M%S}"
        cma_log "archived $CONFIG_DIR -> ${CONFIG_DIR}.removed.*"
    fi
fi

cat <<EOF

[done] account '$ALIAS_NAME' removed.

```

Notes:

- * Shared store `$SHARED_DIR` is untouched – other accounts still work.
- * Reload your shell so the alias change takes effect.

EOF

8.5 `claude-list-accounts.sh`

```
#!/usr/bin/env bash
# claude-list-accounts.sh – Print a status summary of every
# detected
# Claude Code account: which alias maps to it, whether credentials
# are
# present, and whether its shared-state symlinks are intact.

set -euo pipefail

if (( BASH_VERSINFO[0] < 4 )); then
    for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
        [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
    done
    echo "claude-list-accounts requires bash 4+. Install via: brew
        install bash" >&2
    exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

# Build alias -> dir mapping from the managed alias file.
declare -A ALIAS_OF_DIR
if [[ -f "$ALIAS_FILE" ]]; then
    while IFS= read -r line; do
        [[ "$line" =~ ^alias[[:space:]]+([a-zA-Z0-9_-]
            +)=.*CLAUDE_CONFIG_DIR=([^[:space:]]+) ]] || continue
        ALIAS_OF_DIR["${BASH_REMATCH[2]}"]="${BASH_REMATCH[1]}"
    done < "$ALIAS_FILE"
fi

# Items we expect to be symlinks pointing into $SHARED_DIR. NOTE
# (§11.4
```

```

# own-settings): settings.json is DELIBERATELY excluded – each dir
# has its OWN
# real settings.json (per-alias keys stay local), so it is not a
# shared symlink
# and must not be reported as "not linked".
CHECK_LINKS=(projects history.jsonl plugins todos CLAUDE.md)

printf '%-12s  %-45s  %-6s  %-5s  %s\n' \
    "ALIAS" "CONFIG DIR" "CREDS" "LINKS" "NOTES"

mapfile -t accounts < <(cma_detect_accounts)
for dir in "${accounts[@]"; do
    alias_name="${ALIAS_OF_DIR[$dir]:--}"
    if [[ -f "$dir/.credentials.json" ]]; then creds="yes"; else
        creds="no"; fi
    ok=0 total=0 missing=()
    for item in "${CHECK_LINKS[@]"; do
        total=$((total+1))
        if [[ -L "$dir/$item" ]] \
            && [[ "$(cma_realpath "$dir/$item")" == "$(cma_realpath
                "$SHARED_DIR/$item")" ]]; then
            ok=$((ok+1))
        else
            missing+=("$item")
        fi
    done
    notes=""
    (( ${#missing[@]} )) && notes="not linked: ${missing[*]}"
    printf '%-12s  %-45s  %-6s  %-5s  %s\n' \
        "$alias_name" "$dir" "$creds" "$ok/$total" "$notes"
done

echo
echo "Shared store: $SHARED_DIR"
[[ -d "$SHARED_DIR" ]] || echo " (does not exist – run claude-
    unify.sh)"
echo "Alias file: $ALIAS_FILE"
[[ -f "$ALIAS_FILE" ]] || echo " (does not exist – run
    install.sh)"

```

8.6 claude-rollback.sh

```

#!/usr/bin/env bash
# claude-rollback.sh – Convenience wrapper that calls claude-
# unify.sh with
# --rollback. Restores every .preunify.<timestamp> backup created
# by the
# unification run and archives the shared store out of the way.
#
# After this you can re-run install.sh to start over from scratch.

set -euo pipefail

```

```

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
exec "$LIB_DIR/claude-unify.sh" --rollback "$@"

```

8.7 claude-export-docs.sh

```

#!/usr/bin/env bash
# claude-export-docs.sh – Convert the multi-account markdown doc
to
# matching .html and .pdf siblings. Re-runnable; overwrites
outputs.
#
# Order of preference for PDF: pandoc + weasyprint > pandoc +
wkhtmltopdf
# > chromium --headless print-to-pdf > weasyprint on the generated
HTML.
# HTML always uses pandoc with a self-contained option so the file
is
# portable without external CSS/JS.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

DOC_DIR="${DOC_DIR:-$HOME/Documents}"
MD_FILE="${MD_FILE:-$DOC_DIR/Claude_Multi_Account_Fine_Tuning.md}"
HTML_FILE="${MD_FILE%.md}.html"
PDF_FILE="${MD_FILE%.md}.pdf"
DOCX_FILE="${MD_FILE%.md}.docx"
# Optional Word styling: first existing reference doc wins.
Generate a starter
# with: pandoc -o reference.docx --print-default-data-file
reference.docx

```



```

DOCX_REFERENCE="${DOCX_REFERENCE:-}"

[[ -f "$MD_FILE" ]] || cma_die "markdown not found: $MD_FILE"
cma_require pandoc

# --- Preprocess: expand `<!-- INCLUDE: relative/path -->` markers
# ---
# These markers, placed inside a fenced code block in the source
# markdown,
# get replaced with the contents of the referenced file (relative
# to the
# directory containing the markdown). That keeps the .md small
# while the
# generated HTML/PDF stays self-contained.
# Portable temp file: BSD/macOS mktemp has no --suffix. pandoc
# reads the
# format from --from, so the missing .md extension is irrelevant.
PROCESSED_MD="$(mktemp "${TMPDIR:-/tmp}/cma-export.XXXXXX)"
trap 'rm -f "$PROCESSED_MD"' EXIT

# Use only POSIX/2-arg awk match() (RSTART/RLENGTH + substr): the
# 3-arg
# match($0,re,arr) capture form is a GNU-awk extension and breaks
# on the BSD
# awk that ships with macOS.
awk -v base="$(dirname "$MD_FILE")" '
  /<!-- INCLUDE: / {
    if (match($0, /INCLUDE: [^ ]+ -->/)) {
      tok = substr($0, RSTART, RLENGTH) # "INCLUDE: path -->"
      sub(/^INCLUDE: /, "", tok)
      sub(/ -->$/, "", tok)           # -> "path"
      path = tok
      if (path !~ /\^\/) path = base "/" path
      while ((getline line < path) > 0) print line
      close(path)
    }
    next
  }
  { print }
' "$MD_FILE" > "$PROCESSED_MD"

# --- HTML ---
# --standalone makes a complete HTML doc (head/body), --embed-
# resources
# inlines CSS/JS/images so the file is self-contained.
cma_log "rendering HTML -> $HTML_FILE"
pandoc \
  --from=gfm+yaml_metadata_block \
  --to=html5 \
  --standalone \
  --embed-resources \
  --toc --toc-depth=3 \
  --highlight-style=tango \

```

```

--metadata title="Claude Multi-Account Fine Tuning" \
-o "$HTML_FILE" \
"$PROCESSED_MD"

# --- PDF ---
cma_log "rendering PDF -> $PDF_FILE"

render_pdf_via_pandoc_engine() {
    local engine="$1"
    command -v "$engine" >/dev/null 2>&1 || return 1
    pandoc \
        --from=gfm+yaml_metadata_block \
        --pdf-engine="$engine" \
        --toc --toc-depth=3 \
        --highlight-style=tango \
        -V geometry:margin=0.9in \
        -V mainfont="DejaVu Serif" \
        -V monofont="DejaVu Sans Mono" \
        --metadata title="Claude Multi-Account Fine Tuning" \
        -o "$PDF_FILE" \
        "$PROCESSED_MD"
}

render_pdf_via_chromium() {
    local engine
    for engine in chromium chromium-browser google-chrome chrome; do
        if command -v "$engine" >/dev/null 2>&1; then
            # Use the freshly generated HTML so styling matches.
            "$engine" --headless --no-sandbox --disable-gpu \
                --print-to-pdf="$PDF_FILE" \
                --print-to-pdf-no-header \
                "file://$HTML_FILE" >/dev/null 2>&1
            return $?
        fi
    done
    return 1
}

render_pdf_via_weasyprint_on_html() {
    command -v weasyprint >/dev/null 2>&1 || return 1
    weasyprint "$HTML_FILE" "$PDF_FILE"
}

# Try engines in order; the first that succeeds wins.
if render_pdf_via_pandoc_engine weasyprint 2>/dev/null; then
    cma_log "pdf engine: weasyprint (via pandoc)"
elif render_pdf_via_pandoc_engine wkhtmltopdf 2>/dev/null; then
    cma_log "pdf engine: wkhtmltopdf (via pandoc)"
elif render_pdf_via_weasyprint_on_html 2>/dev/null; then
    cma_log "pdf engine: weasyprint on html"
elif render_pdf_via_chromium 2>/dev/null; then
    cma_log "pdf engine: chromium headless"

```

```

else
    cma_die "no usable PDF engine. Install one of: weasyprint,
           wkhtmltopdf, chromium."
fi

# --- DOCX ---
# pandoc writes .docx natively (no external engine). A reference
# doc supplies
# styling (fonts, headings, code blocks) when provided.
cma_log "rendering DOCX -> $DOCX_FILE"
docx_args=(
    --from=gfm+yaml_metadata_block
    --to=docx
    --toc --toc-depth=3
    --highlight-style=tango
    --metadata title="Claude Multi-Account Fine Tuning"
    -o "$DOCX_FILE"
)
# Auto-discover a reference doc if none was passed explicitly.
if [[ -z "$DOCX_REFERENCE" ]]; then
    for cand in "$DOC_DIR/reference.docx" "$LIB_DIR/assets/
               reference.docx"; do
        [[ -f "$cand" ]] && { DOCX_REFERENCE="$cand"; break; }
    done
fi
[[ -n "$DOCX_REFERENCE" && -f "$DOCX_REFERENCE" ]] &&
    docx_args+=(--reference-doc "$DOCX_REFERENCE")
if pandoc "${docx_args[@]}" "$PROCESSED_MD"; then
    cma_log "docx ok${DOCX_REFERENCE:+ (styled via
           $DOCX_REFERENCE)}"
else
    cma_warn "DOCX render failed (pandoc); HTML + PDF are still
           produced"
fi

ls -lh "$HTML_FILE" "$PDF_FILE" "$DOCX_FILE" 2>/dev/null || ls
    -lh "$HTML_FILE" "$PDF_FILE"

```

8.8 install.sh

```

#!/usr/bin/env bash
# install.sh – One-shot bootstrap for the Claude multi-account
# toolkit.
# Idempotent; safe to re-run after pulling updates.
#
# What it does, in order:
# 1. Detect tooling (jq, rsync, awk, pandoc) and fail fast if
#    missing.
# 2. Symlink every script into ~/.local/bin (creates the dir if
#    needed).
# 3. Ensure ~/.local/bin is on PATH for future shells.
# 4. Create the managed alias file at $ALIAS_FILE and source it
#    from

```

```

# ~/.bashrc and ~/.zshrc (whichever exist).
# 5. Run claude-unify.sh to merge any existing account dirs.
# 6. Run claude-export-docs.sh to refresh the PDF/HTML alongside
# the
# markdown doc.

set -euo pipefail

# macOS ships bash 3.2 which lacks `mapfile`. If a newer bash is
# available
# (Homebrew installs it at /opt/homebrew/bin/bash on Apple
# Silicon, or
# /usr/local/bin/bash on Intel), re-exec with it. Otherwise tell
# the user
# how to install it.
if (( BASH_VERSINFO[0] < 4 )); then
  for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
    [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
  done
  echo
  "claude-toolkit requires bash 4+. Install via: brew
  install bash" >&2
  exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
  (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

cma_log "platform: $(cma_os)"

# 1. Required tooling. pandoc is needed only for doc export; the
# rest are
# load-bearing for unify/add-account.
for t in rsync jq awk; do cma_require "$t"; done
command -v pandoc >/dev/null 2>&1 || cma_warn "pandoc not found –
PDF/HTML export will be skipped"

# 1b. Node deps for the optional TOON utility (scripts/toon.mjs,
# and the
# toon_encode.py wrapper that shells out to it). Soft by design:
# the toolkit's
# core (unify/add-account) needs no Node, so a missing npm is a
# warning, not a

```

```

# hard failure. Idempotent – npm install is a no-op once @toon-
# format/toon is
# already present. (node_modules/ is gitignored; this is how the
# dep arrives.)
_cma_repo_root="$(cd "$LIB_DIR/.." && pwd)"
if [[ -f "$_cma_repo_root/package.json" ]]; then
    if command -v npm >/dev/null 2>&1; then
        cma_log "installing Node deps for the TOON utility (npm
            install) ..."
        ( cd "$_cma_repo_root" && npm install --no-audit --no-fund --
            silent ) \
            || cma_warn "npm install failed – scripts/toon.mjs (TOON
                utility) may be unavailable"
    else
        cma_warn "npm not found – scripts/toon.mjs (TOON utility)
            unavailable until you run 'npm install' in
            $_cma_repo_root"
    fi
fi
unset _cma_repo_root

# 2. Symlink the scripts onto PATH.
BIN_DIR="${BIN_DIR:-$HOME/.local/bin}"
mkdir -p "$BIN_DIR"
for f in "$LIB_DIR"/claude-*.sh; do
    name="$(basename "$f" .sh)"
    link="$BIN_DIR/$name"
    if [[ -L "$link" || -e "$link" ]]; then
        if [[ "$(cma_realpath "$link")" != "$(cma_realpath "$f")" ]];
            then
                mv "$link" "${link}.preunify.$(date +%Y%m%d%H%M%S)"
                ln -s "$f" "$link"
                cma_log "linked $link -> $f"
            fi
        else
            ln -s "$f" "$link"
            cma_log "linked $link -> $f"
        fi
    fi
done
# .same loop over the Kimi family commands (v1.27.0) – they end up
# on PATH
# exactly like the claude-* ones, so the docs' `kimi-add-account`
# etc. work
# out of the box.
for f in "$LIB_DIR"/kimi-*.sh; do
    name="$(basename "$f" .sh)"
    link="$BIN_DIR/$name"
    if [[ -L "$link" || -e "$link" ]]; then
        if [[ "$(cma_realpath "$link")" != "$(cma_realpath "$f")" ]];
            then
                mv "$link" "${link}.preunify.$(date +%Y%m%d%H%M%S)"
                ln -s "$f" "$link"
                cma_log "linked $link -> $f"
            fi
        fi
    fi
done

```

```

else
    ln -s "$f" "$link"
    cma_log "linked $link -> $f"
fi
done

# 2b. Build the BUNDLED Go claude-code-router (submodule) and
#       install it as
# `ccr`, so the provider aliases route through OUR vendored router
#       rather than a
#       separately-installed Node one.
#
# NOT best-effort any more (field failure 2026-07-22, operator
#       host, v1.25.4):
# this was `if ! bash ...; then cma_warn ...`, so a build failure
#       only WARNED,
# the install continued, and the banner still printed "[done]
#       installed" while
# the npm @musistudio ccr kept answering for `ccr`. The operator's
#       FIRST
# symptom was a provider alias dying at RUNTIME. An installer that
#       reports
# success over a broken product is a §11.4 PASS-bluff at the
#       install layer.
#
# The failure is RECORDED and re-raised by step 7 rather than
#       aborting here:
# the remaining steps (PATH, aliases, unify) are independent and
#       still worth
# completing, and step 7 then reports EVERY problem at once
#       instead of making
# the operator re-run once per failure.
# Declared together (and BEFORE first use) so `set -u` never trips
#       on an
# append to an unset array, and so step 7 can length-test them
#       safely.
CMA_INSTALL_FAILURES=()
CMA_INSTALL_DEGRADED=()
if ! bash "$LIB_DIR/claude-ccr-build.sh"; then
    CMA_INSTALL_FAILURES+=("bundled claude-code-router (Go) did NOT
        build/install – provider aliases on the router transport
        cannot launch. Fix: ensure a Go toolchain new enough for
        submodules/claude-code-router/go.mod is available
        (install/upgrade Go: https://go.dev/dl/ – or, with an
        older Go already installed and network access:
        GOTOOCHAIN=auto claude-ccr-build), then run: claude-ccr-
        build")
fi

# 3. Make sure ~/.local/bin is on PATH for new shells. We add
#       to .bashrc
# and .zshrc once; existing shells need a manual reload.
# shellcheck disable=SC2016 # $HOME/$PATH are intentionally
#       unexpanded literals written into rc files
PATH_LINE='export PATH="$HOME/.local/bin:$PATH"'

```

```

for rc in "${CMA_RC_FILES[@]}; do
    [[ -f "$rc" ]] || continue
    if ! grep -F -q "$PATH_LINE" "$rc"; then
        # Backup-first, idempotent BEGIN/END block (refuses if it
        # cannot back up).
        cma_rc_append_managed "$rc" path "$PATH_LINE" || true
    fi
done

# 4. Alias file + rc sourcing.
cma_ensure_alias_file

# Re-register any pre-existing aliases from $HOME/.bashrc into the
# managed
# alias file, then comment out the originals so they're not
# duplicated.
migrate_inline_aliases() {
    local rc="$1"
    [[ -f "$rc" ]] || return 0
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    local changed=0

    # `|| [[ -n "$line" ]]' so a final line with NO trailing
    # newline is still
    # processed rather than silently dropped (mirrors lib.sh's rc
    # read loops).
    while IFS= read -r line || [[ -n "$line" ]]; do
        if [[ "$line" =~ ^alias[[:space:]]+(claude[0-9a-zA-Z_-]
        +)=.*CLAUDE_CONFIG_DIR=([[:space:]]\\")+ ]]; then
            local a="${BASH_REMATCH[1]}" d="${BASH_REMATCH[2]}"
            d="${d//\"/}"
            d="${d/#$HOME/$HOME}"
            d="${d/#~/$HOME}"
            cma_write_alias "$a" "$d"
            printf '# migrated to %s: %s\n' "$ALIAS_FILE" "$line" >>
                "$tmp"
            changed=1
        else
            printf '%s\n' "$line" >> "$tmp"
        fi
    done < "$rc"
    if (( changed )); then
        # migrate removes NOTHING (it only comment-prefixes lines in
        # place), so the
        # intended removal count is 0 – cma_rc_safe_rewrite's content-
        # loss gate then
        # REFUSES any candidate that lost a line (e.g. a newline-less
        # final line the
        # old read loop could drop) and parks it instead of
        # publishing. The committer
        # also takes the pristine .cma-orig backup, superseding the
        # bespoke .preunify.
        if cma_rc_safe_rewrite "$rc" "$tmp" 0; then

```

```

        cma_log "migrated inline claude* aliases in $src ->
        $ALIAS_FILE"
    else
        cma_warn "skipped migrating inline aliases in $src (rejected
        or unbackupable)"
    fi
else
    rm -f "$tmp"
fi
}
for rc in "${CMA_RC_FILES[@]}; do
    migrate_inline_aliases "$rc"
done

# 4b. Build + install the Go compatibility proxy (cma-proxy) for
#       provider
# compatibility (helixagent Hermes tool-call recovery + poe/kimi/
#       sarvam
# request-schema fixes). Replaces the former per-provider python
#       proxies.
# Recorded (not warn-and-forget) for the same reason as 2b:
#       without it the
# helixagent/poe/kimi/sarvam aliases silently lose their compat
#       shims, which is
# a DEGRADED product the operator must be told about by name –
#       step 7 reports
# it. It is a DEGRADED (not FAILED) capability: those aliases
#       still launch
# against their direct endpoint, so it must not by itself fail the
#       install.
if ! bash "$LIB_DIR/claude-proxy-build.sh"; then
    CMA_INSTALL_DEGRADED+=("compatibility proxy (cma-proxy, Go) did
    NOT build – helixagent/poe/kimi/sarvam aliases run WITHOUT
    their compat shims (Hermes tool-call recovery + request-
    schema fixes inactive). Fix: install Go, then run: claude-
    proxy-build")
fi

# 4c. Provider session-sync hook + an install-time sync (soft –
#       the host may
# lack keys/network at install time, so a failure here is non-
#       fatal). The hook
# refreshes provider aliases from cache on every new shell (no
#       network) and
# kicks a detached full sync when the cache is stale (§11.4.89).
cma_install_session_hook
if [[ -x "$LIB_DIR/claude-providers.sh" ]]; then
    cma_log "running claude-providers sync (install-time; soft)"
    ( "$LIB_DIR/claude-providers.sh" sync ) || cma_warn "provider
    sync skipped (no keys/network?)"
fi

# 5. Unify whatever accounts exist now.
if (( $(cma_detect_accounts | wc -l) > 0 )); then
    cma_log "running claude-unify.sh"

```



```

"$LIB_DIR/claude-unify.sh"
else
    cma_warn "no ~/${ACCOUNT_PREFIX}* dirs detected – add one with
        claude-add-account.sh"
fi

# 5b. Same unification for the Kimi family (v1.27.0), over
    $SHARED_DIR/kimi/.
# Only when Kimi homes are actually present – a fresh install with
    none should
# keep the same single cma_warn the Claude side emits, not two.
if [[ -x "$LIB_DIR/kimi-unify.sh" ]] && (( $
    (cma_detect_kimi_accounts | wc -l) > 0 )); then
    cma_log "running kimi-unify.sh"
    "$LIB_DIR/kimi-unify.sh"
fi

# 6. Refresh docs if pandoc is available.
if command -v pandoc >/dev/null 2>&1 && [[ -f "$HOME/Documents/
    Claude_Multi_Account_Fine_Tuning.md" ]]; then
    cma_log "running claude-export-docs.sh"
    "$LIB_DIR/claude-export-docs.sh" || cma_warn "doc export failed
        (continuing)"
fi

# 7. SELF-VERIFY, then report honestly.
#
# The banner below is the toolkit's success claim. It is now
    EARNED, not
# printed unconditionally: claude-install-verify probes the REAL
    artifacts the
# runtime will resolve (it does not grep our own source), so
    "[done]" can no
# longer appear over a product whose router is missing or is the
    npm
# doppelganger. Any recorded build failure from 2b, or any verify
    FAIL, exits
# non-zero with the actionable fix.
if [[ ! -f "$LIB_DIR/claude-install-verify.sh" ]]; then
    # Say WHICH file is missing rather than letting bash emit a bare
    # "No such file or directory" that reads like a bug in the
        installer.

    # A checkout without the gate cannot demonstrate a good
        install, so this is
    # a failure, not a skip – never assume-good on a missing
        verifier.
    CMA_INSTALL_FAILURES+=("post-install self-verify is MISSING
        ($LIB_DIR/claude-install-verify.sh) – this checkout cannot
        prove the install is good. Fix: re-pull the repository.")
elif ! bash "$LIB_DIR/claude-install-verify.sh"; then
    CMA_INSTALL_FAILURES+=("post-install self-verify FAILED – see
        the [FAIL] lines above")
fi

```

```

if (( ${#CMA_INSTALL_DEGRADED[@]} > 0 )); then
    printf '\n[warn] claude-multi-account installed with DEGRADED
        optional components:\n' >&2
    for _d in "${CMA_INSTALL_DEGRADED[@]}"; do printf '    - %s\n'
        "$_d" >&2; done
fi

if (( ${#CMA_INSTALL_FAILURES[@]} > 0 )); then
    printf
        '\n[FAILED] claude-multi-account install did NOT complete
        successfully.\n\n' >&2
    for _f in "${CMA_INSTALL_FAILURES[@]}"; do printf '    - %s\n'
        "$_f" >&2; done
    printf '\nNothing above was silently ignored. Fix the items and
        re-run: %s\n' "$LIB_DIR/install.sh" >&2
    printf 'To re-check without a full re-install:  claude-install-
        verify\n' >&2
    exit 1
fi

cat <<EOF

[done] claude-multi-account installed.

    Scripts on PATH:  $BIN_DIR/claude-{unify,add-account,remove-
        account,list-accounts,rollback,export-docs}
    Kimi on PATH:     $BIN_DIR/kimi-{unify,add-account,remove-
        account,list-accounts,rollback,providers}
    Alias file:       $ALIAS_FILE
    Shared store:     $SHARED_DIR

Open a new shell (or run: source
    $ALIAS_FILE) so the aliases load, then:
    claude-list-accounts          # see what's wired up
    claude-add-account            # add another account
                                interactively
    kimi-add-account --alias kimi1 # provision a Kimi Code account
                                (device flow)
EOF

```

8.9 tests/lib/assert.sh

```

#!/usr/bin/env bash
# assert.sh – small assertion helpers for the test suite. Each
# helper
# emits `[PASS]` or `[FAIL]` lines and increments counters in the
# caller.
# A test file should source this once and then call the helpers
# inside
# `it` blocks.

TESTS_PASSED="${TESTS_PASSED:-0}"
TESTS_FAILED="${TESTS_FAILED:-0}"
TESTS_FAILURES=()

```

```

CURRENT_TEST="${CURRENT_TEST:- (unknown)}"

# Mark the start of an individual test case. Use as:  it "merges
            history.jsonl"
it() {
    CURRENT_TEST="$*"
    printf '\n  • %s\n' "$CURRENT_TEST"
}

_pass() {
    TESTS_PASSED=$((TESTS_PASSED + 1))
    printf '    \033[32m[PASS]\033[0m %s\n' "$1"
}

_fail() {
    TESTS_FAILED=$((TESTS_FAILED + 1))
    TESTS_FAILURES+=("$CURRENT_TEST :: $1")
    printf '    \033[31m[FAIL]\033[0m %s\n' "$1"
    [[ -n "${2:-}" ]] && printf '        %s\n' "$2"
}

assert_eq() {
    local expected="$1" actual="$2" msg="${3:-equality}"
    if [[ "$expected" == "$actual" ]]; then _pass "$msg ($expected)"
    else _fail "$msg" "want=$expected got=$actual"; fi
}

assert_file() {
    local path="$1" msg="${2:-file exists}"
    if [[ -f "$path" ]]; then _pass "$msg: $path"
    else _fail "$msg" "missing: $path"; fi
}

assert_dir() {
    local path="$1" msg="${2:-dir exists}"
    if [[ -d "$path" ]]; then _pass "$msg: $path"
    else _fail "$msg" "missing dir: $path"; fi
}

# Portable realpath for the harness. BSD/macOS `readlink` has no
#   -f, so
# `readlink -f` is a hard error there (empty output) – which would
#   make the
# symlink assertions below pass/fail spuriously on macOS. Self-
#   contained
# (mirrors lib.sh's cma_realpath) so assert.sh works even in test
#   files that
# don't source lib.sh (e.g. test_add_remove.sh).
_assert_realpath() {
    local p="$1" t dir base
    while [ -L "$p" ]; do
        t="$(readlink "$p")"
        case "$t" in /*) p="$t" ;; *) p="$(dirname "$p")/$t" ;; esac
    done
}

```

```

done
dir="$(cd "$(dirname "$p")" 2>/dev/null && pwd -P)" || dir="$(
    (dirname "$p")"
base="$(basename "$p")"
printf '%s/%s\n' "$dir" "$base"
}

assert_symlink_to() {
    local link="$1" expected="$2" msg="${3:-symlink target}"
    if [[ ! -L "$link" ]]; then
        _fail "$msg" "not a symlink: $link"
        return
    fi
    local actual; actual="$( _assert_realpath "$link" )"
    local want; want="$( _assert_realpath "$expected" )"
    if [[ "$actual" == "$want" ]]; then _pass "$msg: $link ->
        $expected"
    else _fail "$msg" "want=$want got=$actual"; fi
}

assert_not_symlink() {
    local p="$1" msg="${2:-not a symlink}"
    if [[ -L "$p" ]]; then _fail "$msg" "is a symlink: $p"
    else _pass "$msg: $p"; fi
}

assert_file_contains() {
    local path="$1" needle="$2" msg="${3:-file contains}"
    if grep -F -q -- "$needle" "$path" 2>/dev/null; then _pass
        "$msg: '$needle' in $(basename "$path")"
    else _fail "$msg" "'$needle' not found in $path"; fi
}

assert_file_not_contains() {
    local path="$1" needle="$2" msg="${3:-file lacks}"
    if grep -F -q -- "$needle" "$path" 2>/dev/null; then _fail
        "$msg" "'$needle' found in $path"
    else _pass "$msg: '$needle' absent from $(basename "$path")"; fi
}

assert_jq() {
    local path="$1" expr="$2" expected="$3" msg="${4:-jq check}"
    local actual; actual="$(jq -r "$expr" "$path" 2>/dev/null ||
        echo '<jq-error>')"
    if [[ "$actual" == "$expected" ]]; then _pass "$msg ($expr =
        $expected)"
    else _fail "$msg" "expr=$expr want=$expected got=$actual"; fi
}

assert_lines() {
    local path="$1" expected="$2" msg="${3:-line count}"
    local actual; actual="$(wc -l < "$path" 2>/dev/null || echo 0)"
    actual="${actual// /}"

```

```

if [[ "$actual" == "$expected" ]]; then _pass "$msg ($expected
lines)"
else _fail "$msg" "want=$expected got=$actual lines in $path";
fi
}

# assert_fn_from – assert that a shell FUNCTION was defined by an
EXPECTED file.
#
#   assert_fn_from FN EXPECTED_FILE [MSG]
#       Check FN as it is defined in the CALLER's shell.
#   assert_fn_from --source FILE FN [MSG]
#       Source FILE in a throwaway subshell first, then check FN's
provenance
#       there. For call sites whose `source` lives inside a
#       `( ... )` subshell:
#       only the sourcing is subshelled, so the pass/fail still
lands in the
#       caller's counters and reaches `summary`.
#
# WHY THIS EXISTS – the guard that failed under the conditions of
its own bug.
# The login profile exports BASH_ENV=~/.bashrc, so EVERY non-
interactive bash –
# including run-all.sh's `bash "$f"` per test – sources the
PRODUCTION alias file
# before the test's first line. cma_run and cma_run_provider are
therefore
# ALREADY defined, from the host, in every test shell. A test that
sources its
# sandbox $ALIAS_FILE and then calls the wrapper looks correct,
but the tests
# run with `set +e`: if that source silently fails, execution
falls straight
# through to the fully-working HOST function and the test passes –
while
# grading live host code instead of the code under test. Tests
written to prove
# cma_ensure_alias_file emits a correct body then report green for
exactly the
# regression they exist to catch.
#
# `shopt -s extdebug` makes `declare -F NAME` print "name lineno
file", which is
# the only way to see WHERE a function came from. Naming both the
expected and
# the actual file in the failure message is deliberate: "wrong
provenance" is
# useless without "...it came from the host alias file instead".
assert_fn_from() {
    local file="" fn expected msg src had_extdebug
    if [[ "${1:-}" == "--source" ]]; then
        file="$2"; fn="$3"; expected="$2"; msg="${4:-function
provenance}"
    else

```

```

    fn="$1"; expected="$2"; msg="{3:-function provenance}"
fi

if [[ -n "$file" ]]; then
    # set +u: an alias file may reference unset vars while being
    # sourced.
    src="$( set +u; source "$file" >/dev/null 2>&1
        shopt -s extdebug; declare -F "$fn" 2>/dev/null | awk
            '{print $3}' )"
else
    had_extdebug="$(shopt -p extdebug)"
    shopt -s extdebug
    src="$(declare -F "$fn" 2>/dev/null | awk '{print $3}')"
    eval "$had_extdebug"
fi

if [[ -z "$src" ]]; then
    _fail "$msg" "$fn is not defined at all (expected it from
        $expected)"
elif [[ "$src" == "$expected" ]]; then
    _pass "$msg: $fn came from $expected"
else
    _fail "$msg" "$fn was defined by $src, NOT by the expected
        $expected"
fi
}

assert_exit() {
    local expected="$1"; shift
    local out; out="$("$@" 2>&1)"; local rc=$?
    if (( rc == expected )); then _pass "exit $expected for: $"
    else _fail "exit code mismatch" "want=$expected got=$rc cmd=$*"
        out=$out"; fi
}

summary() {
    echo
    if (( TESTS_FAILED == 0 )); then
        printf '\033[32m✓ %d passed, 0 failed\033[0m\n'
            "$TESTS_PASSED"
        return 0
    fi
    printf '\033[31m✗ %d failed, %d passed\033[0m\n'
        "$TESTS_FAILED" "$TESTS_PASSED"
    printf '\nFailures:\n'
    for f in "${TESTS_FAILURES[@]}; do printf '  - %s\n' "$f"; done
    return 1
}

```

8.10 tests/lib/sandbox.sh

```
#!/usr/bin/env bash
# sandbox.sh – utilities for running scripts against a temp HOME
# so the
# real ~/.claude state is never touched.
#
# Usage from a test file:
#
#   source "$TESTS_DIR/lib/sandbox.sh"
#   make_sandbox          # creates $SANDBOX_HOME, exports HOME,
#                           etc.
#   make_account acct1    # creates a fake ~/.claude-acct1 inside
#                           sandbox
#   make_account acct2 --plugins
#   run_unify             # runs claude-unify.sh against the
#                           sandbox
#   cleanup_sandbox       # rm -rf the sandbox (registered with
#                           trap)

SCRIPTS_DIR="${SCRIPTS_DIR:-$HOME/Documents/scripts}"

make_sandbox() {
    # Remember the real HOME before overriding, so vendored Go is
    # reachable.
    export CMA_REAL_HOME="${HOME:-/home/$(whoami)}"
    SANDBOX_HOME="$(mktemp -d "${TMPDIR:-/tmp}/cma-test.XXXXXX")"
    # Override every env var the toolkit reads from. Pointing HOME
    # at the
    # sandbox is the main switch; the other vars only matter if a
    # caller
    # tests defaults explicitly.
    export HOME="$SANDBOX_HOME"
    export SHARED_DIR="$SANDBOX_HOME/.claude-shared"
    export ALIAS_FILE="$SANDBOX_HOME/.local/share/claude-multi-
account/aliases.sh"
    export DEFAULT_DIR="$SANDBOX_HOME/.claude"
    export ACCOUNT_PREFIX=".claude-"
    export CLAUDE_BIN="/usr/bin/true" # dummy; we never actually
    launch claude
    # Point at the real vendored Go toolchain so ccr-build/proxy-
    build work in
    # the sandbox (the sandbox HOME doesn't have ~/.local/share/
    claude-go).
    if [[ -x "$CMA_REAL_HOME/.local/share/claude-go/bin/go" ]]; then
        export VENDORED_GO="$CMA_REAL_HOME/.local/share/claude-go/bin/
go"
    fi
    mkdir -p "$DEFAULT_DIR" "$SANDBOX_HOME/.local/bin"
    # Self-check: prove the switch actually took before any test
    writes.
    assert_sandboxed
    trap 'cleanup_sandbox' EXIT
}
```

```

# assert_sandboxed – abort loudly unless $HOME is a sandbox from
#   make_sandbox.
#
# This exists because a test that writes to $HOME without a
#   sandbox does not
# merely fail: in the REAL home, ~/.local/bin/claude-* are
#   SYMLINKS into this
# repo (install.sh links every claude-*.sh there), and `cat >`
#   follows symlinks.
# A single unsandboxed stub write therefore destroys the
#   production script it
# points at. That is not hypothetical – it silently truncated
# scripts/claude-session.sh from 201 lines to an 8-line stub and
# scripts/claude-sync-state.sh from 103 lines to 2, which broke
#   every provider
# alias with "No conversation found with session ID:
#   11111111-2222-...".
#
# Exits 99 rather than returning: there is no safe way to
#   continue.
assert_sandboxed() {
  case "${basename "${HOME:-/}"}" in
    cma-test.*) return 0 ;;
  esac
  echo "FATAL: test tried to use HOME=${HOME:-<unset>}, which is
    not a make_sandbox temp dir." >&2
  echo "      Refusing to continue: writes under the real \${HOME}
    follow symlinks into the repo" >&2
  echo "      and would destroy production scripts. Call
    make_sandbox first." >&2
  exit 99
}

# sandbox_stub PATH – install an executable stub (content on
#   stdin) inside the
# sandbox. Use this instead of `cat > "${HOME}/..."` for anything
#   under
# $HOME/.local/bin.
#
# Three guarantees a bare `cat >` does not give:
# 1. $HOME is a real sandbox (assert_sandboxed).
# 2. The target is INSIDE that sandbox – a path that escaped it
#   is a bug.
# 3. An existing symlink is REMOVED before writing, never
#   written through.
# Guarantee 3 is the one that matters: it is what turns
#   "clobbered a
#   production script" into "replaced a link inside a temp
#   dir".
sandbox_stub() {
  local target="$1"
  assert_sandboxed
  case "$target" in
    "$HOME"/*) : ;;

```



```

    *) echo "FATAL: sandbox_stub target escapes the sandbox:
        $target" >&2; exit 99 ;;
esac
mkdir -p "$(dirname "$target")"
# Break the link rather than following it.
[[ -L "$target" ]] && rm -f -- "$target"
cat > "$target"
chmod +x "$target"
}

cleanup_sandbox() {
[[ -n "${SANDBOX_HOME:-}" ]&& -d "$SANDBOX_HOME" ]] || return 0
# Safety net: only delete if the path matches what mktemp would
# have
# produced (cma-test.* somewhere under a tempdir root).
# Different
# systems put mktemp output in different places (/tmp on most
# Linux,

# /tmp/.private/<user>/ on this host, /var/folders/... on
# macOS) so
# we anchor on the cma-test. prefix rather than the parent.
case "$(basename "$SANDBOX_HOME")" in
cma-test.*)
# Go installs its module cache read-only (0444 files, 0555
# dirs), so a
# plain `rm -rf` fails with EACCES partway through and
# leaves a
# ~13.5k-file `go/pkg/mod` skeleton behind on EVERY run of
# any
# Go-building test. Because the sandbox IS $HOME, each such
# test also
# re-downloads the whole cache – 500-850MB per sandbox,
# unshared – so the
# residue is the dominant term in sandbox leakage (109
# orphans totalling
# 35GB were measured before this was fixed).
#

# Restore write permission first so the removal can
# actually complete.
# Safe by construction: this only ever runs inside a path
# whose basename
# matched cma-test.*, i.e. a mktemp dir this suite created.
chmod -R u+w -- "$SANDBOX_HOME" 2>/dev/null || true
rm -rf -- "$SANDBOX_HOME"
;;
*) echo "refusing to rm sandbox at unexpected path:
    $SANDBOX_HOME" >&2 ;;
esac
}

# make_account NAME [--plugins] [--settings JSON] [--history
# "line1|line2"] [--memory KEY:VALUE...]

```

```

make_account() {
    local name="$1" with_plugins=0 settings_json="" history_lines=()
    shift
    local memory_entries=() todo_entries=()
    while (( $# )); do
        case "$1" in
            --plugins) with_plugins=1; shift ;;
            --settings) settings_json="$2"; shift 2 ;;
            --history) IFS='|' read -r -a history_lines <<< "$2"; shift
                2 ;;
            --memory) memory_entries+=("$2"); shift 2 ;;
            --todo) todo_entries+=("$2"); shift 2 ;;
            *) echo "make_account: unknown arg $1" >&2; return 1 ;;
        esac
    done

    local dir="$HOME/${ACCOUNT_PREFIX}${name}"
    mkdir -p "$dir/projects/-home-test/memory" "$dir/todos" "$dir/
tasks" \
    "$dir/plans" "$dir/file-history" "$dir/paste-cache" \
    "$dir/shell-snapshots" "$dir/session-env" "$dir/
telemetry" \
    "$dir/sessions" "$dir/backups" "$dir/cache" "$dir/
plugins"

    # Fake credentials so list-accounts reports CREDs:yes.
    printf '{"account":"%s"}\n' "$name" > "$dir/.credentials.json"
    printf '{"name":"%s"}\n' "$name" > "$dir/.claude.json"

    # A unique transcript so the union-merge step has something to
    merge.
    printf '{"role":"user","msg":"hi from %s"}\n' "$name" \
    > "$dir/projects/-home-test/${uuidgen 2>/dev/null || echo
    "session-${name}").jsonl"

    # Memory + todo content if requested.
    local m
    for m in ${memory_entries[@]+"${memory_entries[@]}"}; do
        local k="${m%*:}" v="${m#*:}"
        printf -- '%s\n' "$v" > "$dir/projects/-home-test/memory/${
k}.md"
    done
    local t
    for t in ${todo_entries[@]+"${todo_entries[@]}"}; do
        printf '{"subject":"%s"}\n' "$t" > "$dir/todos/${uuidgen 2>/
dev/null || echo "todo-${name}-${t}").json"
    done

    # History lines (used to test concat+dedup later).
    if (( ${#history_lines[@]} )); then
        printf '%s\n' "${history_lines[@]}" > "$dir/history.jsonl"
    fi
}

```

```

# Settings JSON if specified, else a minimal default.
if [[ -n "$settings_json" ]]; then
    printf '%s\n' "$settings_json" > "$dir/settings.json"
else
    printf '{"enabledPlugins":{"%s-plugin":true}}\n' "$name" >
        "$dir/settings.json"
fi

if (( with_plugins )); then
    mkdir -p "$dir/plugins/cache/test-marketplace/test-plugin/
        1.0.0"
    mkdir -p "$dir/plugins/marketplaces/test-marketplace"
    cat > "$dir/plugins/installed_plugins.json" <<EOF
{
    "version": 2,
    "plugins": {
        "test-plugin@test-marketplace": [
            {
                "scope": "project",
                "installPath": "$dir/plugins/cache/test-marketplace/test-
                    plugin/1.0.0",
                "version": "1.0.0"
            }
        ]
    }
}
EOF
    cat > "$dir/plugins/known_marketplaces.json" <<EOF
{
    "test-marketplace": {
        "source": {"source": "github", "repo": "example/test"},
        "installLocation": "$dir/plugins/marketplaces/test-
            marketplace"
    }
}
EOF
fi

echo "$dir"
}

run_unify() {
    "$SCRIPTS_DIR/claude-unify.sh" "$@"
}

run_add_account() {
    "$SCRIPTS_DIR/claude-add-account.sh" "$@"
}

run_remove_account() {
    "$SCRIPTS_DIR/claude-remove-account.sh" "$@"
}

```

```

run_list_accounts() {
    "$SCRIPTS_DIR/claude-list-accounts.sh" "$@"
}

run_rollback() {
    "$SCRIPTS_DIR/claude-rollback.sh" "$@"
}

```

8.11 tests/run-all.sh

```

#!/usr/bin/env bash
# run-all.sh – discover every test_*.sh sibling, run each in its
#           own
# subshell, and tally pass/fail.
#
# Exit code is 0 only if every test file's `summary` returned 0.

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
export SCRIPTS_DIR

# SERIALIZATION: two suite runs must never overlap – a repo that
# mutates while
# its own tests execute produces results that cannot be
# reproduced. Silent on
# the normal path; prints only when it actually contends. When
# run-proof.sh is
# the caller this legitimately inherits the lock instead of
# deadlocking on it.
# shellcheck source=lib/suite-lock.sh
source "$TESTS_DIR/lib/suite-lock.sh"
cma_suite_lock_acquire suite

FILES=()
if (( $# )); then
    for arg in "$@"; do FILES+=("$TESTS_DIR/test_${arg}.sh"); done
else
    # Portable discovery: `mapfile`/`readarray` are bash 4+ and
    # absent on the
    # bash 3.2 that ships with macOS, where this test runner must
    # also run.
    while IFS= read -r f; do FILES+=("$f"); done \
        << (find "$TESTS_DIR" -maxdepth 1 -name 'test_*.sh' -type f |
            sort)
fi

PASSED=0 FAILED=0 SKIPPED=0 FAILED_FILES=()

# "${FILES[@]+...}" guards against an empty array under `set -u`
# on bash 3.2,

```

```

# which (unlike bash 4.4+) treats "${FILES[@]}" on an empty array
as unbound.
for f in "${FILES[@]}${FILES[@]}"; do
    [[ -f "$f" ]] || { echo "missing: $f" >&2; FAILED=$((
        FAILED+1)); FAILED_FILES+=("$f"); continue; }
    printf '\n\033[1m==> %s\033[0m\n' "$(basename "$f")"

    # Capture output while still streaming it, so we can
    cross-check the printed
    # result against the exit code (see harness-integrity check
    below).
    _out="$(mktemp "${TMPDIR:-/tmp}/cma-runall.XXXXXX")"
    bash "$f" 2>&1 | tee "$_out"
    _rc=${PIPESTATUS[0]}

    # HARNESS INTEGRITY (defense in depth): a file's exit code is
    the ONLY thing
    # this runner tallies, and `summary` is what turns TESTS_FAILED
    into a

    # non-zero exit. A file that prints [FAIL] but exits 0
    means its summary call
    # is missing/bypassed – the failures would otherwise be counted
    as a PASS.
    # This really happened: test_providers.sh lacked `summary` and
    hid 5 real
    # failures behind a green run. Never trust a 0 that contradicts
    the output.
    if (( _rc == 0 )) && grep -q '\[FAIL\]' "$_out"; then
        printf '\033[31m[HARNESS] %s printed [FAIL] but exited 0 –
            missing summary?\033[0m\n' \
            "$(basename "$f")" >&2
        _rc=1
    fi
    # Visibility: a file that SKIPS on a missing prerequisite exits
    0 and would
    # otherwise be indistinguishable from a genuine pass in "ALL
    GREEN".
    if (( _rc == 0 )) && grep -q '^SKIP' "$_out"; then
        SKIPPED=$((SKIPPED + 1))
    fi
    rm -f "$_out"

    if (( _rc == 0 )); then
        PASSED=$((PASSED + 1))
    else
        FAILED=$((FAILED + 1))
        FAILED_FILES+=("$(basename "$f")")
    fi
done

echo
echo "=====
```

```

echo "Test files: $((PASSED + FAILED))    passed: $PASSED
      failed: $FAILED    (skipped-prereq: $SKIPPED)"
if (( FAILED )); then
    echo "Failed files:"
    for f in "${FAILED_FILES[@]}; do echo "  - $f"; done
    exit 1
fi
echo "ALL GREEN"

```

8.12 tests/test_lib.sh

```

#!/usr/bin/env bash
# test_lib.sh – unit tests for the helper functions in lib.sh.
set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

# shellcheck source=lib/assert.sh
source "$TESTS_DIR/lib/assert.sh"
# shellcheck source=lib/sandbox.sh
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox
# shellcheck source=../lib.sh
source "$SCRIPTS_DIR/lib.sh"
# lib.sh enables `set -e`. The test harness is intentionally
# tolerant of
# non-zero exits (we assert on them), so turn it back off here.
set +e

it "cma_validate_alias accepts well-formed names"
( set -e; cma_validate_alias "claude3" ); assert_eq 0 $? "claude3"
( set -e; cma_validate_alias "work_acct-1" ); assert_eq 0 $?
  "work_acct-1"

it "cma_validate_alias rejects invalid names"
( cma_validate_alias "3claude" >/dev/null 2>&1 ); assert_eq 1 $?
  "rejects digit-leading"
( cma_validate_alias "bad name" >/dev/null 2>&1 ); assert_eq 1 $?
  "rejects spaces"
( cma_validate_alias "" >/dev/null 2>&1 ); assert_eq 1 $?
  "rejects empty"

it "cma_suggest_alias starts at claude1 when nothing exists"
suggestion="$(cma_suggest_alias)"
assert_eq "claude1" "$suggestion" "first suggestion"

it "cma_suggest_alias increments past existing claudeN aliases"
cma_write_alias claude1 "$HOME/.claude-acct1"
cma_write_alias claude2 "$HOME/.claude-acct2"
cma_write_alias claude5 "$HOME/.claude-acct5"    # gap

```

```

suggestion="$(cma_suggest_alias)"
assert_eq "claude6" "$suggestion" "skips past highest, not count"

it "cma_write_alias is idempotent – rewriting same alias doesn't
    duplicate"
cma_write_alias claude1 "$HOME/.claude-acct1" # second write
count="$(grep -c '^alias claude1=' "$ALIAS_FILE")"
assert_eq "1" "$count" "one alias line for claude1"

it "cma_remove_alias removes the line"
cma_remove_alias claude5
assert_file_not_contains "$ALIAS_FILE" "alias claude5=" "claude5
    removed"

it "cma_ensure_alias_file sources from the shell rc file"
# lib.sh manages .zshrc on macOS and .bashrc + .zshrc on Linux
(CMA_RC_FILES).
# Assert against the platform-appropriate target, selected the
same way lib.sh
# selects it, so the test is correct on both OSes.
if [[ "$(uname -s)" == "Darwin" ]]; then RC_FILE="$HOME/.zshrc";
    else RC_FILE="$HOME/.bashrc"; fi
touch "$RC_FILE"
rm -f "$ALIAS_FILE"
cma_ensure_alias_file
assert_file "$ALIAS_FILE" "alias file created"
assert_file_contains "$RC_FILE" "source \"$ALIAS_FILE\" \"rc file
    gets source line"

it "cma_detect_accounts skips the shared store"
mkdir -p "$HOME/.claude-shared" "$HOME/.claude-acct1"
found=(); while IFS= read -r _l; do found+=("$_l"); done <
    <(cma_detect_accounts)
joined="${found[*]:-}"
cond=1; [[ "$joined" == *".claude-acct1"* ]] && cond=0; assert_eq
    0 "$cond" "finds .claude-acct1"
cond=1; [[ "$joined" != *".claude-shared"* ]] && cond=0;
    assert_eq 0 "$cond" "excludes .claude-shared"

it "cma_detect_accounts excludes non-Claude .claude-* dirs
    (e.g. .claude-server-commander)"
# Mimic the real-world false positive seen on mistborn.local: an
MCP server
# config dir whose name happens to start with .claude- but has
only its own
# config files, no Claude markers.
mkdir -p "$HOME/.claude-server-commander"
printf '{}\n' > "$HOME/.claude-server-commander/config.json"
printf '{}\n' > "$HOME/.claude-server-commander/feature-
    flags.json"
found=(); while IFS= read -r _l; do found+=("$_l"); done <
    <(cma_detect_accounts)
joined="${found[*]:-}"

```

```

cond=1; [[ "$joined" != *.claude-server-commander* ]] &&
    cond=0; assert_eq 0 "$cond" "excludes .claude-server-
commander"
cond=1; [[ "$joined" == *.claude-acctl* ]] && cond=0; assert_eq
    0 "$cond" "still finds the legit empty account"

it "cma_detect_accounts includes a populated account dir even if
    it has foreign config too"
# A real account dir with the Claude marker (projects/) shouldn't
    get
# falsely excluded just because some other file happens to be
    there.
mkdir -p "$HOME/.claude-real/projects"
printf '{}\n' > "$HOME/.claude-real/some-other-tool.json"
found=(); while IFS= read -r _l; do found+=("$_l"); done <
    <(cma_detect_accounts)
joined="{${found[*]}:-}"
cond=1; [[ "$joined" == *.claude-real* ]] && cond=0; assert_eq
    0 "$cond" "finds .claude-real"

it "cma_realpath resolves a symlink chain to its canonical target
    (no readlink -f)"
mkdir -p "$HOME/rp/real"
: > "$HOME/rp/real/file"
ln -s "$HOME/rp/real/file" "$HOME/rp/link1"
ln -s "$HOME/rp/link1" "$HOME/rp/link2"
    # chain: link2 -> link1 -> real/file
got="$ (cma_realpath "$HOME/rp/link2")"
want="$ (cd "$HOME/rp/real" && pwd -P)/file"
assert_eq "$want" "$got" "cma_realpath follows the symlink chain"
# A plain (non-symlink) path canonicalizes to itself.
got2="$ (cma_realpath "$HOME/rp/real/file")"
assert_eq "$want" "$got2" "cma_realpath is identity on a real
    path"

it "no runtime script INVOKES 'readlink -f' (absent on BSD/macOS)"
# Strip comments first so explanatory comments mentioning the flag
    don't count;
# we only care about real invocations. kimi-*.sh is in the same
    loop (v1.27.0)
# so the family can never drift into a GNU-only construct.
hits="$ (for f in "$SCRIPTS_DIR"/lib.sh "$SCRIPTS_DIR"/install.sh
    "$SCRIPTS_DIR"/claude-*.sh "$SCRIPTS_DIR"/kimi-*.sh; do
    sed 's/#.*//' "$f"; done 2>/dev/null | grep -c 'readlink
    -f')"
assert_eq 0 "$hits" "zero 'readlink -f' invocations in runtime
    scripts"

it "no committed proof artifact contains a literal secret"
# Regression guard for the H2 incident: live proof files (opcode
    debug config /
# mcp list) once committed a real API key + a DB connection-string
    password.
# We count *suspect* lines rather than print them, so a failure
    never re-echoes a

```



```

# secret into the test log. Provider-key prefixes + URL
# user:password@ are the
# signatures; redacted placeholders carry the word REDACTED and
# are excluded.
proof_dir="$SCRIPTS_DIR/tests/proof"
if [[ -d "$proof_dir" ]]; then
    # Prefixes match only at a TOKEN BOUNDARY (line start or a non-
    # word char
    # before them): a real leaked key appears after a quote/=/:/
    # space (e.g.
    # "api_key":"sk-ant-..."), never embedded mid-identifier. Without
    # the boundary,
    # innocent identifiers captured in build/log noise false-
    # positive – e.g. `re_`
    # inside a Go-cache path `retire_connection_id_frame_test.go`,
    # or `secret_`
    # inside `my_secret_value`.
    leaks="$(grep -rIE \
        -e '(^|^[^A-Za-z0-9_-])(sk-ant-|sk-|gsk_|xai-|hf_|AIza|xoxb-|
        xoxp-|xoxs-|pc-|re_|secret_|ghp_|github_pat_|AKIA)[A-Za-
        z0-9_-]{12,}' \
        -e ':[^:/@ "]+:[^@/ "]{4,}@' \
        -e 'eyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]
        {10,}' \
        "$proof_dir" 2>/dev/null | grep -vc 'REDACTED' || true)"
    [[ -z "$leaks" ]] && leaks=0
    assert_eq 0 "$leaks" "proof dir free of literal secrets
    (suspect-line count)"
else
    _pass "no proof dir on this host (nothing to scan)"
fi

# Regression guard for the token-boundary anchor (a Go-cache path
# `retire_connection_id_frame_test.go` in captured build noise
# once false-
# positived on the `re_` prefix). The scanner MUST ignore a secret
# prefix
# embedded mid-identifier, yet still catch a real key at a token
# boundary.
it "proof-secret scanner: boundary-anchored (embedded prefix
    ignored, real key caught)"
_scan_fx="$(mktemp -d "${TMPDIR:-/tmp}/cma-scanfx.XXXXXX)"
{
    echo "rm: cannot remove '/go/pkg/mod/quic-go/internal/wire/
    retire_connection_id_frame_test.go'"
    echo "my_secret_value_identifier_here = 1"
} > "$_scan_fx/noise.log"
printf '"api_key":"sk-ant-abc123def456ghi789jkl"\n' > "$_scan_fx/
leak.log"
_scan_re='(^|^[^A-Za-z0-9_-])(sk-ant-|sk-|gsk_|xai-|hf_|AIza|xoxb-|
xoxp-|xoxs-|pc-|re_|secret_|ghp_|github_pat_|AKIA)[A-Za-
z0-9_-]{12,}'
_scan_noise="$(grep -cE "$_scan_re" "$_scan_fx/noise.log" ||
true)"
_scan_leak="$(grep -cE "$_scan_re" "$_scan_fx/leak.log" || true)"

```

```

assert_eq 0 "${_scan_noise:-0}" "embedded prefixes (retire_/
    my_secret_) are NOT flagged"
assert_eq 1 "${_scan_leak:-0}" "a real boundary-anchored sk-ant-
    key IS flagged"
rm -rf "$_scan_fx"

# cma_merge_claude_json must UNION the projects subtree across
# accounts while
# keeping each account's private auth keys (userID/
#   oauthAccount/...) to ITSELF.
# A regression here would either lose sessions (no union) or leak
#   credentials
# between accounts (CRITICAL). Locks the property verified by hand
#   this session.
it
    "cma_merge_claude_json: private keys stay per-account;
    projects union (no credential leak)"
if command -v jq >/dev/null 2>&1; then
    mj_a="$SANDBOX_HOME/.mrg-a"; mj_b="$SANDBOX_HOME/.mrg-b"; mkdir
    -p "$mj_a" "$mj_b"
    printf '%s\n' '{"userID":"UID-
        A","oauthAccount":"a@x","projects":{"pa":{"v":1}}}' >
        "$mj_a/.claude.json"
    printf '%s\n' '{"userID":"UID-
        B","oauthAccount":"b@x","projects":{"pb":{"v":2}}}' >
        "$mj_b/.claude.json"
    cma_merge_claude_json "$mj_a" "$mj_b" >/dev/null 2>&1
    assert_eq "UID-A" "$(jq -r .userID "$mj_a/.claude.json")"
        "account A keeps its OWN userID after merge"
    assert_eq "UID-B" "$(jq -r .userID "$mj_b/.claude.json")"
        "account B keeps its OWN userID after merge"
    assert_eq '["pa","pb"]' "$(jq -rc '.projects|keys'
        "$mj_a/.claude.json")" "A sees both projects (union)"
    assert_eq '["pa","pb"]' "$(jq -rc '.projects|keys'
        "$mj_b/.claude.json")" "B sees both projects (union)"
    mj_leak=0
    grep -q 'UID-B\b@x' "$mj_a/.claude.json" && mj_leak=1
    grep -q 'UID-A\b@x' "$mj_b/.claude.json" && mj_leak=1
    assert_eq 0 "$mj_leak" "no cross-account credential leak in
        either direction"
else
    _pass "jq absent – skipping cma_merge_claude_json security test"
fi

it "cma_ensure_alias_file generates cma_run with project-scoped
    cwd-hook resolution"
# Verify the emitted cma_run body has the _cma_hook_root marker
#   (project-local
#   .claude-cwd-hook support) and the three-tier resolution order:
#   1. CMA_CWD_HOOK env var 2. <git-toplevel>/.claude-cwd-hook 3.
#       global fallback
_mig_ph="$ALIAS_FILE.ph"
cat > "$_mig_ph" <<'PHFMT'
export CLAUDE_BIN="/usr/bin/true"

```

```

cma_run() {
    "$CLAUDE_BIN" "$@"
}

alias claude1="CLAUDE_CONFIG_DIR=$SANDBOX_HOME/.claude-1 cma_run"
PHFMT
( ALIAS_FILE="$_mig_ph" cma_ensure_alias_file ) >/dev/null 2>&1
# Extract the regenerated cma_run body and check the markers
_ph_body="$(awk '/^cma_run\(\) ?\{/{f=1} f{print} f&&/^}/{exit}'
    "$_mig_ph")"
grep -q '_cma_hook_root' <<<"$_ph_body"; assert_eq 0 $? "cma_run
    has _cma_hook_root marker (project-scoped hook support)"
# With no CMA_CWD_HOOK set, should check git toplevel first, then
    global
grep -q 'git rev-parse --show-toplevel' <<<"$_ph_body"; assert_eq
    0 $? "cma_run resolves git toplevel for project-local
    hook"
grep -q '\.claude-cwd-hook' <<<"$_ph_body"; assert_eq 0 $?
    "cma_run checks for .claude-cwd-hook in project root"
grep -q 'CMA_CWD_HOOK:-' <<<"$_ph_body"; assert_eq 0 $? "cma_run
    respects CMA_CWD_HOOK env var override"
grep -q '.local/bin/claude-cwd-hook' <<<"$_ph_body"; assert_eq 0
    $? "cma_run falls back to global claude-cwd-hook"
rm -f "$_mig_ph"

# --- wrapper self-heal is unconditional, not marker-driven
    -----
# HISTORY: cma_ensure_alias_file used to decide whether to
    regenerate each
# wrapper by matching ~22 "markers" against the on-disk body, then
    DROPPING the
# function block and re-appending it. That drop-then-re-append
    left the file
# without the function between two separate whole-file writes –
    one half of the
# concurrency race that destroyed the live alias file (see the
    alias-file
# section header in lib.sh). The renderer now always emits the
    current wrapper
# text, so there is no marker list, no drop window, and no stale
    body can
# survive a write.
#
# What still must hold: emitting the current body may never be
    short-circuited
# by a cached version stamp. A corrupted body with an intact stamp
    line would
# otherwise be wrongly trusted and the self-heal skipped
# (test_128k_output_clamp.sh deliberately constructs exactly that
    shape).
# The behavioural half of this is the cwd-hook regeneration test
    above, which
# feeds in a stale cma_run body and asserts the current one comes
    back.

```

```

it "wrapper regeneration has no marker/version-stamp short-
circuit"
_mk_body="$(awk '/^_cma_alias_render\(\) ?\{f=1\} f{print} f&&/
^}/{exit}' "$SCRIPTS_DIR/lib.sh")
$(awk '/^_cma_emit_managed\(\) ?\{f=1\} f{print} f&&/^}/{exit}'
"$SCRIPTS_DIR/lib.sh")"
_mk_stamp="$(grep -ciE 'version.?stamp|CMA_ALIAS_VERSION'
<<<"$_mk_body" || true)"
assert_eq 0 "$_mk_stamp" "no version-stamp short-circuit (would
skip a needed self-heal)"
_mk_cond="$(grep -cE 'grep -q.*cma_run(_provider)?\(\)'
<<<"$_mk_body" || true)"
assert_eq 0 "$_mk_cond" "wrapper emission is unconditional (not
gated on the old body)"

it "\$ALIAS_FILE has exactly one writer"
# Nothing outside cma_alias_commit may write, append to, or rename
onto the
# alias file. That single-committer property is what makes the
exclusive lock
# and the sanity gate impossible to bypass – the incident was two
writers, each
# individually correct, interleaving.
_wr="$(grep -nE '(command)?mv +-?f? *"?\$[A-Za-z_]+ " *"\
$ALIAS_FILE"|> *"\$ALIAS_FILE"|>> *"\$ALIAS_FILE"' \
"$SCRIPTS_DIR/*.sh | grep -v ':[0-9]*: *#' || true)"
_wr_n="$(printf '%s\n' "$_wr" | grep -c . || true)"
assert_eq 1 "$_wr_n" "exactly one writer of \$ALIAS_FILE remains"
$_wr"

# =====
# The ccr self-reference guard: ONE definition, and it matches the
whole shape
# =====
# The original guard was a `case` listing four literal spellings,
DUPLICATED in
# lib.sh's launch gate and providers-verify.sh's Gate 0.
Everything it did not
# literally list went through – including `127.0.1.1:3456`, which
is Debian's
# default loopback for the local hostname, i.e. a form that IS the
gateway on a
# stock install. Both halves matter and both are asserted here:
that the two
# call sites share one definition, and that the definition covers
the shape.
it "the ccr-gateway guard has exactly one definition (no drifting
copy)"
# lib.sh owns it; providers-verify.sh must CALL it, never re-
implement it.
_gw_defs="$(grep -c '^_cma_is_ccr_gateway() {' "$SCRIPTS_DIR/
lib.sh" || true)"
assert_eq 1 "${_gw_defs:-0}" "_cma_is_ccr_gateway defined once in
lib.sh"
_gw_dup="$(grep -c '_cma_is_ccr_gateway() {' "$SCRIPTS_DIR/
providers-verify.sh" || true)"

```

```

assert_eq 0 "${_gw_dup:-0}" "providers-verify.sh does not re-
define it"
_gw_call="$(grep -c '_cma_is_ccr_gateway ' "$SCRIPTS_DIR/
providers-verify.sh" || true)"
_gw_ok=1; (( _gw_call >= 1 )) && _gw_ok=0
assert_eq 0 "$_gw_ok" "providers-verify.sh calls the shared
helper"
# The literal-spelling `case` that under-matched must be gone from
both.
_gw_legacy="$(grep -cE '127\.0\.0\.1:?"\$_ccr_port|\
{CMA_CCR_PORT}' \
"$SCRIPTS_DIR/lib.sh" "$SCRIPTS_DIR/providers-
verify.sh" 2>/dev/null \
| awk -F: '{s+=$2} END{print s+0}')"
assert_eq 0 "${_gw_legacy:-0}" "the duplicated literal-spelling
case is gone"

it "the ccr-gateway guard matches every form that IS the gateway"
# Each of these resolves to the local ccr gateway on port 3456.
Every one of
# them was ALLOWED through by the old guard.
_gw_missed=""
for _u in \
'http://127.0.0.1:3456/v1' \
'http://localhost:3456/v1' \
'http://[::1]:3456/v1' \
'http://0.0.0.0:3456/v1' \
'http://127.0.1.1:3456/v1' \
'http://127.0.0.2:3456/v1' \
'http://127.255.255.254:3456/v1' \
'http://[:ffff:127.0.0.1]:3456/v1' \
'http://[0:0:0:0:0:0:1]:3456/v1' \
'http://[0::1]:3456/v1' \
'http://LOCALHOST:3456/v1' \
'http://localhost:3456' \
'http://user@127.0.0.1:3456/v1' \
'http://user:pw@localhost:3456/v1' \
'http://127.0.0.1:3456?x=1' \
'http://127.0.0.1:3456#frag' \
'http://127.0.0.1:3456/v1?x=1' \
'http://127.0.0.1:3456' \
; do
_cma_is_ccr_gateway "$_u" || _gw_missed="$_gw_missed $_u"
done
assert_eq "" "$_gw_missed" "no gateway spelling slips through"
"missed:$_gw_missed"

it "the ccr-gateway guard does NOT false-positive on legitimate
bases"
# A guard that refuses everything is not a fix. These must all
still launch.
_gw_false=""
for _u in \
'http://127.0.0.1:8080/v1' \

```

```

'http://127.0.0.1:3457/v1' \
'http://localhost:3457/v1' \
'http://localhost/v1' \
'https://api.deepseek.com/anthropic' \
'https://api.z.ai/api/coding/paas/v4' \
'https://openrouter.ai/api/v1' \
'http://192.168.1.10:3456/v1' \
'http://10.0.0.5:3456/v1' \
'https://127.0.0.1.example.com:3456/v1' \
'http://[fe80::1]:3456/v1' \
'http://[2001:db8::1]:3456/v1' \
'https://api.test/v1' \
'http://myhost:3456/v1' \
; do
    _cma_is_ccr_gateway "$_u" && _gw_false="$_gw_false $_u"
done
assert_eq "" "$_gw_false" "no legitimate base is swept up" "false
positives:$_gw_false"

it "the ccr-gateway guard honours a non-default CMA_CCR_PORT"
( CMA_CCR_PORT=9999; _cma_is_ccr_gateway 'http://127.0.0.1:9999/
v1' ); assert_eq 0 $? "9999 is the gateway when
CMA_CCR_PORT=9999"
( CMA_CCR_PORT=9999; _cma_is_ccr_gateway 'http://127.0.0.1:3456/
v1' ); assert_eq 1 $? "3456 is NOT the gateway when
CMA_CCR_PORT=9999"

# =====
# Account detection must not count directories that are no longer
# accounts
# =====
it "cma_detect_accounts ignores archived (.removed.*)
and .preunify.* dirs"
# claude-remove-account's DEFAULT (non---delete) mode renames the
# config dir to
# `

```

```

_det_arch="$( HOME="$_det_home" cma_detect_accounts | grep -c
    'removed\|preunify' || true )"
assert_eq 0 "${_det_arch:-0}" "no archived dir appears in the
    detected set"

# =====
# The renderer must not report success for a write it did not make
# =====
it "_cma_alias_render propagates an output write failure"
# The `{ ... } > \out` status used to be discarded and the
    function always
# returned 0. A candidate truncated by ENOSPC then reached the
    committer as a
# successful render – and against a zero-alias source such a stump
    also clears
# the sanity gate, which only requires the header plus the two
    wrapper opening
# lines and has no aliases left to miss.
rm -f "$ALIAS_FILE"; cma_ensure_alias_file >/dev/null 2>&1
# (a) the output path cannot be opened at all
( _cma_alias_render "$ALIAS_FILE" "" "" keep "$HOME/no/such/dir/
    cand" ) >/dev/null 2>&1
assert_eq 1 $? "an unopenable output path is a failed render"
# (b) a genuine ENOSPC. /dev/full accepts the open and fails every
    write, which
# is the real shape of the hazard. Linux-only; skipped
    elsewhere.
if [[ -w /dev/full ]]; then
    ( _cma_alias_render "$ALIAS_FILE" "" "" keep /dev/full ) >/dev/
        null 2>&1
    assert_eq 1 $? "an ENOSPC write (/dev/full) is a failed render"
else
    _pass "SKIP (no writable /dev/full on this platform)"
fi
# CONTROL: a healthy render still succeeds.
( _cma_alias_render "$ALIAS_FILE" "" "" keep "$HOME/.cand-ok" ) >/
    dev/null 2>&1
assert_eq 0 $? "CONTROL: a normal render still returns success"

# =====
# The alias lock is scoped to the file it guards, not to the host
# =====
it "the alias lock is per-\$_ALIAS_FILE – two alias files never
    contend"
# Deliberately the OPPOSITE of tests/lib/suite-lock.sh, which is
    global by
# design (one suite run per checkout, lock in the git dir). If
    this lock were
# global, every sandbox – and every $HOME on a multi-user box –
    would serialize
# against unrelated runs for up to CMA_ALIAS_LOCK_WAIT.
_lk_a="$HOME/lockscope-a/aliases.sh"
_lk_b="$HOME/lockscope-b/aliases.sh"
mkdir -p "$(dirname "$_lk_a")" "$(dirname "$_lk_b")"

```



```

cat > "$HOME/lockscope-hold.sh" <<'LK_EOF'
#!/usr/bin/env bash
set -uo pipefail
source "$SCRIPTS_DIR/lib.sh"; set +e
_cma_alias_lock_acquire || exit 9
: > "$HOME/.lockscope-held"
sleep 4
LK_EOF
chmod +x "$HOME/lockscope-hold.sh"
rm -f "$HOME/.lockscope-held"
SCRIPTS_DIR="$SCRIPTS_DIR" ALIAS_FILE="$_lk_a" "$HOME/lockscope-
hold.sh" & _lk_pid=$!
_lk_w=0
while [[ ! -f "$HOME/.lockscope-held" ]] && (( _lk_w < 400 )); do
    sleep 0.01 2>/dev/null || sleep 1; _lk_w=$(( _lk_w +
1 )); done
# B must acquire instantly even at zero wait: it protects a
DIFFERENT file.
SCRIPTS_DIR="$SCRIPTS_DIR" ALIAS_FILE="$_lk_b"
CMA_ALIAS_LOCK_WAIT=0 bash -c '
    source "$SCRIPTS_DIR/lib.sh"; set +e; _cma_alias_lock_acquire'
    >/dev/null 2>&1
assert_eq 0 $? "a lock on alias file B is unaffected by a holder
of alias file A"
# CONTROL: the same file DOES contend – otherwise the assertion
above is vacuous.
SCRIPTS_DIR="$SCRIPTS_DIR" ALIAS_FILE="$_lk_a"
CMA_ALIAS_LOCK_WAIT=0 bash -c '
    source "$SCRIPTS_DIR/lib.sh"; set +e; _cma_alias_lock_acquire'
    >/dev/null 2>&1
assert_eq 1 $? "CONTROL: the SAME alias file still contends (the
lock is real)"
kill "$_lk_pid" 2>/dev/null; wait "$_lk_pid" 2>/dev/null

summary

```

8.13 tests/test_unify.sh

```

#!/usr/bin/env bash
# test_unify.sh – exercises claude-unify.sh:
#   * basic merge of two accounts: all shared items become
      symlinks
#   * settings.json enabledPlugins is a strict union
#   * history.jsonl is concatenated and de-duplicated
#   * memory files survive merge (newest account wins conflicts)
#   * plugin manifest installPath is rewritten to the shared store
#   * N>2 accounts also work
#   * a second unify run produces the same state (idempotent)
#   * --rollback restores .preunify.* backups

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

```



```

SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"
# shellcheck source=../lib.sh
source "$SCRIPTS_DIR/lib.sh" # defines cma_realpath etc. used
directly below
# lib.sh enables `set -e`; this harness asserts on failures, so
relax it.
set +e

make_sandbox

# Two fixture accounts with overlapping and unique data.
acct1="$(make_account acct1 --plugins \
  --settings '{"enabledPlugins":\
    {"a":true,"b":true},"effortLevel":"high"}' \
  --history 'cmd1|cmd2|shared-line' \
  --memory 'user_role:role from acct1' \
  --todo todo-a-1)"
acct2="$(make_account acct2 --plugins \
  --settings '{"enabledPlugins":\
    {"b":true,"c":true},"skipDangerousModePermissionPrompt":true}' \
  \
  --history 'cmd3|shared-line|cmd4' \
  --memory 'user_role:role from acct2 (NEWER)' \
  --memory 'project_notes:notes from acct2 only' \
  --todo todo-b-1)"

# Make sure acct2 is the most recently touched so the unifier's
"last
# account wins on conflicts" rule applies to it.
touch "$acct2/projects/-home-test/memory/user_role.md"

# shellcheck disable=SC2119 # test intentionally calls run_unify
with no args
run_unify >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "claude-unify exits 0"

it "every shared item becomes a symlink into the shared store"
# §11.4 own-settings: settings.json is intentionally NOT in this
list – it is each
# dir's OWN real file, asserted separately below. Everything else
(incl. the
# plugin CACHE and history) stays a shared symlink.
for item in projects todos tasks plans file-history paste-cache
  plugins \
    shell-snapshots session-env telemetry sessions backups
  cache \
    stats-cache.json history.jsonl; do
  case "$item" in
    stats-cache.json)

```

```

    # Only fixtures with this file get the symlink; skip if
    absent.
    [[ -e "$SHARED_DIR/$item" ]] || continue ;;
esac
assert_symlink_to "$acct1/$item" "$SHARED_DIR/$item" "$acct1
    $item linked"
assert_symlink_to "$acct2/$item" "$SHARED_DIR/$item" "$acct2
    $item linked"
done

it "settings.json is each dir's OWN real file (§11.4 own-
    settings), not a shared symlink"
# Per-alias permissions/model/hooks must never leak, so unify
    gives every dir its
# OWN settings.json – while plugins (cache) + history stay shared
    symlinks (proven
# in the loop above). The propagated enabledPlugins union lives in
    each own copy.
assert_not_symlink "$acct1/settings.json" "acct1 settings.json is
    OWN"
assert_not_symlink "$acct2/settings.json" "acct2 settings.json is
    OWN"
assert_file "$acct1/settings.json" "acct1 own settings.json
    exists"
assert_file "$acct2/settings.json" "acct2 own settings.json
    exists"
assert_jq "$acct1/settings.json" '.enabledPlugins.a' "true"
    "acct1 own settings carries the enabledPlugins union"
assert_jq "$acct1/settings.json" '.enabledPlugins.c' "true"
    "acct1 own settings gained acct2's plugin via the
    propagated union"

it "shared TEMPLATE settings.json enabledPlugins is the union
    {a,b,c}"
# Unify unions ONLY the enabledPlugins map into the shared
    template so newly-
# enabled plugins propagate to every dir.
assert_jq "$SHARED_DIR/settings.json" '.enabledPlugins | keys |
    sort | join(",")' "a,b,c" "union"

it "§11.4 own-settings: per-alias NON-plugin keys stay local – no
    leak to a sibling dir or to shared"
# effortLevel was set ONLY in acct1;
    skipDangerousModePermissionPrompt ONLY in
# acct2. Under own-settings each keeps its own key, neither leaks
    to the other,
# and NEITHER leaks into the shared template (which carries only
    the plugin union).
assert_jq "$acct1/settings.json" '.effortLevel' "high" "acct1
    keeps its OWN effortLevel"
assert_jq "$acct2/settings.json"
    '.skipDangerousModePermissionPrompt' "true" "acct2 keeps
    its OWN permission flag"
assert_jq "$acct1/settings.json"
    '.skipDangerousModePermissionPrompt // "absent"' "absent"
    "acct2's permission flag did NOT leak into acct1"

```

```

assert_jq "$acct2/settings.json" '.effortLevel // "absent"'
    "absent" "acct1's effortLevel did NOT leak into acct2"
assert_jq "$SHARED_DIR/settings.json" '.effortLevel // "absent"'
    "absent" "effortLevel did NOT leak to the shared template"
assert_jq "$SHARED_DIR/settings.json"
    '.skipDangerousModePermissionPrompt // "absent"' "absent"
    "permission flag did NOT leak to the shared template"

it "history.jsonl contains every unique line, no duplicates"
expected_lines=5
actual_lines="$(sort -u "$SHARED_DIR/history.jsonl" | wc -l | tr
    -d ' ')"
assert_eq "$expected_lines" "$actual_lines" "5 unique lines"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd1" "cmd1
    present"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd4" "cmd4
    present"

it "memory: newest account's version wins on filename conflict"
assert_file_contains "$SHARED_DIR/projects/-home-test/memory/
    user_role.md" "NEWER" "acct2 wins"
assert_file "$SHARED_DIR/projects/-home-test/memory/
    project_notes.md" "acct2-only memory survives"

it "plugin manifest paths are rewritten to the shared store"
assert_jq "$SHARED_DIR/plugins/installed_plugins.json" \
    '.plugins["test-plugin@test-marketplace"][0].installPath' \
    "$SHARED_DIR/plugins/cache/test-marketplace/test-plugin/1.0.0" \
    "installPath rewritten"
assert_jq "$SHARED_DIR/plugins/known_marketplaces.json" \
    '["test-marketplace"].installLocation' \
    "$SHARED_DIR/plugins/marketplaces/test-marketplace" \
    "installLocation rewritten"

it "credentials are NOT shared – each account keeps its own"
assert_not_symlink "$acct1/.credentials.json" "acct1 creds local"
assert_not_symlink "$acct2/.credentials.json" "acct2 creds local"
assert_file_contains "$acct1/.credentials.json" "acct1" "acct1
    creds intact"
assert_file_contains "$acct2/.credentials.json" "acct2" "acct2
    creds intact"

it "unify registers claude<N> aliases for every detected account
    that lacks one"
assert_file_contains "$ALIAS_FILE" "alias
    claude1=\"CLAUDE_CONFIG_DIR=$acct1 cma_run\" \"acct1 gets
    claude1 alias"
assert_file_contains "$ALIAS_FILE" "alias
    claude2=\"CLAUDE_CONFIG_DIR=$acct2 cma_run\" \"acct2 gets
    claude2 alias"
alias_count="$(grep -cE '^alias claude[0-9]+=' "$ALIAS_FILE" ||
    true)"
assert_eq 2 "$alias_count" "exactly two claudeN aliases
    registered"

```

```

it "unify preserves claude<N> basenames and fills gaps with next
    free claudeN"
# A dir literally named ~/.claude-claude4 should be aliased as
    claude4, not
# renumbered. Other dirs consume the remaining free claude<N>
    slots.
c4="$(make_account claude4)"
zacct="$(make_account zebra)"
run_unify >/dev/null 2>&1
assert_file_contains "$ALIAS_FILE" "alias
    claude4=\"CLAUDE_CONFIG_DIR=$c4 cma_run\" \"claude4 dir
    keeps claude4 alias"
assert_file_contains "$ALIAS_FILE" "alias
    claude3=\"CLAUDE_CONFIG_DIR=$zacct cma_run\" \"zebra dir
    gets next free claude3 alias"
alias_count="$(grep -cE '^alias claude[0-9]+=' "$ALIAS_FILE" ||
    true)"
assert_eq 4 "$alias_count" "four claudeN aliases total (1,2,3,4)"

it "re-running claude-unify is idempotent (settings.json byte-
    stable)"
checksum_before="$(sha256sum "$SHARED_DIR/settings.json" | cut
    -d' ' -f1)"
target_before="$(cma_realpath "$acct1/projects")" # readlink -f
    is unavailable on BSD/macOS
# shellcheck disable=SC2119 # test intentionally calls run_unify
    with no args
run_unify >/dev/null 2>&1
checksum_after="$(sha256sum "$SHARED_DIR/settings.json" | cut -d'
    ' -f1)"
target_after="$(cma_realpath "$acct1/projects")"
assert_eq "$checksum_before" "$checksum_after" "settings.json
    unchanged"
assert_eq "$target_before" "$target_after" "symlink target
    unchanged"

it "N>2 accounts: adding a third and re-running unifies it too"
acct3="$(make_account acct3 \
    --settings '{"enabledPlugins":{"d":true}}' \
    --history 'cmd5|cmd1')"
# shellcheck disable=SC2119 # test intentionally calls run_unify
    with no args
run_unify >/dev/null 2>&1
assert_symlink_to "$acct3/projects" "$SHARED_DIR/projects" "acct3
    projects linked"
assert_jq "$SHARED_DIR/settings.json" '.enabledPlugins.d' "true"
    "acct3 plugin merged"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd5" "acct3
    history merged"

it "$11.4 own-settings PROOF: a per-alias settings.json edit does
    NOT leak into another dir or shared"
# Add a unique per-alias NON-plugin key to acct1's OWN
    settings.json AFTER unify,

```

```

# then re-run unify. own-settings must keep it local: acct1
# retains it, acct2
# never gains it, and the shared template never gains it (it
# carries only the
# enabledPlugins union). This is the load-bearing anti-leak
# guarantee of the
# refactor – a real behavioral proof, not a tautology.
jq '.model = "per-alias-model-acct1-only"' "$acct1/settings.json"
  > "$acct1/settings.json.probe" \
  && mv "$acct1/settings.json.probe" "$acct1/settings.json"
# shellcheck disable=SC2119 # test intentionally calls run_unify
# with no args
run_unify >/dev/null 2>&1
assert_jq "$acct1/settings.json" '.model' "per-alias-model-acct1-
only" "acct1 keeps its OWN per-alias model after unify"
assert_jq "$acct2/settings.json" '.model // "absent"' "absent"
"acct1's per-alias model did NOT leak into acct2"
assert_jq "$SHARED_DIR/settings.json" '.model // "absent"'
"absent" "acct1's per-alias model did NOT leak to the
shared template"
# The plugin union still flows to every dir (plugins ARE meant to
be shared).
assert_jq "$acct2/settings.json" '.enabledPlugins.a' "true"
"acct2 still carries the propagated enabledPlugins union"

# — B1 / B2: absorb_default_plugins + link_default_plugin_subdirs
# Use an isolated SHARED_DIR/DEFAULT_DIR/ACCOUNT_PREFIX triple so
these tests
# don't interfere with the main sandbox state or the rollback
assertions below.

b1_sd="$SANDBOX_HOME/.claude-shared-b1"
b1_dd="$SANDBOX_HOME/.claude-b1"
b1_acct="$SANDBOX_HOME/.claude-b1-acct"
mkdir -p "$b1_dd/plugins/cache/foo/1.0.0"
printf 'hello-b1\n' > "$b1_dd/plugins/cache/foo/1.0.0/file.txt"
mkdir -p "$b1_acct"
printf '{"name":"b1"}\n' > "$b1_acct/.credentials.json"
printf '{"name":"b1"}\n' > "$b1_acct/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$b1_acct/settings.json"
SHARED_DIR="$b1_sd" DEFAULT_DIR="$b1_dd" ACCOUNT_PREFIX=".claude-
b1-" \
"$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "absorb_default_plugins (B1): files under DEFAULT_DIR/plugins/
cache are absorbed into shared store"
assert_file "$b1_sd/plugins/cache/foo/1.0.0/file.txt" \
"absorb_default_plugins: file.txt absorbed from DEFAULT_DIR/
plugins/cache into SHARED_DIR"

it "link_default_plugin_subdirs (B2): DEFAULT_DIR/plugins/cache
becomes a symlink into shared store"
b2_is_link=1; [[ -L "$b1_dd/plugins/cache" ]] && b2_is_link=0

```

```

assert_eq 0 "$b2_is_link" "DEFAULT_DIR/plugins/cache is a symlink
after unify"
b2_real="$(cma_realpath "$b1_dd/plugins/cache")"
assert_eq "$(cma_realpath "$b1_sd/plugins/cache")" "$b2_real"
"symlink resolves to SHARED_DIR/plugins/cache"

it "link_default_plugin_subdirs (B2 idempotent): second unify
creates no additional .preunify.* backup"
b2_pre="$(find "$b1_dd/plugins" -maxdepth 1 -name
'cache.preunify.*' 2>/dev/null | wc -l | tr -d ' ')"
SHARED_DIR="$b1_sd" DEFAULT_DIR="$b1_dd" ACCOUNT_PREFIX=".claude-
b1-" \
"$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1
b2_post="$(find "$b1_dd/plugins" -maxdepth 1 -name
'cache.preunify.*' 2>/dev/null | wc -l | tr -d ' ')"
assert_eq "$b2_pre" "$b2_post" "no new .preunify.* backup created
on second unify run (idempotent)"

# — B4: sync_claude_md seed branches

# (b) DEFAULT_DIR has a real CLAUDE.md; no account CLAUDE.md.
# sync_claude_md should seed shared CLAUDE.md from
DEFAULT_DIR.

b4b_sd="$SANDBOX_HOME/.claude-shared-b4b"
b4b_dd="$SANDBOX_HOME/.claude-b4b"
b4b_acct="$SANDBOX_HOME/.claude-b4b-acct"
mkdir -p "$b4b_dd" "$b4b_acct"
printf 'default-memory\n' > "$b4b_dd/CLAUDE.md"
printf '{"name":"b4b"}\n' > "$b4b_acct/.credentials.json"
printf '{"name":"b4b"}\n' > "$b4b_acct/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$b4b_acct/settings.json"
SHARED_DIR="$b4b_sd" DEFAULT_DIR="$b4b_dd"
ACCOUNT_PREFIX=".claude-b4b-" \
"$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "sync_claude_md (B4b): DEFAULT_DIR/CLAUDE.md seeds shared
CLAUDE.md when no account has one"
assert_file_contains "$b4b_sd/CLAUDE.md" "default-memory" \
"sync_claude_md: DEFAULT_DIR/CLAUDE.md content seeded into
shared store"

# (c) No DEFAULT_DIR CLAUDE.md; an account has a real CLAUDE.md.
# sync_claude_md should fall back and seed from the first
account's CLAUDE.md.

b4c_sd="$SANDBOX_HOME/.claude-shared-b4c"
b4c_dd="$SANDBOX_HOME/.claude-b4c"
b4c_acct="$SANDBOX_HOME/.claude-b4c-acct"
mkdir -p "$b4c_dd" "$b4c_acct"
# No CLAUDE.md in b4c_dd (the DEFAULT_DIR).
printf 'acct2-memory\n' > "$b4c_acct/CLAUDE.md"
printf '{"name":"b4c"}\n' > "$b4c_acct/.credentials.json"
printf '{"name":"b4c"}\n' > "$b4c_acct/.claude.json"

```

```

printf '{"enabledPlugins":{}}\n' > "$b4c_acct/settings.json"
SHARED_DIR="$b4c_sd" DEFAULT_DIR="$b4c_dd"
    ACCOUNT_PREFIX=".claude-b4c-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it
    "sync_claude_md (B4c): account CLAUDE.md seeds shared
    CLAUDE.md when DEFAULT_DIR has none"
assert_file_contains "$b4c_sd/CLAUDE.md" "acct2-memory" \

    "sync_claude_md: account CLAUDE.md seeded into shared
    store when DEFAULT_DIR lacks one"

# — R1: history.jsonl merge must not fuse records across a source
#      that lacks a
#      trailing newline. Regression for cat-based concat gluing one
#      file's last line
#      onto the next file's first line.
r1_sd="$SANDBOX_HOME/.claude-shared-r1"
r1_a="$SANDBOX_HOME/.claude-r1-a"
r1_b="$SANDBOX_HOME/.claude-r1-b"
mkdir -p "$r1_a" "$r1_b"
printf '{"n":"r1a"}\n' > "$r1_a/credentials.json"; printf
    '{"name":"r1a"}\n' > "$r1_a/claude.json"
printf '{"n":"r1b"}\n' > "$r1_b/credentials.json"; printf
    '{"name":"r1b"}\n' > "$r1_b/claude.json"
printf '{"enabledPlugins":{}}\n' > "$r1_a/settings.json"
printf '{"enabledPlugins":{}}\n' > "$r1_b/settings.json"
# acct a's history has NO trailing newline (the bug trigger); acct
#      b follows it.
printf '{"e":"one"}\n{"e":"two"}' > "$r1_a/history.jsonl"
printf '{"e":"three"}\n{"e":"four"}\n' > "$r1_b/history.jsonl"
SHARED_DIR="$r1_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r1"
    ACCOUNT_PREFIX=".claude-r1-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it
    "history.jsonl (R1): a source missing its trailing newline
    does not fuse records"
r1_two=1; grep -qxF '{"e":"two"}' "$r1_sd/history.jsonl" 2>/
    dev/null && r1_two=0
r1_three=1; grep -qxF '{"e":"three"}' "$r1_sd/history.jsonl" 2>/
    dev/null && r1_three=0
r1_fused=0; grep -qF '"two"}{"e":"three"' "$r1_sd/history.jsonl"
    2>/dev/null && r1_fused=1
assert_eq 0 "$r1_two" "history line {\"e\": \"two\"} preserved
    intact"
assert_eq 0 "$r1_three" "history line {\"e\": \"three\"} preserved
    intact"
assert_eq 0 "$r1_fused" "no fused two+three record"

# — R2: enabledPlugins union must keep "any true" regardless of
#      account order.
# Regression: rightmost-wins let the lexically-last account's
#      false clobber a

```



```

# plugin an earlier account had enabled.
r2_sd="$SANDBOX_HOME/.claude-shared-r2"
r2_a="$SANDBOX_HOME/.claude-r2-a"
r2_b="$SANDBOX_HOME/.claude-r2-b"
mkdir -p "$r2_a" "$r2_b"
printf '{"n":"r2a"}\n' > "$r2_a/.credentials.json"; printf
    '{"name":"r2a"}\n' > "$r2_a/.claude.json"
printf '{"n":"r2b"}\n' > "$r2_b/.credentials.json"; printf
    '{"name":"r2b"}\n' > "$r2_b/.claude.json"
# a (lexically first) enables foo; b (lexically LAST, the overlay
  account) disables it.
printf '{"enabledPlugins":{"foo":true}}\n' > "$r2_a/
  settings.json"
printf '{"enabledPlugins":{"foo":false,"bar":true}}\n' > "$r2_b/
  settings.json"
SHARED_DIR="$r2_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r2"
ACCOUNT_PREFIX=".claude-r2-" \
  "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "enabledPlugins (R2): a plugin enabled in any account stays
  enabled (any-true union)"
assert_jq "$r2_sd/settings.json" '.enabledPlugins.foo' "true"
  "foo stays true despite last account false"
assert_jq "$r2_sd/settings.json" '.enabledPlugins.bar' "true"
  "bar from last account present"

# — R3: a single malformed settings.json must not abort the whole
  unify.
# Regression: the multi-file jq -s merge ran unguarded under set
  -e.
r3_sd="$SANDBOX_HOME/.claude-shared-r3"
r3_a="$SANDBOX_HOME/.claude-r3-a"
r3_b="$SANDBOX_HOME/.claude-r3-b"
mkdir -p "$r3_a" "$r3_b"
printf '{"n":"r3a"}\n' > "$r3_a/.credentials.json"; printf
    '{"name":"r3a"}\n' > "$r3_a/.claude.json"
printf '{"n":"r3b"}\n' > "$r3_b/.credentials.json"; printf
    '{"name":"r3b"}\n' > "$r3_b/.claude.json"
printf '{"enabledPlugins":{"ok":true}}\n' > "$r3_a/settings.json"
printf '%s\n' '{ this is not valid json' > "$r3_b/settings.json"
SHARED_DIR="$r3_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r3"
ACCOUNT_PREFIX=".claude-r3-" \
  "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1
r3_rc=$?

it "settings.json (R3): one malformed account file does not abort
  unify"
assert_eq 0 "$r3_rc" "unify still exits 0 despite a malformed
  settings.json"
r3_tmp=0; [[ -e "$r3_sd/settings.json.tmp" ]] && r3_tmp=1
assert_eq 0 "$r3_tmp" "no orphaned settings.json.tmp left behind"
# The valid account's settings must NOT become a dangling symlink
  (silent loss):
# the merge must still write shared from the valid file(s) and
  link to it.

```



```

r3_readable=0; [[ -f "$r3_a/settings.json" ]] && r3_readable=1
assert_eq 1 "$r3_readable" "valid account settings.json still
    readable (not a dangling symlink)"
assert_jq "$r3_a/settings.json" '.enabledPlugins.ok' "true"
    "valid account's settings survive a malformed sibling"

# — R4: directory merge must resolve file conflicts by newest
    mtime, not by the
# lexically-last account name. Regression: pass 2 overlaid
    ACCOUNTS[-1]
# (alphabetically last) unconditionally, so a stale lexically-last
    account
# clobbered fresher content from an earlier account.
r4_sd="$SANDBOX_HOME/.claude-shared-r4"
r4_a="$SANDBOX_HOME/.claude-r4-a"
r4_b="$SANDBOX_HOME/.claude-r4-b"
mkdir -p "$r4_a/projects/p/memory" "$r4_b/projects/p/memory"
printf '{"n":"r4a"}\n' > "$r4_a/.credentials.json"; printf
    '{"name":"r4a"}\n' > "$r4_a/.claude.json"
printf '{"n":"r4b"}\n' > "$r4_b/.credentials.json"; printf
    '{"name":"r4b"}\n' > "$r4_b/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$r4_a/settings.json"
printf '{"enabledPlugins":{}}\n' > "$r4_b/settings.json"
# Same file in both. b (lexically LAST) is STALE; a (lexically
    first) is FRESH.
# Set explicit mtimes (touch -t, portable) so a is strictly newer
    than b.
printf 'OLD-from-b\n' > "$r4_b/projects/p/memory/note.md"
printf 'NEW-from-a\n' > "$r4_a/projects/p/memory/note.md"
touch -t 202001010000 "$r4_b/projects/p/memory/note.md"
touch -t 202601010000 "$r4_a/projects/p/memory/note.md"
SHARED_DIR="$r4_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r4"
    ACCOUNT_PREFIX=".claude-r4-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "dir merge (R4): freshest file wins conflicts regardless of
    account name order"
assert_file_contains "$r4_sd/projects/p/memory/note.md" "NEW-from-
    a" \
    "newer file from lexically-earlier account wins (not the stale
        lexically-last one)"

# — R5: rollback must not leave a dangling symlink for an item an
    account never
# had. Such items get a shared-store symlink but NO .preunify
    backup, so the
# restore loop never visits them; once SHARED_DIR is moved aside
    they dangle.
r5_sd="$SANDBOX_HOME/.claude-shared-r5"
r5_a="$SANDBOX_HOME/.claude-r5-a"
r5_b="$SANDBOX_HOME/.claude-r5-b"
mkdir -p "$r5_a/projects" "$r5_b/projects" "$r5_b/todos" # only
    b has todos
printf '{"n":"r5a"}\n' > "$r5_a/.credentials.json"; printf
    '{"name":"r5a"}\n' > "$r5_a/.claude.json"

```

```

printf '{"n":"r5b"}\n' > "$r5_b/.credentials.json"; printf
    '{"name":"r5b"}\n' > "$r5_b/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$r5_a/settings.json"
printf '{"enabledPlugins":{}}\n' > "$r5_b/settings.json"
SHARED_DIR="$r5_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r5"
ACCOUNT_PREFIX=".claude-r5-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1
SHARED_DIR="$r5_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r5"
ACCOUNT_PREFIX=".claude-r5-" \
    "$SCRIPTS_DIR/claude-unify.sh" --rollback >/dev/null 2>&1

it "rollback (R5): no dangling shared-store symlink left for a
    never-backed-up item"
r5_dangle=0; [[ -L "$r5_a/todos" ]] && r5_dangle=1
assert_eq 0 "$r5_dangle" "a/todos (symlink, no backup) is removed
    on rollback, not left dangling"

# — R6: rollback restores the OLDEST .preunify backup (the true
    original) when a
# path has more than one, deterministically (find order is
    otherwise arbitrary).
r6_sd="$SANDBOX_HOME/.claude-shared-r6"
r6_a="$SANDBOX_HOME/.claude-r6-a"
mkdir -p "$r6_a"
printf '{"n":"r6a"}\n' > "$r6_a/.credentials.json"; printf
    '{"name":"r6a"}\n' > "$r6_a/.claude.json"
# Two backups for the same path; the oldest timestamp holds the
    true original.
printf 'OLDEST-original\n' > "$r6_a/memo.preunify.20200101000000"
printf 'newer-backup\n' > "$r6_a/memo.preunify.20240101000000"
SHARED_DIR="$r6_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r6"
ACCOUNT_PREFIX=".claude-r6-" \
    "$SCRIPTS_DIR/claude-unify.sh" --rollback >/dev/null 2>&1

it "rollback (R6): oldest .preunify backup wins when a path has
    several"
assert_file_contains "$r6_a/memo" "OLDEST-original" "the true
    (oldest) original is restored deterministically"

it "--rollback restores backups and archives the shared store"
# shellcheck disable=SC2119 # test intentionally calls
    run_rollback with no args
run_rollback >/dev/null 2>&1
assert_dir "$acct1/projects" "acct1 projects restored as real dir"
assert_not_symlink "$acct1/projects" "no longer a symlink"
cond=1; [[ ! -d "$SHARED_DIR" ]] && cond=0; assert_eq 0 "$cond"
    "shared store moved aside"
shared_archive="$(find "$HOME" -maxdepth 1 -name '.claude-
    shared.removed.*' -type d 2>/dev/null | head -1)"
cond=0; [[ -n "$shared_archive" ]] || cond=1; assert_eq 0 "$cond"
    "archive sibling exists"

# Drift guard: claude-add-account wires CMA_SHARED_ITEMS (lib.sh)
    while

```

```

# claude-unify.sh carries its own SHARED_ITEMS. CLAUDE.md
# documents "keep the two
# lists in sync". They MUST agree EXCEPT for CLAUDE.md, which
# unify handles
# specially via sync_claude_md() (promotion from ~/.claude /
# DEFAULT_DIR - a
# generic rsync/symlink cannot do that). This catches accidental
# future drift
# (a new shared item added to one list only) without forcing the
# two paths to merge.
it "shared-item lists stay in sync: unify SHARED_ITEMS ==
    CMA_SHARED_ITEMS minus the CLAUDE.md special-case"
_cma_items="$(awk '/^CMA_SHARED_ITEMS=\\(/{f=1} f{print} f&&\\)/
{exit}' "$SCRIPTS_DIR/lib.sh" | tr -d '()' | tr ' ' '\n'
| grep -vE 'CMA_SHARED_ITEMS=|^$|^CLAUDE\\.md$' | sort |
tr '\n' ' ')"
_uni_items="$(awk '/^SHARED_ITEMS=\\(/{f=1} f{print} f&&\\)/
{exit}' "$SCRIPTS_DIR/claude-unify.sh" | tr -d '()' | tr
' ' '\n' | grep -vE 'SHARED_ITEMS=|^$' | sort | tr '\n' '
')"
assert_eq "$_uni_items" "$_cma_items" "the two shared-item lists
agree (modulo the intentional CLAUDE.md special-case)"

# -----
# daemon/ + jobs/ (v1.17.0): Claude Code's background-agent
# registry MUST be
# shared - a background agent started under one alias is
# registered in that
# config dir's daemon/ and is otherwise invisible to every other
# alias.
# -----
acct3="$(make_account acct3)"
acct4="$(make_account acct4)"

# Seed the rosters BEFORE the first unify in this section: after
# linking,
# $acctN/daemon is a symlink into shared, so writing to it later
# would
# overwrite the shared file directly (and empty the union inputs).
mkdir -p "$acct3/daemon/dispatch" "$acct4/daemon"
printf 'k3' > "$acct3/daemon/control.key"
printf 'k4' > "$acct4/daemon/control.key"
cat > "$acct3/daemon/roster.json" <<'JSON'
{"proto":1,"supervisorPid":300,"updatedAt":2000,"workers":{"w-a":
{"sessionId":"s-a","updatedAt":2000},"w-share":
{"sessionId":"s-share-NEWER","updatedAt":2000},"w-old":
{"sessionId":"s-old","updatedAt":1000}}}
JSON
cat > "$acct4/daemon/roster.json" <<'JSON'
{"proto":1,"supervisorPid":400,"updatedAt":1500,"workers":{"w-b":
{"sessionId":"s-b","updatedAt":1500},"w-old":
{"sessionId":"s-old-SHOULD-STAY-
STALE","updatedAt":500},"w-share":{"sessionId":"s-share-
old","updatedAt":1000}}}
JSON

```

```
run_unify >/dev/null 2>&1
```

```
it "daemon dir is merged into shared + symlinked from every
    account"
assert_dir "$SHARED_DIR/daemon" "shared daemon dir created"
assert_symlink_to "$acct3/daemon" "$SHARED_DIR/daemon" "acct3
    daemon is a symlink into shared"
assert_symlink_to "$acct4/daemon" "$SHARED_DIR/daemon" "acct4
    daemon is a symlink into shared"

it "daemon/roster.json: workers are UNION-merged (newer updatedAt
    wins per worker), not last-file-wins"
assert_jq "$SHARED_DIR/daemon/roster.json"
    '.workers | keys | sort | join(",")' "w-a,w-b,w-old,w-
    share" "all four workers present (union)"
assert_jq "$SHARED_DIR/daemon/roster.json" '.workers["w-
    share"].sessionId' "s-share-NEWER" "newer updatedAt wins
    the shared worker"
assert_jq "$SHARED_DIR/daemon/roster.json" '.workers["w-
    old"].sessionId' "s-old" "stale worker value does not
    clobber the newer one"
assert_jq "$SHARED_DIR/daemon/roster.json" '.supervisorPid' "300"
    "supervisorPid comes from the newest roster"
assert_jq "$SHARED_DIR/daemon/roster.json" '.updatedAt' "2000"
    "top-level updatedAt is the max"

it "jobs dir is merged into shared + symlinked from every account"
assert_dir "$SHARED_DIR/jobs" "shared jobs dir created"
assert_symlink_to "$acct3/jobs" "$SHARED_DIR/jobs" "acct3 jobs is
    a symlink into shared"
```

```
# -----
# cma_migrate_daemon_dirs_once (lib.sh): pre-v1.17.0 provider dirs
#   carry LOCAL
# daemon registries that must join the shared one – contents
#   merged, rosters
# unioned (never lost), dirs replaced by symlinks, idempotent.
# -----
```

```
it "cma_migrate_daemon_dirs_once: local provider daemon dirs are
    merged, unioned, symlinked, backed up"
```

```
rm -f "$SHARED_DIR/.daemon-migration-done"
```

```
p1="$HOME/.claude-prov-p1" p2="$HOME/.claude-prov-p2"
```

```
mkdir -p "$p1/daemon" "$p2/daemon"
```

```
cat > "$p1/daemon/roster.json" <<'JSON'
```

```
{"proto":1,"supervisorPid":300,"updatedAt":2000,"workers":{"w-
    prov1":{"sessionId":"s1","updatedAt":2000},"w-share":
    {"sessionId":"s-share-NEWER","updatedAt":2000}}}
```

```
JSON
```

```
cat > "$p2/daemon/roster.json" <<'JSON'
```

```
{"proto":1,"supervisorPid":400,"updatedAt":1500,"workers":{"w-
    prov2":{"sessionId":"s2","updatedAt":1500},"w-share":
    {"sessionId":"s-share-old","updatedAt":1000}}}
```

```
JSON
```

```
cma_migrate_daemon_dirs_once
```

```

assert_symlink_to "$p1/daemon" "$SHARED_DIR/daemon" "p1 daemon
    replaced by shared symlink"
assert_symlink_to "$p2/daemon" "$SHARED_DIR/daemon" "p2 daemon
    replaced by shared symlink"
ok=1; compgen -G "$p1/daemon.preunify.*" >/dev/null && ok=0;
    assert_eq 0 "$ok" "p1 original daemon backed up
    (.preunify)"
assert_jq "$SHARED_DIR/daemon/roster.json" '[.workers | keys[] |
    select(. == "w-prov1" or . == "w-prov2" or . == "w-
    share")] | sort | join(",")' "w-prov1,w-prov2,w-share"
    "both providers' workers unioned into the existing
    registry"
assert_jq "$SHARED_DIR/daemon/roster.json" '.workers["w-
    share"].sessionId' "s-share-NEWER" "newer worker version
    wins across providers"
rm -f "$SHARED_DIR/daemon/roster.json" ; _sentinel="$(jq -c
    '.workers|keys' <(echo '{"workers":{}}') 2>/dev/null ||
    true)"
cma_migrate_daemon_dirs_once
ok=1; [[ ! -e "$SHARED_DIR/daemon/roster.json" || "$(jq -r
    '.workers["w-share"].sessionId' "$SHARED_DIR/daemon/
    roster.json" 2>/dev/null)" == "s-share-NEWER" ]] && ok=0
assert_eq 0 "$ok" "second migration run is a no-op (marker file)"
rm -rf "$p1" "$p2"

summary

```

8.14 tests/test_add_remove.sh

```

#!/usr/bin/env bash
# test_add_remove.sh – exercises claude-add-account.sh and
# claude-remove-account.sh end-to-end:
# * add suggests the next free claudeN
# * add creates the dir with the right symlinks
# * add writes a shell-safe alias line
# * add refuses to overwrite an existing dir
# * add accepts a custom alias name
# * remove drops the alias line
# * remove archives the dir by default, --delete removes it

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Start with two existing accounts and a unified shared store.
make_account acct1 >/dev/null # side-effect only; returned path
    not needed

```

```

make_account acct2 >/dev/null # side-effect only; returned path
    not needed
# shellcheck disable=SC2119 # test intentionally calls run_unify
    with no args
run_unify >/dev/null 2>&1

it "claude-add-account --yes uses sensible defaults"
run_add_account --alias claude3 --yes >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "exit 0"
new_dir="$HOME/.claude-claude3"
assert_dir "$new_dir" "config dir created"
assert_symlink_to "$new_dir/projects" "$SHARED_DIR/projects"
    "projects linked"
# $11.4 own-settings: settings.json is each dir's OWN real file
    (NOT a shared
# symlink) so per-alias permissions/model/hooks never leak. It is
    seeded from
# the shared template, so it carries the enabledPlugins map.
    Meanwhile the plugin
# CACHE (plugins/) and history stay SHARED symlinks.
assert_not_symlink "$new_dir/settings.json" "settings.json is OWN
    (not a shared symlink)"
assert_file "$new_dir/settings.json" "own real settings.json
    exists"
assert_jq "$new_dir/settings.json" 'has("enabledPlugins")' "true"
    "own settings.json carries enabledPlugins (seeded from
    template)"
assert_symlink_to "$new_dir/plugins" "$SHARED_DIR/plugins"
    "plugins (cache) still shared"
assert_symlink_to "$new_dir/history.jsonl" "$SHARED_DIR/
    history.jsonl" "history.jsonl still shared"
assert_symlink_to "$new_dir/CLAUDE.md" "$SHARED_DIR/CLAUDE.md"
    "CLAUDE.md linked"
assert_file_contains "$ALIAS_FILE" "alias claude3=" "alias line
    written"
assert_file_contains "$ALIAS_FILE" "CLAUDE_CONFIG_DIR=$new_dir"
    "alias points at new dir"

it "claude-add-account refuses to overwrite an existing dir"
( run_add_account --alias claude3 --yes >/dev/null 2>&1 )
rc=$?
cond=$(( rc != 0 ? 0 : 1 )); assert_eq 0 "$cond" "exits non-zero
    when dir exists"

it "claude-add-account accepts a custom alias name and custom dir"
custom_dir="$HOME/.claude-mywork"
run_add_account --alias mywork --dir "$custom_dir" --yes >/dev/
    null 2>&1
assert_dir "$custom_dir" "custom dir created"
assert_symlink_to "$custom_dir/projects" "$SHARED_DIR/projects"
    "projects linked"
assert_file_contains "$ALIAS_FILE" "alias mywork=" "custom alias
    written"

```



```

it "claude-add-account validates alias names"
( run_add_account --alias "bad name" --yes >/dev/null 2>&1 )
rc=$?
cond=$(( rc != 0 ? 0 : 1 )); assert_eq 0 "$cond" "rejects bad
    alias"

it "claude-add-account is non-interactive without --yes
    (CMA_NONINTERACTIVE=1)"
export CMA_NONINTERACTIVE=1
run_add_account --alias claude4 >/dev/null 2>&1
rc=$?
unset CMA_NONINTERACTIVE
assert_eq 0 "$rc" "exit 0 without --yes"
assert_dir "$HOME/.claude-claude4" "config dir created via
    defaults"
assert_file_contains "$ALIAS_FILE" "alias claude4="
    "alias written non-interactively"
# Regression (cma_run_provider migration mis-fire): adding more
    accounts must
# not chop previously-written alias lines. All earlier aliases
    must survive.
assert_file_contains "$ALIAS_FILE" "alias claude3=" "earlier
    claude3 alias survives"
assert_file_contains "$ALIAS_FILE" "alias mywork="
    "earlier mywork alias survives"

it "claude-remove-account --archive moves the dir aside"
run_remove_account --alias claude3 --archive --yes >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "exit 0"
cond=1; [[ ! -d "$HOME/.claude-claude3" ]] && cond=0; assert_eq 0
    "$cond" "original dir gone"
archived="$(find "$HOME" -maxdepth 1 -name '.claude-
    claude3.removed.*' -type d 2>/dev/null | head -1)"
cond=1; [[ -n "$archived" ]] && cond=0; assert_eq 0 "$cond"
    "archived sibling exists"
assert_file_not_contains "$ALIAS_FILE" "alias claude3=" "alias
    line removed"

it "claude-remove-account --delete actually deletes"
run_remove_account --alias mywork --delete --yes >/dev/null 2>&1
cond=1; [[ ! -d "$HOME/.claude-mywork" ]] && cond=0; assert_eq 0
    "$cond" "dir deleted"
# "find printed nothing" is also what a find that never ran
    prints, so pin the
# find down at both ends: it must exit 0, and the same sweep
    without the
# .removed filter must still see the sandbox's other account dirs.
    Only then
# does an empty archive list mean "--delete really deleted".
_ar_all="$(find "$HOME" -maxdepth 1 -name '.claude-*' | grep -c .
    || true)"
_ar_hits="$(find "$HOME" -maxdepth 1 -name '.claude-
    mywork.removed.*')"; _ar_rc=$?
assert_eq 0 "$_ar_rc" "the archive-sibling find ran (exit 0)"

```

```

cond=1; [[ "${_ar_all:-0}" -ge 1 ]] && cond=0; assert_eq 0
"$cond" "the sweep can see \${HOME}'s account dirs
($_ar_all found)"
cond=1; [[ -z "$_ar_hits" ]] && cond=0; assert_eq 0 "$cond" "no
archive sibling"

it "claude-remove-account rejects unknown alias"
( run_remove_account --alias nonexistent --yes >/dev/null 2>&1 )
rc=$?
[[ $rc -ne 0 ]]; assert_eq 0 $? "unknown alias errors out"

summary

```

8.15 tests/test_list.sh

```

#!/usr/bin/env bash
# test_list.sh - claude-list-accounts.sh reports accurate state.

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Empty-host case first.
it "lists nothing when no accounts exist"
# shellcheck disable=SC2119 # test intentionally calls
# run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
cond=1; [[ "$out" == *"ALIAS"* ]] && cond=0; assert_eq 0 "$cond"
"header always printed"
echo "$out" | grep -E '^-' >/dev/null; rc=$?
cond=$(( rc != 0 ? 0 : 1 )); assert_eq 0 "$cond" "no account rows"

make_account acct1 >/dev/null # side-effect only; returned path
# not needed
make_account acct2 >/dev/null # side-effect only; returned path
# not needed
# shellcheck disable=SC2119 # test intentionally calls run_unify
# with no args
run_unify >/dev/null 2>&1
run_add_account --alias claude3 --yes >/dev/null 2>&1

it "shows every account dir with creds + link status"
# shellcheck disable=SC2119 # test intentionally calls
# run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
assert_file_contains <(printf '%s\n' "$out") ".claude-acct1"
"acct1 listed"

```



```

assert_file_contains <(printf '%s\n' "$out") ".claude-acct2"
    "acct2 listed"
assert_file_contains <(printf '%s\n' "$out") ".claude-claude3"
    "claude3 listed"
assert_file_contains <(printf '%s\n' "$out") "claude3" "claude3
    alias resolved"

it "reports the shared store path"
# shellcheck disable=SC2119 # test intentionally calls
    run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
assert_file_contains <(printf '%s\n' "$out") "$SHARED_DIR"
    "shared store path printed"

it "flags accounts with missing credentials"
# claude3 was created by add-account; it has no .credentials.json.
# shellcheck disable=SC2119 # test intentionally calls
    run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
line="$(echo "$out" | grep ".claude-claude3" || true)"
cond=1; [[ "$line" == *" no *" ]] && cond=0; assert_eq 0 "$cond"
    "claude3 creds:no"
line1="$(echo "$out" | grep ".claude-acct1" || true)"
cond=1; [[ "$line1" == *" yes *" ]] && cond=0; assert_eq 0
    "$cond" "acct1 creds:yes"

summary

```

8.16 tests/test_export.sh

```

#!/usr/bin/env bash
# test_export.sh – claude-export-docs.sh produces HTML and PDF,
    expands
# <!-- INCLUDE: path --> markers, and the output PDF starts with
    "%PDF-".

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

# The doc-export pipeline hard-requires pandoc (cma_require
    pandoc) plus at
# least one PDF engine. On hosts without that tooling (minimal
    servers, a
# stock macOS), SKIP rather than fail – the feature is genuinely
    unavailable,
# matching how test_opencode/verify_opencode_live SKIP when
    opencode is absent.
# Engine list mirrors claude-export-docs.sh's own probe order.
_have_pdf_engine() {

```

```

local e
for e in weasyprint wkhtmltopdf chromium chromium-browser
    google-chrome chrome; do
    command -v "$e" >/dev/null 2>&1 && return 0
done
return 1
}
if ! command -v pandoc >/dev/null 2>&1 || ! _have_pdf_engine; then
    echo
        "SKIP: doc-export prerequisites (pandoc + a PDF engine)
        not installed"
    exit 0
fi

make_sandbox
mkdir -p "$HOME/Documents/scripts"

# Tiny fixture markdown + a fixture script to include. We point
    the
# export script at this synthetic doc instead of the real one.
fixture_script="$HOME/Documents/scripts/sample.sh"
cat > "$fixture_script" <<'EOF'
#!/usr/bin/env bash
echo "SAMPLE_SCRIPT_BODY_MARKER"
EOF

fixture_md="$HOME/Documents/fixture.md"
cat > "$fixture_md" <<'EOF'
---
title: Fixture
---
# Fixture

Body paragraph above the include.

```bash
<!-- INCLUDE: scripts/sample.sh -->

```

End of fixture. EOF

```

it "MD_FILE env override is honored and HTML is generated"
MD_FILE="$fixture_md" "$SCRIPTS_DIR/claude-export-docs.sh"
>/dev/null 2>&1 rc=$? assert_eq 0 "$rc" "export exits 0"
assert_file "$HOME/Documents/fixture.html" "html generated"
assert_file "$HOME/Documents/fixture.pdf" "pdf generated"

```

```

it "markers are expanded in output" assert_file_contains
"$HOME/Documents/fixture.html"
"SAMPLE_SCRIPT_BODY_MARKER" "include expanded"
assert_file_not_contains "$HOME/Documents/fixture.html" "<!--
INCLUDE:" "markers removed"

```

```
it "PDF starts with the %PDF- magic bytes" head -c 5 "$HOME/
Documents/fixture.pdf" | grep -q '^%PDF-' assert_eq 0 $? "PDF
magic header present"
```

```
it "regeneration is non-destructive (file timestamps move
forward)" MD_FILE="$fixture_md" "$SCRIPTS_DIR/claude-export-
docs.sh" >/dev/null 2>&1 assert_file "$HOME/Documents/
fixture.html" "html still present after rerun" assert_file "$HOME/
Documents/fixture.pdf" "pdf still present after rerun"
```

summary

---

## ## 9. Troubleshooting

Symptom	Likely cause	Fix
		-----
		-----
		-----
`claude1` works, `claude2` shows empty memory	Symlinks weren't created in `~/.claude-milos85vasic2nd/`.	Re-run `claude-unify.sh` – it's idempotent.
New account can't see history	Forgot to reload shell after `claude-add-account.sh`.	`exec \$SHELL -l` or `source ~/.local/share/claude-multi-account/aliases.sh`.
Plugin loads fail with "plugin not found"	Manifest path points outside the shared store.	Re-run `claude-unify.sh` – the path-rewrite pass will fix it.
Both accounts see *each other's* drafts in the queue	`sessions/` is shared and two processes are running at once.	Don't run two accounts concurrently, or remove `sessions` from `SHARED_ITEMS`.
`pandoc` fails to find a PDF engine	No weasyprint/wkhtmltopdf/chromium installed.	`brew install weasyprint` or `apt install weasyprint`.
Want to start completely over		
-		
`claude-rollback` then re-run		
`install.sh`.		

## ## 10. Sharing the ecosystem with OpenCode

The same plugin ecosystem that the multi-account setup unifies can also be shared with a host-installed [OpenCode](https://opencode.ai). Claude Code plugins are runtime-specific (hooks + JS), but their portable contents – Anthropic-format **\*\*Skills\*\*** (``SKILL.md``), **\*\*MCP servers\*\*** (``mcp.json``), and the user **\*\*`CLAUDE.md`\*\*** – map directly onto OpenCode's native ``skills.paths``, ``mcp{}``, and ``instructions[]`` config keys.

``claude-opencode-sync`` performs this translation in one idempotent, additive pass (it never clobbers OpenCode's own providers or servers):

```
``bash
claude-opencode-sync --dry-run --stats # preview
claude-opencode-sync # apply (prior config is
backed up)
```

It scans the plugin cache, parses both the wrapped (`{"mcpServers": {...}}`) and bare (`{name:{...}}`) `.mcp.json` shapes, expands `{CLAUDE_PLUGIN_ROOT}`, deduplicates servers shipped by multiple plugins, and applies a conservative enable policy — enabling only no-auth/no-secret servers by default so OpenCode startup stays fast, while configuring all the rest `enabled:false`, ready to opencode mcp auth. On the reference host the latest recorded live run (`scripts/tests/proof/00-summary.txt`, 2026-07-27, opencode 1.18.5) measured `skill_paths=163`, `skills_resolved=1356`, `mcp_total=144` with `mcp_enabled=7` by the default allowlist and `mcp_connected=26`, `mcp_failed=0`.

End-to-end proof, writing inspectable evidence to `scripts/tests/proof/` and a dated `PROOF.md` tying it together:

```
bash scripts/tests/run-proof.sh
```

It runs six legs, each with its own log in that directory: the hermetic sandbox suite (`40-sandbox-suite.log`), live OpenCode verification against the real binary and real config (`41-live-verify.log`), live provider-alias verification (`42-live-providers.log`), live alias verification (`43-live-aliases.log`), alias end-to-end against provider endpoints (`44-alias-e2e.log`), and the constitution/conformance static checks (`45-constitution.log`). It acquires the suite lock at the outermost entry point, and the nested `run-all.sh` inherits that lock rather than deadlocking against it.

Full guide, architecture diagrams, and the enable-policy flowchart live in **OpenCode\_Integration.md** and `docs/diagrams/`.

---

## 11. Provider aliases — running other LLMs through Claude Code

The same alias machinery that powers `claude1..N` also drives **provider aliases**: a Claude Code alias per LLM provider whose key you keep in your keys file, pointed at the strongest model that provider will actually serve you (see *Model tier* below — strongest **paid** when the account has credit, strongest **free** when it does not). The command is `claude-providers` and it is **fully dynamic** — no provider, base URL, or model ID is hardcoded.

### Data flow (nothing hardcoded)

1. `~/api_keys.sh` provides the API-key *variable names* (read by `grep`, never executed).
2. `models.dev/api.json` (fetched + cached, with graceful offline degrade) provides each provider's endpoint, transport hint (npm), and full model catalog with cost/context/reasoning metadata.
3. `providers_resolve.py` resolves each key into a record: provider id, alias, base URL, transport (native iff the provider speaks the Anthropic API, else router), and a strong + fast model chosen from the metadata.
4. The bundled **LLMsVerifier** submodule (or a lightweight HTTP probe) optionally verifies the key + model before the alias is activated.
5. For each provider: a non-secret env file, a `cma_run_provider <id> alias`, a `~/ .claude-prov-<id>` config dir linking all shared items (so every plugin is available), and the always-on plugin set are generated.

Two tiny editable inputs cover the only things data can't infer: `providers/key-aliases.json` (key-name → provider normalization) and `providers/overrides.json` (per-provider pins, e.g. a short dseek alias or a native `/anthropic` endpoint).

### Model tier — strongest paid if funded, strongest free if not

Model choice is **credit-aware**, and this is a hard rule for every provider alias the toolkit generates:

Credit state of the provider account	Model the alias runs
Credit / purchased tokens available	the strongest <b>paid</b> model that provider serves and that passes verification
No credit	

Credit state of the provider account	Model the alias runs
	the strongest <b>free</b> model (free tier, \$0 cost) that passes the same verification
Unknown / undeterminable	treated as <i>no credit</i> — the free choice

Three things are worth being explicit about.

**Why "unknown" resolves to free.** The two mistakes are not symmetric. Pinning a paid model on an unfunded key produces a 402/403 the moment you launch, and you are left holding a dead alias at the exact moment you wanted to work. Pinning a free model on a funded key costs you some capability and nothing else, and the next claude-providers sync upgrades it as soon as the credit signal is readable. So an inconclusive billing probe, a provider that exposes no balance endpoint, or an offline catalog all fall to the conservative branch.

**It is a floor, not a cap.** Within whichever tier applies the selection is still the *strongest* model available: the rule constrains which models are eligible, it does not lower the bar inside that set. It applies to both slots — the strong model (ANTHROPIC\_MODEL) and the fast/background model (ANTHROPIC\_SMALL\_FAST\_MODEL) — and to every alias produced by sync --multi.

**A human pin always wins.** If scripts/providers/overrides.json names a strong\_model / fast\_model for a provider, that is what runs; the tier logic only decides the unpinned case. Pinning is the supported way to say "I know what I'm paying for on this provider, stop deciding for me."

The tier split constrains — rather than replaces — the capability ranking the resolver already applied (reasoning-capable first, then newest release, then largest context window, with output cost as a flagship tie-break for the strong slot; cheapest tool-call-capable model for the fast slot). Verification is unchanged and unconditional: a free model is not exempt from the sentinel and tool-calling probes, so "cheapest that works" never degrades into "cheapest".

The enforcement mechanism (in providers\_resolve.py and in the bundled **LLMsVerifier**, which carries the corresponding detection) landed alongside this section. What is written above is the **behavioural contract**; consult the source for the current function, flag, and field names.

## Transports

- **native** — claude runs directly with ANTHROPIC\_BASE\_URL, ANTHROPIC\_AUTH\_TOKEN, ANTHROPIC\_MODEL, ANTHROPIC\_SMALL\_FAST\_MODEL.
- **router** — launches through a bundled Go fork of claude-code-router (ccr code), which translates Anthropic ↔ OpenAI/Gemini. The provider entry is written into the ccr config with the live key at launch (chmod 600), so the toolkit itself never persists secrets.

Both transports export the model into the environment. `lib.sh` exports `ANTHROPIC_MODEL`, `ANTHROPIC_SMALL_FAST_MODEL`, the four `ANTHROPIC_DEFAULT_{OPUS,SONNET,HAIKU,FABLE}_MODEL` tier maps, and `CLAUDE_CODE_MAX_CONTEXT_TOKENS` for **both** — it used to do so only in the native branch, which left router aliases resolving Claude Code's own tier ids client-side and dying in an unbreakable compaction loop. Extending those exports to the router transport is also what exposed the router defect described next: it is the change that put a raw vendor-prefixed catalog id into the request body for the first time.

## Router selector semantics — the / that killed 16 of 24 aliases

A route in the router's config is `provider,model` on disk. But the router *also* honours a **client-supplied selector** in the request body's `model` field, so a caller can pin an exact upstream for a single request instead of taking the server-configured `Router.default.parseExplicitSelector` (`submodules/claude-code-router/internal/router/selector.go`) recognises **two** separators — a comma and a slash — and whichever appears **leftmost** decides the split, so one string can never be parsed two different ways. If neither separator is present, or either half would be empty, it is not a selector at all and normal routing applies.

The two separators look symmetric and are not, and treating them as if they were is what broke the alias fleet:

- A **comma** is unambiguous. It is this project's own on-disk route syntax and it appears in no model id anywhere. So when a comma-form selector names a provider that is not configured, the caller really did ask for something absent, and the request **must fail loudly**. Quietly redirecting it to `Router.default` would send the request — and the bill — to an upstream the caller never chose.
- A **slash** is ambiguous. It is simultaneously Node CCR's `Provider/model` wire format *and* the near-universal vendor/model catalog convention: `deepseek-ai/...`, `Qwen/...`, `nvidia/...`, `zai-org/...`

**The live defect (2026-07-27).** Once `lib.sh` began exporting `ANTHROPIC_MODEL` for the router transport too, Claude Code put the raw catalog id in the request body. The router read the **vendor** as a provider name, found no such provider in the per-alias config, and returned an unknown-provider error instead of falling through to `Router.default`. The gateway answered **503 in 1-4 ms — before any network call** — and Claude Code retried with backoff until the 180-second launch bound expired (kilo logged 11 gateway 503s spanning 179 s; 8-12 per affected alias in `~/.claude-code-router/<id>/service.log`). Blast radius: **16 of the 24 verified aliases** — every one whose model id carries a vendor prefix — including `helixagent` (`HelixAgent/HelixLLM`).

What made this so hard to see is that **the on-disk route was correct the whole time**. `"kilo,nvidia/nemotron-..."` parses fine: its comma comes first, so the leftmost rule splits on the comma exactly as intended. The config read back healthy while every launch died.

**The fix decides from evidence, not from the separator.** On the slash form:

- if some configured provider actually serves the **whole string**, it is a catalog id that merely looks like a selector, and it routes normally;
- if nobody serves it, the unknown-provider error still stands — so `Ghost/whatever` cannot silently reach `Default`;
- if the vendor prefix **collides** with a real provider name (provider `nvidia` serving `nvidia/nemotron-...`), it resolves to the whole id, because the split half is absent from that provider's `Models` while the full string is present.

The comma form's loud failure is unchanged.

**Why every earlier gate was green — this is the part to internalise.** The provider-verification stack has four layers, and only the last one touches the code path that broke:

Layer	What it does	Could it see this?
1. Existence ( <code>providers-verify.sh</code> )	curls the provider's own <code>base_url</code> — sentinel echo + tool-calling probe	<b>No</b> — never touches the ccr route path
2. Semantic ( <code>providers-semantic.sh</code> )	curls the same <code>base_url</code> through <code>LLMsVerifier</code> 's code-visibility driver	<b>No</b> — same reason
3. Status/config checks		



Layer	What it does	Could it see this?
	reads <code>status.json</code> and the generated per-alias config	<b>No</b> — the config was correct
4. Live TUI launch ( <code>verify_superpowers_tui.sh</code> )	launches real Claude Code <i>through the alias</i> , i.e. through <code>ccr</code>	<b>Yes — only here</b>

Layers 1-3 were green throughout, and honestly so: the provider endpoints really did work. The request never reached them. That asymmetry is the entire reason the layer-4 leg exists, and the reason `claude-release-gate.sh` is mandatory before a release commit — a suite that only probes upstreams verifies the upstreams, not the product.

## Token-limit guards — the arithmetic, and which direction is dangerous

Every provider alias exports two guards before launching Claude Code, and they are **co-derived, not independent**, so the pair can never sum past the provider's context window:

- `CLAUDE_CODE_MAX_OUTPUT_TOKENS` — the output cap.
- `CLAUDE_CODE_AUTO_COMPACT_WINDOW` — the auto-compaction **trigger**.

Leaving either unset is not a safe fallback. Claude Code's own default for an unknown model is 128000 output tokens, so "no export" is a request for 128000.

The derivation runs in two stages (`lib.sh`, the `cma-token-guards` block):

1. **Carve.** The output cap is carved *out of* the context rather than trusted alongside it: `out = min(catalog output, context - CMA_INPUT_FLOOR)`, bounded by the CLI's own 128000 ceiling and floored at 8192. `CMA_INPUT_FLOOR` defaults to **160000**.
2. **Window, with a tool-schema reserve.**  
`window = context - out - CMA_TOOL_TOKEN_BUDGET`, where `CMA_TOOL_TOKEN_BUDGET` defaults to **80000**. The reserve exists because the endpoint counts *tool schemas* as input too, and a window computed as `context - out` pretends the request is only text plus output.
3. **Compression-loop guard.** If that window falls below `CMA_MIN_COMPACT_WINDOW` (default **120000**), Claude Code would trigger compaction on every single request — the system prompt and tool schemas alone exceed the window — and

never get any work done. So the window is bought back out of the output cap, and **in that branch the effective output cap becomes**

$$\text{out} = \text{context} - \text{CMA\_MIN\_COMPACT\_WINDOW} - \text{CMA\_TOOL\_TOKEN\_BUDGET}$$

For a 262144-token provider: the carve gives  $\text{out} = 102144$ , whose window is  $262144 - 102144 - 80000 = 80000$  — below the 120000 minimum — so the guard rebalances to  $\text{out} = 262144 - 120000 - 80000 = 62144$ ,  $\text{window} = 120000$ , and  $62144 + 120000 + 80000 = 262144$  exactly.

**Be exact about which direction is dangerous.** An underestimated tool budget does **not** merely "compact earlier". Because the effective output cap is  $\text{context} - \text{min\_window} - \text{budget}$ , too small a budget **inflates** the output cap, the request overflows the context, and the launch dies with a 400. Measured live on a host with **227 enabled plugins**, whose real tool input is **158112** tokens against the 80000 default:

$$46536 \text{ text} + 158112 \text{ tools} + 62144 \text{ output} = 266792 \quad \text{on a } 262144 \text{ window}$$

Note further what that host cannot fix by tuning:  $158112 + 120000$  already exceeds 262144, so **no output cap makes a 262144-context provider fit at all** there. The remedies are to shrink the enabled plugin surface or to use a wider-context provider (nvidia at 1000000 and xiaomi at 1048576 fit it comfortably). Override per host with CMA\_TOOL\_TOKEN\_BUDGET in the provider env or the shell.

A context too small to reserve the budget yields no window at all. That is deliberate: such a provider cannot serve Claude Code's tool prefix regardless, and the launch must fail loudly rather than be lied to by a guard that cannot hold.

Also note what the window is not. It is a **compaction trigger**, not a hard cap — it cannot bound the tool schemas themselves, which are assembled before any conversation exists to compact. That is precisely why the separate tool budget is needed.

## **HelixAgent: a local GPU backend, and why auto-start is opt-in**

The helixagent alias routes Claude Code to a local HelixLLM backend (<http://127.0.0.1:7061/v1>, router transport — repointed 2026-09-03 off :18434, which is the llama.cpp coder container, a different service), pinned in scripts/providers/helixagent.json to context\_limit **229376** and max\_output **8192**. The context is sized

for the multi-turn *agent loop*, not for the first request: each executed tool feeds its output back and the loop accumulates toward ~182K tokens, which overflowed the earlier 180224 pin.

That backend shares a single GPU with **HelixCode**, and the two cannot run at once — HelixCode drives HelixLLM in *coder* mode (-c 24576 --parallel 8), while helixagent needs *claude* mode (-c 229376 --parallel 1). Coder mode is the default operational state, so helixagent is honestly demoted to unverified and refused by the launch gate until the operator flips the backend with `helix_code/scripts/helixllm-mode.sh claude` (that companion repo, not this toolkit).

**CMA\_HELIX\_AUTOSTART — opt-in, default OFF.** The launch wrapper can start HelixLLM for you, but it will not do so unless you ask:

```
CMA_HELIX_AUTOSTART=1 helixagent
accepted: 1 | true | yes | on
```

Anything else — including the unset default — is OFF. The reason is that booting a Helix service is a heavyweight, machine-wide side effect: HelixLLM claims the single GPU, and starting it in claude mode **evicts whatever HelixCode had running in coder mode**. A provider alias must not do that to a host merely because someone typed an alias name. When auto-start is disabled and the backend is not answering, the wrapper still says so and still prints the exact command to run — the operator keeps both the information and the choice. With it enabled, the wrapper locates `helixllm-mode.sh`, starts claude mode, and polls `/health` for up to 120 s before giving up with a diagnostic.

## Safety

Provider config dirs are **excluded from account auto-detection**, so they never merge into real-account auth and never disturb `claude-unify` / `claude-add-account`. Existing `claudeN` accounts keep working unchanged, and new accounts are still created the same way.

During `claude-providers sync --multi`, the API key is passed to `model_verify.py` via the **CMA\_PROBE\_KEY environment variable** — never as a command-line argument. This means the key never appears in `ps aux` or `/proc/<pid>/cmdline`, preventing accidental secret exposure to other local users or process-monitoring tools. The key is injected per-invocation and is never written to the alias file, the env file, or any repository path.

## Commands

```
claude-providers sync # discover + create/refresh every
 provider alias
claude-providers list # alias, provider, transport,
 strong/fast model
claude-providers show <id> # one provider's resolved env
claude-providers remove <id> # remove alias + env, back up the
 config dir
claude-providers add --from-key VAR --id PROVIDER # register a
 mapping, then sync
```

## Session color — auto-applied (v1.10.0)

Each provider session is now tagged with the alias's deterministic color **automatically**. The toolkit writes an agent-color record (`{"type": "agent-color", "agentColor": "<color>", "sessionId": "<id>"}`) into the session's `.jsonl` — the same record Claude Code's `/color` command writes — via `claude-session apply-color`, called by the `cma_run_provider` wrapper. This is the **only** non-interactive way to set the prompt color: `/color` itself is TUI-only and `claude -p '/color x'` is a no-op, with no CLI flag, settings key, or env var. (That survey was performed against Claude Code **2.1.195**; the reference host now runs **2.1.220**, and the finding has not been re-surveyed against it.) Verified live that the record is written, is idempotent (re-applied only when the color changes), and persists across `claude --resume`. The toolkit cannot observe the TUI, so confirm the prompt-bar rendering visually on first launch. (Documented in `docs/Provider_Aliases_User_Guide.md` and `docs/SESSION_COLOR.md`.)

Full details, overrides, verification, and troubleshooting live in `docs/Provider_Aliases_User_Guide.md`; diagrams in `docs/diagrams/provider-aliases.md`.

## 12. Per-project auto-session & per-alias color

A **bare** alias launch — native (`claude1`, `claude2`, ...) or provider (`deepseek`, `kim-for-coding`, ...) with no arguments — resumes, or the first time creates, **one long-lived Claude session per project root**. The session id is derived deterministically from the project root path (the git working-tree root if you are inside a repo, otherwise `$PWD`), so every alias — and every subdirectory of the repo — maps to the *same* session: switch aliases and you continue the same ongoing work.

The session is named in lowercase kebab-case from the project root directory's basename (e.g. `claude_toolkit` → `claude-toolkit`; `Android 15` → `android-15`). Whitespace and special characters are

sanitized to hyphens, and leading/trailing separators are trimmed. The name is applied on **both create and resume**, so a session created *without* a name — by an older wrapper or a plain claude invocation — finally gets named the next time you bare-launch an alias into it.

The moment you pass any argument (a prompt, -p, --resume, --session-id, any flag), the wrapper steps aside and your arguments go to claude verbatim — no auto-session is injected.

**Color is auto-applied (v1.10.0).** Each alias maps deterministically to one of Claude Code's 8 prompt colors, and the toolkit now applies it for you: on a bare launch the wrapper writes an agent-color record (`{"type": "agent-color", "agentColor": "<color>", "sessionId": "<id>"}`) into the session's `.jsonl` — the exact record Claude Code's `/color` command writes — via `claude-session apply-color`. It runs **before** launch (to colour a resumed session) and **after** exit (to colour a freshly-created one). It is idempotent: the record is re-written only when the color actually changes, so switching aliases re-colors the *same* session and the file never grows unbounded. This is the **only** non-interactive mechanism: `/color` is TUI-only and `claude -p '/color x'` is a no-op — there is no CLI flag, `settings.json` key, or environment variable for it (surveyed against claude 2.1.195; the reference host now runs 2.1.220, against which the survey has not been repeated). Verified live that the record is written, is idempotent, and persists across `claude --resume`. The toolkit cannot programmatically observe the TUI, so confirm the prompt-bar rendering visually on first launch.

Implemented by `scripts/claude-session.sh`, driven by the `cma_run / cma_run_provider` alias wrappers in `scripts/lib.sh`. Full details, the naming/color tables, and live verification are in **docs/SESSION\_COLOR.md**.

## 13. TOON utility

`scripts/toon.mjs` (and its Python wrapper `scripts/toon_encode.py`) encode JSON into **TOON** (Token-Oriented Object Notation) — a compact encoding that cuts token use for structured data in LLM prompts by declaring array fields once. See **docs/TOON\_Integration.md**.

## 14. Authoring rule: the generated alias file is self-contained

This section is for anyone **editing the toolkit**, not for operators. It records a constraint that is invisible from the source file and that has now broken twice.

`cma_run_provider` — the launch wrapper for every provider alias — does not run from `lib.sh`. It is **emitted into the alias file** by a quoted heredoc in `lib.sh`:

```
cat <<'CMA_PROV_BODY_EOF'
... the entire cma_run_provider body ...
CMA_PROV_BODY_EOF
```

That alias file (`~/.local/share/claude-multi-account/aliases.sh`) is sourced by the user's **interactive shell**, and it **never sources `lib.sh`**. So at the moment this body actually executes, none of `lib.sh`'s helpers exist — `cma_log`, `cma_warn`, `cma_die`, `cma_require` and friends are all absent.

**The rule: inside the heredoc body, never call a `lib.sh`-only helper bare.** A bare call does not degrade gracefully. It prints

```
cma_warn: command not found
```

and **destroys the message it was supposed to carry**. That is not hypothetical: it shipped in v1.26.6 and is preserved in this repository's own evidence — `scripts/tests/proof/providers-helixagent-superpowers.txt` has that exact string on line 3, sitting precisely where the operator should have read the instruction to flip HelixLLM out of coder mode.

Two accepted idioms, and the difference between them:

```
Acceptable floor – guards against the shell error, but SILENTLY
DROPS the text.
command -v cma_log >/dev/null 2>&1 && cma_log "..."

Required where the message IS the feature – prints either way.
_hl_log() { if command -v cma_log >/dev/null 2>&1; then
 cma_log "$1"; else printf '[cma] %s\n' "$1" >&2; fi; }
_hl_warn() { if command -v cma_warn >/dev/null 2>&1; then
 cma_warn "$1"; else printf '[cma warn] %s\n' "$1" >&2;
 fi; }
```

The first form is the older convention and is enough to keep the shell quiet. It is *not* enough for a message the operator cannot proceed without — the HelixLLM coder-mode remedy is the only thing telling them what to do, so dropping it is as bad as garbling it. Use the fallback emitters there.

scripts/tests/test\_alias\_file\_selfcontained.sh enforces the floor mechanically. It lints the **generated** alias file (not the source) for bare calls to any lib.sh-only helper — folding multi-line \ and &&|| continuations first, since the real guard idiom spans lines and a naive per-line lint would report guarded code as bare — and then behaviourally executes the emitters in a shell with no lib.sh present, to prove the text actually arrives.

## 15. Bundled submodules and their pins

The toolkit vendors four submodules. Pins as of claude\_toolkit-1.26.7 (git submodule status):

Submodule	Pin	Role
submodules/LLMsVerifier	2a105fe7 (llmsverifier-v1.12.2-12-g2a105fe7)	The model-verification and semantic-code-visibility drivers behind provider verification layers 1-2.
submodules/claude-code-router	9accd18 (v0.4.9-5-g9accd18)	The Go router (ccr) behind the <b>router</b> transport — including the selector fix in §11.
submodules/go	go1.26.4	Vendored Go toolchain, built by claude-go-build.sh into ~/.local/share/claude-go.
submodules/challenges	2e3ef88 (helixcode-v1.1.0-18-g2e3ef88)	Conformance/challenge corpus used by the static checks.
submodules/containers	a432efa	Not on any toolkit code path.

Two pinning rules that are not obvious:

- **submodules/go must track a point release on release-branch.go1.N, and is pinned to whatever submodules/claude-code-router/go.mod requires.** Never bump it to origin/master: that is the unreleased next-major dev tree, and it carries no VERSION file — which claude-go-build.sh parses and hard-fails without. A bump must be followed by claude-go-build, claude-ccr-build, and the live router legs.
- **The vendored toolchain is opt-in.** install.sh only *symlinks* claude-go-build; it never builds Go from source, because downloading and compiling an ~80 MB compiler is not something an alias installer should do unasked. Consequently ~/.local/share/claude-go does **not** exist on a normal install, and every consumer of VENDORED\_GO must probe for the binary's

existence (command `-v`) rather than testing the variable — the variable is assigned a path string unconditionally and is therefore never empty, so a `${VENDORED_GO:-go}` fallback can never fire on its own.

## 16. Kimi Code: accounts and the family model

The Kimi Code CLI is a first-class sibling of Claude Code. The **family model** is an orthogonal axis to the provider backend: the provider engine (verification, model selection, aliases) stays shared, and each family supplies its own agent binary, home env var, account prefix, user-scope root, and launcher functions. The diagram `docs/diagrams/kimi-family.{mmd,svg}` shows the naming contract at a glance.

	Claude family	Kimi family
agent binary	<code>claude</code>	<code>kimi</code>
home env var	<code>CLAUDE_CONFIG_DIR</code>	<code>KIMI_CODE_HOME</code>
account prefix	<code>.claude-</code>	<code>.kimi-code-</code>
user-scope root	<code>~/.claude</code>	<code>~/.kimi-code</code>
account aliases	<code>claude1...N</code>	<code>kimi1...N</code>
shared store area	<code>\$SHARED_DIR/**</code>	<code>\$SHARED_DIR/kimi/**</code>

### 16.1 The namespace contract

- `claudeN` / `kimiN` — **native accounts** (a full per-account home).
- `<id>` (no prefix) — **always Claude Code** over that backend.
- `kimi-<id>` — **always Kimi Code** over the **same** backend.
- `kc-<id>` — Claude Code over a **Kimi-native** backend (legacy renamed).

`kimiN` account aliases must never be named `kimi-*/kc-*` (those are reserved provider namespaces), and `kc-*` ids never get a `kimi-kc-*` twin (the Kimi agent on a Kimi-native backend is the `kimiN` account).

### 16.2 Kimi accounts

`kimi-add-account --alias kimi1 --login` provisions a Kimi account at `~/.kimi-code-kimi1` and drives Kimi's **interactive device flow** (`kimi login`) — headless login is a non-goal. The mirror account commands are `kimi-remove-account`, `kimi-list-accounts`, `kimi-unify`, and `kimi-rollback`.



kimi-unify merges KIMI\_SHARED\_ITEMS = (AGENTS.md, plugins, skills, sessions, session\_index.jsonl) into \$SHARED\_DIR/kimi/. Sessions are workdir-keyed and home-independent, so cross-account resume works through the shared symlinks. AGENTS.md is promoted from the newest account and symlinked from every Kimi home — the ~/.claude/CLAUDE.md promotion analog. **Never merged** (per-account private): config.toml, credentials/, oauth/, device\_id, bin/, logs/, tui.toml. There is **no kimi-sync-state** — Kimi has no .claude.json analog; continuity comes entirely from the shared sessions/.

## 16.3 kimi-<id> provider aliases

For every provider id that verifies, sync/--multi also emits alias kimi-<id>="cma\_run\_kimi\_provider <id>", gated by the **same** status.json record as the Claude twin (single shared gate — refuse non-verified unless --force). The launcher renders ~/.kimi-prov-<id>/config.toml (provider type = openai|anthropic by base shape, base\_url, api\_key from the same env record, max\_context\_size from the same derived limits, default\_model) and runs KIMI\_CODE\_HOME="\$HOME/.kimi-prov-<id>" kimi -m "<host>/<strong-model>". There is **no cma-proxy** on this path (the Kimi CLI speaks the wire protocol natively) and **no CLAUDE\_CODE\_\* guards**. One strong model per kimi alias — fast pairing is a non-goal. Emit/suppress with --kimi-aliases / --no-kimi-aliases (default on); kimi-providers list carries an agent (claude/kimi) column.

## 16.4 Legacy rename (kimi-\* → kc-\*)

The legacy aliases that opened **Claude Code** on a **Kimi-native backend** are renamed **once** (automatically at sync, or via kimi-providers migrate-names): kimi-for-coding→kc-for-coding, kimi-for-coding2→kc-for-coding2, kimi-for-coding-highspeed→kc-for-coding-highspeed, kimi-k3→kc-k3, kimi-k2p7→kc-k2p7. The kimi- prefix now means only "Kimi Code CLI agent". The rename is idempotent (second run is a no-op) and reversible via backup\_and\_remove backups. Update scripts or muscle memory that reference the old aliases.

---

*Last revised 2026-09-05 for release claude\_toolkit-1.27.0 (OpenCode integration added 2026-06-06; provider aliases added 2026-06-16; runtime sync via cma\_run/claude-sync-state, CMA\_PROBE\_KEY secret-hygiene model, and cma\_realpath BSD portability hardening added 2026-06-28; per-dir own settings.json, router selector semantics, co-derived token guards, CMA\_HELIX\_AUTOSTART, and the self-contained alias-file authoring rule added 2026-07-27; Kimi Code accounts + kimi-<id> provider aliases and the kimi-\*→kc-\* rename added 2026-09-05). Maintain by editing this markdown*

*file and re-running MD\_FILE="\$PWD/  
Claude\_Multi\_Account\_Fine\_Tuning.md" bash scripts/claude-export-  
docs.sh.*